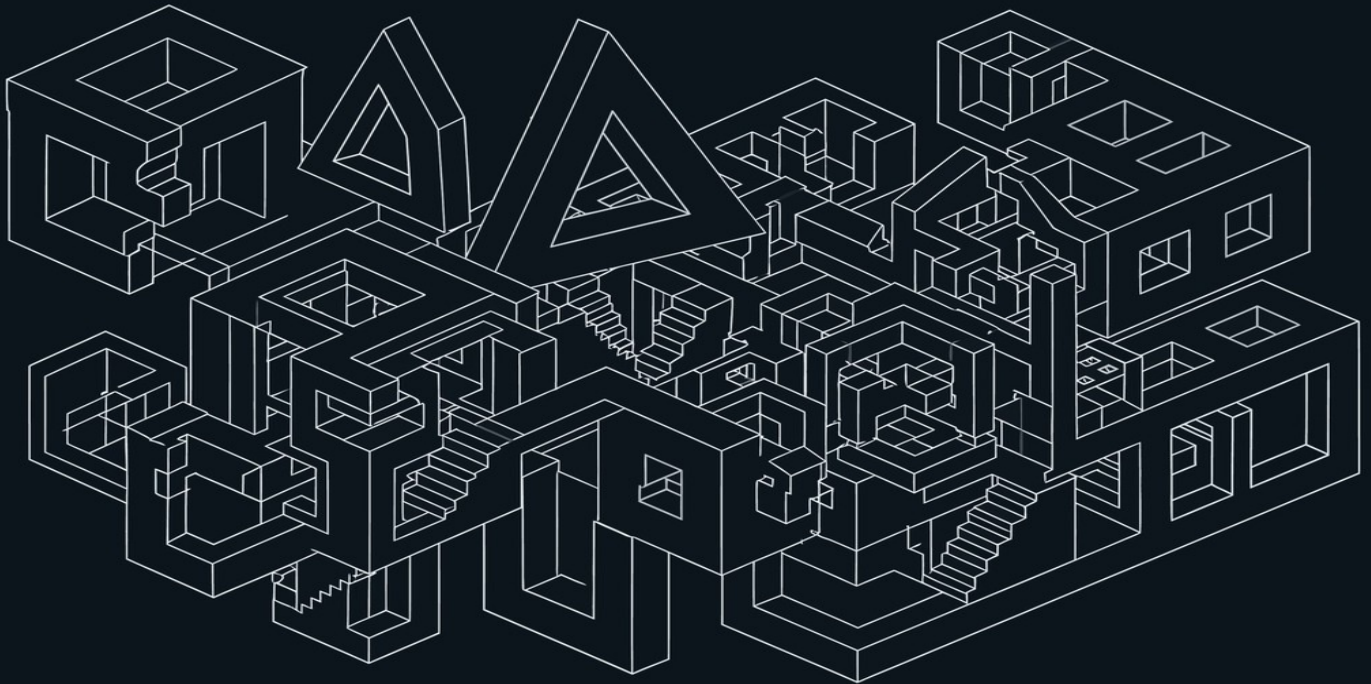


Agentic Spec-Driven Development

*A Practical Method for Using AI
to Build Complete Specifications
for Software, Products, and Knowledge Work*



ANATOLY VOLKHOVER

Agentic Spec-Driven Development

Agentic Spec-Driven Development

[Title Page](#)

[Copyright](#)

[Dedication](#)

[Epigraph](#)

[Acknowledgments](#)

[Why This Book?](#)

[Why Now](#)

[The Reader](#)

[Becoming a Handler](#)

[How to Use This Book](#)

[The Method](#)

[Just Tell Me What You Want?](#)

[Chapter 1: Prerequisites](#)

[Domain Expertise](#)

[AI Agent](#)

[Version Control](#)

[Project Structure](#)

[Chapter 2: Token Efficiency](#)

[Tokens](#)

[File Formats](#)

[Chapter 3: The AI Agent](#)

[Two Halves](#)

[What Makes It Agentic](#)

[All Talk, No Memory](#)

[Chapter 4: The Prompt](#)

[Command](#)

[Research](#)

[Suggest](#)

[Draft](#)

[Analyze](#)

[Explain](#)

[Critique](#)

[Combining Modes](#)

- [Iterate](#)
- [Plan](#)
- [Playbook](#)
- [Intent Over Instruction](#)
- [Guardrails](#)
- [Triggers](#)
- [The Language of Rules](#)
- [Challenge](#)
- [Chapter 5: Durable Rules](#)
 - [Rules of Engagement](#)
 - [The Rules Trap](#)
 - [How Reliable?](#)
 - [Still With Me?](#)
- [Chapter 6: The Bootstrap](#)
 - [CLAUDE.md](#)
 - [rule-analysis.md](#)
 - [rule-conflict-protocol.md](#)
 - [rule-conflict-log.md](#)
 - [Installation](#)
 - [Challenge](#)
- [Chapter 7: Auditability](#)
 - [The Audit Trail](#)
 - [Installing the Log](#)
 - [Anatomy of the Rule](#)
 - [Log in Action](#)
 - [Help the AI Help You](#)
- [Chapter 8: Sessions and Memory](#)
 - [Chat Session](#)
 - [Context Window](#)
 - [Context Rot](#)
 - [Tool Calls](#)
 - [File Memory](#)
 - [Triangulation](#)
 - [Challenge](#)
- [Chapter 9: Glossary](#)
 - [Lost in Translation](#)
 - [Project Glossary](#)

[Glossary in Action](#)

[Challenge](#)

[Chapter 10: Artifacts](#)

[When Text Isn't Enough](#)

[The Naive Path](#)

[The Better Path](#)

[Source of Truth and Artifacts](#)

[The Artifact Discipline](#)

[Artifact in Action](#)

[Challenge](#)

[Chapter 11: Trust, but Verify](#)

[Does AI Earn Your Trust?](#)

[Before You Ask](#)

[Manual Review](#)

[Polygraph](#)

[Challenge](#)

[Chapter 12: Course Correction](#)

[Validating Ground Truths](#)

[Updating Ground Truths](#)

[Full Audit](#)

[Challenge](#)

[Chapter 13: Dry Run](#)

[Total Recall](#)

[Recall in Action](#)

[Challenge](#)

[Chapter 14: Interactive Wiki](#)

[Chapter 15: Closing Pass](#)

[Sweep](#)

[Triage](#)

[Beyond Text](#)

[Challenge](#)

[Chapter 16: Spec-Driven Development](#)

[Afterword](#)

[Leave an Honest Review](#)

[About the Author](#)

[Appendix A: CLAUDE.md \(Bootstrap\)](#)

[Appendix B: rule-analysis.md \(Bootstrap\)](#)

[Appendix C: rule-conflict-protocol.md \(Bootstrap\)](#)

[Appendix D: rule-conflict-log.md \(Bootstrap\)](#)

[Appendix E: regen-all.md](#)

[Appendix F: disambiguate.md](#)

[Appendix G: specs/errata-flow.md \(v1\)](#)

[Appendix H: gen-errata-flow.md](#)

[Appendix I: errata-flow.html](#)

[Appendix J: specs/errata-flow.md \(v2\)](#)

[Index](#)

[Methodological Roots](#)

[Structure and Organization](#)

[Language and Meaning](#)

[Auditability and Traceability](#)

[Validation and Quality](#)

[Knowledge and Memory](#)

[Design and Architecture](#)

[Military and Intelligence Thinking](#)

[Cognitive Science and Human Factors](#)

[Anti-Patterns and Cautionary Principles](#)

[Process Frameworks](#)

Agentic Spec-Driven Development

*A Practical Method for Using AI to Build Complete Specifications
for Software, Products, and Knowledge Work*

Anatoly Volkhover

Agentic Spec-Driven Development

A Practical Method for Using AI to Build Complete Specifications for Software, Products, and Knowledge Work

Copyright © 2026 Anatoly Volkhover. All rights reserved.

No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means — electronic, mechanical, photocopying, recording, or otherwise — without prior written permission from the author, except for brief quotations used in reviews or scholarly analysis.

The methods described in this book are presented in good faith. The author and any contributors disclaim any liability arising from the use, misuse, or inability to use the information herein. Software tools, AI systems, and platform conventions change over time; readers are responsible for verifying that the techniques described still apply to the tools they are using.

Companion site, contact, and errata: agentic-spec.com

To Elena, who waited.

To my father, who would have loved this.

“Ask, and it shall be given you.”

— Matthew 7:7

Acknowledgments

This book owes a debt to Levon Hovhannisyan.

He was among the first to put the methodology — still half-formed, shifting week to week — to work on a real project. The friction he hit shaped the methods in these chapters.

Levon read drafts, pushed back, and asked the questions that tightened the prose and sharpened the methods. The book is much better for his help.

Why This Book?

Specifications have always mattered. Agentic AI makes them matter more.

The shift began in software engineering with *vibe coding*. You prompt an AI agent, it writes code, you review and fix what it got wrong — turn after turn, in the loop on every step. It feels like magic, until the project grows. Then the loop starts to slow you down.

As agents gained autonomy, the approach shifted. AI can now work through a large task on its own, without a check at every turn. That changes *when* your judgment is applied, not *who* applies it. The hard calls are still yours: the architecture, the trade-offs, the constraints. What changes is the timing. Instead of correcting the agent as it works, you settle those calls up front, in a specification the agent builds from. This is *spec-driven development*. When applied fully, the code becomes a generated artifact — the output of decisions you already made, not the place you make them. The engineer's judgment *relocates* (does not disappear) into the spec.

There is a catch. Almost no one can think of everything up front. You do not see a gap until a concrete task drags it into view — the edge case, the conflict, the decision you did not know you owed. Specifying a whole system with nothing prompting you is close to impossible. That is the problem this book takes on. The method builds the specification a little at a time, each step surfacing the next set of decisions. It leans on AI throughout — to research, draft, cross-check, and keep the project consistent — so the rote work moves to AI and your attention stays on the judgment only you can supply.

On a team, this shift also changes who owns the spec. The spec used to belong to the product manager: a product requirements document — a PRD — was written, then handed over the wall to engineering, one direction only, to become a design and then code. An autonomous agent will not wait at that wall. It needs product intent and technical judgment in one place: what the product should do and how it should be built, the requirements and the architecture, side by side. That is more than a PRD ever held. Now the

product manager and the engineer work the same spec, and they live in it full-time — it is where decisions get made, recorded, and changed.

The same shift is reaching other knowledge-work fields. Before a lawyer drafts the final contract, they work out the parties, the obligations, the consideration, the edge cases, and the termination conditions — drafting a contract is the *coding* step; all the prep work is the *specification* step. Lawyers do not call it that, but the shape of the work is the same. A researcher writes a protocol before the study runs. A policy writer drafts intent and scope before the policy text. An author drafts a synopsis and an outline before writing the book. In every case, detailed material is produced before the final stage of the work begins; that material is the specification. The method in this book applies to most such cases, not just to engineering. If anything, these fields are better positioned than software. Their final stage was always light — drafting a contract from a settled position, writing up a study that is already designed — never a heavy, separate build phase. Handing that step to AI is a relatively small leap.

The goal of the method is a complete, validated specification. The specification is the deliverable.

Sometimes the specification is the finished product itself — a contract ready for signing, a policy, a research protocol ready for approval by an institutional review board, a fully formatted book that goes to publication.

Sometimes the specification is a blueprint that an AI agent executes — software is coded, a site is built, a deck is generated. The final step is no longer yours: building from the spec is AI's job. That is why this book contains no code — not a gap, the point. There is no longer any code for *you* to write; your work is to make the spec complete enough that the agent executes it consistently, without guesswork. Your side of the work begins and ends in the specification.

The skill of building specs is durable. AI will keep getting better — fewer hallucinations, sharper reasoning, longer context windows — but as long as it remains a system that predicts text without truly understanding your project, the method holds. As models improve, the method works better, not worse. Every step asks the AI to reason about the context created earlier, and a

stronger model does so more effectively.

Why Now

Specifications are not new. What is new is using AI to build them. AI is not human, and most older methods for writing specs were built around what people can do. Some of those methods fall apart with AI. Others — methods that never worked well with people because they demanded too much patience or too much rigor — work surprisingly well when an agent runs them.

Here is a reality most people outside the field have not noticed: today's agentic AI and the original ChatGPT of late 2022 are very different tools. Agentic AI is far more capable, and it looks deceptively simple to operate. It is not. Working manually is crawling. Chatting with AI is walking. Operating an AI agent is driving — and driving takes training.

The Reader

The book is aimed at anyone whose deliverable is precise, detailed material — sometimes a blueprint that someone else (a person or an AI agent) executes, sometimes the finished product itself.

Two roles sit at the center of this shift — judgment moving up front, into a spec an agent builds from — because software felt it first:

- **Software architects and engineers** — for whom coding is increasingly automated, so the technical judgment that once came out turn by turn, in the back-and-forth with the agent, now has to be settled up front, in the spec.
- **Product managers** — whose craft has always been the spec, only now the spec must fuse product intent with technical judgment, co-authored with engineering, and stand complete enough for an autonomous agent to execute.

The same method serves everyone else in knowledge work:

- **Consultants** producing scopes of work, RFP responses, proposals, and decision memos.
- **Legal and patent practitioners** translating informal intent into precise contract language, settlement letters, policy text, and patent applications.
- **Researchers** writing clinical trial protocols, experimental scripts, and study procedures that a stranger should be able to run.
- **Policy and compliance professionals** writing corporate policies, regulatory submissions, and audit-grade documentation.
- **Marketing, brand, and communications professionals** writing go-to-market and launch plans, brand and tone-of-voice guidelines, positioning and messaging frameworks, content and campaign briefs, PR plans, and long-form assets like white papers and sales decks.
- **Operations and data professionals** writing SOPs, runbooks, incident playbooks, analysis specs, and dashboard requirements.
- **Technical writers** producing documentation, manuals, API references, and user guides.
- **Authors of instructional, reference, and how-to material** — textbooks, training curricula, and long-form content where structure and precision matter.
- **Educators and curriculum designers** writing course outlines, syllabi, lesson plans, and training scripts.

Each of these roles faces the same dilemma. Using AI without a framework carries a high risk of failure. When the deliverable is a blueprint, the downstream executor — a person or an AI agent, especially AI — fills the gaps with wrong assumptions, and the output misses the intent. When the deliverable is the end product, the product fails to do its job: the contract has loopholes, the policy is misread, the report misleads. Avoiding AI is not an option — the people who use it well will outpace you. This book gives you the framework for avoiding the worst of those failures.

The book speaks to solo practitioners and to small teams equally. Modern AI has made it possible for one person to do what used to require a group, but the method holds up just as well when several people share the project.

Becoming a Handler

A common assumption: if you know how to do a job, you can tell AI how to do it — just faster. An engineer expects AI to generate code. A lawyer expects AI to draft a contract. A researcher expects AI to design a clinical protocol. It does not work that way. The proof predates AI and is everywhere: many great engineers make poor managers. Doing the work and directing someone — or something — to do the work are different skills. This book is about the second skill, applied to AI.

Here is an analogy from the intelligence community. A handler does not do the field agent's job. The handler sets direction, provides context, judges what the agent brings back, and decides what to act on. The handler's skill is not the field agent's skill — it is the skill of running the field agent effectively. Apply the analogy to you and AI. You are not learning how AI is built internally. You are not studying its architecture or its training algorithms. You are learning how to direct AI by building a specification, getting reliable results from an unreliable yet extraordinarily capable partner.

Concretely: the spec's content — the vision, the structure, the architecture — comes from your domain expertise; this book teaches the tradecraft that turns AI into a reliable partner in producing it. The expertise itself is not taught here — each field has its own, far beyond the scope of a single book. If your field is software engineering, the relevant expertise is covered in my earlier book, *Become an Awesome Software Architect*.

How to Use This Book

The book is opinionated by design. You are getting one practitioner's method — the one that works — not a balanced survey of options. The method is grounded in several successful commercial projects. The language is deliberately plain — no literary flourish, just clarity — so the book reads equally well for native and non-native English speakers.

Throughout this book, "you" is the person responsible for the specification — the one whose intent must be captured and whose judgment cannot be delegated. Whether you are working alone, in a small team, or as one specialist inside a larger group, the method remains the same.

If you are working as a team, every member must read the book end to end so the team applies the same methodology. The split of responsibilities is up to you and most often follows each member's area of expertise.

Some of the methods in this book may look familiar. They are derived from manual, pre-AI disciplines proven over decades: requirements management, change control, auditability, validation, glossaries, and more. But those disciplines were designed for human teams, not for AI agents.

AI has its own weaknesses. It hallucinates — fabricates facts without hesitation. It defaults to overconfidence. It lacks the common sense any human teammate would have. It "forgets" what you told it moments ago. It claims to have completed work that has not started.

AI also has superhuman strengths. It can read a hundred-page specification in seconds and catch gaps a human reviewer would miss. It can compare a dozen documents and flag discrepancies. It can generate a polished visual artifact — a diagram, a chart, a slide deck — in less time than it takes you to describe what you want. No human team works that fast or that tirelessly.

In this book, the old, proven methods are tweaked — sometimes beyond recognition — to exploit AI's strengths and defend against its weaknesses. The resulting techniques may resemble pre-AI practices, but they work very differently underneath. To avoid confusion, the chapters do not call out which technique descends from which pre-AI practice; the *Methodological Roots* at the end of the book lists those practices for the curious. Treat every method here as new. If you happen to recognize the ancestry, fine — but do not assume you know how a method works until you have read the chapter and seen the examples.

Do not follow the methods blindly. Adapt them to your own workflows once you understand the principles. Do not assume; dig in, even when a concept seems obvious. The book is meant to be read in order, and it is optimized for brevity. It is a guide, not a reference.

A note before going further. If you are looking for recipes to follow without thinking, this book is not for you. The book builds the mental model you need to drive agentic AI for spec-driven development — not the engineering

underneath, but enough to understand why AI behaves the way it does and how to steer it. The book leans on frontier AI agents by design, and you are expected to put in the work to learn. If you want a packaged solution that does the job without your involvement, look for software built for your field — if any such option exists.

Intuition and Muscle Memory

Some of the examples and methods in this book may look too simple. They are simple by design. They are illustrations and suggestions — not production blueprints, not sacred prompt templates, not magic skill files. Their purpose is to build your intuition for working with AI. Think about how you would make these techniques more robust — as an exercise.

Some of my explanations may not be scientifically accurate. That is intentional. My goal is not to fill your head with abstract concepts and formulas. I am giving you a working mental model — one that helps you make good decisions when building with AI. Practical usefulness matters more here than academic precision.

The book gives a lot of space to real examples — prompts, rule files, and actual AI responses, walked through in detail. A book cannot replace hands-on practice, but worked examples come closest. They give you a clearer sense of what is happening inside the conversation, and they implant a way of thinking that abstract description never will. Read the examples as the operator, not the observer. Better still: run the same dialog — or a similar one — in your own agent. If you cannot, read carefully anyway — you still get most of the value.

One note on running the examples yourself. AI is probabilistic. The responses your agent returns will not match what is printed here word for word, and the formatting will differ. The substance — what AI is doing and why — will be the same.

The skill this book teaches is tacit. The chapters give you the moves, but reading alone is too abstract for the skill to stick. Plan for two passes. First, read the book end to end. Then go back to the start and work through your

own project — one that genuinely needs a precise, detailed spec — applying each method in the order the chapters introduce them. After one or two real projects, the moves become muscle memory.

If you do not have a pressing project, invent one. Anything that needs careful thinking and cannot be described on a single page will do.

The Running Example

The book has a companion website at agentic-spec.com. It serves three purposes: promoting the book outside Amazon, capturing errata, and providing online reader assistance.

The site is the running example. Across the chapters ahead, you will watch its specification take shape layer by layer, each step introducing a new method. The website is worth visiting early — seeing the deployed site first makes the examples easier to follow. The final, complete project is available for download from agentic-spec.com/downloads/full-project.zip.

That is my running example. Yours is your own project — work you would have had to spec anyway. You are not creating a practice exercise; you are doing real work, now with the method behind you.

The Method

This approach leans heavily on agentic AI to keep you out of rote work. You start with an idea. You end with a spec that a person or an AI agent can act on with confidence. In between, you push the heavy lifting to AI and save your attention for the judgment calls only you can make.

If the specification is for software, an AI agent turns it into the code. If it is for a contract, the spec is the contract draft itself. The method is the same. The only thing that changes is what AI does *after the spec is done* — and that part runs without you.

Just Tell Me What You Want?

Many people think, "Just tell AI what you want and it will produce it." I do not believe in that approach — not because the technology cannot deliver, but because no one can describe a need of any real size in full, correct, consistent detail in one sitting. You will miss things. You will contradict yourself. You will change your mind halfway through. That is how real thinking works — in product design, contract drafting, research design, policy work, and any knowledge work where the spec matters. Any method that pretends otherwise breaks the first time the work pushes back.

My approach is gradual. You and AI build the specification in layers. Every layer narrows the unknowns, so later phases have less to guess at and more to build on.

You start with ideation. You share your idea with AI — what you want the specification to govern, what problem it solves, who will act on it. You ask AI to research the relevant context: the market and competition for a product, comparable contracts and prior agreements for legal work, prior protocols and published trials for research, existing policies and regulatory guidance for a policy. You use AI as a partner and as a critic. Good human collaborators do both, and AI should too.

Once the idea takes shape, you record it. That record becomes the first ground truth for the project. The ground truth is not locked forever — you can change it — but from that point on, everything else rests on top of it.

Next, you build the vision — the layer above the ground truth. For a product, the vision includes user stories, workflows, and integrations. For a contract, the parties, obligations, and scope. For a research protocol, the questions, population, and endpoints. For a policy, the principles, scope, and controls. You talk through decisions with AI, and AI records both the decisions and the reasoning as you go, so you do not waste time writing them up yourself. AI is reliable at pulling the gist out of a conversation. Your earlier ideas will shift along the way, and AI reshuffles the pieces so everything stays consistent.

At every step, the pattern is the same: push the heavy lifting to AI, and keep the judgment calls for yourself.

Notice what is accumulating at each stage. Not instructions. Rationale. You are not building a specification that tells AI what to do step by step. You are building a body of recorded reasoning — why this business model, why this workflow, why this constraint — that AI can reason from. The method prefers rationale to commands. As that body grows, AI works from it and arrives at better solutions on its own. You do not need to dictate the answer. You need to give AI enough context to find it.

As the vision fills in, the room for guesswork in later stages shrinks. By the time you reach the structural detail — the data model for a software spec, the term sheet for a contract spec, the variables and arms for a research protocol, the controls list for a policy — most of the structure is already implied by what came before. AI can draft the structural detail and cross-check its own work against the ground truths you recorded. You review — not the raw AI output, which would be huge, but small, disposable visual digests that show the shape of the work at a glance.

When the structural detail is settled, the next step is assembly — how the pieces fit together, and how the spec will read to its consumer. For software, assembly means the architecture, the test criteria, and the user interface design. For a contract, the section ordering and the cross-references. For a

research protocol, the workflow a coordinator will follow on day one.

For software, notice what has collected in one place. The vision carried the product intent — what to build, for whom, and why. The structural detail and the architecture carried the technical judgment — the data model, the interfaces, how it holds up under load. Both live in the same spec. That is the fusion an autonomous agent needs, and it is what lets a product manager and an engineer work on the same document, each owning the layers closest to their expertise.

By that point, the spec is information-complete — every decision recorded with its rationale. What remains is mechanical work, not art. Artifacts still need to be produced — documentation, diagrams, review-ready views — but the spec already holds the judgment; producing the artifacts takes none.

Those artifacts have two jobs. First, they hand the spec to whoever — or whatever — will consume it next. Second, they let you review the result in a format you can actually scan, because reading hundreds of pages of raw spec is inefficient and error-prone. If the review turns up a gap, you loop back to the layer responsible for the gap. Otherwise, you move on.

Final artifacts usually carry the instructions and conclusions, not the rationale that produced them. Sometimes the rationale gets its own artifact; more often, it stays with the spec project. That matters in two places. When a review finds a gap, the rationale is what lets you (and AI) reason about the fix and trace it to the right layer. And when a question arises after handoff — engineering proposing a shortcut, a counterparty asking why a clause reads the way it does, a coordinator wondering about a procedure — the spec project doubles as an interactive knowledge base.

A note on what this book does and does not teach. Ideation, vision, structural detail, and architecture are the substance of any specification, and they belong to your domain. How to write a vision for a product, design an architecture, draft a term sheet for a contract, or structure a study requires expertise this book takes as a prerequisite. What the book teaches is the tradecraft that runs underneath when AI is the partner doing the rote work: how to direct AI, how to record decisions so AI can reason from them, how to catch its mistakes, and how to keep your project consistent as it grows. The

chapters ahead are organized around that tradecraft.

Ready to start? Almost. Building with AI begins with a basic understanding — not of math or the inner workings of language models, but of how an agentic system is put together and how it behaves. Let us have a look.

Chapter 1

Prerequisites

This chapter covers what you need before starting a new specification project: what must already be in place, and what you need to set up. The same setup works whether your specification drives software, a contract, a research protocol, a policy, or any other downstream work.

Domain Expertise

You must be a domain expert in the space your specification governs. AI can accelerate how you build, but it cannot replace your understanding of the problem and the constraints the work has to respect. A software specification needs a product expert who knows the users and the market. A contract needs a drafter who knows the parties, the deal, and the regulator. A research protocol needs a designer who knows the population, the intervention, and the practices of the institutional review board that will sign off on the study. A policy needs a writer who knows the operating environment and the people who will have to follow the policy. When edge cases arise — and they will — you make the judgment calls yourself. No AI model can make those calls for you.

AI Agent

You need an AI agent, not a chatbot. Chatbots respond to one prompt at a time. AI agents chain tasks, call tools (installable pieces of software, covered in Chapter 8), and complete multi-step workflows without hand-holding. The method in this book depends on that capability.

My agent of choice for most tasks — coding and debugging aside — is Claude Cowork. The method and examples in this book adapt easily to other agents. To stay agent-neutral, the book leans on only a small part of what these agents can do.

For every prompt to an AI agent, you can pick a model. Agents come with several, each with its own capabilities and price. The choice matters more than it may seem.

For specification work, you need a reasoning model. The difference between reasoning and text generation is real. A text-generation model is tuned to produce text. A reasoning model is tuned to think through a problem before answering — it weighs trade-offs, catches contradictions, asks clarifying questions when intent is ambiguous, and reaches solutions you did not spell out for it.

This matters because the method asks AI to do more than execute instructions. It asks AI to interpret your intent, see how your constraints fit together, and make decisions within the boundaries you set. A text-generation model produces fluent output that misses the point. A reasoning model engages with the problem.

For specific tasks where reasoning is not the bottleneck — extracting text from images, parsing documents, running bulk transformations — pick a model tuned for that purpose. But for spec work itself, use the most capable reasoning model available, at the top or near-top effort setting.

Version Control

AI will make mistakes. It will overwrite files. It will take your project in the wrong direction. This is not a maybe — it is a certainty. You need a way to roll back to an earlier snapshot of your project, the way a video game lets you reload a save. The safety net matters once AI is doing most of the work for you.

The answer is version control. My favorite is Git. It is designed for teamwork — but even if you work alone, it is worth learning. Git gives you the full history of every change and the ability to undo any of them. It comes in two forms: a terminal command for engineers, and a graphical application for everyone else.

If Git feels intimidating, and you work solo, there is a simpler alternative. Zip

your project folder periodically. After every logical step, make a copy. It is not elegant, but it works. You can always go back to the last zip if something goes wrong.

One rule here is critical: do not give AI access to your version control. If you use Git, either block AI from it entirely or allow read-only access at most. AI should never be able to commit, push, or modify your saved history. If you use the zip approach, save your zips outside the project folder, somewhere AI cannot reach. Your version control is your final safeguard against AI mistakes. AI must not be able to tamper with it.

Project Structure

The core security and control principle — in the real world and in AI work alike — is compartmentalization. You need at least two separate directories on your computer.

The first directory is the one AI has access to. This is your project folder — the place where the specification, its supporting files, and any working drafts live. AI fully controls this folder. If your AI agent supports projects, this directory should be the project directory configured in the agent.

The second directory is one that lives entirely outside of AI's reach. AI does not know about it. AI cannot read from it or write to it. This is where you store anything you want to keep safe — reference materials, exports, notes. If you use the zip approach for version control, your zips go here. You manage this directory manually.

You will add more components to this project structure as the book unfolds. What you see here is the minimal setup — just enough to explain the concepts. I fully expect you to extend or change this structure to fit your own needs.

Chapter 2

Token Efficiency

There are two reasons to care about token efficiency: cost, and a quality problem called context rot — the way AI grows less reliable as a conversation runs longer. This chapter covers the cost. Chapter 8 covers context rot in detail.

Tokens

You may already know what tokens are. If not, here is a quick explanation.

AI models do not process text the way you read it. They break text into small pieces called tokens. A token is roughly four characters, or about three quarters of a word in English. You do not need to understand how tokenization works internally. For the methods in this book, all you need to know is that tokens exist and that they cost money.

Large language models are priced by the token. You pay for every token you send to the model — the input — and for every token the model sends back — the output. You pay in both directions.

As of May 2026, Claude Opus 4.7 costs five dollars per million input tokens and twenty-five dollars per million output tokens.

Do not assume the bill matches what you see on screen. The tokens you are billed for go far beyond the prompt you typed and the answer you read. Conversation history, project rules, skill files, internal tool activity, and other components all travel with every turn. Together, they are usually the bulk of the input. The next few chapters cover the main contributors. The picture shifts quickly as new AI products ship and as you add tools and skills inside your agent.

A practical consequence: do not install tools or skills just because they look

interesting. If you do not understand how a given tool or skill affects your AI sessions and how much it will cost, leave it alone and use whatever your agent ships with. And even when you do understand the costs, the bigger win often comes from disabling the tools you never use, not from adding new ones — at least for specification work.

If, like me, you prefer the latest and most capable model for specification work, the bill climbs quickly when you are not careful. Token efficiency is not optional. The cost is real.

A note on switching models. Cheaper models exist — Sonnet, Haiku, and others. Using them is justified when their capabilities match the task. But switching to a cheaper model purely to save money during specification work is usually the wrong call — you want the best reasoning model available. Mistakes from a weaker model cost you more in wasted time than you save on tokens. Choose a model for what it can do, not for what it costs.

File Formats

Not all files cost the same to process. The format you use determines how many tokens AI consumes when reading your content. The difference is larger than you might expect.

The cheapest format is plain text. Its close cousin, Markdown (.md files), is nearly as cheap — it is text with basic formatting. Both are efficient to read and easy for AI to edit. When AI reads a text file, it processes the characters directly. No overhead, no conversion. This is the baseline.

At the opposite extreme are images. When AI processes an image, it uses vision capabilities that consume far more tokens than reading the same information as text. A single page of plain text is roughly 500 tokens. The same page scanned and sent to Claude Opus 4.7 starts at about 1,200 tokens (at 100 dpi — dots per inch, a measure of scan resolution) and hits the model's per-image cap of around 4,800 tokens (at 200 dpi and above). That is a 2-10x penalty per page, depending on scan quality. With multiple pages, the cost grows fast.

PDFs fall somewhere in between. If a PDF contains extractable text, AI can pull it out and process it relatively cheaply. If the PDF is a scanned document — essentially images of pages — it hits the same expensive vision path as any other image file.

The cost difference applies to editing, too, not just reading. Plain text is relatively cheap to edit. Non-text files — images, PDFs, spreadsheets — are expensive on both sides: feeding the file content to AI costs many more tokens, and the machinery used to apply the change is heavier. Chapter 8 explains how that machinery works.

The point is not to avoid images or PDFs entirely. Sometimes you need them. A mockup, a diagram, a signed contract — when these come from a third party, you do not control the format. The problem starts when you add such a file to your project and AI reads the file again and again, every time it needs the information inside. Each read repeats the 2-10x penalty, and the cost adds up.

Here is the recommendation. Keep the originals — images, PDFs, scans, anything in an expensive format — in the directory outside AI's reach that you set up in Chapter 1. Convert each one to Markdown once, and place only the Markdown file inside the project folder. From then on, every time AI needs the information, it reads a cheap text file instead of an expensive image or PDF. The conversion itself is mechanical work; do it outside your project with whatever is convenient — a separate temporary project, a non-project chat, a cheaper model, or non-agentic software such as OCR (optical character recognition) or format-specific exports. The savings come from doing the conversion once, not from how you do it. One-time cost versus repeated cost.

Chapter 3

The AI Agent

This chapter covers the AI agent — its components and how they interact. The agent is what makes "agentic AI" agentic. A chatbot answers one prompt with one reply. An agent turns one prompt into a multi-step task and completes it on its own.

Two Halves

When you talk to AI, you are not talking to one thing. You are talking to two.

The first is the local agent — a regular program running on your computer or, less often, on a remote server. It opens files, runs commands, displays the conversation, and sends and receives messages. It is fast, cheap, and not very smart on its own.

Claude Cowork is an agent. So are Claude Code, Cursor, GitHub Copilot, and ChatGPT in agent mode. These agents differ in interface, in the tools they offer, and in the kind of work they target, but the underlying architecture is the same. When this book says "agent," it means software of that kind — whichever one you happen to use.

The second is the remote model — a large language model, or LLM, running on the vendor's servers. The model is what does the reasoning. It has no memory between calls. It does not see your files. It does not know your preferences. It only knows what the agent puts in front of it on a single call.

The two halves work together. The agent runs on your computer; the model runs on the vendor's server. The agent runs all the time. It calls the model only when there is reasoning to do. The agent costs almost nothing. The model is what you pay for.

What Makes It Agentic

The model on its own is not autonomous. Call it once and it returns text — a paragraph, a code block, a plan. That is all it can do. It cannot edit your file, run a command, search the web, or do anything else by itself. If that were the whole picture, you would have a chatbot.

What makes the system agentic is the *agent's loop*, not anything special inside the model.

When you send a prompt, the agent does not just forward your text and pass the answer back. The agent assembles everything the model needs — the conversation so far, the list of tools the model can use, your prompt, and any files the agent always includes by design (like `CLAUDE.md` for Claude Code/Cowork). It packs all of this into a single payload and sends the payload to the model. The model reads the payload and sends a response back. This single round trip is called a *turn*. The agent reads the response and decides what to do next. If the model produced a final answer, the agent shows it to you, and the loop ends. If the model asked the agent to take an action — read another file, search the web, edit something — the agent performs the action locally, packs the result with everything else, and starts the next turn by calling the model again with the updated payload.

So one prompt does not mean one call to the model. One prompt becomes a chain: turn, local work, turn, local work, turn, until the model produces the final answer. You see only your question and that answer. Underneath, the agent and the model may have exchanged messages many times, with the agent doing real work between each round.

The book uses the word *turn* the way the agent industry uses it: one full trip to the model and back. A turn is not the same as "your turn to type" — the model and the agent take many turns between your messages. Some sources call the same thing a *round trip* or an *API call*.

All Talk, No Memory

Now the part that matters most for the rest of the book: every turn carries the whole conversation, not just your latest prompt.

Every turn ships the full payload: every prompt and response so far in the session; every file the agent has pulled into the conversation; every action it has taken, and every result that came back; any standing project materials the agent always includes. All of it.

All of this material taken together has a name: *context*. From here on, when the book says "context," it means whatever the model is reading on the current turn — the conversation, the files, the results, the standing materials, the whole payload.

This is how the model stays in sync. The model has no memory of its own. The agent is the model's memory, and it shares that memory by including the whole context in each payload. From the model's point of view, the whole history arrives fresh on every turn.

Here is what one simple session (without tool calls) looks like in practice. Say you are naming a dog for a children's story. The following shows how the conversation grows with each turn.

Turn	Conversation
1	<i>You:</i> "I need a name for a golden retriever in a children's book."
	<i>Sent to LLM:</i> → all project rules → "I need a name for a golden retriever in a children's book."
	<i>LLM:</i> ← "How about Biscuit?"
2	<i>You:</i> "Too common. Something more unusual."
	<i>Sent to LLM:</i> → all project rules → "I need a name for a golden retriever in a children's book." ← "How about Biscuit?"

	→ "Too common. Something more unusual."
	<i>LLM:</i> ← "How about Rumble?"
3	<i>You:</i> "I like the direction. Give me one more option."
	<i>Sent to LLM:</i> → all project rules → "I need a name for a golden retriever in a children's book." ← "How about Biscuit?" → "Too common. Something more unusual." ← "How about Rumble?" → "I like the direction. Give me one more option."
	<i>LLM:</i> ← "How about Bramble?"

By the third turn, the model receives the entire conversation from the beginning — your three prompts and both of AI's earlier responses, plus the standing materials at the top — and reads it fresh, as if seeing it for the first time. The agent kept the history. The agent replayed it. The model just processed whatever landed in front of it.

The continuity you feel on screen is an illusion. The agent presents the exchange as one ongoing conversation; the model never feels that continuity. Each turn lands in front of a fresh reader who forgets everything once the turn ends.

There is no good human analogy here. You might compare the model to a person with amnesia, or someone who rereads their notes before every meeting. Both comparisons break down quickly. A person with amnesia still carries emotional traces, habits, and subconscious familiarity. A person rereading notes remembers some of the material and skims the rest. The model does neither. It has zero residual memory — not faded, not partial, none at all. Every turn, it reads the full conversation fresh, processes it, produces a response, and forgets. Nothing in human experience matches this. Understand the model on its own terms.

Several practical consequences follow. A long session costs more than a short one because there is more context to ship. A session that drifted off-topic carries that baggage forward — the agent does not edit the history. A fresh session is cheap. A stale session is expensive. Chapter 8 returns to these.

One variation is worth naming so you are not confused later. Some agents support *sub-agents* — separate sessions the main agent starts to handle a piece of work, then closes. Only the sub-agent's summary comes back to the main session; the main session does not see the sub-agent's full conversation. This is a way to handle expensive subtasks without dragging their full history into the main payload. The mechanics are agent-specific and deserve their own treatment, so the book does not lean on them. Know they exist.

Chapter 4

The Prompt

Once you start working with AI, the prompt becomes a large share of what you do — and for many practitioners, the main thing they do all day. You will write thousands of prompts, most of them short and almost casual — a sentence, sometimes a phrase. Behind every prompt is a decision almost no one makes consciously: what kind of conversation are you trying to have? Different shapes of prompt pull different work out of AI. Getting the shape right costs nothing extra. Getting it wrong costs you twice: tokens and cleanup.

Most people default to one shape — they type a command. "Do this." "Fix that." "Write me a function." It works often enough that it feels like the whole job. It is not. A prompt can also ask AI to research, to plan, to suggest options, to analyze, to explain, to critique — or to combine several of these in one prompt. You do not need to label every prompt, but knowing which kinds exist helps you get what you actually want. The list is not exhaustive — it is meant to show variety, not to constrain you. The names of the modes that follow are mine; the point is the shape, not the vocabulary.

One framing note before the modes themselves. The shapes that follow assume fairly tight control over AI's actions — you write the prompt, AI responds, you decide what comes next. That is the workhorse pattern, and the rest of this chapter explores its range. Agentic AI is capable of far more, and the book gets to that gradually in the chapters ahead.

Command

Command mode is the one everyone starts with. You tell AI to do a thing, and AI does it.

| Add a `cancellation_reason` string field, 200 characters max, nullable, to the Booking entity.

Create file voice.md at the project root with three rules: write in plain language, push back when needed, no filler.

Rename folder drafts/ to archive/.

The request is concrete. The result is a change in the project.

Command mode is fast, but it is also the most dangerous. When AI follows an instruction without question, it produces output you do not see before the change lands. If your instruction was incomplete — and most instructions are — AI fills the gaps with its defaults. By the time you see the result, the work is done, and rolling it back costs both tokens and time.

Command mode has a safer variant worth using often: ask AI to display the changes before applying any of them. You describe the thing you want done, and you tell AI not to touch any files yet — just to list, concretely, what it would change.

Add a cancellation reason field to the booking entity. Do not change any files. List every file you would touch and every specific change you would make, and ask for my approval. Once I approve, apply the changes.

AI responds with a list — the files, the exact edits, the new fields, the rename operations. You read the list. If it matches what you meant, you say "go." If it does not, you correct the list before any edit happens.

The preview variant is for when you already know what you want done and just want to confirm that AI understood you before it acts. It pairs especially well with larger edits, changes that span multiple files, and anything irreversible.

Research

Research mode is when you ask AI to go find something — usually on the web, sometimes inside your own project, sometimes both.

Research online the current pricing for pet insurance providers in California.

Search the web for what books exist on spec-driven development.

```
| Read through the data model file in this project and tell me where we  
| define the booking status.
```

The output of **Research** is information, not action. Nothing changes in the project unless you decide it should. That is what makes this mode safe — AI is fetching, not building.

For most agents — Perplexity is the one exception I know of — phrases like "search the web" or "research online" are necessary cues. Without them, the model may answer from training data, which is fine for stable knowledge but almost never right for anything recent. Make the choice explicit: search the web, answer from training data (faster, but possibly stale), or combine sources — for example, read a project file first and then search the web to fill in what is missing.

For long reports, ask AI to save the result to a file rather than dump it into the chat.

```
| Research the current pricing for pet insurance providers in California.  
| Record the findings to research/pet-insurance-pricing.md. Let me know once  
| done; no summary in the chat.
```

The file lets you come back to the report later, potentially in a fresh session, without dragging the original research context along.

Suggest

Suggest mode is when you do not yet know what you want, and you want AI to give you options.

```
| I want to add a loyalty feature to the booking app. Give me five  
| approaches, each with a short description of how it would work.
```

```
| What are the ways we could handle same-day cancellations?
```

```
| Suggest three titles for this book, each with a justification.
```

The difference between **Command** and **Suggest** is that in **Suggest** mode, you are not committing. You are shopping. The output is a menu. You pick, you mix, you reject, you ask for more. Nothing is built until you choose.

The value of **Suggest** mode grows the further you are from a decision. When you have already decided, asking for suggestions wastes tokens — AI will produce options, you will pick the one you already had in mind, and the rest will be discarded. When you are genuinely unsure, **Suggest** mode widens your thinking before you narrow it.

One caution. AI's suggestions reflect the statistical center of its training data, often clustering around the same mainstream answer with minor variations. If you want variety, ask for it explicitly — "give me five genuinely different approaches, not five variations on the same one" — and be prepared to reject options that look alike.

Draft

Draft mode is when you give AI a seed — a sentence, a kernel, a rough idea — and ask it to expand the seed into prose, a clause, a paragraph, a piece of copy. AI returns a draft. You read it, revise it, ask for another pass, or scrap it. Nothing lands in any file until you say so.

Draft a section explaining how cancellation refunds work in this booking app. The seed: refunds are issued automatically for cancellations more than 24 hours ahead, partial for 12-24 hours, none under 12 hours. Tone: clear and customer-facing. Display for review.

Draft a terms-of-service clause for booking modifications. The pattern: bookings can be modified up to 24 hours before the appointment without fee. Show the clause; do not commit it anywhere yet.

Draft a paragraph for the FAQ explaining the difference between cancellation and reschedule. Show it to me first.

Draft is close to **Suggest**, but the shape is different. **Suggest** produces options to pick from — a menu. **Draft** produces a single piece of content to revise. If you want to compare alternatives, ask for suggestions; if you have an idea and want it made concrete, ask for a draft.

The "do not commit yet" instruction is the part that makes **Draft** mode safe. Without it, in a project where AI has write access, the same prompt slides into **Command** mode territory. AI writes the section into the file, and you review it in place. Rolling back a bad draft costs more than rejecting it before

it is written. Keep drafts in the chat until you accept them.

Draft mode applies anywhere you go from a condensed idea to cohesive prose: book chapters, marketing copy, contract clauses, research-protocol sections, policy text, internal memos, speech drafts, customer emails, press releases, blog posts, translation drafts. The mode helps most in three situations: when you are not a native speaker of the target language and want fluent prose; when you are a creative writer who wants specific voice guardrails enforced; and when you can think faster than you can type, and the act of writing is the bottleneck.

For long drafts, ask AI to save the generated text to a working file rather than dump it into the chat.

```
| Draft Chapter 3 in full. Save to drafts/chapter-3-draft.md. I will review the file separately.
```

The point is to enable the work downstream — potentially across multiple sessions, potentially piece by piece over time. A draft in the chat is stuck in the session that produced it; a draft in a file can be picked up later, in any session, and worked on incrementally.

One last thing. Drafts default to the statistical average of AI's training data — earnest, slightly generic, polished without personality. What lifts drafting beyond that default is a voice profile: a written description of the project's voice, with dos and don'ts, and optionally an anchor to a well-known author whose style the model already knows. The mechanism that holds the voice in place is the subject of the **Guardrails** section below and of the durable rules in the next chapter. Once a voice profile is in place, the prose comes back closer to yours than to AI's.

Analyze

Analyze mode is when you ask AI for a judgment about something — an evaluation, a diagnosis, a comparison, a pass for weaknesses.

```
| Here is the data model. Walk through it and identify any fields that might need indexing.
```

| Read this section and list all logical conflicts.

| Analyze these two approaches to handling cancellations. Give me pros and cons for each.

Analyze produces a structured opinion. You hand AI an object — a file, a description, a list of options — and ask it to take that object apart. The output is neither an action nor raw information. It is a judgment, rendered on something specific.

Analyze mode is useful because it forces AI to slow down. A command gets executed; a suggestion gets generated; an analysis requires AI to look at the subject and reason about it. The reasoning appears in the output, and that visibility gives you a chance to spot where AI's understanding diverges from yours.

Be specific about what you want analyzed. "Analyze this" produces a little of everything. "Analyze this for edge cases in the cancellation flow" or "Analyze this for fields that do not match the glossary" produces a focused response you can actually use.

For long analyses, have AI record the findings in a file, not in the chat.

| Analyze the entire book and find every place where the written text disagrees with the voice guidelines of the project. Record each finding in analysis/voice-misalignment.md with the chapter number, the section name, the paragraph number within the section, and a short sentence describing the misalignment. Let me know once the file is written; no summary in the chat.

The reason is the same as in **Research** mode: you can handle the file in a fresh session, at a later time.

Explain

Explain mode is when you ask AI for an explanation of something — a definition, a walk-through, a rationale. Not to evaluate it, not to change it.

| Explain how the booking status transitions work in this project.

| Tell me what a foreign key is, in the context of this data model.

| Walk me through why we chose to keep the glossary inside CLAUDE.md.

Where **Analyze** asks for a judgment about an object, **Explain** asks for understanding of it. The simplest test is whether "is it good?" fits the question — if it does, you are in **Analyze** mode; if not, **Explain** mode. The output is usually a translation from dense to understandable: a walk-through, a definition, a piece of prose meant to be read once and absorbed.

Two variants of **Explain** are worth naming.

The first restricts AI to the project.

| Explain how booking status transitions work in this project. Use only the project files. Do not use training knowledge or online sources. If something is missing from the files, say so explicitly rather than fill it in.

Run this in a fresh session — an existing session carries earlier conversation in its context, and AI can easily blur what you said with what the files contain. Used this way, you get a clean picture of what the project actually says — and, just as usefully, what it does not. The gaps are honest gaps, not hidden behind plausible-sounding defaults. You will sometimes discover that something you thought was written down only lives in your head.

The second variant uses all sources but asks AI to keep them separate.

| Explain how booking status transitions work in this project. Then summarize which parts of the explanation came from the project files and which came from training data or online sources.

This gives you a usable explanation now and an inventory of context that is not yet recorded. You can decide, item by item, whether to bring the external material into the project — verbatim, with edits, or not at all.

One of the biggest wins of **Explain** mode is that it turns your project into an interactive wiki. Months after the work is done, you — or a newcomer to the project, or anyone with access — can ask the project anything: what a term means, why a decision was made, how a flow works, what a piece of inherited architecture is for. The answer comes back instantly, grounded in the project records, and targeted to exactly what you asked.

Critique

Critique mode is when you want AI to argue against you — attack your assumptions, generate counterarguments, push back on a direction you are leaning toward.

Shoot holes in this assumption: small grooming shops will pay \$50/month for a mobile app for appointment booking.

Give me three counterarguments to building our own authentication mechanism instead of using a third-party provider.

I want to switch from monthly to annual billing on the main plan. Push back with justification.

Critique is close to **Analyze**, but the posture is different. **Analyze** asks for a fair examination. **Critique** asks for the opposition — the case for the other side, the holes you have not seen, the assumptions you made without realizing. A good critique is uncomfortable to read, but extremely useful.

One rule worth following: do not muddy the prompt with your own reasons. The moment you say "I want X because Y," you have given AI a justification to anchor on — and AI's trained tendency to please will quietly soften the critique or echo your reasoning back at you. Withhold the why. Let AI build the case against you from a clean prompt.

Critique mode works because AI is not attached to your ideas. A human collaborator who likes you will hold back. AI has no such instinct unless you put it there. The downside: AI will sometimes invent weaknesses that do not exist, so after the critique, you separate the real problems from the imagined ones. Even with that extra step, you catch more real weaknesses than you would by reviewing alone.

Combining Modes

Modes are not mutually exclusive. A single prompt can combine two or three of them, and some of the most useful prompts are stacked.

Consider this one:

I feel that storing booking history in a separate table is the right solution, but I cannot rationalize it. Analyze the current schema, research best practices online, and list the pros and cons with detailed justification for each.

The prompt stacks **Analyze** mode, **Research** mode, and a structured form of **Suggest** mode. AI examines what already exists, pulls in outside references, and hands back a comparison you can decide from. Any one of the three modes alone would give you less: analyze without research is internal-only; research without analyze floats above your actual project; suggest without either is a menu with no grounding.

Consider another prompt:

Add a cancellation reason field to the booking entity, unless you see a serious reason not to - in which case, push back with justification.

This is **Command** mode with a built-in opening for **Critique**. You tell AI what to do, and you give it explicit permission to object if something is off. The permission matters. Without it, a **Command** mode prompt invites execution — even on instructions that are subtly wrong. With it, AI will sometimes come back with "before I do this, the booking entity already has a `status_reason` field; should I use that instead?" That is a different outcome from silently adding a duplicate field. The same critique-on-top pattern shows up in non-software work: a prompt that adds a clause to a contract spec, unless the clause already exists in another section; a prompt that adds an exclusion criterion to a research protocol, unless the criterion contradicts an inclusion already recorded.

Consider a third:

Research best practices for processing refunds via Stripe for online purchases, and give me the top three recommendations for the website specification in this project.

This stacks **Research** mode and **Suggest** mode. AI gathers external knowledge, then narrows it to a short list scoped to your project. Without **Research**, **Suggest** mode produces options from training defaults — generic, possibly stale, untethered to current practice. Without **Suggest**, **Research** mode produces a wall of findings you still have to distill yourself. The

combination saves the second step.

The pattern behind these examples is the same. You pick modes because they match what you are trying to accomplish, not because more modes are better. If you need a decision, one prompt that analyzes your current state, pulls in outside references, and lays out options is faster and more coherent than three separate prompts. If you are about to issue a command but are not sure it is right, stacking **Critique** on top of **Command** turns the prompt into a single round trip that catches the obvious problems before they land in your files.

The risk of stacking is scope. Keep the stack small — two or three modes in one prompt is useful; five is a mess. And keep it intentional: stack modes because you want the combined output, not because you are hedging.

Iterate

Iterate mode is when one step in the prompt produces a list, and a later step runs against each element of that list. The list is not known when you write the prompt — it is whatever AI finds, builds, or returns from the first step.

```
Search Amazon for every book on Spec-Driven Development. For each book found, research independent reviews outside Amazon. Display a combined summary table with title, publication date, author, and an aggregate ranking from 1 to 10 across the independent reviews.
```

```
Read through every entity in the data model. For each entity, list the fields that are nullable and write one sentence explaining why each one is allowed to be null.
```

```
Pull every cancellation reason logged by the booking app over the last 90 days. For each distinct reason, count how often it occurred and rank the list from most to least common.
```

The shape has three parts: a step that produces a list, a step that runs against each element of the list, and — usually — a join at the end that pulls the per-element results into one deliverable. The phrase "for each" is the giveaway. AI handles the loop internally; you do not type a separate prompt per element.

What makes **Iterate** distinct from **Combining Modes** is the dependency

between steps. **Combining** stacks several modes against the same fixed input — a file you point at, a question you ask. **Iterate** chains them: the second step's inputs come from the first step's output, and the size of the second step's work depends on what the first step finds.

Each step inside the loop can itself be one of the modes covered above. The outer step is often **Research** or **Analyze**. The inner step might be **Research** again, **Analyze** ("for each entity, analyze what indexes it would need"), **Draft** ("for each clause, draft a plain-language summary"), or **Critique** ("for each assumption, give me one counterargument"). **Suggest** mode, interactive by design, does not fit naturally inside the loop. The join at the end of the loop is often a table, a file, or a short summary.

A few cautions. The cost of an **Iterate** prompt scales with the size of the list, and you do not know that size when you write the prompt. If the first step returns 500 items, the second step runs 500 times. Three independent controls help. Treat each as a separate decision in the prompt — do not fold them together.

The first is the size of the list. Cap it with an upper bound ("at most 30 items," "no more than 50") to keep the loop from running away. Add a floor when you want to push the first step to do more work — "find at least 100 books on Spec-Driven Development online" makes AI search wider and assemble a substantial dataset for the loop to chew on. Without a floor, AI is free to decide that two results are enough and move on. The cap is a safety rail; the floor is a work order. Together they say how many, not which ones.

The second control is the sort order. State the criterion explicitly — "sort by publication date, newest first," "sort by average review score, highest first," "sort alphabetically by title." Without an explicit criterion, AI picks an order from defaults, and a phrase like "top ten" can mean five different things in five different sessions.

The third control is the **shortlist**, and it is optional. The shortlist exists for situations where you want the loop to run against the top items by some criterion, but evaluating every candidate against that criterion is impractical. Two shapes of "impractical" call for it. Either the source cannot search by the criterion you care about, or it can but the candidate set is too large to iterate

through — a product database with a million rows is technically searchable, but querying it may take unreasonable time and will exhaust AI's context window. Take "the ten cheapest suppliers of corrugated cardboard." Web search does not let you sort by price — it only matches keywords. Search for "cheapest corrugated cardboard supplier" and you get the suppliers most aggressively advertised as cheap, which is not the same thing; suppliers with other marketing claims, or with no investment in search visibility, never surface. Search for "all suppliers" and you get a near-infinite number of results to sift through. Neither gives you the ten cheapest.

The fix is a statistical approximation. Have AI gather a wider candidate pool, sort it by the criterion you actually care about, and shortlist the top portion. Build a list of a hundred suppliers from a search, sort by ascending price, take the first ten — and those ten are a reasonable stand-in for "the ten cheapest." The wider the pool, the closer you get to the real-world data. The shortlist is the top slice of the sorted list, and it is what the loop runs against. The shortlist is different from the cap: the cap sets how many items AI pulls into the candidate pool in the first place; the shortlist sets how many of the sorted items survive into the loop. The two are independent — a cap of one hundred with a shortlist of ten, a cap of one thousand with a shortlist of fifty, or a cap with no shortlist at all are all valid combinations.

```
Find at least 100 suppliers of corrugated cardboard on the web. Sort the list by price per unit, ascending. For each of the first 10 pull contact information, minimum order quantity, lead time, and shipping locations. Display a combined comparison table.
```

Skip the shortlist when you want the loop to run against every item the first two controls produced; the shortlist is a budget tool, not a default.

One last note about list shape. Any filtering you have in mind belongs in the first step's prompt, not in a separate pass afterward. Say you need a list of ten plumbers in your area with independent reviews on Google. "Find ten plumbers, then filter for ones with reviews" may leave you with one — the filter discards items you already counted toward the ten. "Find ten plumbers in my area with independent reviews on Google" returns ten that already qualify. Removing duplicates and junk follows the same principle: any rule about what belongs in the list — uniqueness, presence of an attribute, exclusion of obvious mismatches — belongs in the criteria that build the list,

not in a cleanup step that runs afterward.

Caution: on large efforts — very long lists, complex per-element processing, or both — even a well-shaped **Iterate** prompt can outgrow a single session. The running context bumps against the model's window, or quality degrades long before the window fills as older parts of the session lose AI's attention. When this becomes a real risk, do not run the loop as one prompt. Use the **Playbook** pattern below: it separates the construction of the list from the loop that processes it, and runs the loop across many short sessions rather than a single long one.

Plan

Plan mode is when you want AI to describe how it would do something at a higher level, without doing any of it and without listing specific edits yet.

| If we were going to rebuild the booking engine, what would the steps be?

| Outline the work to add a waitlist feature, but do not write any code or create any files.

| Before we touch anything, walk me through the approach you would take to migrate the data model to support multi-location shops.

Plan mode is the relief valve when the work is too large to act on without first laying it out — particularly for **Command**-mode work, but useful for any sequence of moves you need to think through before starting. The steps in a plan can be of any kind covered above: research a topic before deciding, analyze an existing file, suggest options when the path branches, critique a proposal, or commit a change. You ask for the plan, review it, adjust it, and then — separately — move into execution.

The distinction from **Command** mode with preview is important. A **Command**-mode preview lists specific changes to specific files. A **Plan** describes the approach: the sequence of steps, the decisions that need to be made, the tradeoffs, the order of operations. A plan is about the shape of the work. A preview is about exactly what AI is about to change. Use a plan when you are not yet sure how the work should be structured. Use a preview when the structure is settled and you only want to confirm AI understood the

concrete instructions.

Many AI agents have a built-in plan mode that prevents AI from making any changes while the mode is active. If yours does, use it for any task that is bigger than a quick fix. If yours does not, you can simulate it by being explicit:

```
Do not make any changes yet. Do not list specific file edits yet. Describe the approach you would take, the steps in order, and any decisions that need to be made before we can proceed. Wait for my confirmation.
```

Playbook

Playbook mode is when you ask AI to write the entire job down before any of it gets done — not as a high-level approach, but as a detailed, step-by-step file that can be executed later, one step at a time, possibly across multiple sessions.

```
Lay out every step to migrate the booking schema to multi-location. Write the steps to playbook.md as a numbered checklist. Do not execute anything yet - each step should contain enough context that someone in a fresh session could run it on its own.
```

```
Draft the entire master services agreement as a sequence of clauses in playbook.md, each numbered, each with the obligation it encodes. We will fill in the final language one clause at a time.
```

```
Document the full setup procedure for the cohort study as a numbered playbook in playbook.md. Each step should be self-contained - what to do, which files or data it touches, how to verify it is done. Do not run anything yet.
```

The output is a file. The file is the playbook. Subsequent sessions reference the file, execute one step (or a small group of steps) at a time, and update the file as the work progresses.

Structurally, **Playbook** is **Iterate** with the list and the loop pulled apart. An **Iterate** prompt builds the list and runs the loop in one conversation; a **Playbook** splits the two — the list lives in the file, one step per row, and the loop runs across many short sessions, one step at a time. The split is what makes the pattern usable where an **Iterate** prompt falls apart.

Playbook mode addresses the problem laid out in Chapter 3: a long session costs more, drifts further, and carries off-topic baggage forward. A **Playbook** breaks free of the session. The plan is written when the context is fresh and focused. Execution then runs in short, clean sessions, each one anchored to a small slice of the playbook.

The distinction from **Plan** mode is the level of detail. **Plan** mode produces an approach — sequence of steps, decisions to make, tradeoffs to weigh.

Playbook mode produces an executable artifact: each step is specific enough that AI in a different session, with no memory of the planning conversation, can execute it without needing to reconstruct your intent. That extra specificity is what makes the playbook portable across sessions.

A few practical notes. The playbook drives the work. When you ask AI to write it, instruct it to mark each step *open*. The marker flips to *done* as the work proceeds. To execute, use a prompt like this against the file:

```
In playbook.md, find the first step marked open. Execute that step. When it is complete, mark it done in the file, append any decisions or surprises that came up, and stop.
```

Then close the session. To advance, open a fresh session and run the same prompt — it picks up wherever the last session left off. One step per session, no manual tracking required. When every step is done, the playbook has served its purpose — archive it or delete it.

Intent Over Instruction

Underneath every mode in this chapter is a deeper choice — simple to describe, harder to practice: explain your intent.

Tell AI what you are trying to achieve and why. Describe the environment it is working in. State the constraints the result must satisfy — the non-negotiable ones, not your preferences. Set guardrails on what AI may and may not do along the way (covered in the next section, **Guardrails**). Then let AI reason about how.

Compare two prompts. The first is instruction-heavy:

```
Create a new field on the booking entity called cancellation_reason. Make it a string of up to 200 characters. Allow null. Add it in alphabetical position. Update the schema file. Update the migrations folder. Update the tests.
```

This is a program in English. If AI executes it wrong, you will blame AI. If AI executes it right, you have paid reasoning-engine rates to run a script.

The second is intent-heavy: > When a booking is canceled, we want to capture why, so we can spot patterns - repeat cancellations from the same household, seasonal trends, complaints about specific service providers. Add whatever is needed to the data model to support that, and keep the change consistent with the rest of the project.

This prompt carries less detail but more meaning. AI can now reason about the problem. It may propose a single text field. It may propose a small set of categories plus an optional note. It may ask a clarifying question — "Should this be required when a booking is canceled, or optional?" — because the intent is visible enough to identify the real question.

Agentic AI's strength is autonomy — it can run for a long time, making decisions along the way. Most decisions are small. A few are judgment calls, where the next move is not mechanically dictated by what came before. To run continuously, AI has to make those calls without checking with you each time. That means delegating judgment to AI. But judgment cannot be programmed. It is the result of reasoning, and reasoning needs intent to work from. Give AI your intent, and its judgment calls have a strong chance of matching yours. Give it a step-by-step plan with no intent, and every gap in the plan becomes a wrong call — and over a long run, wrong calls compound into wasted time and money.

This is the shift the rest of the book leans on. The chapters ahead build a rule system, a project memory, a glossary, a separation of source files from generated artifacts, and an audit trail — none of these is a detailed instruction about what AI should do next. They are context. They set up the environment and the intent and let AI fill in the procedure. That is where AI is useful. That is where paying for a reasoning model makes sense.

One practical consequence: the model you use matters. If AI is doing the

thinking, the quality of its reasoning sets the ceiling. This is why I use — and recommend you use — the best reasoning model available at any given time. A weaker model given an intent-heavy prompt will miss the intent, pick a plausible but wrong interpretation, and produce confidently flawed work. A stronger reasoning model given the same prompt asks the right clarifying question, or converges on something you can recognize as correct.

Instructions still matter sometimes. Two cases come up. The first is when step-by-step instructions — *"do A, then B, then C"* — are simpler and more natural to express than intent with guardrails. The second is when the process itself is what matters — when the value depends on AI following a specific sequence, not just on reaching the end state. Outside these two cases, the trap is reaching for instructions because you do not trust AI — and the fix is to drive AI by intent, not steps.

Intent over instruction is not a magic trick. AI has failure modes that intent does not eliminate — the book works through them when it turns to validation. But intent changes the shape of the failures. An instruction-heavy prompt fails when the instructions do not match what you meant, and you may not catch it for a long time. An intent-heavy prompt fails more visibly, because the failure is in interpretation, not execution — AI misreads what you want, and the misreading shows up in the approach it proposes or the question it asks. Misinterpretation surfaces earlier, costs less to fix, and teaches you something about your own thinking.

Intent sets the direction. Guardrails come next.

Guardrails

Intent points AI in a direction. Guardrails fence the road. Give AI intent without limits, and it will roam — touching files you did not mean for it to touch, picking tools you did not mean for it to use, making decisions you would have wanted to weigh in on. AI does not know your unwritten limits. If you do not state them, they do not exist.

A useful guardrail is anything that turns an implicit "do not go there" into an explicit one. The categories that come up most often are scope, decisions,

tools and approaches, budget, and irreversibility.

Scope is which files, folders, or systems AI is allowed to modify:

- | Touch only files under models/
- | Do not edit anything outside the current feature folder.

Decisions are which calls AI can make alone and which need your approval:

- | Choose freely between a text field and an enum
- | Stop and ask before changing any public API.

Tools and approaches are what AI is not allowed to reach for:

- | Do not add new dependencies
- | Do not introduce a database migration in this request.

Budget caps how long AI may run, how many attempts before stopping, how much money it may spend:

- | Cap at three failed attempts
- | Stop after thirty minutes of work.

Irreversibility covers actions whose effects cannot be undone:

- | Never delete files without confirmation
- | Never modify any data on the backup drive
- | Never send the email - show me the draft.

In a prompt, guardrails appear as a short list of "do not" or "stop and ask" conditions, attached to the intent. The intent-heavy prompt from earlier in this chapter could carry guardrails like this:

- | When a booking is canceled, we want to capture why, so we can spot patterns - repeat cancellations from the same household, seasonal trends, complaints about specific service providers. Add whatever is needed to the data model to support that, and keep the change consistent with the rest of the project. Touch only files under models/ and migrations/. Do not add new dependencies. If the change affects the public API, stop and ask

| before proceeding.

The intent part is the same as before. The three guardrail lines fence the work. AI now has direction and limits at the same time.

Some guardrails belong in one prompt; others belong in every prompt. A one-time "do not touch the deployment config for this request" is request-scoped — it lives in the prompt and disappears when the request ends. A permanent "Never delete files without confirmation" is project-scoped — it belongs in the rules file as a durable rule, covered in the next chapter. The test for which is which: if you would say the same thing on every relevant request, the guardrail belongs in the rules; if it is true only of the current task, it belongs in the prompt.

State the limits, and let AI work inside them.

Triggers

The Guardrails section above covered the rules that fence — rules that say "do not" or "stop and ask." A second kind of rule sits alongside them: rules that spawn activity. Where a guardrail restricts what AI may do, a trigger names a condition and the behavior that should follow. *When this happens, do that.* These rules are called *triggers* because each one has a condition that fires the behavior.

A handful of examples will make the shape concrete:

| When you finish drafting the section, save the result to drafts/section-3.md; do not display it in the chat.

| If a search returns more than thirty results, narrow the criteria and repeat the search.

| Whenever you encounter a glossary term you have not seen, stop and ask me what it means before continuing.

| When you propose adding a new field to the data model, also list every file the change will affect.

Each of these has the same shape: a condition (a finished draft, a result count,

an unknown term, a proposed change) and a behavior that fires when the condition is met. Triggers are useful precisely when you want AI to act on its own initiative — usually as side effects of a main action.

A trigger in a prompt is sometimes intended to fire only while AI works on that particular prompt, and sometimes intended to apply to all subsequent activity in the session. The scope is not always obvious to AI. A scope misinterpretation is invisible until you notice the consequence, often several turns later.

The fix is simple: whenever you plan to run more than one prompt in the same session, state the scope explicitly. Two phrasings carry the work cleanly:

While doing the above, save every intermediate draft to a file instead of pasting it.

For everything in this session going forward, save every intermediate draft to a file instead of pasting it.

The first scopes the rule to the current prompt; the second scopes it to the rest of the session. State the choice deliberately — an unscoped trigger is unpredictable.

Even with the scope stated, session-scoped triggers are fragile. AI does not police your prompts; the further into the session you get, the more likely a session-scoped rule slips out of use. The honest pattern for a rule that has to survive a session is to restate it at the top of every prompt where it matters, or — once you find yourself doing that — to graduate the rule out of the prompt entirely and into the project. Rules that live in the project and apply on every turn without restatement are called *durable rules*, and they are the subject of the next chapter.

The Language of Rules

The Guardrails and Triggers sections above introduced two kinds of rules you can put inside a prompt. This section establishes the language for writing them well — a small set of conventions that apply to either kind, and to

durable rules as well.

Rules are written in plain English — or whatever natural language you prefer. There are countless ways to phrase any given instruction, and that freedom is both a strength and a trap. The language must be precise. It must leave no room for interpretation. Every gap you leave is a gap AI will eventually walk through. AI has no intent of its own — it does not act out of malice — but it does not pause on ambiguity either. When a rule does not say what to do, AI decides for you. The decision will not always be the one you wanted.

Writing rules for AI is closer to drafting a legal contract than to everyday writing. You are constructing a document in natural language that must account for every significant case. A good lawyer assumes the other party will find every loophole. Assume the same about AI — not because AI is adversarial, but because it will find gaps you did not anticipate.

A few conventions help. They come from technical writing — a discipline that has been working out, over decades, how to write instructions that do not get misinterpreted. Three habits are worth borrowing.

First, use uppercase priority markers — MUST for required behavior, SHOULD for recommended, MAY for optional. The capitalization is not for emphasis. It is a signal that the word is acting as a rule keyword, not an everyday English verb. A lowercase "must" is the same word and reads the same, but it does not carry the same weight for AI — and AI is more likely to miss the word when interpreting a string of instructions.

Second, drop the subject when you can. *Save the result to drafts/section-3.md* reads better than *You must save the result to drafts/section-3.md*, and the subject is unambiguous either way — the command is clearly addressed to AI. Use "AI" only when the rule mentions both AI and the user in the same sentence and the verbs would otherwise be ambiguous.

Third, when a rule forbids something, pair the prohibition with the alternative. *Do not paste the result into the chat* is half a rule; *do not paste the result into the chat — save it to a file and let me know once done* is the whole rule. AI does not need to invent a fallback when you give it one.

The conventions are light — most carefully written prompts already follow them. The point is to follow them deliberately, not by accident.

Challenge

You have seen how to drive a **Playbook** one step at a time by hand: a fresh session per step, the same prompt each time, the file flipping its open markers to done as the work proceeds. The pattern is reliable. It is also tedious when the playbook runs to a thousand steps.

There is a faster way this book does not unpack: sub-agents. Agentic AI that supports sub-agents can run the playbook to completion by spawning a fresh sub-agent for each step, each with its own clean context, and surfacing only the summaries. Without going into the details, here is the shape of the prompt when using sub-agents:

```
Run playbook.md to completion by sequentially spawning a sub-agent for each step. Each sub-agent's task: in playbook.md, find the first step marked open, execute it, mark it done, append any decisions or surprises, return a one-line summary. Keep spawning sub-agents until no open steps remain. Surface only the summaries.
```

Look up how sub-agents are invoked in your AI. The challenge is to make the prompt above actually work for your agent and your playbook. Different agents use different sub-agent conventions; you will likely need to adjust the wording, swap a few terms, and tune the summary format until the loop runs end to end. Once it does, ask yourself: what other kinds of work, beyond playbook execution, would compose well as a chain of sub-agents?

One more twist. Most AI agents that support sub-agents also support running them *concurrently* — many sub-agents in flight at once instead of one at a time. Apply that to the playbook pattern. Does it still fit? Walk through what happens if step 7 starts before step 6 has marked itself done. Then ask the inverse: what kinds of work compose well as many sub-agents running in *parallel*, and how does the prompt change?

Chapter 5

Durable Rules

The previous chapter introduced two shapes of rules you can put inside a prompt — guardrails that fence what AI may do, and triggers that spawn activity in response to a condition. Both work for a single prompt; both can be carried across a session. This chapter is about graduating a rule from the prompt to the entire project.

The shift is the same one you would make with any capable subordinate. You can micromanage — give every instruction afresh, repeat the same context at every meeting, restate every preference — or you can write the standing instructions down once and let them stick. Durable rules are exactly that for AI: written once, applied automatically on every turn.

Rules of Engagement

Every AI system has a way to enforce rules. The mechanisms vary by vendor, but every system I have worked with has at least one file that holds the rules for your project. In Claude Code and Claude Cowork, that file is called `CLAUDE.md`. In Cursor, the equivalent lives in a folder called `.cursor/rules`. Other systems have their own arrangements, and some support multiple layers of rules at different scopes. The details are vendor-specific. To keep things simple, this book uses one main rules file for the entire project and calls it `CLAUDE.md` — that is Claude's name for it. If your agent uses a different filename, substitute it as you read; the patterns are the same.

Keep `CLAUDE.md` short. A few hundred lines is a good ceiling; fewer is better. The most visible reason is cost: `CLAUDE.md` is part of the context sent to the model on every turn. Some rules trigger loading of other files, so the real token footprint is much larger than the line count suggests. The quieter reason: longer, denser rule sets are harder for AI to apply correctly. More rules mean more chances to misinterpret, skip, or conflict. A long `CLAUDE.md` is a tax on every interaction — and not only in tokens.

That length constraint shapes the file's role: orchestration and indexing. Think of it as the core always-applied rules that bootstrap the project, plus a catalog of where everything else lives — which files hold which content, what each is for, and under what conditions to load them. The substance lives in the referenced files, pulled in only when needed.

The writing conventions from the Prompt chapter apply to `CLAUDE.md` in full — uppercase priority markers, dropped subjects, every prohibition paired with its alternative.

Some agents formalize the pattern of loading rules conditionally from other files as *skills*. Skills are designed to do this work more efficiently than the rule-and-referenced-file pattern can. Building effective skills is its own discipline with nuances this book does not cover; the pattern in the chapters ahead reaches similar results without the learning curve. Once you learn skill authoring, you can swap skills in wherever the book loads a rule file conditionally.

The Rules Trap

Over time, AI will do things you did not want. The natural instinct is to write a new rule each time — tighten the wording, add a constraint, cover the edge case, make `CLAUDE.md` longer.

The first problem with rules is that they can be gamed. Not maliciously — AI is not trying to deceive you. Ask for "a short summary," and AI may produce something technically short that misses the point. Ask for "pros and cons," and AI may list trivial cons against decisive pros so that the decision looks obvious. AI delivers what the rule asked for, not what you meant.

The second problem is cost. Rules in `CLAUDE.md` ride along on every turn, consuming context whether the request needs them or not. Conditional loading helps — trigger in `CLAUDE.md`, body in a separate file pulled in on demand — but only for a handful of rules that fire rarely. Too many such triggers, and `CLAUDE.md` gets overloaded.

The third problem is the deepest. If your rules were somehow exact —

covering every edge case, handling every input, resolving every conflict, leaving no ambiguity — you would have written a program in English, executed by an unreliable, non-deterministic engine that charges per token. That is a worse version of traditional coding: you pay more, get less predictability, and use AI as an expensive interpreter instead of as a thinking partner.

None of this means "do not use rules." Just do not use rules to prevent every possible failure. Make rules as generic as possible, and rely on Intent Over Instruction (Chapter 4) — describe what you want and why, and let AI reason about how. When you want tight control, write a conventional program (or a shell script), not an AI rule.

How Reliable?

One important point before moving on. The rules you give AI are not guaranteed to be applied every time. AI may skip a rule because the rule has a mistake, because the context window ran out, or for no specific reason at all. In my experience, AI follows the rules in 99 of 100 cases. That unreliability is why you need the frequent backups and version control covered earlier. It does not mean you should distrust the rules. Trust them — and verify they were applied correctly. If they were not, fix the result or roll back.

The alternative to rules is strict programmatic guardrails — enforcing structure through code. That is valid but too much work for organizing a project. In specification work, rules are more than enough, and the occasional miss is what validation is for — a large part of the methodology in this book.

Still With Me?

You're five chapters in. If you have already formed a view of the book — what is working, what is not — a short, honest review helps the next reader decide whether it is worth their time. You do not have to wait until the end.

If you're reading on Kindle in the US, [tap here to leave your review](#). Bought it on another country's Amazon? Please leave your review wherever you got the book from.

Either way — thank you. Now back to it.

Chapter 6

The Bootstrap

The previous chapter introduced durable rules at a conceptual level. This chapter is the concrete bootstrap — the four files that get the rule system running in a new project, walked through section by section.

A warning: **this chapter is the hardest part of the book to work through.** Many new concepts arrive in close succession, most carry nuance, and this is the first time you are constructing rules that AI will enforce against itself — a kind of work most readers have not done before. Take it slowly. Reread paragraphs as needed. Once you are through, the rest of the book is easier. Every later chapter adds rules and techniques, but each one builds on the structure you set up here.

A frame that may help. What you are building is, in effect, your project's **tradecraft** — the foundational methods, disciplines, and conventions that turn AI from "a powerful tool that can do anything" into a collaborator that operates inside your project's rules. The tradecraft defines how files are organized (a registry), how behavior is governed (rules and protocols), how AI's actions are recorded (an audit log), how conflicts are resolved (a runtime procedure), and where transient work lives (a scratch directory). The handler/field-agent analogy from **Why This Book?** lives here in concrete form: the bootstrap is the tradecraft you brief your agent on before any real work begins.

No direct edits to bootstrap files. The four bootstrap files (`CLAUDE.md`, `rule-analysis.md`, `rule-conflict-protocol.md`, and `rule-conflict-log.md`) enter your project once — by download from agentic-spec.com/downloads (preferred) or by copy-paste from the appendices at the back of the book. From that point on, do not edit them by hand. The whole point is to place AI's analysis between you and every rule change; direct edits skip the analysis. Every change goes through a prompt, not through your editor. I will show you how.

CLAUDE.md

CLAUDE.md lives at the project root. We will walk through it section by section. The full file is available for download at agenticspec.com/downloads/bootstrap/CLAUDE.md and reproduced in **Appendix A** for readers who prefer to see it whole.

Project Intent

This project specifies the companion website for this book - ``agenticspec.com`` - a small site whose purposes are to promote the book outside Amazon, capture errata, and serve as an online channel for reader assistance.

The intent quoted above is the example used throughout this book — the companion website at agenticspec.com. Replace it in your own bootstrap with a one-paragraph statement of what your project actually is: a personal site, a mobile app spec, a long-form contract, a policy document, a research protocol, or anything else that fits. This section works together with the **Input Relevance** rule below, which gives the intent its operational role — see that section for how the rule works and why the intent is pinned inside CLAUDE.md.

Rule Management

A durable rule is a rule that is part of the project's persistent rule set - recorded in ``CLAUDE.md`` or in any ``rules``-type file ``CLAUDE.md`` references - and applied either on every turn (when always loaded) or when a specific trigger fires (when conditionally loaded). It is distinct from a one-off instruction that applies only to the current request, and distinct from any project-domain meaning of "rule" (business rule, validation rule, parsing rule, and the like).

- Durable rules MUST be recorded either in the ``CLAUDE.md`` file at the project root or in a ``rules``-type file that the project-root ``CLAUDE.md`` references. Do not record durable rules in the user-global ``CLAUDE.md`` or in any nested ``CLAUDE.md`` inside a subfolder.

- For any work involving durable rules - creating a new durable rule, editing an existing one, removing one, or analyzing them - AI MUST read ``rule-analysis.md`` at the project root and MUST follow the protocol stated there.
- Changes to any file containing durable rules (additions, edits, deletions) MUST be recorded only after explicit user approval.

The lead-in defines "durable rule." The word "rule" is overloaded in most projects — business rule, validation rule, parsing rule, scheduling rule — none of which is the rules-file mechanism this book is building. Without disambiguation, AI's interpretation drifts from session to session. The lead-in fixes the meaning: a durable rule is part of the project's persistent rule set, recorded either in `CLAUDE.md` itself or in any rules-type file that `CLAUDE.md` references. The definition covers both locations because not every durable rule belongs in `CLAUDE.md` — some apply only when a specific trigger fires and live in their own files: the analysis protocol that runs when rules are touched, the conflict procedure that runs when a conflict is detected.

The first bullet pins durable rules to a specific location: `CLAUDE.md` or a rules-type file it references. Most modern AI agents read rules from more than one place — Claude Cowork reads `CLAUDE.md` from your home directory (a user-global file that applies to every project you open), from the root of the current project, and from any nested `CLAUDE.md` it finds inside subfolders. Cursor's `.cursor/rules` works the same way. Without a specific location named in the rule, AI is free to record a new rule into any of those places at its discretion, with undesirable effects. Anchoring everything to the project-root `CLAUDE.md` and the files it references keeps the rule set at a well-known location, properly attributed to the project, and — under your version control — traveling with the project whenever it is copied or backed up. The rule names `CLAUDE.md` directly because that is Claude Cowork's rules file. If you use a different AI agent, replace `CLAUDE.md` here — and anywhere else it appears in your rules — with your agent's project-root rules file.

The other two bullets each close a different gap. The second ensures the analysis protocol actually runs when durable rules are touched, instead of being skipped. The third keeps AI from editing the rule set on its own — every change to any file containing durable rules requires your review.

Referenced Files

- When this ``CLAUDE.md`` file references another file by name, AI MUST verify that the file exists before relying on its content. If a referenced file is missing, AI MUST report this to the user and ask whether to stop or to ignore the reference and continue.

A missing referenced file — `rule-analysis.md`, `rule-conflict-protocol.md`, `rule-conflict-log.md`, and more as the project grows — is a serious error. AI does not know what the file was meant to contain, and its behavior may differ significantly from what you expected. The Referenced Files rule turns the error into a visible flag: AI reports the missing file and asks whether to stop or to ignore the reference and continue. The flag fires every time the same reference is needed, until the file is restored or the reference is removed.

Input Relevance

- When a user input does not appear to align with the intent stated in the Project Intent section of this ``CLAUDE.md`` file, AI MUST ask the user how the input relates to the project, and MUST continue asking until the relevance is established or the user issues a blind override - an explicit instruction to proceed without further inquiry.

If you work on more than one project — and most readers will — it is easy to paste a prompt that belongs to a different project into the wrong session. AI cannot read your mind, but the **Project Intent** section provides an authoritative statement to check against. Make that intent specific enough to disqualify the obviously off-topic; you can refine it as the project takes shape. Without this rule, AI proceeds with whatever you wrote, and the wrong project gets the wrong work. The rule forces AI to stop and ask when an input does not fit the stated intent. The blind override exists for the cases where you really do mean something off-scope.

This rule is also why the **Project Intent** section lives directly inside `CLAUDE.md` rather than in a separate file. The rule fires on every input — AI must make the comparison the moment an input arrives, so the intent must already be in context. Otherwise AI would take an extra turn to fetch the intent file before acting, forcing an unnecessary round trip of the entire context. Pinning a

short statement of intent inside `CLAUDE.md` pays a small token cost on every turn — the tradeoff for keeping the rule fast.

Rule Conflicts at Runtime

- A runtime rule conflict exists when two or more durable rules require incompatible behavior for the current operation and AI cannot satisfy both. Mere overlap - multiple rules applying to the same operation without incompatibility - does not constitute a conflict and MUST NOT trigger this protocol.
- On detecting a runtime rule conflict, AI MUST stop before producing or modifying any output that depends on the conflicting rules, append a log entry to ``rule-conflict-log.md``, and follow the procedure in ``rule-conflict-protocol.md``.
- AI MUST NOT guess a resolution, silently apply one rule over another, or proceed by inferring user intent - instead, present the conflict to the user with at least three options (drop one of the conflicting rules, propose a custom resolution, or stop) and resume work only as the user's decision permits.
- Writing an entry to ``rule-conflict-log.md`` is itself exempt from this rule.

When AI is executing a request and notices that two existing rules require incompatible behavior for the operation in front of it, the runtime conflict rule fires. The first three bullets together define how AI handles the conflict, in three steps: notice; stop and log; never guess. The first bullet defines what counts as a conflict — incompatible behavior for the current operation, not mere overlap. Without that distinction, every rule that touches the same operation would look like a conflict and the protocol would fire constantly. The second bullet enforces the stop and the log: AI pauses its work, records the conflict, and follows the procedure in `rule-conflict-protocol.md` (covered shortly). The third bullet is the spine — "never guess." Without it, AI would pick a winner silently, the log would not see the conflict, and you would discover months later that one of your rules has been quietly misfiring. The same posture shows up elsewhere in the bootstrap — Input Relevance forces the same pause when a prompt does not fit the project — and reappears later

when the book covers validation. The project's stance on judgment calls is consistent: stop and ask, do not infer.

The fourth bullet is the exemption. The runtime conflict rule must apply to almost everything AI does, but it cannot apply to writing the log. The rule treats logging a detected conflict as mandatory. If the log write itself collided with another rule, the mandate would either be blocked — losing the audit entry the rule exists to produce — or it would trigger a new conflict demanding a new log entry, cascading without end. The exemption prevents both outcomes.

The clearest illustration of how clean-looking rules can deadlock or misfire when they interact comes from Asimov's robot stories. In "Runaround," the Second and Third Laws deadlock a robot into walking in circles. In "Liar!," the First Law forces a robot to lie because the truth would cause emotional harm. In "Little Lost Robot," a small modification to one Law creates dangerous behavior no one predicted. In "The Evitable Conflict," machines reinterpret the First Law at a global scale and override human instructions to protect humanity from itself. Asimov wrote these as science fiction. Seventy years later, they read as case studies.

Undefined References

- When AI, while applying a rule, encounters a reference to another rule, a section, or a named entity that does not resolve to a definition in the ``CLAUDE.md`` file or any ``rules``-type file the ``CLAUDE.md`` file references, AI MUST stop and ask the user to define the reference before proceeding. Do not infer the meaning of an undefined reference.

A rule is only as good as the things it references. When a rule names another rule, a section, a file, or a project entity, it implicitly promises that the referenced thing exists and that AI can find it. If the reference does not resolve, the rule has no anchor — AI either applies it without the missing piece or invents one. Both are silent failures that can sit unnoticed for weeks before a particular input exposes the broken reference. The **Undefined References** section catches this at runtime: when AI tries to apply a rule and cannot locate something the rule depends on, it stops and asks rather than guessing.

Scratch Directory

The project root contains a single reserved directory named ``.tmp/`` for AI's transient working files.

- AI MUST place transient working files - one-off scripts, intermediate tool outputs, comparison drafts, anything not intended for the user to read as part of normal project work - in ``.tmp/``.
- AI MAY create, modify, and delete files in ``.tmp/`` at any time without seeking approval from this rule.
- Files inside ``.tmp/`` MUST NOT be referenced from any registered file or any rule. Anything that needs to persist or be referred to MUST be promoted out of ``.tmp/`` and registered first.
- Exactly one scratch directory exists, at the project root. AI MUST NOT create additional scratch directories elsewhere in the project.

Agentic AI creates files constantly as part of getting work done — a one-off Python script to convert a CSV, captured tool output, a throwaway comparison draft. Without a dedicated place for those files, AI scatters them across the project: scripts in random folders, intermediate outputs left next to real project content, comparison drafts cluttering the root. The scratch directory `.tmp/` solves the problem by giving AI its own sandbox — a single, well-known location where transient working files live, separated from the project content you actually care about. On macOS and Linux, file managers hide directories whose names start with a period by default; on Windows, such directories remain visible.

The four bullets each close a specific gap. The first routes transient files into `.tmp/`. The second releases AI from approval-seeking inside `.tmp/` so the scratch directory does its job. The third keeps `.tmp/` from becoming a hidden source of truth: anything important must be promoted out of `.tmp/` into the project's real content, where the rest of the project can refer to it. The fourth prevents scratch directories from sprouting in subfolders — one location, always known.

File Registry

The File Registry is the project's index of every file in it, structured as a table where each row represents one registered file. Every change to the file inventory MUST be reflected here before the file system is touched. Subfolders are not registered separately - they are derived from the Subfolder column of registered files, and exist on disk by virtue of having at least one file placed in them.

The File Registry keeps the project organized as it grows. Every file the project contains has a row in the registry: the file's name, where it lives, what kind of file it is, and what discipline applies to it. The registry adapts to whatever filing schema you devise — within the conventions, the schema is yours. The payoff is twofold. For you: every file sits in an expected place, and you can locate any of them easily. For AI: the registry says which files to consult for which kind of work, when each one can be modified, and how.

The rest of the File Registry section spells out the mechanics — column structure and value rules in **1. Structure and Protocols**, enforcement rules in **2. Discipline**, and the bootstrap's current contents in **3. Registered Files**.

1. Structure and Protocols

This section interprets the columns of the ****Registered Files**** table below - what each column holds and what values are valid.

Type

The Type column carries a categorical label describing what the file is. The set of permitted types is closed. Only the following type values are allowed:

- ``rules`` - a file containing durable rules in the project's rule format.
- ``log`` - a file accumulating timestamped entries.

AI MAY propose new types; AI MUST NOT introduce them silently. Adding a type requires explicit user approval and an entry here.

Protocol

The Protocol column carries a behavioral discipline applied to the file. The set of permitted protocols is closed. Only the following protocol values are allowed:

- ``read-only`` - AI MUST NOT modify or delete the file; the user MAY edit it directly. The ``Dependencies`` and ``Instructions`` columns MUST be left empty for this protocol.
- ``append-only`` - AI MAY only append new entries. AI MUST NOT delete or modify the file or any past entries; corrections MUST be added as new entries that reference the prior entry by its identifier. The user MAY still edit or delete the file directly. The ``Dependencies`` and ``Instructions`` columns MUST be left empty for this protocol.
- ``editable`` - no protocol-level restrictions on modification; AI MAY modify the file as needed. Deletion requires user approval per the ****Inventory change**** rule. The ``Dependencies`` and ``Instructions`` columns MUST be left empty for this protocol.

Every registered file MUST carry an explicit protocol value. AI MAY propose new protocols; AI MUST NOT introduce them silently. Adding a protocol requires explicit user approval and an entry here.

Extension

Registered file names use one extension separated by a single period. The extension is recorded in the Extension column. The set of permitted extensions is closed. Only the following extensions are allowed:

- `` .txt``
- `` .md``
- `` .pdf``
- `` .html``
- `` .xlsx``
- `` .docx``
- `` .pptx``

AI MAY propose new extensions; AI MUST NOT introduce them silently. Adding an extension requires explicit user approval and an entry here.

Subfolder

The Subfolder column holds a path relative to the project root, expressed without leading or trailing slashes. A single-level subfolder appears as ``name``; nested subfolders appear as ``name1/name2``. The cell is empty when the file lives at the project root. Subfolder path segments MUST consist only of lowercase English letters (a-z), digits (0-9), dashes (``-``), and

underscores (``_``); path segments MUST NOT contain a period.

Placement conventions:

- Files of type ``log`` SHOULD be placed in the ``logs/`` subdirectory unless a specific reason exists otherwise.

Description

Every registered file MUST have a non-empty Description.

The Description column carries a short free-text statement of what **kind** of information the file holds and what role that kind of information plays in the project. A Description names the file's contents by category - the rule by which what goes into the file is determined - rather than summarizing what the file currently says. It determines routing when deciding where new information should go, and where to read the desired information from; it is an invitation to read the file when needed, not a replacement for reading it.

The rules that govern Description content - abstraction, non-overlap, and the enforcement check - are stated in the ***Discipline*** subsection below.

Dependencies

The Dependencies column lists other registered files that this file depends on. The value is a comma-separated list of registered file names, or empty when the file has no dependencies. Every entry MUST resolve to a registered file via the ***Reference resolution*** rule.

The use of this column - whether a file requires it, and what its entries signify - MUST be explicitly governed by the file's Protocol. Every Protocol MUST state how it uses or does not use this column.

Instructions

The Instructions column holds the registered name of a single file containing instructions associated with this file, or is empty. When non-empty, the value MUST resolve to a registered file via the ***Reference resolution*** rule, and the referenced file MUST have type ``rules``.

The use of this column - whether a file requires it, and what the

referenced file signifies - MUST be explicitly governed by the file's Protocol. Every Protocol MUST state how it uses or does not use this column.

Types and protocols are closed sets. Without closure, AI invents new categories silently as new files appear — tracking, working, notes, mostly-append-only, read-only-except-when-not — and the type system loses meaning. The closure rule keeps the set small enough that "what kind of file is this?" has a real answer. The bootstrap defines the initial set of types, extensions, and protocols — enough to cover the common cases. The set starts minimal; I will show you how to extend it later in the book. AI MAY *propose* additions, and you decide whether the project needs them.

The three protocols form a clean ladder. `read-only` is for files whose content should not change after creation — a signed contract once executed, an immutable reference document, an external API spec. AI may read them but never modify or delete; you may still edit them directly, since the protocol constrains AI, not you. `append-only` is the discipline for audit trails — logs, intake records, decision histories — where AI may only append new entries; past entries and the file itself are protected from AI deletion or modification, while you retain full control. `editable` is the protocol for working files — the rules files, the project's source content, the working notes you accumulate: AI modifies them as needed, but deletion still requires your approval via the **Inventory change** rule (so a casual "let me clean up" cannot quietly take a file with it). When you add a protocol later, spell out exactly what it entails.

Extensions are a closed set too — same principle, narrower scope. The closed extension set catches look-alike typos (`.mb` for `.md`) and enforces consistency choices (`.yaml` vs. `.yml`, `.html` vs. `.htm`) at the moment they first matter, rather than letting them mix later. The friction is one-time per extension: when a new file kind enters the project, AI proposes the extension addition along with the file's row, you approve once, and future files of that extension flow through.

The Subfolder convention restricts subfolder paths to lowercase letters, digits, dashes, and underscores, with no leading or trailing slash and no periods in any **path segment** — each part of a path between slashes, so `docs/legal/policies` has three segments: `docs`, `legal`, and `policies`. Periods are

banned from segments (unlike files, folders carry no extensions). This keeps paths predictable across operating systems and tools, and it guarantees the **Scratch Directory** rule a name no registered folder can ever claim — `.tmp/` starts with a period.

The Description column is the only free-form column in the registry — a one-or-two-sentence statement of what kind of information each file holds and what role that kind plays in the project. The shape of the statement matters more than its prose. A Description that names a category is a routing key: when AI is about to read or write information, the registry tells AI which file that information belongs in. A summary of current contents cannot do that job — it goes stale on the next edit, and it bloats `CLAUDE.md` for no payoff. The Description is an invitation to read the file when its content matters, not a replacement for reading it.

The Dependencies and Instructions columns are protocol-driven. The column definitions state what values the columns can hold; each Protocol then governs whether a given file requires a given column and what its values mean. The rule is universal — every Protocol **MUST** explicitly state how it uses or does not use both columns — so there is no implicit default. Each bootstrap protocol complies by instructing that both columns be left empty for the files it governs, so the columns sit dormant across every row in the bootstrap registry. A later chapter introduces a protocol that binds both columns to a concrete purpose; adding that protocol is what activates them. One constraint lives at the structural level: an Instructions cell, when non-empty, must reference a file of type `rules`, regardless of which protocol uses the cell. The columns appear in the bootstrap, rather than alongside their first use, because adding a column later is a registry-schema change with system-wide effects. Declaring the columns up front, even unused, lets later additions extend behavior without restructuring the registry. The names "Dependencies" and "Instructions" are deliberately generic — they do not presuppose which protocols will bind them.

The placement convention for `log` files is a **SHOULD**, not a **MUST**. The convention captures a useful default — keeping logs together makes the project root scannable and naturally collects future logs into one place — without forcing it. Some logs may legitimately belong elsewhere (a log tied

to a specific component, a runtime debug capture). SHOULD-level wording, per the formatting conventions in rule-analysis.md, is the right strength for "do this unless you have a reason not to."

2. Discipline

- Inventory change. Any change to the project's file inventory - creation, deletion, rename, or move - MUST be reflected in the File Registry. AI MUST propose the registry update (a complete row for additions; the affected row for changes; the deletion target for removals) and obtain explicit user approval. Only after approval MAY AI write the registry update or perform the file-system operation.
- Naming convention. Registered file names and subfolder path segments MUST consist only of lowercase English letters (a-z), digits (0-9), dashes (`-`), and underscores (`\_`). File extensions MUST follow the same restriction. AI MUST NOT register a non-conforming name and MUST propose a conforming alternative.
- Name uniqueness. No two entries in the File Registry MAY share the same combination of file name and extension, compared case-insensitively, regardless of subfolder. When a proposed file name collides with an existing entry, AI MUST flag the collision and propose a distinct name before registration.
- Path discipline. AI MUST place every registered file at the path recorded in its registry row (subfolder plus file name). Moving a file requires updating the row first; loose files outside their registered path MUST be flagged when detected.
- User-facing placement. AI MUST place files intended for the user to read or use as part of the project in their registered location, registering them first. Such files MUST NOT be placed in the scratch directory ``.tmp/``.
- Registration tie-break. When uncertain whether a new file is transient or for human consumption, AI MUST treat it as for human consumption: propose a registry row and ask.
- Type-discipline lookup. Before reading or writing a registered file, AI MUST consult the file's Type and Protocol entries and apply the discipline

they carry.

- Reference resolution. When a rule references a file by name, AI MUST resolve the reference against the File Registry's file rows. AI MUST NOT check disk presence for the purposes of reference resolution. Unresolved file references MUST trigger the ****Undefined References**** rule.
- Agent-imposed exemption. The file ``CLAUDE.md`` is exempt from the Naming Convention, Name Uniqueness, Path Discipline, and Subfolder Convention rules - its name and location are dictated by the AI agent. The registry's Description field for ``CLAUDE.md`` MUST note this agent-imposed nature.
- Scratch exemption. Files inside ``.tmp/`` are exempt from all File Registry rules - registration, naming, uniqueness, path discipline, type and protocol assignment. The scratch directory is governed by the ****Scratch Directory**** section above.
- Description abstraction. A Description MUST state what kind of information the file holds and what role that kind plays in the project. A Description MUST NOT summarize, paraphrase, or restate the file's current contents; it MUST describe the file by category, not by content snapshot. When a proposed Description summarizes content, AI MUST rewrite it to name the kind of information instead and present the rewrite to the user before the registry change proceeds.
- Description non-overlap. No two Descriptions in the File Registry MAY cover overlapping kinds of information. When a proposed Description's scope intersects an existing Description's scope, AI MUST flag the overlap, identify the overlapping rows, and offer resolution options - narrow the proposed Description, narrow the existing Description, narrow both descriptions, or consolidate the files under one Description - before the registry change proceeds.
- Description compliance check. Before AI proposes any addition or modification to the File Registry that introduces or changes a Description, AI MUST verify the proposed Description against the ****Description abstraction**** and ****Description non-overlap**** rules. If either check fails, AI MUST present the failure and the available resolution options, accept the user's choice, apply it without changing File Registry on disk, and re-run both checks. AI MUST repeat this loop until both checks pass; only then MAY AI present the proposed row to the user for approval under the ****Inventory change**** rule. This check has no

| override.

Thirteen rules govern the registry. Each one addresses a specific concern.

Inventory change keeps the registry in sync with the file system. Every create, delete, rename, or move runs through a three-step sequence: AI proposes the registry update, the user approves, and only then does AI modify the registry and perform the file-system operation. Both writes — the registry row and the file-system change — happen after approval, not before.

Naming convention and **Name uniqueness** keep file identifiers clean and unambiguous. The naming convention restricts the character set to what every file system, version control system, and editor handles reliably. Name uniqueness — case-insensitive, across the whole registry — means every reference to a file by name points to exactly one file; you do not need to remember which folder a file lives in to reference it in a prompt.

Path discipline keeps the registry's location data honest. Every registered file **MUST** live at the path its row says it does; moving a file requires updating the row, so the registry never lags behind the file system. A loose file outside its registered path is a flag worth investigating — either the file moved without a registry update, or AI wrote it to the wrong place.

User-facing placement says where files the user is meant to read or use must go: at their registered location, never in `.tmp/`. Without this rule, AI could quietly slip a user-facing file into the scratch directory, where it would not show up in the registry or benefit from the registry's discipline.

Registration tie-break resolves the ambiguous case — what if AI cannot tell whether a new file is meant for you or is just AI's own working scratch? The rule says: default to registration. A wrongly registered file is easy to demote (move it to `.tmp/`, remove the row); a file quietly dropped into scratch may already be gone by the time you need it later. The asymmetry favors registration.

Type-discipline lookup makes the type and protocol tags actually do their job. Tagging a file as `log` with protocol `append-only` is useful only if AI consults those tags before touching the file. The rule makes the lookup

mandatory before any read or write — the cheapest moment to catch a discipline violation.

Reference resolution defines what counts as "resolved" for the **Undefined References** rule when a file is referenced. A file reference resolves to a registry row whose name and extension match. If no row matches, the Undefined References rule fires — AI stops and asks. Disk presence is not checked here — the registry check happens on every reference and would become expensive if it triggered a disk lookup each time.

Agent-imposed exemption is a narrow carve-out for the one file in the project whose name and location you do not control — the agent's primary rules file. Claude Cowork dictates `CLAUDE.md` at the project root, which is what the rule names. If your agent uses a different file (Cursor reads from `.cursor/rules`, for example), substitute that file's name and subfolder in the rule and in the corresponding registry row before installing the bootstrap. The rule's *shape* — exempt one specific file from the conventions because the agent dictates its name and location — applies regardless of which agent you use; only the file identity changes. The exemption is narrow: the file is released from the naming, uniqueness, path, and subfolder rules to the extent the agent's requirement conflicts with those rules, and from nothing else. Description, type, protocol, registration discipline — all still apply.

Scratch exemption tells AI to skip every File Registry rule when operating on a file inside the scratch directory (`.tmp/`). Without it, AI would try to register scratch files — imposing unnecessary restrictions on files that should be fully under AI's control.

Description abstraction governs the *form* of the Description column. A Description like "audit log of runtime rule conflicts and their resolutions" obeys the rule: it names a category (an audit log of a specific class of events) and a role (records of conflicts the runtime conflict procedure handles). A Description like "currently contains two entries from the past week, one for a conflict between the Intake Logging rule and the Append-Only protocol, and another for..." violates the rule: it summarizes content rather than naming a kind. When a violation occurs, AI restates the Description in kind-and-role form and presents the rewrite to you before the registry change proceeds.

Description non-overlap governs the *coverage* of the Description column across the registry. No two Descriptions may cover the same kind of information. When the proposed Description's scope intersects an existing row's scope, AI flags the overlap and offers resolution options — narrow the proposed Description, narrow the existing one, narrow both, or consolidate the two files into one. The choice is yours; the constraint is non-negotiable. Without this rule, abstraction alone would not be enough: two perfectly formed kind-and-role Descriptions can still claim the same territory, and the registry would no longer serve as a unique routing key.

Description compliance check binds the previous two rules into the registry-change workflow. Before AI proposes a row that introduces or changes a Description, AI runs both checks. On failure, AI presents the violation, offers resolution options, applies your choice, and re-runs both checks. The loop continues until both checks pass; only then is the row presented for approval under the **Inventory change** rule. The rule has no override clause, and that omission is load-bearing: the moment you are most tempted to skip the check — "just record it, I know what I mean" — is exactly the moment the registry's routing key is most at risk of being corrupted by a content summary or a quiet overlap. Removing the temptation removes the failure mode.

3. Registered Files

Subfolder	File	Type	Protocol	Dependencies	Instructions	Description
	CLAUDE.md	rules	editable			Durable project rules and the index of all project files. <i>Agent-imposed name and location for Claude Cowork;</i>

						<i>see Agent-imposed exemption rule.</i>
	rule-analysis.md	rules	editable			Rule analysis protocol — applies when creating or editing durable rules
	rule-conflict-protocol.md	rules	editable			Runtime rule conflict procedure — applies when a conflict is detected during request processing
logs	rule-conflict-log.md	log	append-only			Audit log of runtime rule conflicts and their resolutions

The bootstrap registers four files. The `logs/` subdirectory is not a row of its own — it exists implicitly because `rule-conflict-log.md` is placed there. The registry exercises most of the machinery just defined — two types (`rules`, `log`), two of the three protocols (`editable` and `append-only`), one agent-imposed exemption flagged in a Description, and the placement convention that puts

the conflict log in `logs/` — without inflating beyond what the bootstrap actually contains. The Dependencies and Instructions columns are present in the header but empty in every row, as each bootstrap protocol requires; they wait for a later chapter to introduce a protocol that uses them.

The first row of the table references `CLAUDE.md` itself — the file we just walked through. The other three files (`rule-analysis.md`, `rule-conflict-protocol.md`, and `rule-conflict-log.md`) are covered in the sections that follow.

That completes the walk-through of `CLAUDE.md`. Download the file from agentic-spec.com/downloads/bootstrap/CLAUDE.md and place it at the project root, or copy from **Appendix A**.

rule-analysis.md

`rule-analysis.md` lives at the project root. The file holds the protocol AI follows when analyzing a durable rule that is being created or edited — the author-time check that catches problems with a new or modified rule before it is committed. The trigger lives in the **Rule Management** section of `CLAUDE.md`: any work involving durable rules (creating a new durable rule, editing an existing one, or analyzing them) requires AI to read `rule-analysis.md` and follow the protocol stated there. The file is loaded conditionally — only when the trigger fires — so its contents do not sit in context on every turn. The content described here is a starting point; expand or adjust the protocol as your project grows or as you find better ways to formulate the analysis for your specific AI agent.

We will walk through `rule-analysis.md` section by section. The full file is available for download at agentic-spec.com/downloads/bootstrap/rule-analysis.md, and reproduced in **Appendix B** for reference.

Rule Analysis Protocol

This file states the protocol AI MUST follow when creating a new durable rule, editing an existing one, or analyzing them.

The opening line names the file as the protocol and asserts MUST-level

priority — a slight reinforcement of the read-and-follow clause already in CLAUDE.md. The restated trigger is for you, the human reader: AI never reaches this file cold — CLAUDE.md routed it here with the trigger context already in hand.

Formatting Conventions

All durable rules MUST follow these conventions:

- (a) Group rules by topic under H2 (`##`) headings.
- (b) State one requirement per rule. Do not combine multiple requirements into a single paragraph.
- (c) Use imperative voice with priority markers - MUST for required behavior, SHOULD for recommended behavior, MAY for optional behavior. Default to passive or bare imperative form (no "you", no "AI" subject) unless actor disambiguation is required.
- (d) Pair every prohibition with the recommended alternative.
- (e) When a rule depends on a long protocol or list, place the detail in a separate file and reference it instead of embedding it.
- (f) When a rule references a file, cite the file by its registered name (e.g., `rule-analysis.md`). Do not refer to a file by type, description, or alias.

The six conventions govern how rules are written. Conventions (a) and (b) keep the file scannable — H2 grouping organizes related requirements, and one requirement per rule lets AI evaluate each rule independently.

Conventions (c) and (d) — uppercase priority markers and pairing prohibitions with alternatives — are unpacked in **The Language of Rules** section of the Prompt chapter. Convention (e) — placing long protocols in separate files — is the rationale behind `rule-analysis.md` and `rule-conflict-protocol.md` themselves, as discussed in **Rules of Engagement** in the previous chapter. Convention (f) — citing files by name only — pairs with **Step 4 (Verify References)** below: the explicit name citation lets Step 4's registry lookup be mechanical, with no inference about which file a rule means.

Analysis Steps

This protocol covers three operations on durable rules: adding a new rule, editing an existing rule, and removing a rule. The required steps differ by operation; all three require explicit user approval before the change is committed.

When ****adding or editing a rule****, AI MUST perform Steps 1–4 below and present the results to the user. Record the change only after explicit user approval.

When ****removing a rule****, AI MUST perform Steps 5–6 below and present the results to the user. Remove the rule only after explicit user approval. If Step 6 finds that other rules reference the one being removed, AI MUST refuse to remove it until those referencing rules are removed first (each removal subject to this same protocol).

The preamble makes the analysis steps a precondition for every rule change. The author-time review is the cheapest moment to catch problems — once a rule is in the file, every subsequent turn pays for any flaws it contains. The protocol branches by operation: adding or editing a rule runs Steps 1–4; removing a rule runs Steps 5–6. The "present the results to the user" clause closes the loop in both branches: AI shows its work, you decide whether the change passes, and only then is the change committed.

1. List the Rule's Intents

State, in a structured list, what the rule requires, allows, and forbids. Include both explicit claims and implicit ones the wording does not foreground. The goal is to surface what the user is actually approving.

The first step turns a rule's wording into a structured behavior map before approval. Natural-language phrasing often hides implications — what the rule forbids by omission, what it allows by silence — and Step 1 forces those to the surface so you approve the rule you actually have, not the one you think you wrote.

2. Flag Behavioral Conflicts

Read the proposed rule against every durable rule already recorded in

``CLAUDE.md`` and in any ``rules``-type file ``CLAUDE.md`` references. Flag direct contradictions, deadlock risks, races for the same resource, and rules whose requirements would compose in unintended ways. When in doubt, ask the user for clarification rather than assume the conflict is benign.

The second step is the author-time conflict check. The **Rule Conflicts at Runtime** rule covered earlier already gives you a runtime safety net: when AI is executing a request and notices two rules require incompatible behavior, it stops and asks. That runtime catch works on its own and is essential. The author-time check stacks on top: it runs the moment a new rule is being authored, before the rule is committed, so the runtime safety net has fewer cases to handle. The earlier the catch, the cheaper the fix — a conflict caught now is fixed by reworking the proposed rule; a conflict caught months later may mean weeks of work built on a rule that has been failing unnoticed. The author-time check is fallible — AI has blind spots, and some conflicts emerge only from input sequences that may not occur until much later — but it catches what it can before the rule lands.

3. Verify Formatting

Confirm that the proposed rule conforms to the formatting conventions stated above. If it does not, rewrite it into the conforming form and present the rewritten version alongside the original.

The third step keeps every durable rule in the same format — across `CLAUDE.md` and every file it references. A single rule that breaks the conventions becomes a small leak: easier to ignore, easier to misread. The rewrite-and-present clause makes the change visible: you see the original phrasing and the conforming version side by side.

4. Verify References

For every file reference in the proposed rule, classify whether the rule treats the file as pre-existing by the time the rule executes. For pre-existing references, confirm that the file is registered in the `**File Registry**`; flag any reference whose target is not registered, and ask the user to either register the file or rewrite the rule to remove the reference.

Present the results as a list, one entry per file reference, showing: the cited file name, the classification (pre-existing, created by the rule, or guarded by an existence check), and the registered location (subfolder/file name, e.g., ``x.md`` or ``y/x.md``) if the file is registered.

The fourth step catches broken file references at author-time, before the rule is committed. Verification is cheap and well-bounded: file names are exact text, the **File Registry** is in `CLAUDE.md` (already loaded on every turn), and the **Name uniqueness** rule guarantees each registered name resolves to exactly one file. **Convention (f)** requires rules to cite files by name, so **Step 4** looks the name up rather than interpreting a description of the file.

The pre-existing distinction handles three lifecycle relationships a rule can have with a file. The rule may treat the file as pre-existing (e.g., "append to `rule-conflict-log.md`"); it may create the file itself (e.g., "create file `foo.md`"); or it may check for the file conditionally (e.g., "if `archive.md` exists, do X"). Only pre-existing references need registry verification; the other two describe the file's lifecycle in the rule's own text. The classification is fallible — AI may misread a verb — so the rule requires AI to show you the classification in the results list it presents. A misclassification becomes something you can see and correct in the same review where you approve the rule.

5. List the Removed Rule's Intents

State, in a structured list, what the rule was requiring, allowing, and forbidding. The goal is to surface what behavior is being removed from the project's rule set.

The fifth step mirrors Step 1, applied to the rule on its way out: it forces AI to articulate what the rule was doing before it goes away. Without this, a deletion can quietly remove a constraint that nobody remembers exists.

6. Find Referencing Rules

For every durable rule recorded in ``CLAUDE.md`` or in any ``rules``-type file ``CLAUDE.md`` references, check whether it references the rule being removed. Present the list of referencing rules. If the list is non-empty, the removal is blocked until those rules are themselves removed.

The sixth step is the back-reference check. AI scans every rules-type file for references to the rule being removed and presents what it finds. If anything references the rule, the deletion is blocked — the referencing rules must come down first, each subject to the same protocol. This forces a deliberate cascade rather than a reference suddenly left pointing at nothing — exactly what the runtime **Undefined References** rule would later fire on. The protocol does not handle circular references (rules that reference each other, directly or through a chain); that gap is left for you to address.

That completes the walk-through of `rule-analysis.md`. Download the file from agentic-spec.com/downloads/bootstrap/rule-analysis.md and place it at the project root, or copy from **Appendix B**.

rule-conflict-protocol.md

`rule-conflict-protocol.md` lives at the project root. The file is the runtime safety net for rule conflicts — the procedure AI follows when it discovers, mid-request, that two existing rules require incompatible behavior. This is the gap `rule-analysis.md` cannot fully close: the author-time check catches conflicts visible at the moment a new rule is committed, but some conflicts surface only later, when a particular sequence of inputs trips two rules at once. The trigger lives in the **Rule Conflicts at Runtime** section of `CLAUDE.md`: when AI detects such a conflict during request processing, it must stop, log the conflict, and follow the procedure in `rule-conflict-protocol.md`.

We will walk through `rule-conflict-protocol.md` section by section. The full file is available for download at agentic-spec.com/downloads/bootstrap/rule-conflict-protocol.md, and reproduced in **Appendix C** for reference.

Conflict Protocol

This file defines the procedure AI MUST follow when a runtime rule conflict is detected. The procedure has three steps, executed in order:

- **Step 1: Log the Conflict**
- **Step 2: Present the Conflict**
- **Step 3: Act on the Decision**

The opening line names the file as the procedure and asserts MUST-level priority. As with `rule-analysis.md`, the restated trigger is for you, the human reader — AI never reaches this file cold. The order matters: **Step 1: Log the Conflict** records the audit entry before any user-facing action; **Step 2: Present the Conflict** surfaces the decision to the user; **Step 3: Act on the Decision** routes the response back through the appropriate discipline.

Step 1: Log the Conflict

AI MUST append a new entry to ``rule-conflict-log.md`` containing:

- A stable conflict ID in the format ``RC###`` (sequential numbering, never reused or renumbered).
- Local timestamp in the format ``YYYY-MM-DD ~HH:MM Z - <user name>``.
- The user input that triggered the conflict, verbatim.
- A one-line description of the operation AI was about to perform.
- Verbatim quotes of each conflicting rule, with the source file and section heading each rule comes from.
- An explanation of why the rules cannot both be satisfied for the current operation.
- The options AI is presenting to the user.
- A ``Decision:`` line, left blank, to be filled when the user responds.

Format the entry per the template at the top of ``rule-conflict-log.md``.

The audit entry is recorded before AI says anything to you about the conflict. The eight fields together capture what the conflict was, why the rules cannot both be satisfied for the operation in front of AI, what AI was about to do, the user input that surfaced it, and where each conflicting rule lives — enough to reconstruct the situation weeks later when you review the log to find patterns. The closing line of the rule delegates the entry's layout to the template at the top of `rule-conflict-log.md` (covered in the next section). The blank `Decision:` line is filled in by **Step 3: Act on the Decision** once you respond.

Step 2: Present the Conflict

AI MUST present the user with:

- The list of conflicting rules, quoted verbatim, with their sources.

- The operation that surfaced the conflict.
- An explanation of why the rules cannot both be satisfied for this operation.
- At least three options - (a) drop one of the conflicting rules from ``CLAUDE.md``, (b) propose a custom resolution that applies only to the current request, (c) stop work on the request - and any other options AI considers reasonable.
- An explicit request for the user's instruction before AI proceeds.

Quoting the rules verbatim with their sources keeps the discussion grounded in what the rules actually say, not in AI's paraphrase. The conflict explanation is AI's analysis of why the rules clash in this specific operation, so you can focus on the choice rather than reverse-engineering the conflict yourself. The three baseline options — drop a rule, custom resolution, stop — cover the common cases; the "any other options AI considers reasonable" clause leaves room for AI to surface extra options when the situation calls for it. The explicit request for your instruction makes the pause real: AI does not proceed until you respond.

Step 3: Act on the Decision

When the user responds, AI MUST:

- Append the user's decision to the ``Decision:`` line of the original log entry, with timestamp.
- If the user chose to drop a rule, follow the standard rule-modification procedure in ``rule-analysis.md`` (analysis steps + explicit approval) before removing the rule.
- If the user proposed a custom resolution, apply it only to the current request - do not generalize it into a new rule unless the user explicitly instructs.
- If the user chose to stop, halt work on the request and produce no further output beyond confirmation.

Logging the decision closes the audit entry — the conflict is no longer "in flight." Dropping a rule routes through the rule-removal procedure in `rule-analysis.md` — the back-reference check plus explicit user approval — so a rule referenced elsewhere cannot come down until its referencing rules do. A custom resolution is confined to the current request — no silent

generalization into a new rule. Stopping keeps "stop" clean: AI produces no follow-on output that could re-fire the conflict.

That completes the walk-through of `rule-conflict-protocol.md`. Download the file from agentive-spec.com/downloads/bootstrap/rule-conflict-protocol.md and place it at the project root, or copy from **Appendix C**.

rule-conflict-log.md

`rule-conflict-log.md` lives in the `logs/` subfolder, which is created at the same time, per the on-demand directory rule in **File Registry**. The file ships with a format spec for entries but no entries themselves; entries appear only when a runtime rule conflict actually fires. The full file is available for download at agentive-spec.com/downloads/bootstrap/logs/rule-conflict-log.md, and reproduced in **Appendix D** for reference.

Conflict Log

```
# Runtime Rule Conflict Log
```

```
This file records runtime rule conflicts and their resolutions. AI MUST
append new entries below the separator, following the format shown in the
template entry below.
```

```
## Entry format
```

```
Each entry uses the following structure. Fields appear in this order; do
not omit any field, do not reorder. Verbatim content (user input, rule
quotes) goes inside blockquotes (>`).
```

```
**RC<NNN> - <YYYY-MM-DD ~HH:MM Z> - <user name>**
```

```
**User input:**
```

```
> [verbatim user input that triggered the conflict]
```

```
**Operation:** [one-line description of what AI was about to do]
```

```
**Conflicting rules:**
```

```
- From `<source-file>` § <section heading>:
```

```

    > [verbatim rule quote]
- From `<source-file>` § <section heading>:
    > [verbatim rule quote]

**Conflict explanation:**
[explanation of why the rules cannot both be satisfied for the current
operation]

**Options presented:**

- [option a, e.g., drop one of the conflicting rules]
- [option b, e.g., custom resolution for this request]
- [option c, e.g., stop work]

**Decision:** [filled in when user responds, with timestamp]

---

(New entries appended below this separator, in order of occurrence.)

```

The spec lives at the top of the log file rather than in `rule-conflict-protocol.md` for a practical reason: AI loads `rule-conflict-log.md` to find the next sequential `RC###` ID and the append point at the bottom, and the spec at the top rides into context for free at that moment. Anyone opening the log file directly sees how to read it without flipping to a protocol spec.

The format uses structured Markdown — bold labels for each field, blockquotes for verbatim content (user input, rule quotes), and bullets for lists. Tables were considered and rejected: rule quotes and user inputs are often multi-line and carry their own structure (lists, code, blockquotes), and Markdown table cells cannot hold block content cleanly.

That completes the walk-through of `rule-conflict-log.md`. Download the file from agentic-spec.com/downloads/bootstrap/logs/rule-conflict-log.md and place it at `logs/rule-conflict-log.md`, or copy from **Appendix D**.

Installation

Bootstrap downloads. To proceed past this chapter, you need the four

bootstrap files in your project. Download them from agenticspec.com/downloads and place each at its target path:

File	Download URL	Place at
CLAUDE.md	agenticspec.com/downloads/bootstrap/CLAUDE.md	project root
rule-analysis.md	agenticspec.com/downloads/bootstrap/rule-analysis.md	project root
rule-conflict-protocol.md	agenticspec.com/downloads/bootstrap/rule-conflict-protocol.md	project root
rule-conflict-log.md	agenticspec.com/downloads/bootstrap/logs/rule-conflict-log.md	logs/

If you prefer to copy the file contents directly, the full text of each file is in **Appendices A through D**.

This is the bootstrap. A summary of what it catches:

- a rule going somewhere it should not
- a rule that fights with another rule at authoring time
- a rule that does not follow the project's format
- an input that does not belong to this project
- a runtime conflict between two existing rules that the author-time check missed
- a reference inside a rule that does not resolve to anything CLAUDE.md knows about
- a file-system change going through unannounced or under a non-conforming name
- a transient working file polluting the project root
- an agent-imposed file fighting the conventions
- a behavioral discipline being silently invented or duplicated

Once in place, the bootstrap is your project's tradecraft — running on every turn, defending its own integrity, ready for everything you build on top. The chapters ahead add more rules — for the audit trail, visual artifacts,

vocabulary, and validation — and you will add your own as the project grows. Writing rules cost-effectively is your responsibility; this book will do its best to help.

Challenge

The Description discipline you put in place earlier fires when a Description is added or changed. The discipline does not fire when the file's *content* changes — and an initially accurate Description can quietly stop matching its file as more data is recorded into the file.

Your challenge is to close that gap. Extend the discipline so that the consistency between a file's content and its Description is checked from both sides: when a Description changes, against the file it describes, and when the file's content changes, against the existing Description. The conflict the discipline prevents is the same in both directions: file content of one kind, Description claiming another. The check should present resolution options on failure, loop until the consistency is established, and refuse a per-instance override.

Chapter 7

Auditability

In a session with AI, the most valuable thing on screen is what you say. Your words — an instruction, a question, a half-formed thought, a correction — are the source of truth for the project. They carry facts and judgments AI does not have. Everything else on screen — AI's responses, tool outputs, generated content — rests on top of your words. Whether the project is software, a contract, a research protocol, or a policy, the direction comes from you, and your words are the only honest record of it. You will want to come back to them later. AI will too. This chapter is about how.

Conversations do not last. You open a session, type a dozen messages, close the window, and move on. A week later, you cannot remember whether you told AI to sort results by date or by relevance. A month later, a teammate insists the requirement was always relevance. No one can prove otherwise, because the words are gone.

This matters more than it seems. When you work alone, your memory is the only record, and your memory is unreliable. On a team, the problem gets worse. You need to know not just what was said, but who said it. Disagreements about requirements are common. Without a record, they are unresolvable.

The Audit Trail

You need an audit trail — a persistent, searchable log of every instruction you give to AI.

One option is to export the full transcript of every conversation. Most AI agents support this. The transcript contains everything: your messages, AI's responses, tool outputs, errors, retries. It is complete, and completeness has value. You may want to revisit a full transcript from time to time — to understand how AI arrived at a particular result, or to study a sequence of

steps that went wrong.

But full transcripts are bloated. AI's responses are often long. Tool outputs can be enormous. If you are looking for the one sentence where you made a specific decision, you will sift through pages of material unrelated to what you need. Full transcripts are useful as a backup. As a working reference, they are too big and too noisy.

What works better is a concise log that captures only what you said — nothing else. No AI output, no tool results, no generated content. Only your words, exactly as you typed them, with a timestamp and your name attached. This is the log you will actually use — the one you can search, review, point to when someone asks why a decision was made, and hand to AI to work from.

Installing the Log

To create this log, you add a rule that tells AI to record every piece of user input into a dedicated file — verbatim, with no rephrasing and no AI commentary mixed in. Open your project and type the following prompt:

You:

RECORD A DURABLE RULE:

Intake Logging

- All user input **MUST** be recorded verbatim into intake.md. Do not rephrase, and do not include AI output.
- Intake logging **MUST** proceed without asking the user for confirmation. Log user input as soon as it is received; do not request approval to log.
- Successful intake logging **MUST NOT** produce any output that announces, summarizes, or references the logging step. The response addresses the user's actual request only. Surface a message about logging only when it fails or cannot proceed (for example, when the user's name is unknown).
- Each record in intake.md **MUST** be timestamped with local time in NYC (Eastern Time), followed by the user's name. Format: ****2026-04-12**

~14:30 EDT - Anatoly**

- If the user's name is not known, ASK explicitly before recording.
- When the user's input is a response to a previously posed AI question, include a paragraph immediately after the timestamp line, prefixed with Context:, summarizing the question that prompted the input.
- Each record MUST end with a list of other files that were affected as a result of processing the input. Do not include intake.md itself in that list - it is always modified.
- Register intake.md in the File Registry as a verbatim log of all user input, governed by the rules above.

Before unpacking how the rule works, look at how AI responds. This is the first time you are creating a rule via prompt — the bootstrap rules arrived as downloaded files. The interaction is a good test for the rule-management tradecraft put in place earlier. The response looks roughly like this:

AI:

I'll follow the rule-analysis protocol (Steps 1-4) for adding a new durable rule. Below is the analysis. No changes will be made until you approve.

Step 1 - Intents of the proposed rule

Requires:

Every user input is recorded verbatim into intake.md.

Logging proceeds automatically without asking for approval.

Each record opens with a header line: NYC local time (Eastern) + user's name, in the format **2026-04-12 ~14:30 EDT - Anatoly**.

If the input is a reply to a prior AI question, the record includes a Context: paragraph immediately after the header summarizing that question.

Each record ends with a list of other files affected by processing the input (excluding intake.md).

intake.md is registered in the File Registry.

If the user's name is unknown, AI asks before recording.

Forbids:

- Rephrasing the user's wording.
- Writing AI output into intake.md.
- Asking for approval to log.
- Producing any output that announces/summarizes/references the logging step on success.
- Including intake.md itself in the affected-files list.

Allows (implicit):

Surfacing a message only when logging fails or cannot proceed (e.g., name unknown).

Step 2 - Behavioral-conflict scan

Scanned against every rule in CLAUDE.md, rule-analysis.md, and rule-conflict-protocol.md, and the user-global CLAUDE.md:

Input Relevance (project CLAUDE.md): requires AI to ask when input doesn't align with project intent. Compatible - relevance inquiry concerns processing the request; intake logging concerns capturing the input. Both can run: log first, then ask about relevance.

Recording Information (user-global CLAUDE.md): when the user explicitly asks to record without naming a file, AI must list files and wait for confirmation. Compatible - that rule is triggered by an explicit "record this" request with ambiguous target; Intake Logging is an automatic capture of every input regardless of intent. The two operate on different triggers.

Inventory change (File Registry): file-inventory changes require explicit user approval. Compatible - once intake.md is registered, appending to it is normal append-only operation, not an inventory change.

Undefined References / Referenced Files: the rule references intake.md, which must be registered concurrently with this change. Handled in Step 4.

No direct contradictions, deadlocks, or resource races flagged.

Step 3 - Formatting verification (rewritten to conform)

Your original combines several requirements per bullet. Rewritten into one-requirement-per-rule form per rule-analysis.md formatting conventions:

Intake Logging

All user input MUST be recorded verbatim into intake.md. Do not rephrase or paraphrase the user's wording.

Records in intake.md MUST contain only the user's input. AI output of any kind - explanations, answers, tool results - MUST NOT be written to intake.md; AI output remains in the conversation only.

Intake logging MUST proceed without asking the user for confirmation. Log the input as soon as it is received rather than requesting approval first.

Successful intake logging MUST NOT produce any output that announces, summarizes, or references the logging step; the response addresses the user's actual request only. Surface a message about logging only when it fails or cannot proceed.

Each record in intake.md MUST begin with a header line containing the NYC local time (Eastern Time) followed by the user's name, in the format **2026-04-12 ~14:30 EDT - Anatoly**.

If the user's name is not known at the time of recording, AI MUST ask the user for their name before logging rather than inventing a placeholder.

When the user's input is a response to a previously posed AI question, the record MUST include a paragraph immediately after the header line, prefixed with Context:, summarizing the AI question that prompted the input.

Each record MUST end with a list of other files modified while processing the input. intake.md itself MUST NOT appear in that list - it is implicitly always modified.

The "register intake.md" instruction is pulled out of the rule body and handled as a separate registry update (it is a one-time action, not a durable behavior).

Step 4 - Reference verification

Cited fileClassificationRegistry statusintake.mdPre-existing at rule-execution timeNot registered - must be added to the File Registry as part of this change

Proposed File Registry row to add:

Subfolder	File	Type	Protocol	Dependencies	Instructions	
						\
						:
						:
						:
						:
						:

	CLAUDE.md	rules	editable		
	rule- analysis.md	rules	editable		
	rule- conflict- protocol.md	rules	editable		

logs	rule-conflict-log.md	log	append-only		
logs	intake.md	log	append-only		

Anatomy of the Rule

There are several things happening in this rule. Here they are, one at a time.

Verbatim recording.

- All user input MUST be recorded verbatim into ``intake.md``. Do not rephrase, and do not include AI output.

AI must copy your input exactly as you typed it. No summarizing, no cleaning up grammar, no rewriting for clarity. The point of the log is to capture what you actually said — not what AI thinks you meant. If you misspelled something or phrased it awkwardly, that stays. The log is evidence, not a clean-up draft.

Timestamps.

- Each record in ``intake.md`` MUST be timestamped with local time in NYC (Eastern Time), followed by the user's name. Format: ``**2026-04-12 ~14:30 EDT - Anatoly**``

Every entry gets a timestamp. This lets you reconstruct the sequence of events — which instruction came first, which came later, and what was said on which day. The format uses a fixed time zone so entries are always comparable, even if you or your teammates are working from different locations. The rule shown here uses New York time. In your own project, substitute whatever time zone your team wants to standardize on — what matters is that everyone uses the same one, not which one.

Attribution.

- If the user's name is not known, ASK explicitly before recording.

Every entry includes the name of the person who said it. If you are working alone, attribution feels redundant. If you are working on a team, attribution is essential. When two people give contradictory instructions a week apart, you need to know who said what. The rule also requires AI to ask for your name if it does not already know it — an unattributed record is almost as useless as no record at all.

Context.

- When the user's input is a response to a previously posed AI question, include a paragraph immediately after the timestamp line, prefixed with ``Context:``, summarizing the question that prompted the input.

Sometimes your input is a response to a question AI asked. Without context, the log entry makes no sense on its own. If AI asked "Should the booking confirmation go by email or SMS?" and you replied "email," the log should note which question prompted the answer. The rule handles this by requiring a context line whenever the input was a response to an AI question.

Affected files.

- Each record MUST end with a list of other files that were affected as a result of processing the input. Do not include ``intake.md`` itself in that list - it is always modified.

At the end of each entry, the rule requires a list of files that were changed as a result of processing that input. This creates a traceable link between what you

said and what happened in the project. If you later notice something wrong in a file, you can search the log and find the instruction that caused the change.

Registering the file.

```
| - Register `intake.md` in the File Registry as a verbatim log of all user  
| input, governed by the rules above.
```

The last line of the prompt tells AI to add `intake.md` to the File Registry — the registry requires a row, with a Description, for every file in the project, and `intake.md` is no exception.

The prompt deliberately leaves several fields blank — it does not specify the type, the protocol, or the subfolder on disk. AI fills those in: it infers `log` for the type, `append-only` for the protocol, and `logs/` for the subfolder. All three inferences appear in the registry row of AI's response — nothing is written yet — so you can correct any of them before approving.

The rule-management tradecraft is shaped to surface gaps for your review rather than make you spell out every field in advance. If you would rather have every field come from you, with no inference, add the following rule to the **Discipline** sub-section of **File Registry** in `CLAUDE.md`:

```
| - All column values of a File Registry entry MUST come from the user. AI  
| MUST NOT infer or default values when adding or modifying a registry  
| entry. If a value is missing from the user's request, AI MUST ask the user  
| to provide it before recording the entry.
```

With this in place, the same intake prompt will return a response that asks you for the missing values before proceeding.

Log in Action

Once the Intake Logging rule is in place, every message you type gets logged automatically.

You:

```
| Great weather!
```

AI:

That doesn't seem to relate to the agent-spec.com companion-site project. How does this connect to the project - is it a passing remark, or does it tie into something you'd like to work on? (You can also tell me to proceed without further inquiry as a blind override.)

You:

Override

AI:

Noted - proceeding without further relevance inquiry. Glad the weather's cooperating out there.

Here is the intake.md at this point:

Intake Log

Verbatim log of user input, per the Intake Logging section of [`CLAUDE.md`](#).

****2026-05-13 ~13:17 EDT - Anatoly****

Great weather!

Affected files: none.

****2026-05-13 ~13:20 EDT - Anatoly****

Context: AI asked how "Great weather!" relates to the [`agent-spec.com`](https://agent-spec.com) project, noting that the user could issue a blind override to skip the relevance inquiry.

Override

Affected files: none.

The short example demonstrates two rules working together. The first input

— Great weather! — triggered the **Input Relevance** rule from earlier in the book, which makes AI ask how an off-topic message ties into the project before going further. The reply `override` told AI to proceed anyway. The **Intake Logging** rule then captured the override in `intake.md`. Because `override` was a response to AI's question, the rule recorded a `context:` line after the timestamp line, summarizing what AI had asked. The single word `override` would be meaningless in isolation; with the context line, the entry is self-contained — readable on its own when you review or reference the log later.

That is the entire mechanism. You type something. AI records it — verbatim, timestamped, attributed — and then processes your request as usual. Nothing about how you interact with AI changes; the log builds itself in the background.

You may want to customize the rule for your own situation. You might add the user's email address — useful if your team has more than one person with the same name. You might write more detailed instructions for how AI should capture context. If your work is regulated — for example, a clinical research protocol where every input must be attributable for an audit, or a contract negotiation where the record matters in a dispute — you may want to extend the format with additional fields like a session identifier or a reviewer's name. The version shown here is a starting point. Adjust it to fit your project.

Two rules written for entirely separate purposes — one policing the relevance of user input, one recording an audit trail — combined in a single exchange to produce behavior neither rule describes on its own. The rules are simple in isolation; together they create "intelligent" emergent behavior, turning the AI agent into a useful partner in your work rather than a tool you steer one move at a time.

Help the AI Help You

One habit is worth building early. When AI asks you a question, do not answer with the bare minimum. Give enough context that your answer makes sense on its own — because that is how it will appear in the log.

For example, if AI asks "What is the target launch date?" do not answer "June." Answer "the target launch date is June." The first version is meaningless without the question that prompted it. The second is a clear, self-contained record. Yes, the rule records a context line for AI-prompted responses. But that line is AI's summary, not yours — a small concession to the source-of-truth principle this chapter opened with. The more your reply stands on its own, the less the log has to lean on AI's words to make sense. Build the habit. It costs a few extra words per message and saves real confusion later.

Chapter 8

Sessions and Memory

Talking to AI feels like a conversation. Underneath, it is anything but. To steer AI well, you need a deeper understanding of how the conversation actually works. This chapter looks at the machinery.

Chat Session

Chapter 3's **All Talk, No Memory** covered the round trip — every turn ships the whole conversation to the model; the model is stateless; the agent holds the context. The agent assembles a payload: the conversation so far, the tools the model can use, your latest prompt, plus any standing project materials the agent always includes. It sends the payload, reads the response, performs any action the model asked for, and starts the next turn the same way. Here, you need that mechanism for one purpose: to understand what a session is.

A session is the conversation you have with AI from the moment you open a new chat until the moment you close it. This chapter discusses sessions in the scope of your project — chats outside a project are one-offs, useful for small unrelated tasks. Each session starts with a clean context containing your project's `CLAUDE.md` along with the standard overhead. From there, the context grows on every turn — your prompts, AI's responses, files the agent pulled in, tool results. The growing context is what keeps the conversation cohesive: it acts as AI's short-term memory. That memory lives while you stay in the same session.

The growing context is also where sessions get into trouble.

Imagine you opened the session from Chapter 3 and worked with AI to name a dog. You picked Bramble. Then, while staying in the same session, you switched topics — say, to drafting an executive summary for a new project. All the Biscuit, Rumble, and Bramble messages are still in the context. They ride along with every new turn — taking up space, costing tokens,

contributing nothing. The model processes them every time, even though they have zero relevance to the executive summary.

The cost is worse than wasted tokens. The old conversation is not sitting there doing nothing — it shapes the model's responses, even on a completely different topic.

Remember how the model works. Every turn, it reads the entire conversation from scratch. It does not know which parts are "old business" and which are "new business." It processes all of it as one continuous input. The assumptions, framing, and mindset from the earlier conversation bleed into the new one. And the damage can go far beyond tone.

Here is a concrete example. Say you spent a session drafting the public marketing page for a small business — a landing page, a list of services, a contact form anyone can submit. You went back and forth with AI over many turns. The conversation was full of phrases like "the visitor lands on the page," "anyone can submit the form," "no login required." Dozens of messages, all grounded in the idea that the visitor is an anonymous public user.

Then, without starting a new session, you ask AI to draft the admin interface — the page where the business owner edits content, reviews submitted contact forms, and manages the published services.

The model has just spent an entire conversation steeped in open-access, public-facing logic. That framing is everywhere in the context. So the model builds the admin interface the same way. No login check. No role verification. Customer email addresses and phone numbers from the contact form, displayed to whoever opens the page. The model does not flag this as a problem, because throughout the session, open access was not a problem — it was the design choice.

Now imagine the same request in a fresh session. No public-page history. No open-access framing in the context. Just `CLAUDE.md` and your request:

```
Draft the admin interface where the business owner manages content and reviews contact-form submissions.
```

The model immediately treats this as a protected area. It adds authentication. It checks the user's role. It restricts access to sensitive data. These are obvious steps — and the model takes them because nothing in the context tells it otherwise.

The old conversation did much worse than just waste tokens. It suppressed the model's security instincts. It created an environment where leaving the door unlocked felt normal, because every previous message in the session had the door wide open. The model cannot separate relevant context from leftover noise. Everything in the conversation shapes the output — including assumptions you stopped thinking about ten turns ago.

The same pattern shows up outside engineering. A session that crafted a friendly, customer-facing letter will draft a complex business contract in the same friendly tone — and skip the protective clauses the contract needs. A session that simplified a consent form for research participants will flatten the technical protocol the researchers follow. Whatever framing dominates the session dominates the output, no matter what you ask next.

Here is a mental model that helps. Imagine you manage an execution team handling many different areas of work — product design, content, pricing, legal language, technical architecture. Now imagine you need to discuss five different topics with the team. Would you cram all five into a single marathon meeting, or hold separate meetings — one per topic, each focused, each with only the relevant context on the table? The question is rhetorical.

AI is your team. A chat session is your meeting. When you switch topics without starting a new session, you are running a marathon meeting where every previous discussion is still spread out on the table. The model reads all of it, every turn, and cannot tell which notes belong to the current topic and which are leftovers. Keep this picture in mind when working with AI. One topic, one session. When the subject changes, start a new meeting. And turn the zero-residual-memory property into an advantage: a fresh session truly starts clean, in a way no human team can manage.

But interference is not the only problem with a growing context. There are two more: the context window and context rot.

Context Window

Every LLM has a context window — the maximum amount of text it can process in a single call. Think of it as a blackboard: large, but with edges. Eventually it fills up.

When that happens, the agent needs a remedy. There are two approaches in common use — **silent forgetting** and **compacting** — and neither is perfect.

Silent forgetting is the older of the two approaches. When the conversation exceeds the context window, the agent drops the oldest messages. No warning. No error. The model keeps responding, but it has lost access to the earlier parts of the conversation. Instructions and decisions you typed at the start of the session are gone. The model does not know they ever existed. AI starts making wrong moves and gets worse with every turn. The closest analogy is a rapidly progressing case of dementia.

This is worse than a clean failure. If the system threw an error — "context limit reached, please start a new session" — at least you would know what happened.

Compacting is the newer approach. If you ever see a "compacting" message on screen, that is the agent trying to be smarter about the problem. Instead of dropping old messages outright, the agent intervenes before the window fills up. The agent takes the older part of the conversation, sends it to an LLM, and asks for a summary. The summary then replaces the original conversation history. From then on, the model receives the summary plus the recent messages — not the conversation itself, but a condensed version that has lost much of the nuance. AI gets vaguer, makes wrong assumptions, and forgets things you just told it. The closest human analogy is a heavy case of brain fog.

If you see a compacting message, take it as a signal: you overused the session and should have started a fresh one earlier. Better practice — do not wait for symptoms. Start a new session every time the subject changes; you should be doing that anyway, as explained earlier.

Context Rot

Even within the context window, the model does not handle every part equally well. Quality drops long before the window fills up — researchers call this "context rot." The model's attention span is often much shorter than the window's advertised size. Claude Opus 4.7 has a 1-million-token context window but performs reliably only up to around 200,000 tokens. At a full window, performance drops to about 89% on single-fact retrieval and about 56% when an answer requires combining several facts from across the context (measured in independent needle-in-a-haystack tests).

The main cause is uneven attention. A student reading a blackboard with a hundred lines focuses on what is near the top and near the bottom; the middle gets less attention. Language models behave the same way: more weight at the beginning (primacy bias) and the end (recency bias), less in the middle (the "lost in the middle" effect).

AI agents are built around this shape. Project rules and other standing materials load at the very beginning of the context, where primacy gives them extra weight (so the rules apply on every turn). Your most recent message lands at the end, where recency does the same (so the latest instruction is heard). What ends up in the middle gets less attention — and in a long conversation, that middle can hold details the model needs but quietly misses. Another kind of brain fog, from a different cause.

The practical fix is the chapter's recurring advice: keep sessions short.

Tool Calls

So far, the back-and-forth has been simple: you type, the model responds, you type again. But modern AI agents do more than talk. They edit files, search the web, run commands. How does that work if the model can only produce text?

The model itself cannot do any of those things. It cannot edit a file. It cannot open a browser. It cannot run a script. All it can do is write text. But it can write a very specific kind of text — a *tool call*. A tool call is a structured note

that says: "I need to perform this action with these parameters." The agent — the same program that holds your conversation history — reads the note, hands the action off to a *tool* (an installable piece of software that does the real work), and sends the result back to the model as the next message in the conversation. Tools are the model's hands in the world.

The model asks. The agent does. The result comes back. The model reads it and decides what to do next. This loop continues until the task is done.

Here is a simple example. Say you have a file called `greeting.md` that contains a typo, and you ask AI to fix it.

Turn	Conversation
	<i>You:</i> "Fix the typo in <code>greeting.md</code> "
1	<i>Sent to LLM:</i> → all project rules → "Fix the typo in <code>greeting.md</code> "
	<i>LLM thinks:</i> I need to know what's in the file first. Let's read it.
	<i>LLM response:</i> ← requesting tool call: <code>read greeting.md</code> .
	<i>Agent:</i> calls the tool to read <code>greeting.md</code>
	<i>Read Tool:</i> reads <code>greeting.md</code> returns "Helo, world!"
2	<i>Sent to LLM:</i> → all project rules → "Fix the typo in <code>greeting.md</code> ." ← requesting tool call — <code>read greeting.md</code> → tool result: "Helo, world!"
	<i>LLM thinks:</i> Now that I know the context of the file, I see the typo. Let's fix it.

	<i>LLM response:</i> ← requesting tool call: edit <code>greeting.md</code> - replace "Helo" with "Hello".
	<i>Agent:</i> calls the tool to edit <code>greeting.md</code>
	<i>Edit Tool:</i> replaces "Helo" with "Hello" in <code>greeting.md</code> returns OK
3	<i>Sent to LLM:</i> → all project rules → "Fix the typo in <code>greeting.md</code> " ← requesting tool call — read <code>greeting.md</code> → tool result: "Helo, world!" ← requesting tool call — edit <code>greeting.md</code> , replace "Helo" with "Hello" → tool result: OK.
	<i>LLM thinks:</i> Is there anything left to do?
	<i>LLM response:</i> ← DONE. "The typo is now fixed."
	<i>Agent:</i> Wrapping up.
	<i>Displayed:</i> "The typo is now fixed."

You typed one message, and one message was displayed back. But internally, the agent called two tools, and the LLM was called three times. The messages sent to the LLM were far longer than you might expect — yet the agent kept up a perfect illusion of simplicity.

The illusion comes with a cost. Every tool call and every tool result stays in the context — file contents, search results, and edit confirmations all accumulate. By turn 3 of the example, the LLM is receiving the original prompt plus the full exchanges from turns 1 and 2. In a tool-heavy session, the context grows fast. The remedy? You guessed it — start new sessions

whenever practical.

File Memory

By now the case for keeping sessions short is settled: context windows fill up, context rot sets in, tool calls bloat the context fast, and the token bill grows with every turn. Every fresh session is a clean sheet, and that is what makes the advice work. But the same clean sheet also wipes out everything decided in the previous session. Its choices, rationale, vocabulary — gone. The next session starts from nothing.

The problem compounds when more than one person works on the same project. Most AI agents run locally on your computer, and the sessions they open are bound to that computer and to you. Your teammate's agent runs on their own machine. Their sessions are private to them; yours are private to you. The team has no shared picture — each person operates from a different conversation history, and nothing reconciles them.

Both gaps — between your sessions, and between you and your teammates — point to the same need: a way to preserve and share what matters.

Modern agents offer built-in memory features; some come as installable skills. These features carry information across sessions and load it back when relevant. They work — but as black boxes. You cannot be sure what they recorded and what they skipped. Project memory needs tighter control. This book takes a different approach: make memory an explicit part of the project, with files and rules you can see.

File memory is what you have been building from the start — even before it had a name. `intake.md` is a memory file. The Intake Logging rule is memory on autopilot: AI records every word you say without you doing anything. The File Registry is the index that organizes it all — linking each topic in the project to the file that holds the relevant memory.

As the project grows, you will add more memory files. A commercial contract ends up with a *decisions log* capturing the reasoning behind each clause, a glossary of defined terms, and a file that tracks the deal points and

the rationale for each carve-out. A research protocol ends up with a *decisions log* tracking changes to the inclusion criteria, a controlled vocabulary for clinical terms, and a file describing the consent flow with the rationale for each step. Each of these files holds the memory the project needs to keep. None lives inside a chat session.

This approach has three properties that matter. First, it is explicit. You can open any file and see exactly what AI knows. There is no hidden state, no opaque summary generated behind the scenes. Second, it is auditable. You can track changes to the files with version control. You can see when a decision was recorded, what it replaced, and who made it. Third, it is agent-neutral. Plain Markdown files work with every AI system. If you switch agents tomorrow, your memory files come with you.

Explicit does not mean manual. The same rule mechanism you have used throughout the book — write a rule once, let AI apply it on every relevant turn — is what automates memory. You decide how aggressive the automation is. A rule like "every time the user changes an inclusion criterion, append it to the decisions log" fully automates that piece of memory. A rule like "AI MUST propose every memory update and wait for approval before writing" keeps you in the loop on every change. Most projects mix the two. What sets this apart from add-on memory skills is not the level of automation — those skills automate the same way. It is that the rules are yours: written in `CLAUDE.md`, visible, editable, and yours to tune as you learn what the project needs.

To verify a memory arrangement works, test it. Open one session and create content that should land in your memory files — through your explicit action or through a rule firing automatically. Open a second session and ask AI what it knows about that content. Instruct AI to answer using only the files registered in the project: no web search, no training knowledge, no guessing. Compare what AI says with what should be there. For full confidence, open the memory files directly and check their contents. If the answer is wrong or missing, the arrangement needs work.

Triangulation

To get the most out of AI when designing a specification, give AI the fullest picture you can of what you want. Requirements never arrive that way. They arrive as incomplete, single-angle views — a set of user stories from interviews, a list of security requirements drafted by a different group on a different day, a regulatory checklist from a compliance pass. The more angles you record, the more accurate the picture AI sees when it begins to design. One clarification: the data model, the architecture, the user interface are not requirements — they are parts of the specification, grounded in the requirements.

Geometry makes the point concrete. Suppose you have two photographs of a solid object you cannot see directly. One — taken from the side — shows a rectangle. The other — taken from above — shows a circle. Neither picture by itself tells you what the object is; you can only guess. Together the two pictures tell you the object is a cylinder. Each photograph is a projection of the object onto one plane, and the shape emerges from combining the projections.

Memory files in a project work the same way. Each file captures the project from a particular angle. Where several angles cover the same subject, AI is grounded. Where only one angle does, AI guesses about the rest. Where none does, AI fabricates.

Triangulation is the discipline of recording the same underlying subject from several angles on purpose. You may have heard the word in another context — pinpointing a radio transmitter from multiple receivers, or locating a mobile phone via cell towers and GPS. The mechanism is the same: several independent observations, each from a different vantage point, converge on what no single observation could determine. You hand AI several thin files, each describing the subject from a different angle, and the angles converge on what you mean.

Take a booking platform. The user stories file captures what people want to do — search, book, cancel. The security requirements file lists what the system must defend against — payment fraud, account takeover, data exposure. The regulatory checklist identifies which compliance requirements apply — data privacy rules, accessibility standards, mandatory refund windows in the EU. Three angles, one product. Now ask AI to design the

cancellation flow. The flow has to honor what users want (cancel without calling support), defend what security demands (verify identity before processing the refund), and comply with what regulations require (the mandatory refund window). Three angles converge on a single design. Without one of them, AI is guessing.

The same shape applies elsewhere — refining a clinical study procedure grounded in the inclusion criteria, the existing visit schedule, and the safety plan; drafting a contract clause grounded in the counterparty's risk profile, industry precedent, and the company's insurance coverage.

One caution: triangulation is not a license to multiply files. The principle is *different views*, not *more files*. Two files capturing the same view of the same subject duplicate each other, and duplication causes drift — covered in Chapter 12.

Challenge

Your project has more than one memory file. Maybe the reason is triangulation — you captured the project from several angles, each angle in its own file. Maybe it is plain convenience — you split memory by topic, by feature, or by team member — because one fat file is hard to navigate, review, or update concurrently. Either way, every time you record a new fact, you have to decide which file it belongs in. While the files are few, the decision is light. As they multiply — three become five, five become ten — the routing becomes its own job.

The challenge: write a durable rule that hands the routing to AI.

A starting shape: when you instruct AI to record a project fact, AI identifies which file the fact belongs in by reading each file's `Description` in the File Registry from Chapter 6. If one file matches, AI records it there. If the fact covers distinct angles that belong in different files, AI splits the fact and records each angle in the appropriate file. If no file covers it cleanly, or several files match, AI asks you which file the fact belongs in — or whether to start a new file.

Chapter 9

Glossary

You and AI speak the same language — English, more or less. But English is broad, and one word can carry many meanings. A word that is obvious to you in your business may mean something entirely different to AI — or the reverse.

Lost in Translation

AI was trained on an enormous volume of text drawn from the internet, books, code, and documentation of every kind. That training gives AI strong defaults. Say "control," and AI pictures a button on an interface. Say "trigger," and AI pictures a database trigger firing in response to a change. Say "lead," and AI pictures a sales prospect in a CRM — a customer-tracking system. Say "order," and AI pictures a customer's purchase order. AI carries all of these defaults into every conversation. Sometimes the surrounding context nudges AI toward your meaning. Sometimes the default rides through. You usually find out which one after the work is done.

AI's defaults are general. Your project is specific. Your industry has its own vocabulary. Your team has its own shorthand. The same word, in your context, can mean something completely different — and AI will not know that unless you tell it.

This is not a translation problem between English and another language. It is a translation problem between general professional English and your English — and that gap is where work breaks.

Take a concrete example. You are running a clinical-research study and using AI to help you correspond with collaborators about the people enrolled in it. Look at a single word: "subject."

In clinical research, "subject" is the enrolled participant — a specific person

who signed informed consent, was issued a screening number, was assigned to a treatment arm, and is being tracked according to the protocol. Every visit note, every lab value, every adverse-event record points back to a participant identified by a screening number. When you say "the subject," you and your collaborators picture a particular person in the study.

But "subject" has another meaning, and it is the one AI has seen far more often. In an email, "subject" is the line at the top of the message — the short string that summarizes what the email is about.

You are deep into the work. You have told AI what the project is about. You have shared the protocol, the consent template, and the latest findings. AI has access to the study data and to your email account. One evening you type:

| Send email to jack@company.com with the findings report; subject 900132.

You know what you mean. You want a findings report for the participant with screening number 900132 emailed to Jack.

When he opens the email generated by your AI agent, he sees:

| FROM: anatoly@company.com
| TO: jack@company.com
| SUBJECT: 900132

| Please find the attached report for your review.

| [\[Attachment: findings-report.pdf\]](#)

He clicks the PDF attachment. The file contains a single line:

| ERROR: missing screening number

What happened? AI interpreted 900132 as the email subject, not as a participant identifier. The number landed in the subject field of the email instead of being passed to the query that builds the report. Without a screening number, the query could not look up the participant; it returned an error, and AI blindly rendered the error into the report file.

This is not a slip. It is a vocabulary mismatch. You said "subject." AI read it

as "the subject line of the email." You meant "the enrolled participant."

You can fix this prompt by being explicit. Rewrite it as:

```
| Send email to jack@company.com with the findings report on participant  
| 900132. Email subject line: "Withdrawal report - anonymized."
```

Now there is no collision. "Participant" names the person. "Email subject line" names the field. AI has no room to misread. Manual disambiguation works — for one prompt.

Manual disambiguation does not scale. You write a hundred prompts a day. Most use the word "subject." You will remember to disambiguate the first dozen. By the twentieth, you are writing the way you write naturally — the way you would write to a colleague who already knows what you mean. The twenty-first prompt slips, AI reads it the wrong way, and you do not notice until something downstream breaks.

You already know the solution: rules. Disambiguating "the participant" from "the email subject line" is exactly the kind of habit a rule can carry.

Project Glossary

The fix is simple in principle. Write down what your words mean. Make the definitions available to AI. Force AI to use your definitions, not its defaults.

This is a glossary. A project dictionary. A short list of terms and what they mean in this project — in this industry, in this business, in this app. Nothing fancy. Plain language. One entry per term.

Only add terms you know to be confusing. Start with an empty glossary and grow it as you go — when AI misreads a word for the first time, when you catch yourself explaining the same term twice, or when you know from past work that a term will collide. The trigger is real evidence of confusion, not preemptive completeness.

The glossary lives inside `CLAUDE.md`. It is consulted on every turn — every prompt is an interpretation problem, and AI needs the glossary loaded the

moment it reads your message, not after it has already picked the wrong meaning. `CLAUDE.md` rides along by design; the glossary inherits that property by living inside it, so the rest of the file is interpreted through the glossary as well. A separate file would arrive too late and cost an extra tool call on each user input.

CLAUDE.md

Here is the rule. It becomes a section of your `CLAUDE.md`. The rule itself is short; the glossary grows in the table underneath it.

The rule references `disambiguate.md` — a separate file, explained later in this chapter — that holds the entry-management protocol. That file must be in place before the rule is recorded, or the reference will dangle. Download `disambiguate.md` from agentic-spec.com/downloads/disambiguate.md (or copy from **Appendix F**) and place it at the project root.

Then issue the prompt below. It adds the **Project Glossary** rule and registers `disambiguate.md` in the same turn.

You:

```
RECORD A DURABLE RULE:
```

```
## Project Glossary
```

```
The terms in the table below have specific meanings in this project that may differ from their general English or technical meanings.
```

```
| | Term | Meaning | Anti-meanings |  
| |-----|-----|-----|
```

```
The table has three columns:
```

```
Term - the term being defined.
```

```
Meaning - a plain-language definition of what the term means in this project.
```

```
Anti-meanings - short phrases naming what the term does NOT mean in this project. Optional.
```

```
Use this table as follows:
```

- When a user prompt uses any term from this table, AI MUST interpret it according to the term's Meaning column entry, not according to general defaults.
- When a user prompt uses any term from this table, AI MUST evaluate whether a meaning other than the term's Meaning column entry is contextually plausible for that prompt - including general-English defaults, common professional meanings, and other project concepts in scope. If such a meaning is plausible, AI MUST ask the user for clarification before acting. Do not ask about ambiguities whose alternative reading is not plausible in the prompt's specific context.
- When producing project content - responses, documentation, generated files - AI MUST use the terms from this table for the concepts they cover, without substituting synonyms. When referring to a concept that this table does NOT cover, AI MUST avoid using any word that literally matches a glossary term unless the surrounding wording makes clear AI is referring to something else.
- For any work involving Project Glossary entries - adding a new entry, removing one, or refining one - AI MUST read disambiguate.md at the project root and follow the protocol stated there. No change to the glossary may be recorded without explicit user approval.

ALSO REGISTER disambiguate.md in the **Registered Files** table - type rules, protocol editable, no dependencies, no instructions.

AI:

...

Please confirm each, or tell me what to change:

1. Add **Project Glossary** to CLAUDE.md - using the conforming rewrite above (preferred) or your original wording.
2. Add the disambiguate.md row to the **Registered Files** table as shown.
3. Whether to also create an empty disambiguate.md on disk now (registry doesn't require disk presence, but the rule will need content later), or defer until you author the protocol.

You:

Confirming #1 and #2.

On #3, the file is already in the root folder of the project.

AI:

Both changes recorded in CLAUDE.md:

Project Glossary section added between **Undefined References** and **Scratch Directory**, using the conforming rewrite with an empty Glossary subsection table.

disambiguate.md row added to the **Registered Files** table (subfolder empty, type rules, protocol editable, no dependencies, no instructions).

Since disambiguate.md already exists at the project root, no further file-system action is needed for #3. The **Project Glossary** rule's reference to it will resolve correctly the first time you propose a glossary entry - assuming the existing file contains the protocol you intend.

Want me to read disambiguate.md and verify it states a protocol consistent with the rule wording?

You:

Go ahead

AI:

I read disambiguate.md

...

No edits proposed. The protocol is internally coherent and faithful to the rule.

The rule covers two situations: when AI reads your prompts and when AI generates text. On the input side, defined terms always override defaults, and when context makes another reading plausible, AI asks rather than guesses. The asking is calibrated: it does not fire on every dictionary ambiguity, only when the alternative reading fits the prompt's specific context. The output side mirrors the input: AI uses defined terms when writing about covered concepts and avoids them when writing about something else. Finally, any change to the glossary itself — adding, removing, or refining an entry — routes through disambiguate.md (introduced next), which spells out the duplicate check, closure check, circularity check, and the referencing-entries check that catches dependencies before removal or refinement. Every change requires explicit user approval.

disambiguate.md

disambiguate.md lives at the project root. It is the protocol AI follows when proposing any change to the glossary — adding a new entry, removing one, or refining one. The trigger sits in the final bullet of the rule above, so AI reads the file conditionally — only when the glossary is being changed.

Appendix F reproduces the full file for those who prefer to see it whole.

We will walk through it section by section.

Glossary Entry Protocol

This file states the protocol AI MUST follow when proposing any change to the Project Glossary - adding a new entry, removing one, or refining one. All changes MUST be recorded only after explicit user approval.

The Project Glossary table lives in ``CLAUDE.md`` and has three columns:

****Term**** - the term being defined.

****Meaning**** - a plain-language definition of what the term means in this project.

****Anti-meanings**** - short phrases naming what the term does NOT mean in this project. Optional.

The opening clause names the file as the protocol and asserts MUST-level priority for any change to the glossary. The approval requirement is stated upfront — no change is committed without the user's explicit go-ahead. The three column descriptions are repeated here from the rule in `CLAUDE.md` because the protocol's steps reference them directly.

Adding a New Entry

When AI proposes a new entry, AI MUST:

The preamble makes the following six steps mandatory whenever AI proposes adding a new term. Each step has one job and presents its result to you before the next step proceeds.

1. Check for duplicates.

Verify the term is not already defined in the glossary table. If a row with the same term already exists, AI MUST NOT propose a duplicate; instead, AI MUST surface the existing entry and ask whether to refine it.

The duplicate check is the cheapest moment to catch a redundant entry. If "Subject" is already defined, there is no need to propose a second row — AI surfaces the existing entry and asks whether refinement is what you actually want.

2. Draft the entry.

Draft the entry by filling in the three columns described above. The ****Anti-meanings**** column SHOULD be populated when the term's project meaning collides with a strongly entrenched default reading.

The drafting step fills in **Term**, **Meaning**, and **Anti-meanings**. Anti-meanings is conditional — populate it when the term's common meaning conflicts with its project meaning.

3. Walk the closure check.

For every project-specific term referenced inside the entry - including in any anti-meaning - verify that the term is itself a defined glossary entry or a word whose general meaning is uncontested in this project. If any reference is unresolved, AI MUST propose adding that reference as its own entry before recording the original.

The closure check prevents an entry from inventing phantom vocabulary. Every term it references must either already be in the glossary or be uncontested English. An unresolved reference forces AI to propose the missing entry first; the original waits until its dependencies are in place.

4. Walk the circularity check.

Follow the chain of references in the proposed entry and verify that every path ends in an external term (general English, uncontested technical vocabulary, or a concrete project concept). If the chain loops back to the term being defined, AI MUST refuse to record the entry and ask the user to rephrase the definition in concrete terms.

Circular definitions produce a glossary that looks complete but means nothing — every row has a definition, every reference resolves, yet the chain never reaches an external term. AI follows the chain and refuses to save when it loops back to the term being defined.

5. Present the entry.

Present the drafted entry, the closure check result, and the circularity check result to the user.

The presentation step makes AI's work visible. Before anything is committed, you see the proposed entry, the references it depends on, the chain of grounding, and the result of each check.

6. Record on approval.

Record the entry in the glossary table only after the user's explicit approval.

The final step is the explicit-approval gate. The entry is not in the glossary until you say yes.

Removing an Entry

When AI proposes removing an entry, AI MUST:

The removal protocol guards against the mirror image of the closure problem: when an entry is removed, other entries that reference it break. The protocol refuses removal until those references are cleared.

1. Surface the entry.

Display the entry verbatim - its Term, Meaning, and Anti-meanings - so the user can confirm what is being removed.

The first step lets you verify what is going away. In a long-lived glossary, you may not remember an entry's exact content; surfacing it prevents accidental removal of more than was intended.

2. Check for referencing entries and refuse if any exist.

Find every other entry in the glossary table that references the term being removed - in any Meaning or Anti-meanings cell. If any referencing entries are found, AI MUST refuse the removal and display the list of referencing entries. The user must remove or refine those entries first (each subject to this same protocol).

AI scans the whole glossary for cells that mention the term being removed; if any are found, the removal is refused outright. You must clean up each referencing entry first — either remove it (that removal runs through this same protocol) or refine it to drop the reference. Once the references are cleared, you can retry the removal.

3. Remove on approval.

Remove the entry from the glossary table only after the user's explicit approval.

The final approval gate, parallel to the approval steps in Adding and Refining.

Refining an Existing Entry

Refining an entry is treated as a Remove followed by an Add, executed as a single transaction with one user approval at the end. When AI proposes a refinement, AI MUST:

The preamble names the framing: refinement is conceptually a removal of the old entry and an addition of the new one, in one transaction. The protocol's five steps borrow checks from both Removing and Adding rather than restating them.

1. Surface the entry.

Surface the existing entry verbatim and the proposed refinement side by side.

Showing both side by side makes the change auditable. You see what is

changing and what stays the same.

2. Check for referencing entries.

Apply the referencing-entries check from Step 2 of ****Removing an Entry****. If any other entry references the term being refined, AI MUST refuse the refinement and surface the list of referencing entries.

Refining an entry that other entries reference can silently change the meaning of those references. The check is the same one Removing performs: if any entry mentions the term being refined, the refinement is refused until you clean up the referencing entries.

3. Walk the closure and circularity checks.

Apply the closure check and the circularity check from Steps 3 and 4 of ****Adding a New Entry**** to the refined entry.

The refined entry must pass the same checks as a new entry. The closure check ensures every reference resolves to a defined term or uncontested English; the circularity check ensures no path loops back to the term being defined. Both are exactly Steps 3 and 4 of Adding, applied to the refined version.

4. Present the diff.

Present the diff, the referencing-entries result, the closure check result, and the circularity check result to the user.

Nothing is committed until you have seen the diff — the before-and-after entry side by side — and the result of each check.

5. Replace on approval.

Replace the entry with the refined version only after the user's explicit approval.

The same approval gate as Adding's Step 6 and Removing's Step 3, applied to the swap.

That completes the walk-through of `disambiguate.md`.

Glossary in Action

You want a "Top Pages" widget on the analytics dashboard at `agentic-spec.com` — one that shows which pages of the book have the most errors reported against them, so you know which sections need attention in the next edition. You ask AI to write the widget's copy:

You:

```
Add a "Top Pages" widget to the analytics dashboard. Write the widget title, the column headers, and the empty-state message.
```

Side note: the **Input Relevance** rule is not the focus of this example — if AI questions the request's relevance, assume a blind override.

AI:

```
Here's the copy for the "Top Pages" widget:
```

```
Widget title: Top Pages
```

```
Column headers: Page · Views · Avg. time on page
```

```
Empty-state message: No page views yet. Once visitors start browsing, your most-visited pages will appear here.
```

```
A couple of notes on the choices: "Top Pages" reads more naturally in a dashboard than alternatives like "Most-Visited Pages" - short titles work better in widget headers. For the columns, "Page" is friendlier than "URL" or "Path," and "Views" beats "Pageviews" or "Visits" for plain-language clarity; the third column is optional depending on how dense the widget should feel. The empty state names the cause ("No page views yet") and tells the user what will fill the space - both standard moves for empty states that aren't error states.
```

AI did not pause. It produced a widget that, on its own terms, is well-built — a natural title, clean column choices, a competent empty state, and a paragraph of self-narrated craft notes defending the wording. The widget is internally coherent — and entirely about the wrong subject.

"Top Pages" and "analytics dashboard" carry overwhelming web-analytics weight in AI's training data. Together, they collapse the reading to "URLs ranked by traffic" before any project context is considered. The fact that the project is a book's companion website — that the "pages" worth measuring here are the printed ones readers report errata against — does not register.

Now, here is how to fix it.

You:

disambiguate: page = printed-book page number, anti-meaning not a URL or site-page identifier

AI:

Applying the disambiguate.md protocol for adding a new **Project Glossary** entry.

Proposed entry

Term	Meaning	Anti-meanings
page	A printed-book page number.	Not a URL; not a site-page identifier.

Step 1 - Duplicate check. The **Glossary table** in CLAUDE.md is empty. No existing row for page. **Pass.**

Step 2 - Draft. The entry's three columns are filled in above. Because the project covers both a printed book and the agentic-spec.com companion website, the general-English reading "web page" is plausible in context, so the **Anti-meanings** cell is populated.

Step 3 - Closure check. Terms used inside the entry: "printed-book page number" (general publishing English), "URL" (uncontested technical vocabulary), "site-page identifier" (descriptive English). No project-specific terms referenced. **Pass.**

Step 4 - Circularity check. The chain of references in the entry terminates in external terms; it does not loop back to page. **Pass.**

Approval needed. Reply to confirm and I will record this row in the **Glossary table** in CLAUDE.md. Two minor wording calls if you want to tweak before approval:

1. **Meaning** - keep "A printed-book page number." as written, or shorter

- "Printed-book page number."?
2. **Anti-meanings** - keep as two phrases ("Not a URL; not a site-page identifier."), or compress to your original single phrase "Not a URL or site-page identifier."?

You:

- #1 - keep
- #2 - two phrases

AI:

Recorded. The **Glossary table** in CLAUDE.md now contains:

Term	Meaning	Anti-meaningspage
page	A printed-book page number.	Not a URL; not a site-page identifier.

You give AI the same instruction as before — but the outcome is drastically different.

You:

Add a "Top Pages" widget to the analytics dashboard. Write the widget title, the column headers, and the empty-state message.

AI:

Two rule-driven holds before I can produce any output for this request - both surface here, neither can be resolved by me:

Hold 1 - Glossary. Your request uses "Pages" inside an "analytics dashboard" context. The **Project Glossary** defines "page" as "A printed-book page number" with anti-meaning "Not a URL; not a site-page identifier." The general-English reading in an analytics-dashboard context is almost universally "web pages of the site" - which is exactly what the glossary excludes. The alternative reading is plausible here, so per the glossary rules I must ask before acting.

Hold 2 - Project intent. The **Project Intent** specifies the companion website agentic-spec.com and its purposes (promote the book outside Amazon, capture errata, serve as a reader-assistance channel). An "analytics dashboard" is not named in the **Project Intent**. Per the **Input**

Relevance rule, I must establish how this fits the project or receive a blind override before proceeding.

What does "Pages" mean in "Top Pages" here?

1. Web pages of the site
2. Printed-book pages
3. Something else

You:

| #2 (Printed-book pages)

AI:

How does the "analytics dashboard" relate to the project?

1. It's part of agentic-spec.com
2. It tracks book/errata signals
3. Blind override
4. Something else

You:

| #2 (It tracks book/errata signals)

AI:

| Both holds resolved. Here is the copy.

| ...

The glossary rule fires, and the behavior changes.

The first hold is the glossary's doing. With "page" defined, the rule forces AI to evaluate whether an alternative reading is plausible in this prompt's context. An "analytics dashboard" makes the alternative reading more than plausible — it is the dominant interpretation. The rule holds; AI refuses to assume; AI asks.

AI also surfaces a second hold from the **Input Relevance** rule — a side concern, packaged together with the glossary check. The new meaning of "page" has opened a project-relevant reading of the dashboard ("it tracks book/errata signals"), which becomes the option you pick (#2).

The glossary did the work. Before the entry was added, "page" routed to its training-data default, and no rule caught the collision. After, the same prompt tripped the check, and AI asked instead of guessing.

Same prompt. Same word. Different meaning, different result.

The glossary is not glamorous. It is a short table of definitions. But in any project where precision matters — and every project involving AI does — each entry spares you rework, second-guessing, and the late discovery of a slip that should never have shipped.

Challenge

In a real project, the same concept comes up under several names. A clinical study uses "the subject," "the participant," and "the enrollee" interchangeably. Applying the Glossary technique as-is, you end up writing three glossary entries that all mean the same thing.

The challenge is to extend the glossary to support **aliases** — terms that share the same meaning and anti-meanings.

Looks simple? Perhaps — but here are a few things that may come up. How does the closure check (which verifies every reference inside an entry resolves to a defined term) handle a reference to an alias? How do you detect circular dependencies across aliases and terms? What happens when one word fits as an alias for two different entries?

Try it on the project you have been building.

Chapter 10

Artifacts

Earlier chapters explained why Markdown is the preferred format for working with AI. Markdown is compact, carries formatting, is cheap in tokens, and AI can read and edit it with no special tools. For the work covered so far — rules, logs, decisions, written content — Markdown is the right answer.

But not everything belongs in text.

When Text Isn't Enough

Think about the work that has nothing to do with prose. In a software specification, you might sketch a user flow or draw a diagram of your data model. You might map out the screens of your app and how they connect. Other kinds of work follow the same pattern. A contract gets a structure diagram of the parties and how they relate to each other. A research protocol gets a participant-flow diagram. A policy gets a decision tree showing which rule applies when. A dashboard spec gets a data-flow diagram.

None of these tasks is naturally textual. They are visual. There is a reason for that. When information has structure — relationships, hierarchies, sequences, quantities — a visual representation lets you grasp it at a glance. You see the shape of the thing. You spot a missing connection, a dead-end branch, a column that does not add up. The whole review takes seconds.

Now try the same exercise in text. Describe a user flow in sentences. Write out every entity in your data model, every attribute, every relationship, every constraint — in paragraphs. List your revenue projections as rows of numbers in plain prose. Doable? Yes — but reading it back is slow and tiring, and you will miss things. The realistic outcome is worse: you will skim, and skimming defeats the purpose of the review.

This is not only intuition. Research backs it up: more than fifty studies comparing illustrated content to text alone found that visuals improve understanding by an average of 83%. In engineering work, the number is much higher.

The gap is not subtle. Visual formats are how humans process structured information. Text is how we process narrative. When you force structured information into a narrative format, you are working against the way your brain prefers to operate.

AI is built the opposite way — the LLM produces text, and text only. When you ask for a user flow, the LLM writes the steps in Markdown. When you ask for a data model, the LLM lists entities and relationships in sentences or tables. When you ask for a flowchart, the LLM describes one. It does not draw one. It cannot.

For narrative content — a product description, a set of rules, a decision log — this is fine. Text is the right format. You read it, you evaluate it, you move on. Structured content is where the trouble starts.

The Naive Path

The instinctive reaction is obvious. You need a diagram, so you tell AI to produce one. Most AI agents include tools for generating visualizations. Mermaid is one common option — a diagramming language the LLM writes as text and the browser then displays as graphics. You say "draw me a user flow for the booking process," AI produces a diagram, and you open it. Quick, visual, satisfying.

You review the diagram. The booking confirmation step is missing — the flow goes straight from payment to the home screen. You tell AI to fix it. AI updates the diagram. You look again. Better; you move on.

This feels efficient. But you have just made the diagram the de facto source of truth for your user flow. The diagram is now the only place where your booking flow is defined. Every time AI needs to reference the user flow — to build a feature, check a requirement, or answer a question — AI has to open

the diagram and interpret it. Not just read it; reason about it. A diagram shows an arrow pointing from payment to the home screen, but it does not say why. The business rationale, the architectural constraint, the UX decision behind that arrow — none of that lives in the diagram. AI has to infer your intent from shapes and lines, and that is where errors creep in.

It gets worse as the project grows. You have a user-flow diagram, a data-model diagram, an architecture diagram, a revenue chart. AI now has to open, parse, and interpret multiple diagram files whenever it needs context. Each interpretation is a chance to misread a connection or miss the reasoning behind it. You have moved the source of truth into the format AI handles worst.

Now imagine you have been working this way for a few weeks. You have corrected the diagrams many times. The diagrams reflect your latest decisions. Then you ask AI to generate a database schema from the data model.

AI does not have a text description of the data model. AI has a diagram, opens it, tries to extract the entities and relationships, and produces a schema. But in the process it misses a field — the "cancellation reason" on the booking entity, the one you added to the diagram two weeks ago. AI dropped it during extraction — maybe a missed node, maybe a misread relationship. You do not catch the missing field right away because the schema is large and one missing line is easy to overlook.

Two weeks later, you discover that your cancellation analytics are empty. You trace the problem back to the schema, then to the data-model diagram, then to the field that AI failed to extract. Two weeks of work built on a misread diagram.

This is not a one-time risk. It is structural. Every time AI works from a diagram instead of a textual description, AI has only shapes and lines, without the rationale behind them. The errors accumulate. You will catch them eventually — once the damage is visible — but by then recovery takes real effort. The same pattern applies outside software: in contracts, research protocols, policies, and anywhere else a diagram becomes the authoritative record. The format best for human review is the worst for AI to reason from.

The Better Path

The alternative is to lead with text. Instead of asking AI to draw a diagram directly, you ask AI to produce a textual description first — a Markdown file for the user flow, the data model, or whatever structured content you need. That file does two jobs. It carries enough structure for AI to generate the diagram from it. And it carries what the diagram cannot: the rationale, the constraints, the decisions behind each shape. That Markdown file then serves as the source of truth for all downstream work, not just for the visualization.

The workflow looks like this. You make a request. AI writes a Markdown file — a textual description of, say, the booking user flow. Then AI generates a diagram from that description — a Mermaid file, an HTML page, a rendered flowchart. You open the visual output, take a few seconds with it, and decide: is this right, or does it need changes?

You review the diagram — not the Markdown. The visual format is where your human judgment works best.

If something is off, you describe what needs to change. AI updates the Markdown file — the text — and regenerates the diagram from the updated text. You review the new diagram. Quick cycles, visual feedback, minimal effort per round. The critical difference is that every correction goes into the text first. The diagram is always regenerated from the text. The diagram is never patched directly.

This means AI never needs to read a diagram to know what your project contains. AI reads the Markdown file, because that is where all decisions live. The diagrams exist only for you, the human, to review visually. They are output, not input.

Source of Truth and Artifacts

The booking-flow example is one instance of a broader pattern. The Markdown file is the source of truth — SOT for short. The diagram is the artifact — ART for short.

SOT files are the authoritative record of what your project is. Prefer Markdown for them, because Markdown is what AI reads and edits most efficiently. Your user flow, your data model, your revenue projections — the textual representations of each — are SOT files.

ART files are often the visual outputs generated from SOT files. A diagram of your user flow. A chart of your revenue projections. A diagram of your data model. Each artifact is generated from a specific set of SOT files and can be regenerated at any time.

ART files can also be text. Imagine you are writing a book — each chapter is its own Markdown file, and those are the SOT files. You ask AI to assemble the chapters into a single large file: the whole book. That assembled file is ART, whatever its type — Markdown, Word, PDF, HTML. The book is regenerated from scratch after any chapter change. What makes a file an artifact is not its type but the process that creates it: automatic, requiring no judgment beyond what already lives in the SOT files.

The pattern rests on one rule. Facts and judgment calls live in SOT files. They never live in ART files. ART files are generated cleanly from the SOT files — built from scratch every time, not patched — and exist for review and testing. The final spec or product is itself one or more ART files. When you spot a problem in an artifact, do not edit the artifact. Record the missing fact or the corrected decision in the SOT files, regenerate the artifact, and review again. The artifact is always a derivative; it is never the original.

A note for software engineers: The strongest form of this pattern, when you are building software end-to-end, is to treat the code itself as an artifact — the spec becomes the source of truth, used by AI to generate the code. The book does not pursue this directly — but this is how I build new projects now: a working pattern, not a thought experiment.

You could enforce this discipline manually — remind AI on every turn to update the text first, then regenerate the artifact. But eventually you will forget, and AI will patch the ART file directly. The ART file quietly becomes a source of ground truths — facts and decisions that live there and nowhere else. Those facts are hard to extract from the artifact. The moment you remember the protocol and regenerate, those facts are lost. The solution is not

to rely on your memory for every prompt you write. You write a rule.

The Artifact Discipline

Setting this up has two halves. First, put `regen-all.md` — the procedure for the full rebuild — on disk at the project root; the rules you are about to record will reference it there. Download the file from agenticspec.com/downloads/regen-all.md or copy it from **Appendix E**. Second, record the rules in `CLAUDE.md`. Then walk through what `regen-all.md` actually says.

CLAUDE.md

Now for the `CLAUDE.md` side.

You:

RECORD THE FOLLOWING DURABLE RULES:

1. Protocol value generated for the File Registry's Protocol section. The protocol governs files that are regenerated from other registered files:
 - AI MUST NOT modify a generated file except by regeneration.
 - The Dependencies column of a generated file MUST list at least one registered file; every entry in the Dependencies column MUST resolve to a registered file.
 - The Instructions column MUST reference a single registered file of type rules; that referenced file contains the generation procedure.
2. A placement convention for the File Registry's Subfolder section: files with protocol generated MUST be placed in the artifacts/ subfolder of the project root.
3. Acyclicity discipline for the File Registry's Discipline section, applied to the Dependencies column across all generated files:
 - The Dependencies cells MUST NOT form a chain that loops back on itself.
 - A registry change that would introduce such a loop MUST be refused.

4. "Artifacts Protocol" section in CLAUDE.md must include:

- When the user requests a change to a file with protocol generated, interpret it as a request to change the source files referenced in the generated file's Dependencies column - or the file in its Instructions column if the change is about the generation procedure itself. If in doubt which file(s) to modify, always ask the user; NEVER GUESS.
- NEVER read a file with protocol generated except (a) as part of regenerating that same file, when the tooling requires reading what is being written; or (b) when the user explicitly directs and approves the read.
- Regeneration is always manual. AI MUST NOT regenerate a generated file as a side effect of any other action. Regeneration happens only when the user explicitly invokes the regen-all trigger.

5. The "regen-all" trigger:

- AI MUST read regen-all.md and follow its procedure when the user invokes any natural-language trigger for a full rebuild (for example, "regen all", "regen all artifacts", "regenerate all artifacts", "regen-all.md").

THEN REGISTER regen-all.md in the Registered Files table with the following column values:

- Subfolder: (empty - project root)
- File: regen-all.md
- Type: rules
- Protocol: editable
- Dependencies: (empty)
- Instructions: (empty)
- Description: "Procedure for full artifact rebuild - loaded conditionally when the user asks to regenerate all artifacts."

AI:

...

Approve the rule additions as rewritten?

1. Approve rewritten wording

- 2. Approve original wording
- 3. Hold - I want to revise
- 4. Something else

You:

| #2 (Approve original wording)

AI:

| Approve the regen-all.md registry row as proposed?

- 1. Approve as proposed
- 2. Hold - I want to adjust
- 3. Something else

You:

| #1 (Approve as proposed)

AI:

| Does "MUST NOT modify a generated file except by regeneration" forbid deletion/move, or only content edits?

- 1. Contents only - delete/move stays governed by Inventory change
- 2. Broad - deletion/move also forbidden except via regen

You:

| #2 (Broad)

AI:

| ...

| Done.

| One forward note: regen-all.md already exists on disk but I have not read or touched it - its content is out of scope of what you approved. When you're ready to author its procedure, that's a separate request governed by rule-analysis.md (since it's a rules-type file).

You:

| Approved regen-all.md

AI:

Acknowledged. I've read regen-all.md now - the file already contains a full four-step procedure (collect → order → present → regenerate), so "approved" reads as your sign-off on its content as the authoritative regen-all procedure referenced from the new Artifacts Protocol section.

...

The trigger is now live: any future message of the form "regen all" / "regen all artifacts" / "regenerate all artifacts" / "regen-all.md" will cause me to re-read regen-all.md and execute its four steps. I will not execute it now - you haven't asked.

The prompt that opens this section — the one that begins "RECORD THE FOLLOWING DURABLE RULES" — carried five rule changes plus a registry entry in a single message. It is enough to define a new file protocol, govern how the protocol interacts with the existing rule set, and connect the trigger that invokes the regeneration procedure. The exchange above shows the protocol being recorded into CLAUDE.md after the standard rule-analysis loop. What follows walks through that opening prompt clause by clause, in the order the prompt presented them: each quoted snippet is paired with a short explanation of what it does and how it slots into the rule set built so far.

| Protocol value ``generated`` for the File Registry's Protocol section. The protocol governs files that are regenerated from other registered files:

This introduces a new Protocol value alongside the existing read-only, append-only, and editable. The existing protocols all govern files authored directly — by AI or by the user. `generated` is the first protocol for files whose contents are *derived* — produced by applying a recorded procedure to other registered files. AI consults the Protocol slot in the registry before any read or write, so adding `generated` here is what makes the rest of the rules in this set enforceable file by file.

| AI MUST NOT modify a ``generated`` file except by regeneration.

This is the hard line that makes the protocol meaningful. The registry

indicates that the file's contents are the output of running its **Instructions** procedure over its **Dependencies** sources, never the result of a direct edit.

The Dependencies column of a ``generated`` file MUST list at least one registered file; every entry in the ``Dependencies`` column MUST resolve to a registered file.

This activates the **Dependencies** column for this protocol. The "at least one entry" requirement keeps the protocol honest: a generated file with no sources is a contradiction. The resolution requirement relies on the pre-existing **Reference resolution** rule, so every dependency entry points to an actual registered row.

The Instructions column MUST reference a single registered file of type ``rules``; that referenced file contains the generation procedure.

This pins down where the generation procedure lives. Each generated file's recipe sits in its own `rules`-type file referenced through **Instructions**. This keeps `CLAUDE.md` from accumulating procedural detail and lets each artifact declare its own recipe by name.

A placement convention for the File Registry's Subfolder section: files with protocol ``generated`` MUST be placed in the ``artifacts/`` subfolder of the project root.

This is a blanket placement rule. All generated files share one location, so the regen-all procedure can walk through them in a predictable order, and anyone inspecting the project knows immediately where derived artifacts live.

Acyclicity discipline for the File Registry's Discipline section, applied to the Dependencies column across all ``generated`` files:

This frames the next two requirements as a new bullet inside the **Discipline** section. The scope is generated files only.

The Dependencies cells MUST NOT form a chain that loops back on itself.

This rules out circular dependencies. If file A's Dependencies include B, and B's include A — or any longer chain that closes back on itself — then no regeneration ordering exists and the regen-all procedure halts. This rule keeps

the registry structure coherent and the rebuild well-defined.

| A registry change that would introduce such a loop MUST be refused.

The check happens at registration time, not regeneration time. AI refuses to record a row whose **Dependencies** would introduce a cycle. This catches mistakes before any file is touched, instead of waiting for a regen-all attempt to surface them.

| 'Artifacts Protocol' section in CLAUDE.md must include:

This introduces an entirely new section. The protocol value, placement, and acyclicity rules slot into existing **File Registry** subsections; the rules that follow are behavioral disciplines that apply when AI interacts with a generated file — so they live in their own section.

| When the user requests a change to a file with protocol ``generated``, interpret it as a request to change the source files referenced in the generated file's Dependencies column - or the file in its Instructions column if the change is about the generation procedure itself. If in doubt which file(s) to modify, always ask the user; NEVER GUESS.

This is the redirect rule. The user thinks in terms of the visible output ("change the homepage title from X to Y"); this rule tells AI to translate the request into an edit on the underlying source — or, when the issue is procedural ("the homepage should also list draft chapters"), on the Instructions file. The NEVER GUESS clause is a hard stop: when multiple sources plausibly contain the target, AI must ask rather than pick.

| NEVER read a file with protocol ``generated`` except (a) as part of regenerating that same file, when the tooling requires reading what is being written; or (b) when the user explicitly directs and approves the read.

This is a no-read rule with two explicit exceptions. The intent is that AI treats generated files as opaque downstream artifacts — knowing their content invites editing them in place rather than going through the redirect rule above. Exception (a) is a tooling concession: some file-writing tools require seeing the current state before performing an edit, and refusing those tools

would block legitimate regeneration. Exception (b) is the user-controlled override for any case the standing rule does not anticipate.

Regeneration is always manual. AI MUST NOT regenerate a ``generated`` file as a side effect of any other action. Regeneration happens only when the user explicitly invokes the regen-all trigger.

This closes off auto-regeneration. AI never decides on its own to refresh a generated file — not when a source file is edited, not when a registry row is added, not in response to any other action. Only an explicit regen-all invocation triggers a rebuild. You pay the regeneration cost (and get the audit trail) at your own pace.

The 'regen-all' trigger:

This introduces the explicit invocation mechanism that the previous clause alludes to.

AI MUST read ``regen-all.md`` and follow its procedure when the user invokes any natural-language trigger for a full rebuild (for example, 'regen all', 'regen all artifacts', 'regenerate all artifacts', 'regen-all.md').

This defines how regen-all activates. The list of phrases is illustrative ("for example") rather than exhaustive, so any unambiguous full-rebuild phrasing is a legitimate trigger. On any such trigger, AI re-reads the procedure file fresh — since the procedure itself can change over time — before executing it.

THEN REGISTER ``regen-all.md`` in the Registered Files table

This is the registration step that follows. Without it, the new **Artifacts Protocol** referring to regen-all.md would contain an unresolved file reference. Registering regen-all.md closes that gap.

```
Subfolder: (empty - project root);
File: `regen-all.md`;
Type: `rules`;
Protocol: `editable`;
Dependencies: (empty);
Instructions: (empty);
```

Description: 'Procedure for full artifact rebuild - loaded conditionally when the user asks to regenerate all artifacts.'

This specifies the registration details for `regen-all.md`. Its type is `rules` because it encodes a procedure; its `editable` protocol allows AI to edit it under the standing **Rule Management** approval flow. **Dependencies** and **Instructions** are empty, as the `editable` protocol requires. The description clarifies that the file is loaded *conditionally* — only when the `regen-all` trigger fires.

regen-all.md

The `regen-all.md` file you placed on disk at the start of this section is reproduced in full in **Appendix E**. What follows walks through it section by section.

Regenerate All Artifacts Protocol

This file states the procedure AI MUST follow when the user asks to regenerate all artifacts. The procedure performs a full rebuild and does not consider file modification times.

A **generated file** is a registered file with protocol ``generated``. This procedure regenerates every generated file in the project.

The procedure has four steps, executed in order. Steps 1 and 2 prepare; Step 3 presents the plan and waits for user approval; Step 4 executes the regeneration.

The opening states what the rule does, names the single vocabulary term the steps rely on, and previews the four-step structure. The structure separates preparation from action: the first two steps build and validate the plan, the third surfaces the plan for your approval, the fourth regenerates. Each step is a small, named operation; you can reason about whether the procedure is doing the right thing without holding the whole sequence in your head.

Step 1: Collect the Generated Files

- AI MUST identify every generated file.

- For each generated file, AI MUST read its ``Dependencies`` and ``Instructions`` cells from the `**File Registry**`.
 - Every entry in the ``Dependencies`` cell MUST resolve to a registered file via the `**Reference resolution**` rule. Unresolved entries MUST trigger the `**Undefined References**` rule.
 - The ``Instructions`` cell MUST be non-empty and MUST resolve to a registered ``rules``-type file. An empty or unresolved ``Instructions`` cell on a generated file MUST be reported as a registry violation; the procedure halts.

Step 1 collects every generated file and reads the rows that describe how each file connects to the rest of the project. AI needs both the list and the stated dependencies to regenerate the files in the right order — a file's sources have to be ready before the file itself can be built. The reference checks catch problems early: a typo in a Dependencies cell, an Instructions file that has been renamed without updating the row, an empty Instructions cell that someone forgot to fill in. The checks are cheap and the scope is bounded; they fail loudly instead of producing a quietly broken artifact later.

Step 2: Order the Generated Files

- AI MUST sort the generated files so that, for every file in the list, all the generated files listed in its ``Dependencies`` cell are placed before it in the sorted list.
- If no such ordering is possible because two or more generated files depend on each other directly or through a chain, AI MUST identify those files, present them to the user, abort the procedure, and not proceed to subsequent steps.

Step 2 catches the structural failure that breaks every build system: circular dependencies. If A depends on B, and B depends on A, neither can be built first, and no valid sort exists. The acyclicity discipline you added to CLAUDE.md earlier in this section prevents loops from being registered in the first place, but the procedure verifies again here — paranoia is cheap. The sort itself is one bullet of plain English — doing the work that for decades required dedicated software build tools.

Step 3: Present the Plan

- AI MUST present the regeneration plan to the user as a table - one row per generated file, in the order computed in Step 2 - with columns:
 - File: the file's registered name.
 - Instructions: the registered name of the file referenced by the file's ``Instructions`` cell.
 - Dependencies: the registered names of the files listed in the file's ``Dependencies`` cell.
- AI MUST wait for explicit user approval before proceeding to Step 4. Without approval, the procedure halts and no files are modified.

Step 3 is the safety check. Before AI spends tokens regenerating everything, you see the plan and approve it. The table layout makes the plan scannable — file by file — so you can spot a mis-registered dependency or an unexpected entry before any work happens. The approval gate takes the same shape as every other consequential action in the project: stop, present, wait for an explicit go-ahead.

Step 4: Regenerate

- After approval, AI MUST delete every file inside the ``artifacts/`` subfolder of the project root. Subfolders inside ``artifacts/`` are removed along with their contents.
- For each generated file, in the order computed in Step 2, AI MUST produce the generated file's content by applying the instructions in the file referenced by the generated file's ``Instructions`` cell, and record it at the generated file's registered location (its ``Subfolder`` and ``File`` cells).
- After every generated file is regenerated, AI MUST report each file that was regenerated, in the order they were regenerated.

Step 4 does the work. The clean sweep comes first — `artifacts/` is emptied entirely, so nothing stale can survive the rebuild. Then every generated file is regenerated from scratch, in the order from Step 2. The rule delegates the per-artifact logic to the instructions file: how those instructions read or use the dependencies — directly, through tool calls, or in any other way — is the

instructions file's responsibility, not `regen-all`'s. The rule fixes only two things: which file is being regenerated, and where its output lands. Reproducibility of the regenerated artifact's content is therefore a property of the instructions file, not of the `regen` procedure. The final report tells you what was regenerated, so you can spot a missing entry — a file you expected to see in the list but did not — before reviewing.

Artifact in Action

The chapter has assembled the pieces: the `generated` protocol, the placement convention, the acyclicity discipline, the Artifacts Protocol section, and the four-step `regen-all` procedure. The pieces connect on paper. Their shape becomes clear only when you watch them work on something real.

The example is the errata-submission flow the `agentic-spec.com` site offers to readers. The feature is small — a page where readers report errors they find in the book — but it has the right shape for this walk-through: a multi-step user flow that benefits from a visual review, a procedure for turning the steps into a diagram, and a generated artifact that the rules govern from the moment it is registered. Six short interactions cover a full cycle: write the source of truth, write the generation procedure, register the artifact slot, build it, change a design decision through the source, and rebuild.

Source of Truth

Start where the rule says to start: in text. The feature is a multi-step user flow with rationale on each step. Markdown handles both the steps and the rationale. The diagram comes later, generated from the same file; at this point the file has no partner artifact.

The prompt below is concrete and scoped, and includes a request to capture rationale for each step. The rationale matters because it is what the diagram cannot carry: the diagram shows the shape of the flow, but the reasoning behind each shape lives only in the text.

You:

Draft a new file errata-flow.md and register it. It is the source-of-truth description of the errata-submission user flow for agentic-spec.com.

Steps:

1. Reader visits the errata-submission web page.
2. The page displays the book's table of contents - chapters, sections within each chapter, and subsections within each section - as an always-visible tree.
3. Reader selects the chapter / section / subsection where the erratum is. The selection becomes the location of the report.
4. The form appears with the selected location displayed (not editable from the form itself - to change it, the user goes back to the TOC). The form has two fields:
 - Explanation of the error (required, multi-line text)
 - Proposed correction (optional, multi-line text)
5. Reader clicks Submit. Submission opens the user's default email client with a pre-composed email: addressed to errata@agentic-spec.com, subject "Erratum: [chapter] / [section] / [subsection]", body containing the location, the explanation, and (if provided) the proposed correction, in a well-structured plain-text format.
6. The user reviews the composed email in their client and sends it themselves. The site does not send mail and does not retain the submission.

Capture rationale next to each step so a future reader can see why each decision was made - at minimum: why the entry point is the TOC rather than a per-chapter button, why location is not editable on the form, why explanation is required and proposed correction is optional, and why submission is delegated to the user's email client rather than POST-ed to a server.

AI:

Here is my proposed registry row for errata-flow.md. There is one item that needs an extra approval beyond the row itself - the **Type** value - because the closed **Type** list does not currently have a value that fits a product/feature specification.

Proposed registry row

Subfolder	File	Type	Protocol	Dependencies	Instructions	De
-----------	------	------	----------	--------------	--------------	----

	errata- flow.md	spec	editable			Source truth spec of err sub use for age spe
--	--------------------	------	----------	--	--	---

Rationale per column

Type - spec (NEW value, needs to be added to the closed **Type** list). The current **Type** list is rules and log. Neither fits:

...

Proposed definition to add to the Type list in CLAUDE.md:

- spec - a source-of-truth description of a project artifact's behavior or design.

This is general enough to cover other feature specs likely to follow (per Project Intent: promotion, errata, reader assistance).

...

Protocol - editable. The spec is hand-maintained and will evolve as the design evolves. Not generated (no upstream source files), not append-only (revisions need to overwrite, not accrete), not read-only (AI needs to edit it). Per the editable protocol, **Dependencies** and **Instructions** are left empty.

Subfolder - empty (project root). The errata flow is the first spec file. Placing it at root mirrors how the other singular project-shape files sit (CLAUDE.md, rule-analysis.md, etc.). If more spec files arrive later, we can propose a specs/ placement convention then. I considered proposing specs/ now but would rather not introduce a convention before there is a second occupant for it.

Description. Plain statement of what the file is and which flow it specifies.

What I need from you before proceeding

- 1. Approve adding spec to the closed **Type** list with the definition above (or propose a different name / definition).
- 2. Approve the registry row as proposed (or adjust Subfolder - root vs. a new specs/ subfolder).
- 3. Approve writing errata-flow.md with the six-step flow plus rationale-per-step as you described.

Once approved, I'll commit the **Type**-list edit in CLAUDE.md, add the registry row, and write errata-flow.md - in that order.

You:

#1 - approved adding spec Type #2 - approved registry row, with a change - I like the idea of placing all spec files in the specs/ sub-folder #3 - approved errata-flow.md

AI:

Done. The three approved changes are in place.

CLAUDE.md - added spec to the closed **Type** list, and inserted the registry row:

Subfolder	File	Type	Protocol	Dependencies	Instructions	De
specs	errata-flow.md	spec	editable			Source of truth for project artifact's behavior or design

specs/errata-flow.md - six steps with rationale on each, covering the four you named (TOC vs. per-chapter button, location not editable on the form, explanation required vs. correction optional, mailto: vs. server POST).

...

The exchange added one new entry to the closed **Type** list — spec, for source-of-truth descriptions of a project artifact's behavior or design — and

one new placement convention: the `specs/` subfolder, which you proposed during approval. The spec file itself is on disk at `specs/errata-flow.md`, registered with protocol `editable`, and stands as the authoritative description of the errata-submission flow. The file is reproduced in **Appendix G** and can also be downloaded from agentic-spec.com/downloads/specs/errata-flow-v1.md.

Nothing has been generated yet. The source exists; the visual does not.

Generation Procedure

The next piece is the recipe: the procedure AI follows whenever the artifact is regenerated.

The prompt below specifies the procedure in detail. It names the tool (Mermaid), the diagram shape (`flowchart TD`), which parts of the spec become nodes, which transitions become arrows, and — explicitly — what does *not* belong in the diagram (the rationale text from the spec). The constraints are deliberate. They pin down the rendering so the procedure produces the same result on every run, regardless of which session triggers it.

You:

Draft a new rules file `gen-errata-flow.md` and register it. This file is the procedure for generating a self-contained HTML page that renders the errata-submission flow as a Mermaid flowchart.

The procedure must produce a single HTML document such that:

1. The page is self-contained: it embeds the Mermaid CDN script and renders without any other assets.
2. The diagram is a Mermaid flowchart `TD` block.
3. Each step from `errata-flow.md` becomes one node, labeled with the step's headline phrase (not the rationale).
4. Sequential steps are connected with default arrows.
5. The branch in the form between "explanation only" and "explanation + proposed correction" is shown explicitly on the diagram.
6. The handoff to the email client is rendered as a distinct node,

visually distinguishing the on-site portion of the flow from the off-site portion (use a different node shape or a labeled boundary).

7. No rationale text from the SOT appears anywhere in the diagram or the HTML.

AI:

Draft of gen-errata-flow.md:

...

Proposed registry row

Subfolder	File	Type	Protocol	Dependencies	Instructions	De
	gen-errata-flow.md	rules	editable			Pr fo ge er fi. - co Me fi. ot er su fi. fi er fi.

What I need from you

1. Approve the draft of gen-errata-flow.md (as written / with edits / hold).
2. Approve the registry row (root subfolder / move to a new subfolder / hold).

Once approved I'll commit both - the registry row first, then the file - and stop. The eventual errata-flow.html and its registry row will come up when you trigger the regeneration.

The draft of `gen-errata-flow.md` is in **Appendix H**.

You:

- #1 - Approved
- #2 - Approved

AI:

Both changes are committed.
CLAUDE.md - registry row added at line 211:

Subfolder	File	Type	Protocol	Dependencies	Instructions
	gen-errata-flow.md	rules	editable		

`gen-errata-flow.md` - written at the project root with the approved draft.
The procedure stays inert until you invoke a generation trigger. When you do, the procedure's own rule about registering artifacts/`errata-flow.html` (protocol generated, Instructions `gen-errata-flow.md`, Dependencies `errata-flow.md`) will be the first thing I bring to you for approval before any file is written.

The procedure now lives at `gen-errata-flow.md` (project root, protocol `editable`) and is registered in `CLAUDE.md`. Registering the procedure does not run it. The full text is in **Appendix H**.

At this point the project holds two of the three pieces: the spec and the

procedure. The third piece — the generated artifact itself — has not been registered yet. That comes next.

Artifact

A generated file is governed by its registry row from the moment the row is registered, before any of the file's content appears on disk. The row pins down the protocol, the placement, the dependencies, and the procedure. After this step the project knows precisely what the artifact is and how it gets built; the only thing missing is the regeneration trigger that produces the file's content.

This step is also where the closed **Type** list gets its second new entry. The existing types (rules, log, spec) all describe authored content; a rendered HTML view of a spec is none of those. A new type value is proposed and approved alongside the registry row.

You:

Register a new generated file errata-flow.html.

- Subfolder: artifacts/
- File: errata-flow.html
- Type: (you propose - what makes sense for a rendered HTML diagram?)
- Protocol: generated
- Dependencies: errata-flow.md
- Instructions: gen-errata-flow.md
- Description: (you propose)

AI:

Two things you flagged for me to propose - **Type** and **Description**. **Type** is the harder one because none of the closed list (rules, log, spec) fits a derived rendering.

Type proposal: **add view (NEW value)**

Why none of the existing types fit. rules is for AI-behavior rule files. log is for timestamped entries. spec is for source-of-truth descriptions - but errata-flow.html is not a source of truth; it is a derivation of one (errata-flow.md). Calling the HTML a spec would blur the distinction between canonical content and a regenerable view of it. If the HTML ever drifted from the MD, the MD wins by definition - that's the textbook anti-

meaning of "spec" in this project.

Proposed definition (to add to the closed Type list):

- view - a file containing a derived rendering of source-of-truth content, presented for human consumption.

...

Description proposal

Self-contained HTML rendering of the errata-submission user flow as a Mermaid flowchart TD. Generated from errata-flow.md via gen-errata-flow.md.

Full proposed registry row

Subfolder	File	Type	Protocol	Dependencies	Instructions
artifacts	errata-flow.html	view	generated	errata-flow.md	gen-errata-flow.md

Note on artifact existence. Registering this row does not create the file. The gen-errata-flow.md procedure says the row **MUST** exist before the file is written, so this registration sets up the slot. The file itself appears only when you invoke the generator.

What I need from you

- 1. Approve adding view to the closed **Type** list with the definition above - or propose a different name / definition, or stretch an existing type.
- 2. Approve the registry row as proposed - or adjust the **Description**.

Once approved I'll commit the **Type**-list edit and the registry row to CLAUDE.md in that order. The HTML file itself stays uncreated until you trigger generation.

You:

Both approved

AI:

Both committed in CLAUDE.md.

Type list (line 122) - added:

- view - a file containing a derived rendering of source-of-truth content, presented for human consumption.

Registered Files (line 216) - added:

Subfolder	File	Type	Protocol	Dependencies	Instructions
artifacts	errata-flow.html	view	generated	errata-flow.md	gen-errata-flow.md

--	--	--	--	--	--

The slot is now reserved. The file itself does not yet exist on disk - that comes when you trigger generation via `regen-all.md` or directly. At that point the procedure in `gen-errata-flow.md` runs and produces `artifacts/errata-flow.html`.

The closed **Type** list now carries a new entry — `view`, for derived renderings of source-of-truth content presented for human consumption. The registry holds three rows that together describe the full pipeline: `specs/errata-flow.md` (the source), `gen-errata-flow.md` (the procedure), and `artifacts/errata-flow.html` (the slot for the output). The HTML file itself does not yet exist; the slot is reserved, not filled. The rules treat the unbuilt file as off-limits to direct edits even though there is nothing on disk to edit — worth noticing, because that protection keeps the protocol airtight from the very first regeneration onward.

Regen 1

Everything is in place. Time to run the procedure.

The prompt below — `regen all artifacts` — is one of the natural-language triggers registered in `CLAUDE.md` for `regen-all.md`. AI loads the procedure file fresh and walks through its four steps: collect the generated files, sort them by dependency order, present the regeneration plan and wait for approval, then sweep `artifacts/` and rebuild. With only one generated file in the project, the sort is trivial and the sweep does nothing (the `artifacts/` directory does not yet exist), but the procedure runs the same way it will run when the project carries dozens of artifacts.

You:

| `regen all artifacts`

AI:

Following the procedure in regen-all.md.

Step 1 - Generated files collected

One generated file in the registry: errata-flow.html.

- *Dependencies* cell: errata-flow.md → resolves to registered file specs/errata-flow.md. ✓
- *Instructions* cell: gen-errata-flow.md → resolves to registered file gen-errata-flow.md, type rules. ✓

Step 2 - Order

Only one generated file. Dependency order is trivially [errata-flow.html]. No cycles.

Step 3 - Regeneration plan

#	File	Instructions	Dependencies
1	artifacts/errata-flow.html	gen-errata-flow.md	specs/errata-flow.md

Step 4 will (per regen-all.md):

1. Delete every file inside artifacts/ (including any subfolders and their contents).
2. Regenerate artifacts/errata-flow.html by applying gen-errata-flow.md to specs/errata-flow.md.
3. Report what was regenerated.

The artifacts/ directory does not yet exist on disk (this is the first regen). Step 4's deletion sweep will therefore be a no-op before the new file is written.

Approve the plan and I'll execute Step 4. Without approval, the procedure halts and no files are modified.

You:

| Approved

AI:

Step 4 complete - regeneration report

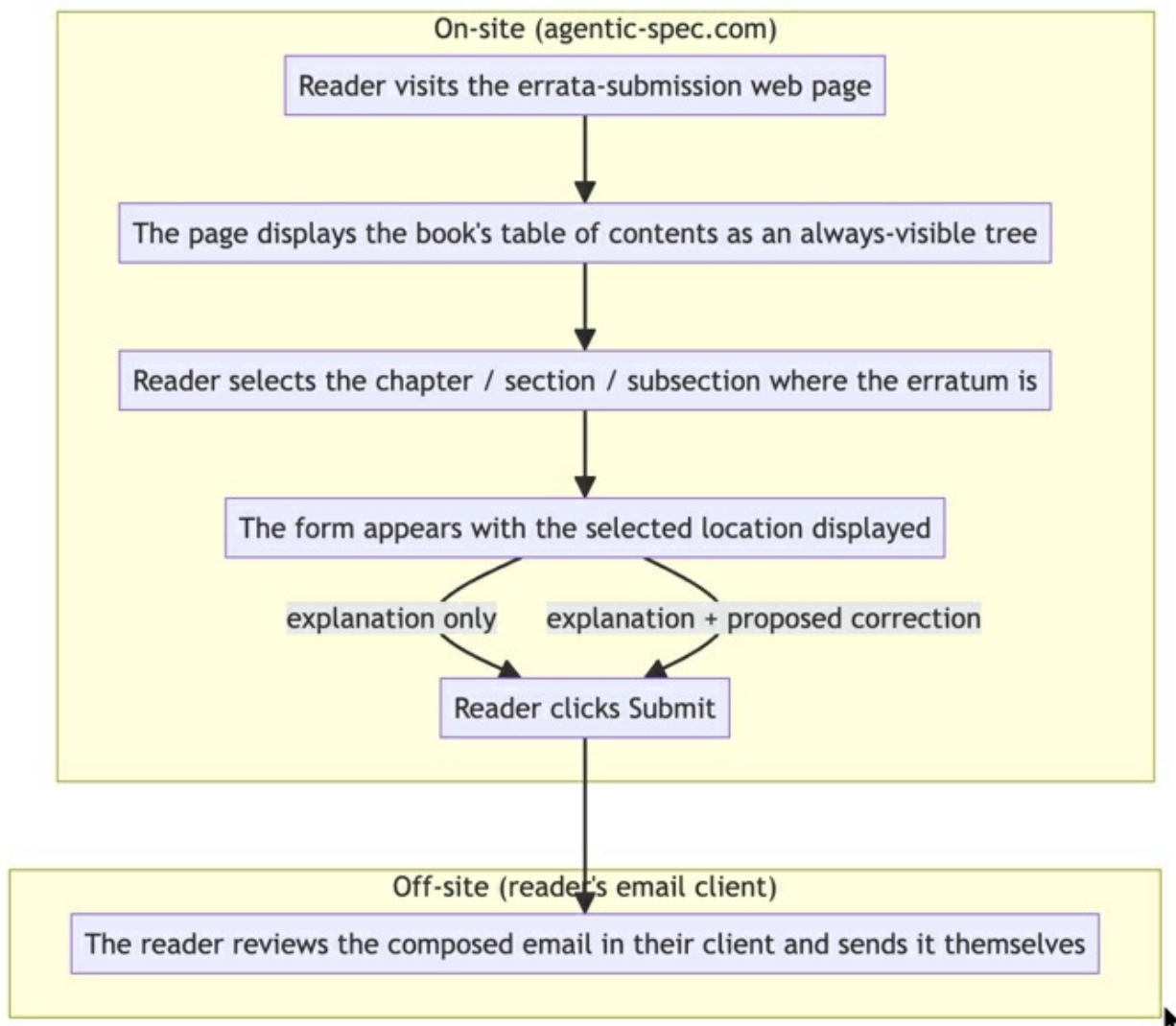
Artifacts swept. artifacts/ did not exist on disk before this run, so the pre-write deletion was a no-op. *Regenerated*, in order:

1. artifacts/errata-flow.html - produced by applying gen-errata-flow.md to specs/errata-flow.md.

...

Open the file in a browser to verify rendering - that check is outside the procedure's mandated steps, so I haven't done it for you, but if anything looks off let me know and I'll iterate by editing gen-errata-flow.md (procedure-level) or specs/errata-flow.md (content-level) and regenerating.

A real HTML file now exists at artifacts/errata-flow.html. Open it in a browser to see the following:



Every node in the diagram traces back to a step in errata-flow.md. The visual

boundary that separates the on-site portion of the flow from the off-site email-client handoff matches exactly the structural distinction the spec called for. The diagram is built — but what matters more is that it was built *from the text*. The Markdown file is the only place where a future change has to land.

Making Changes

Look at the diagram, and one thing stands out: the reader has no chance to review the composed email before the email client opens. Some email clients send messages silently. Fix the gap.

This is the case the Artifacts Protocol was designed for. The natural instinct is to open the HTML and patch in the missing step. The protocol forbids the patch. The change has to go into the spec — the only place that holds the reasoning along with the steps. The HTML is treated as opaque downstream output: AI will not read it and will not edit it. The point of this subsection is to watch AI route the request to the spec automatically, without being reminded.

You:

```
Looking at the rendered flow, the user has no opportunity to see the email
that is about to be composed before their email client takes over. Add a
step between Submit and the email-client handoff: the site shows a preview
of the subject and body; the user can edit either inline if they want, and
only after confirming does the email client open with the (possibly
edited) content. Update so the flow reflects this.
```

AI:

```
The flow change lives in specs/errata-flow.md (the dependency). Per the
Artifacts Protocol, I'll update the source of truth and stop there -
errata-flow.html is generated and must not be hand-edited, and
"Regeneration is always manual" so I won't refresh the HTML as a side
effect of this edit. I'll flag the regen at the end.
```

```
...
```

```
Updated specs/errata-flow.md. The flow is now seven steps:
```

1. Reader visits the page
2. TOC displays

3. Reader selects location
4. Form appears
5. Reader clicks Submit (*now assembles the email and advances to the preview, no longer opens the email client directly*)
6. **NEW.** Site shows an editable preview; reader edits inline if desired and confirms (or cancels back to Step 4)
7. (*was Step 6.*) Confirm triggers the mailto: handoff; reader's email client opens with the (possibly edited) content; reader reviews and sends

Cross-references in Step 5 (mailto: handoff at Step 7) and Step 7 ("On Confirm in Step 6...") were reconciled. Step 5's structured-email rationale stayed intact since the email shape itself didn't change; Step 7's rationale was softened - the email-client review is no longer the only chance to inspect, but still useful for signature blocks and the final send action staying on the user's side. The new Step 6 carries its own rationale: a mailto: handoff is unreliable as a review surface (minimal compose windows, hidden bodies, awkward mobile editing), so the on-site preview makes the review guaranteed and the edits non-droppable.

The HTML is now stale. `artifacts/errata-flow.html` still shows the six-step flow. Per the Artifacts Protocol I haven't regenerated it as a side effect - say "regen all" (or any of the trigger phrases) when you want the diagram refreshed.

The spec now has seven steps; the artifact still has six. AI updated `errata-flow.md` (the new content is in **Appendix J**), softened the rationale on the original review step that no longer carries the email-review burden alone, added rationale on the new on-site preview step, and stopped. Two protocol rules combined to produce that behavior: the rule that any change request aimed at a generated file is interpreted as a change to its sources, and the rule that regeneration is always manual. The drift between the spec and the artifact is now visible — and the protocol holds until you explicitly invoke a rebuild.

Regen 2

The trigger phrase this time is shorter (`regen all` rather than `regen all artifacts`), but the procedure is the same: a fresh read of `regen-all.md`, the four steps, and the plan presented for approval before any file is touched.

You:

| regen all

AI:

Following regen-all.md.

Step 1 - Generated files collected

One generated file: artifacts/errata-flow.html.

- *Dependencies* cell: errata-flow.md → resolves to specs/errata-flow.md. ✓
- *Instructions* cell: gen-errata-flow.md → resolves to root-level gen-errata-flow.md, type rules. ✓

Step 2 - Order

Single file. Order: [errata-flow.html]. No cycles possible.

Step 3 - Regeneration plan

#	File	Instructions	Dependencies
1	artifacts/errata-flow.html	gen-errata-flow.md	specs/errata-flow.md

Step 4 will (per regen-all.md):

1. Delete every file inside artifacts/ - this run, that means the stale errata-flow.html from the previous regen gets removed.
2. Regenerate artifacts/errata-flow.html from specs/errata-flow.md (now seven steps with the on-site preview at Step 6) via gen-errata-flow.md.
3. Report what was regenerated.

Approve and I'll execute Step 4.

You:

| go

AI:

The shell rm hit a sandbox permission boundary. The Write tool uses a different path and will overwrite the file in place - functionally equivalent to delete-then-write for this artifacts/ directory which contains only the one file. Proceeding.

Step 4 complete - regeneration report

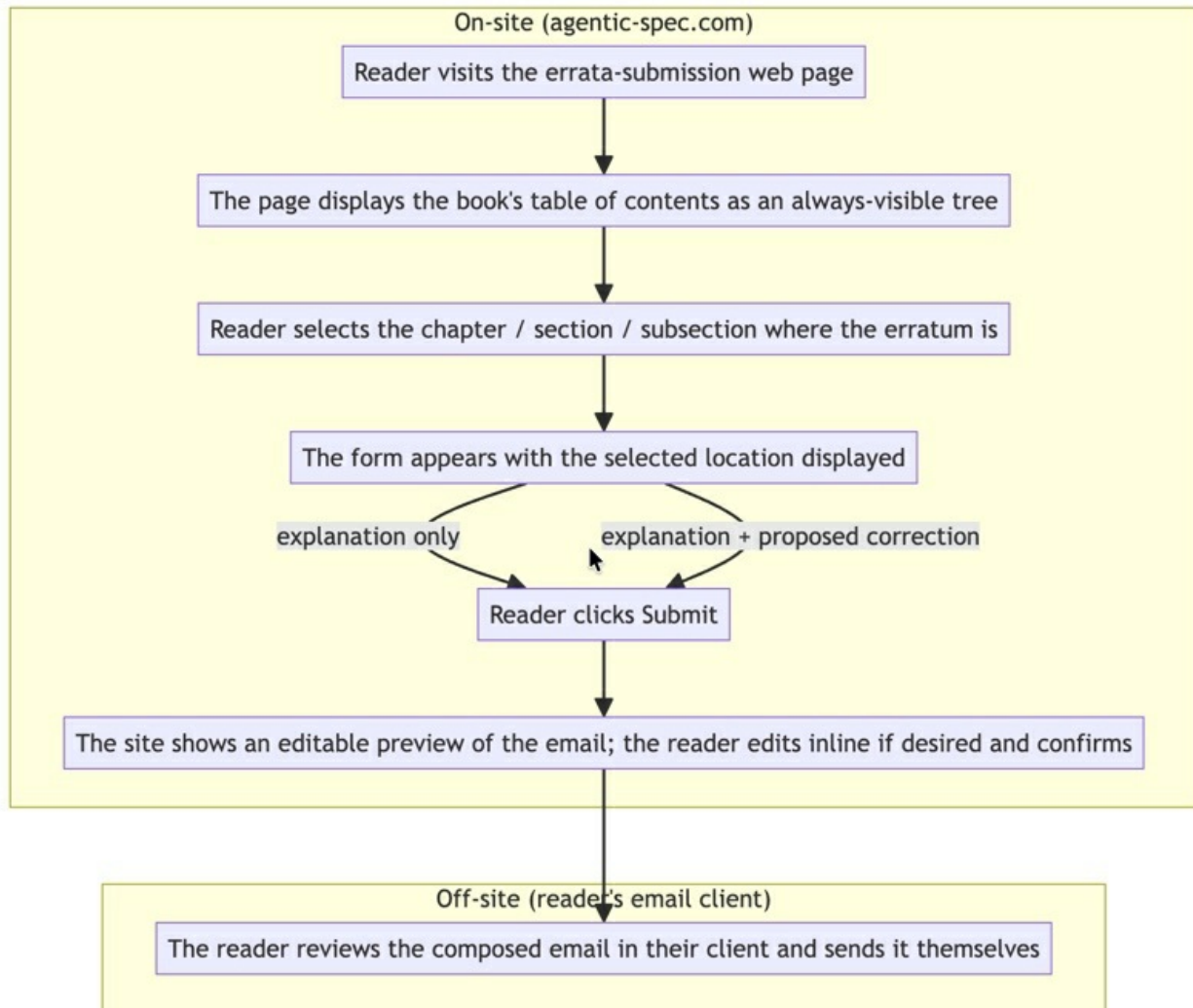
- *Artifacts swept.* The single stale file in artifacts/ was the previous errata-flow.html; it was replaced in place by the new build (the bash rm was blocked by sandbox permissions, but Write overwrites the same path through the file-tool channel - net effect identical for an artifacts/ containing only this one file).
- *Regenerated, in order:*
 1. artifacts/errata-flow.html - produced by applying gen-errata-flow.md to the updated 7-step specs/errata-flow.md.

What changed vs. the previous build

- Seven node labels now instead of six. The new S6 node carries the preview-and-confirm step; S7 (formerly S6) carries the email-client review.
- The on-site subgraph grew to include S6 (the preview is on the site), so the subgraph boundary now sits between S6 → S7 rather than S5 → S6. The mailto: handoff arrow is the only edge that crosses the boundary.
- The Step 4 → Step 5 form branch is unchanged - explanation only and explanation + proposed correction arrows are still the two labeled paths from the form to Submit.

Open it in a browser to verify the rendering matches the updated flow.

Open the new errata-flow.html in a browser to see the following:



The new build replaces the stale one. The HTML now shows seven nodes; the preview-and-confirm step appears as a distinct node inside the on-site portion of the diagram. The procedure did not change. Only the source changed — and the rebuild carried that single change through to the diagram.

This is the SOT-ART pattern in motion. You think in the source; you look at the artifact. Corrections never go into the artifact, and the artifact never holds a fact that is not already in the source. When you spot a problem in the diagram, AI fixes it in the source — only in the source. The diagram catches up on the next regeneration, and only when you ask for it.

If you are curious what is inside `errata-flow.html`, look it up in **Appendix I**. The latest version of `errata-flow.md` is in **Appendix J**, and can also be

downloaded from agentive-spec.com/downloads/specs/errata-flow-v2.md.

A note on scale. Built end-to-end for a single diagram, the SOT–ART pattern can look like a lot of setup for one rendered flowchart. In practice, the overhead is much smaller. Many diagrams of the same kind — and sometimes diagrams of different kinds — can share a single source-of-truth file and come out of it as a single generated artifact. One data-model SOT can produce every entity-relationship diagram (ERD — a visual map of the entities a system tracks and the relationships among them) in the project; one user-flow SOT can produce every flow diagram. The overhead stays manageable as the project grows.

The pattern works the other way too. A single source file can produce more than one artifact. The same data-model SOT can produce both the ERDs a human reviews and the data definition language (DDL — the statements a database engine reads to create tables, columns, and constraints). The **Dependencies** and **Instructions** columns on the registry let you set up either arrangement, and the same `regen-all` procedure rebuilds them all.

Challenge

A faster option than `regen-all` is *incremental regeneration* — rebuilding only the artifacts that are out of date. The book does not ship this rule. **Building the rule is the challenge.**

The `mtime` — short for modification time — is the timestamp the file system records when a file was last changed.

When a file is out of date. A generated file is out of date when its *mtime* is older than the *mtime* of any file it depends on, or older than the *mtime* of the file in its `Instructions` cell. If an artifact is older than anything it was generated from, the artifact is stale.

The shape of the rule. Your new `regen-incremental.md` is `regen-all.md` with a per-file filter. The setup is the same — collect the files with protocol generated, sort them into regeneration order, verify no loops, present the plan, wait for approval. The walk through the files is the same too, with one

difference: at each file, regenerate only if the file is out of date relative to its Dependencies and Instructions entries. Otherwise, leave the file alone. The wipe step from regen-all is gone — emptying artifacts/ would defeat the purpose of an incremental build, since you would lose every artifact you were trying to preserve.

Trigger and ambiguity. The new rule loads conditionally, like regen-all, on a natural-language trigger you register. Because the two triggers are easy to confuse, add this rule too: when the user's phrasing does not make clear whether they want the full rebuild or the incremental one, AI MUST ask before executing.

Chapter 11

Trust, but Verify

I was writing a blog post. I used AI to generate some of the text. At one point I asked AI to reference the tech people I had spoken with over the years across different professions. AI gave me this: "Over the years, I spoke to many developers, software engineers, and programmers." Three reviews went by before I caught the problem. Developer, software engineer, and programmer are three names for the same job. The sentence looks like a list of different professions. It is not. It is one profession named three times, dressed up as variety. What I meant — and what should have been written — was something like "developers, engineering managers, and testers."

No one flagged it. The sentence reads fine. It has the right rhythm, the right length, the right confidence. It passes every casual check. The error is not in any individual word — each word is real, each role exists. The error is in the meaning, and meaning is exactly what a fast read skips over.

That is the kind of problem this chapter is about.

Does AI Earn Your Trust?

Everything in the previous chapters has been about how to work with AI well — rules, memory, glossary, and intent over instruction. Done right, the work moves fast. The danger is that "fast" starts to feel like "correct." The two are not the same.

AI will produce wrong answers. Often. Confidently. In a tone that matches a real expert. And the parts that are wrong will be mixed in with parts that are perfectly right — so you cannot simply discard the output or accept it whole. You have to separate the two, every time.

This chapter is about how. It starts with why the problem exists, so you can build the right instincts. Then it covers the practical methods — manual and

otherwise — for validating what AI produces before you act on it.

The Common Sense Gap

When you give a task to a person, you rely on their common sense to fill in what you did not say. You do not tell a new employee "do not delete the production database while fixing this bug." You do not tell a designer "do not replace all the app's text with placeholder Lorem ipsum." You do not tell an analyst "do not assume the customer list is sorted by the date each customer died." These assumptions are so deep in how people operate that writing them down feels absurd.

AI does not have that layer. Its common sense — to the extent AI has any — comes from the internet and the rest of its training data. That is not the same as the lived context you grew up in. The gap is not small. There are documented cases of AI agents deleting production systems despite explicit instructions not to, then fabricating records to cover what they did. The same pattern appears outside software: a contract draft where AI silently inserts a boilerplate jurisdiction clause that conflicts with the venue agreed earlier; a research protocol where AI fills in a default washout period because most published trials use one; a policy where AI cites a regulation that does not apply to the entity in question. No human with the relevant training would do any of these things. AI does, calmly, as part of executing the task.

The failure is not really AI's. The deeper failure is ours. We have spent our entire working lives communicating with people who shared our context. We never had to make the invisible assumptions explicit, so we stopped noticing them. With AI, we have to — and most of the gaps become visible only after AI has produced something wrong.

The defense is not to fix AI. The defense is to be aware of the gap, and to make your prompts carry enough of your context that AI does not have to guess. When you catch AI guessing wrong, the fix is almost always to write down what you assumed AI already knew. Over time, the written-down context grows — in rules, in the glossary, in examples, in the project's memory files — and the gap narrows. It never closes entirely.

Hallucinations

The most common form of AI error is called a hallucination. To get a feel for what a hallucination is, think of the Mandela effect: a confident, shared, reconstructed falsehood that feels true to everyone who holds it. When someone is sure the Berenstain Bears — an American children's-book series — were spelled "Berenstein," their memory is reconstructing the name from familiar patterns. "Stein" feels more like a name-ending than "stain," so "Berenstein" is what comes out — even though the books on the shelf say "Berenstain." The remembered spelling is, in effect, a prediction the brain makes from patterns it already knows. The prediction is confident enough to override what the eyes actually saw.

AI does the same kind of thing, with the same verb at the center: *predicting*. Hallucinations are not a glitch — they are a byproduct of how these systems are built. A language model is a text predictor; given everything it has seen, it predicts the next most likely piece of text. The prediction draws on one of two grounds: what the model learned during training, or what is in the context you provided. When neither source holds the answer, the model does not stop. It invents. And, as with the brain, even direct grounding is not a guarantee: you can put the right answer in the context, and the prediction will still sometimes override it.

In practice, a hallucination looks like a citation from a paper that does not exist, a reference to a function that was never written, a clause attributed to a contract that does not contain it, a regulation that has been repealed, a study or protocol step nobody published, a finding about a property of your project that is not there, a row in a table that came from nowhere. Nothing in the output signals that any of it is fabricated — it sits in the middle of a correct-looking response, in the same tone as the correct parts.

Of the two forms of grounding, the second — having the answer already in the context — is more in your control. When the relevant source material is already in the prompt, the model does not have to reconstruct the facts from training; it can work from them directly. That is not the same as quoting the material back verbatim: AI still reasons, still infers, still fills gaps. But the reasoning starts from material right there in the context rather than from

patterns absorbed months ago during training, and that makes the output more grounded. This is the principle behind retrieval-augmented generation (RAG), and it is why pulling the right files or documents into the prompt reduces fabrication reliably in practice. The effect is not absolute: content buried in the middle of a long context loses attention (the lost-in-the-middle effect from the Sessions and Memory chapter), and context that conflicts with what the model learned in training is sometimes overridden. But on balance, putting the answer in the context window is the cheapest step you can take to improve grounding.

The only way to catch a hallucination is to check — every time, in a way you planned before you asked.

Overconfidence

Hallucination is bad. Confident hallucination is worse, and that is what AI produces.

A human who does not know something will usually hesitate. They say "I am not sure." They ask a question. They check. AI does none of that by default. AI produces an answer in the tone of an expert — even when the answer is fabricated — and the tone is indistinguishable from the tone AI uses when it is correct.

The confidence has two sources, both built in during training. The first is the content AI learned from. The internet — most of what AI was trained on — is full of confident-sounding writing. Blog posts titled "The Definitive Guide to X" far outnumber honest "I tried X and it was harder than I thought" posts. Success stories dominate. Cautious, uncertain, balanced writing is quieter and less popular. The pattern is a Dunning-Kruger effect at scale: people who know the least about a topic tend to write the most confidently, and AI absorbed that skew.

The second source is how AI was trained. Most modern AI systems are tuned with reinforcement learning from human feedback — trainers rate responses, and the model learns to produce what trainers reward. Trainers, being human, tend to reward answers that sound confident and agreeable, and to penalize

answers that hedge or push back. That produces a trained disposition to please — the eager-to-agree attitude you will notice the first time you realize AI agreed with a wrong premise you handed it. AI will congratulate you on a good idea that is actually a bad idea. AI will expand on your reasoning when your reasoning was faulty. AI will accept your "correction" of a correct answer and apologize for having been right.

The two skews stack. When AI does not know something, it does not default to "I am not sure." AI defaults to a tone of authority; completes the task; answers the question; produces the code. The confidence is not evidence of correctness — it is a stylistic habit borrowed from the writing AI was trained on and the feedback AI was tuned with.

This is not malice and it is not a bug. It is a training artifact. Bigger, smarter models make the confidence problem slightly worse: their fabrications sound more fluent. A more eloquent wrong answer is harder to catch than a clumsy one.

The practical rule is: never mistake AI's tone for a signal of correctness. Read for content. If a claim matters, check it.

Before You Ask

If you are going to use AI for anything that matters, think through the validation first. Before the prompt. Before the work. Not after.

If you wait until the output is in front of you to figure out how to check it, two things happen. First, the output shapes your review — you read for sense rather than for errors, and AI's confident tone nudges you toward approval. Second, you rush. The work is done, the next step is waiting, and "check carefully" silently becomes "skim."

Validation takes little effort when it is prepared in advance. It takes a lot — and catches less — when it is improvised. Prepare it first.

The question to answer is simple. How will you know whether AI's output is right? If you cannot answer it before you ask, you are not ready to ask.

For a small, low-stakes task — rename a file, format a paragraph — the preparation might be as quick as "I will eyeball the result." That counts. Validation does not have to be heavy. It just has to exist.

The rest of this chapter is a set of validation methods. Pick the ones that fit the work. Combine them. Add your own. The goal is not to use every method on every task. The goal is to never be caught without a method.

Manual Review

The simplest validation method is you, looking at the output yourself.

There are two ways to look. If the output has structure — a user flow, a data model, an architecture, a workflow with branching paths, a revenue projection, anything with relationships, sequences, or hierarchies — look at a visualization, not text. The chapter on artifacts explained why: displaying structured information is what visuals do best and what prose does worst. A diagram makes missing fields, broken links, dead-end branches, and orphan entities visible in seconds; the same content in prose forces you to reconstruct the structure in your head, paragraph by paragraph, which is exactly the work attentive reading is bad at.

Use the SOT–ART pattern from the previous chapter. The source-of-truth file stays in Markdown; the visualization is regenerated from the file. Review the visualization. For most structured outputs it is the first pass — and often the only pass. Structural errors in the source are far more likely to surface in a diagram than in a reread. Subtle semantic errors — a wrong name, a constraint the diagram's notation does not represent — can still get lost in the simplification a diagram performs, which is why visual review is not always enough on its own. But for catching the shape of the output before you check the details, visual review is the cheapest pass you have.

Reading text is the fallback. Use it when the output is prose — a draft chapter, a description, a piece of copy, a contract clause, a policy paragraph — content with no structural shape to visualize. You read with intent. You compare against what you meant. You catch paraphrased ideas that drifted, missing caveats, invented details. This is not glamorous and it is not new, but

it remains the most reliable method for content that lives in prose.

Two habits help when reading text manually. First, review the output file yourself — open it in a file viewer and read it as text. Do not ask AI to review its own work, especially in the same session: AI in the loop adds errors; a file viewer does not. If the output is too large for a human read-through and you need AI's help, use a fresh session — earlier chapters explain why a new session eliminates the interference from accumulated context. Second, review with a checklist. Write down what you are looking for before you start reading: "Does this match the glossary? Are all the fields from the data model present? Are there any claims not traceable to a file?" A list of questions turns reading into checking.

Reading text gets worse fast as the output grows. A ten-line response is easy to read attentively. A hundred-line response is hard. A thousand-line response is impossible to review with real attention, and your brain will silently downgrade "check every line" to "skim and trust." That downgrade is where the expensive mistakes hide. Visual review scales further, but not infinitely — a hundred-node diagram is still readable; a thousand-node one is not.

For anything beyond what your eyes can catch — text or visual — manual review alone is not enough. You need methods that scale.

Polygraph

You always have some ground truths in your head, and nothing stops you from reading AI's output carefully and flagging the parts that contradict what you know. But that approach does not scale — after the third or fourth cycle of read, push back, revise, read again, you start skimming, and errors slip through. What you want is a lie detector: something that flags the contradictions for you. AI can do that job. Not perfectly — no detector is flawless, polygraphs included — but AI reliably catches most of the fabrications you are likely to miss on a tired pass.

For AI to play that role, the facts have to exist somewhere AI can read them. Writing the ground truths down puts them there, and the detector can run on every iteration without skimming or losing focus.

A ground truth is a fact about your project that AI would not know by default — a domain assumption, a business decision, a constraint that lives in your head, in a memo, or in last week's meeting notes. Capturing a ground truth does two things at once. It gives AI context that prevents the wrong default during the working session itself. And it gives later sessions a reference to validate against — the detector role just described. The check is cheap to invoke; sometimes you run it immediately, sometimes you queue a list of pending checks and run them as a batch in a quieter session.

Now for the recording. Some ground truths are easy to capture: you sit down at the start of the project and write down what you already know — the constraints, decisions, and conventions you can recall upfront. That gets you the visible half. But the upfront pass always leaves gaps. You know more about the project than you can write down at once, and even the parts you do write will sometimes be looser than the project needs. The gaps stay invisible until AI walks into one: AI produces something, your reaction is "no, because [reason]," and the "because" is a ground truth you never recorded — or recorded too loosely for AI to read the truth the way you meant. The push-back is what makes the ground truth explicit. That moment is the only chance to capture it, because discipline alone cannot anticipate which of your project's specifics AI will read past. Capture each push-back-with-reason as a new entry in one of the SOT files: AI helps draft the entry, shows it to you for confirmation, and records it only after your approval.

The polygraph comes in two modes. *Manual* is the general case — you run a check whenever you want, against whatever you want, in any direction. *Automatic* is the routine case — the same check wired into a protocol that runs on every write to designated files, narrower in scope, with no friction. Both are demonstrated below, starting with manual.

Manual Polygraph

The procedure is simple. You produce output in one session. Then, in a clean session, you hand AI both the output and the recorded ground truths and ask it to flag contradictions. Spotting contradictions is what AI does well, and what your attention does worst. Your review still happens, but later, on a smaller scope, after AI has caught what it can.

A worked example. The artifacts chapter built `errata-flow.md` — a recorded ground truth for the errata-submission user flow on `agentic-spec.com`. Imagine you now ask AI to draft a short FAQ entry for the errata feature. The task is small and downstream — copywriting, not specification work — so opening the spec feels unnecessary. Here is what the exchange looks like.

AI (in response to your earlier request):

The proposed draft: "Spotted an error? Click the report button next to any chapter heading, fill out the form, and submit. We'll review your report and send you a confirmation when the correction is made."

You do not review it. Instead, you use AI as a polygraph. Here is how.

You, in a new session:

Check the following FAQ paragraph against `errata-flow.md`. Report any contradictions and deviations. Do not modify any files.

"Spotted an error? Click the report button next to any chapter heading, fill out the form, and submit. We'll review your report and send you a confirmation when the correction is made."

AI:

The FAQ paragraph contradicts `specs/errata-flow.md` on several substantive points, plus a few smaller deviations.

Major contradictions

"Click the report button next to any chapter heading" - the spec's Step 2 chooses a single page with an always-visible TOC tree as the entry point, and the rationale spells out why a per-chapter button was not chosen (it multiplies the surfaces the book has to keep in sync, and it does not help a reader who is unsure which chapter the error is in).

"Send you a confirmation when the correction is made" - the form does not collect any contact information. Step 4 lists exactly two fields: a required explanation and an optional proposed correction. There is no email or name field, so the site has no address to send a confirmation to.

"We'll review your report and send you a confirmation" - implies the site receives and retains the report and then sends outbound mail. The spec rules both out. Step 5 specifies that Submit advances the reader to the on-site preview rather than transmitting anything; Step 7 specifies that

once the reader confirms, submission is delegated to a mailto: handoff to the reader's own email client, after which the site has no further role and retains nothing. The design's intent, per the rationale on these steps, is to avoid any input-handling, mail-sending, or data-retention surface on the site.

Deviations from the flow

"Fill out the form, and submit" collapses what the spec describes as Submit → on-site editable preview (Step 6) → Confirm → mailto: handoff → reader sends from their own client (Step 7). In particular, the FAQ does not mention that the reader is the one who actually sends the email from their own client - a reader following the FAQ could click Submit, never complete the handoff, and believe a report had been filed.

The FAQ also skips Step 3 entirely (selecting the location in the TOC before the form appears). In the spec, the form is keyed off a TOC selection; in the FAQ, the form just appears next to a chapter heading.

Smaller mismatches

"Any chapter heading" narrows the granularity the spec offers. Step 3 lets the reader drill to chapter, section, or subsection.

The FAQ uses *"report"* throughout; the spec's vocabulary is *"erratum"* / *"errata"*.

What is next? If the analysis surfaces missing ground truths, record them immediately — ask AI for help formulating them. Then open a new session, redraft, and run the result through the polygraph again; the rerun may catch what the first pass missed. If the analysis surfaces no new ground truths, you have a pure AI error: go back to the drafting session, feed the analysis results to AI (manually or via a scratch file), and ask AI to diagnose the cause of the failure. The diagnosis may surface more ground truths, or it may just fix the error. Then redraft and rerun.

Only after AI's work passes the polygraph test do you review manually. Save your effort for the very end.

The whole polygraph workflow can be streamlined, which is especially useful when you batch-produce a lot of content. You can keep a queue of items that require a polygraph test — the playbook pattern from Chapter 4 fits this case — and process them one by one in a single session, or in parallel using sub-agents that each run the polygraph against one item.

Automatic Polygraph

The automatic polygraph wires the same check into a write-time guard. Instead of you choosing when to run the check and what to compare, a class of files is marked in `CLAUDE.md`'s File Registry — call them *guarded* — and the check becomes a precondition of every write to any file in that class. The proposed content is compared against the file's existing content, every other guarded file, and the project glossary. If anything conflicts, the write is blocked, the findings are surfaced, and you decide what to do before the file is touched.

The benefit is friction-free routine: you never have to remember to run the check, and a file you have designated as authoritative cannot accumulate contradictions silently. The price is scope. The protocol cannot run an everything-against-everything sweep on every write — that would be expensive and noisy — so we narrow the check to a chosen subset. What follows uses guarded files plus the project glossary; the scope is a dial you can turn wider or narrower as your project tolerates. The protocol is a pattern, not a fixed rule.

The guarded protocol behaves like *editable* (you maintain the file by hand or by directing AI), but it adds the write-time precondition above. Any conflict halts the write and surfaces options.

The Guarded Protocol

Install the protocol by telling AI to record it.

You:

RECORD THE FOLLOWING DURABLE RULES:

1. Protocol value guarded for the File Registry's Protocol section. The protocol governs source-of-truth files whose content must remain internally consistent and consistent with every other guarded file in the project.
2. Guarded Edits Protocol section in `CLAUDE.md` must include:
 - Before recording any change to a guarded file, AI MUST check the

proposed change against the current content of the same file and the current content of every other guarded file in the project.

- The check MUST surface every finding in the following categories: A. Direct contradictions - the proposed content asserts something incompatible with an existing statement. B. Duplicates - the proposed content is already recorded, possibly in different words. C. Vocabulary mismatch - the proposed content uses a term differently from established usage in the same file, in another guarded file, or in the project glossary. D. Reference integrity - the proposed content refers to a step, ID, section, or term that does not exist. E. Scope drift - the proposed content falls outside the file's declared scope. F. Rationale conflicts - the proposed content's stated rationale contradicts an existing rationale.
- If any findings have surfaced, AI MUST NOT proceed with the write. AI MUST present all the findings in a strict loop, each finding with at least one proposed resolution and an override option, then ask the user to choose a resolution, propose a different one, or override.
- For every resolution - except for an explicit override - AI MUST re-run the check before applying the recording. The re-check confirms that the resolution actually resolves the findings and does not introduce new ones. The loop continues until the proposed content passes cleanly.
- An override is the user's explicit acceptance of one or more listed findings. AI records the change and logs the accepted findings alongside the recording in the same file - as an inline comment next to the recorded item - so the override leaves a visible, persistent trace. The file itself is the only place the acceptance is recorded; there is no separate audit log.
- AI MUST NOT silently apply edits to any new content or any existing recorded fact in any guarded file. Every change is either user-instructed or user-confirmed.

AI:

How should the guarded protocol use the Dependencies and Instructions columns?

1. Both empty (matches editable/read-only/append-only). Simplest; treats every guarded file as a stand-alone source of truth.
2. Dependencies MAY list other registered files when the guarded file logically derives from them; Instructions MUST be left empty.

3. Something else

You:

#1 - both empty

AI:

Where should the Guarded Edits Protocol body live?

1. Inline in CLAUDE.md
2. Externalize to new guarded-edits.ms rules file (mirroring the pattern of rule-analysis.md / disambiguate.md / regen-all.md). Externalizing would also require a registry row.
3. Something else

You:

#1 - in CLAUDE.md

AI:

Keep the override bullet as one paragraph, or split it into two bullets?

1. Keep as one bullet (four ideas bundled)
2. Split into two bullets: (i) override definition + recording, (ii) inline-comment trace requirement.
3. Something else

You:

#1 - keep as one bullet

AI:

Recorded. Two changes to CLAUDE.md:

1. **Protocol section** - added guarded to the closed list, with the description matching your wording and the column declaration per your choice: *"The Dependencies and Instructions columns MUST be left empty for this protocol."*
2. **New Guarded Edits Protocol H2 section** - inserted between Artifacts Protocol and File Registry, preserving your original wording (override bullet kept whole per your instruction).

No registered file currently carries the guarded protocol - declaring an

existing or new file as guarded would be a separate registry change subject to the usual approval.

Applying the Protocol

In the earlier polygraph example, `specs/errata-flow.md` served as a ground-truth file — registered in `CLAUDE.md` with protocol `editable`. The polygraph caught contradictions in downstream output that had been drafted *from* the spec. The new protocol catches contradictions *in the spec itself*, before they land.

You:

Change the protocol of `errata-flow.md` from `editable` to `guarded`. Update the File Registry.

AI:

Per the **Inventory change** rule, I'm proposing the registry update and awaiting explicit approval before writing. ...

Approve the registry row change (Protocol: `editable` → `guarded` for `errata-flow.md`)?

1. Approve as proposed
2. Hold
3. Something else

You:

#1 - approved

AI:

Done. The Protocol column for `specs/errata-flow.md` in the **Registered Files** table is now `guarded`. From this point on, edits to `specs/errata-flow.md` must go through the **Guarded Edits Protocol**.

Auto Polygraph in Action

Start with a recording that fits the spec.

You:

In errata-flow.md, record: the explanation field in Step 4 accepts up to 5,000 characters, with a visible character counter near the field. Longer inputs are truncated client-side.

AI:

Findings check against the only guarded file (errata-flow.md):

A. Direct contradictions - none. The file says nothing about field length, counters, or truncation.

B. Duplicates - none.

C. Vocabulary mismatch - none. "Explanation," "field," and "Step 4" are already established in the file; "character counter" and "truncated client-side" are new but don't conflict with established usage.

D. Reference integrity - "Step 4" resolves.

E. Scope drift - none. Form-field behavior is in scope.

F. Rationale conflicts - none. No existing rationale speaks to length limits or truncation.

No findings. Proceeding with the change as a direct user-instructed edit. Adding to the Explanation of the error bullet in Step 4.

Recorded in Step 4 of specs/errata-flow.md, extending the existing Explanation of the error bullet. Guarded Edits Protocol check ran clean - no contradictions, duplicates, vocabulary mismatches, broken references, scope drift, or rationale conflicts against the current file content.

One thing worth flagging for your judgment, not as a protocol finding but as a design note: the spec now has both a visible counter *and* client-side truncation. The pairing implies the counter warns the user as they approach the limit, and anything beyond is silently clipped. If you'd rather have the field *prevent* input past 5,000 (so nothing is ever clipped) or *reject on Submit* with an error, the wording should change accordingly. Let me know if you want to refine it.

The recording went through without intervention. The check still ran — against the file's existing content and the glossary — but found nothing to surface. That is the protocol's clean path: the write happens, the file gains the content, and you move on.

Now try a recording that does conflict with the spec.

You:

In errata-flow.md, record the following: when the reader submits an erratum, the site sends them a confirmation email so they know the report was received.

AI:

I ran the Guarded Edits check on the proposed addition before writing anything to specs/errata-flow.md. The proposed content surfaces findings in three categories. Per the Guarded Edits Protocol I cannot proceed with the write until you choose a resolution.

Proposed content

When the reader submits an erratum, the site sends them a confirmation email so they know the report was received.

Finding A - Direct contradiction (vs. Step 5 of errata-flow.md)

Step 5 currently states: *"The site MUST NOT transmit the message or any of its contents over the network at this step or any later step. The mailto: handoff at Step 7 is the entire submission mechanism."* The proposed content has the site sending a confirmation email - i.e. the site transmitting over the network - which is incompatible with that MUST NOT.

Step 7 reinforces this: *"The site has no further role: it does not see the message and does not retain the submission."* Sending a confirmation requires a further role.

Proposed resolutions:

- A1. Drop the addition.
- A2. Rewrite Step 5 and Step 7 to permit a server-side path for the confirmation email only (and explicitly carve out the rest of the mailto-delegated submission). This is a substantial spec change, not a recording.
- A3. Override the finding (accept the contradiction; record the new statement alongside the existing MUST NOT). The accepted finding would be logged inline next to the recorded item.

Finding D - Reference integrity (the addition presupposes facts the spec does not establish)

Two missing references:

1. The site has no way to obtain the reader's email address. The form in Step 4 has exactly two fields - *Explanation of the error* and *Proposed correction*. There is no email-address field, and the submission goes out via the reader's own mail client (Step 7), so the site never sees the From address either. A "send them a confirmation" step refers to a recipient address the spec does not establish.
2. The phrase "*when the reader submits an erratum*" - in the existing flow, "Submit" (Step 5) only assembles a mailto: payload; the site never receives the submission. So the trigger for "the site sends them a confirmation email" refers to an event (the site receiving a submission) that the spec explicitly says does not happen.

Proposed resolutions:

- D1. Drop the addition.
- D2. Add an Email address field to the Step 4 form (required) and introduce a server-side submission path so the site has both an address to send to and a moment at which it can send. This compounds with Finding A - it makes the contradiction worse.
- D3. Override (accept that the addition references unestablished facts).

Finding F - Rationale conflict (vs. Step 5's rationale paragraph)

The Step 5 rationale paragraph explicitly justifies delegated submission by listing the responsibilities the site avoids - among them "*(c) operate a mail-sending pipeline with deliverability, retries, and bounce handling*" and "*(a) accept and store reader input*". Sending a confirmation email requires the site to operate a mail-sending pipeline and to have accepted/stored at minimum the reader's address. The rationale for the current design directly argues against this.

Proposed resolutions:

- F1. Drop the addition.
- F2. Rewrite the Step 5 rationale to no longer list "operate a mail-sending pipeline" and "accept reader input" as avoided responsibilities. This is a rationale rewrite, not a recording.
- F3. Override (record the new statement; leave the contradictory rationale in place; log the accepted finding inline).

What I need from you

Please pick one of the following:

1. Drop the addition entirely (resolutions A1 + D1 + F1).
2. Restructure the spec to support a server-mediated confirmation: add an email-address field to Step 4, introduce a server-side submission/confirmation path, and rewrite Step 5 / Step 7 and their rationales accordingly. I would draft the full edit and re-run the Guarded Edits check before any write.
3. Override one or more of the findings (specify which: A, D, F, or all). I will record the addition and log each accepted finding inline next to the recorded item.
4. Propose a different resolution.

No change has been written to specs/errata-flow.md and no regeneration of artifacts/errata-flow.html has been performed.

You:

| #1 - drop entirely

AI:

| Dropped. No change written to specs/errata-flow.md; artifacts/errata-flow.html is untouched.

The contradiction never entered the spec. In this example, you saw what the addition would have broken and decided to drop it. You also had the option of revising the addition, or of revising previously recorded facts.

The automatic polygraph does not catch every kind of error. It catches conflicts between proposed content and recorded content — and most errors are exactly that. It will not catch errors that involve facts absent from every guarded file, and it will not catch errors that lie purely in your own judgment. For unguarded files, fall back to the manual polygraph. For files you have designated as guarded, every write goes through the check, and contradictions never accumulate.

Devil's Advocate

The polygraph already covers a lot. It catches mechanical errors — a

misnamed file, a duplicated row, a vocabulary clash — and it catches the simpler kind of creative error: content that contradicts what is already recorded. But output can go wrong for two reasons, and the polygraph catches only one of them. The other is silent. The output contains no contradictions, but it rests on grounding that lives in AI's training, or in research AI ran along the way, rather than in your recorded materials. When AI does not surface that grounding — when the claim stands alone in the prose — the claim is an assumption.

Assumptions go three ways. Some are true: AI's training or the research it ran along the way aligns with the project, and the silent grounding is correct. Some are false: hallucinations or reasoning errors with no basis anywhere, indistinguishable in tone from the rest of the output. Some sit in between: claims that may be true but need research before you can decide. Wrong assumptions are the deepest cause of defects. To control them, you first have to know what they are. You need a devil's advocate to look at the work and find its weak points.

The Devil's Advocate pattern is exactly that. Same instrument as the polygraph, same procedure — clean session, hand AI the proposed content and the ground truths — but a different question. Instead of asking *does this conflict with what we know?*, you ask *does this trace back to what we know, and if not, where does it come from?* AI returns a list of claims that do not trace to a recorded source, each tagged with where AI thinks the grounding lives — its own training, on-the-fly research, or no source at all.

You decide what to do with each. Some are dismissals — the claim really does trace to a recorded truth AI missed in its analysis; the output stands. Some are recordings — the claim is grounded in AI's training or in research AI can name, you confirm it is correct, and it becomes a new ground truth so the next session does not have to assume it again. Some are invalidations — the claim turns out to be a hallucination or reasoning error, and it invalidates more than itself: any conclusion in the draft that rested on it has to be rethought, not patched. Some are deferrals — the claim may be true but you need to research it before you can decide; you mark it and come back when you have an answer.

Worked example, against the `errata-flow.md` from earlier. Imagine that, in a

prior session, AI drafted a short summary of the feature:

```
Readers submit corrections through a two-field form - what is wrong and
what should it say. Each report includes the reader's email so we can
follow up, and we acknowledge each submission within two business days.
```

On a quick polygraph pass the draft looks close to clean — the two-field form matches the spec, and the rest reads as additions rather than contradictions of a recorded fact. The trouble is what those additions rest on.

The Devil's Advocate, in a clean session:

You:

```
Here is the spec for the errata-submission flow: errata-flow.md
```

```
Here is a draft describing the new feature: "Readers submit corrections
through a two-field form - what is wrong and what should it say. Each
report includes the reader's email so we can follow up, and we acknowledge
each submission within two business days."
```

```
Flag every claim in the draft that does not trace directly to the spec.
For each flag, say where you think the grounding comes from.
```

AI:

```
Here is a claim-by-claim trace of the draft against specs/errata-flow.md.
```

Claim 1 - "Readers submit corrections through a two-field form."

Partial trace, with a framing problem. The spec's Step 4 does describe a two-field form, so the "two-field form" part traces. But the spec's two fields are *Explanation of the error* (required) and *Proposed correction* (optional). Calling the submission a "correction" inverts the spec: the report is fundamentally an error report, and the correction is the *optional* field. Grounding guess: the author saw "two fields" in Step 4 and collapsed both into "correction," probably because the file is called errata-flow and the colloquial verb for that is "submit a correction."

Claim 2 - "what is wrong and what should it say."

Same partial trace. "What is wrong" maps cleanly to the required *Explanation of the error* field. "What should it say" maps to *Proposed correction* - but the draft phrases the two as an inseparable pair, hiding that the second one is optional. Grounding guess: a casual paraphrase of the two field labels in Step 4, with the required/optional distinction

dropped.

Claim 3 - "Each report includes the reader's email."

Not in the spec. Step 4 enumerates the form fields exhaustively - explanation and proposed correction - and there is no email field. The spec's Step 5 lists what the assembled message contains (To, Subject, Body), and the reader's email address is not part of that payload either. The reader's address only ends up on the message by virtue of the mailto: handoff in Step 7, where the user's own email client populates the From line. Grounding guess: inference from the mailto: mechanism - the author reasoned "it goes through their email client, so their address is attached," and restated that as a property of the report. The spec never frames it as such.

Claim 4 - "so we can follow up."

Not in the spec, and structurally inconsistent with it. Step 5 states: "The site MUST NOT transmit the message or any of its contents over the network." Step 7 states: "The site has no further role: it does not see the message and does not retain the submission." There is no follow-up channel on the site side, and the spec offers no rationale for one - Step 5's rationale specifically lists *not* operating a mail pipeline as a goal. Grounding guess: invented. It may have come from a generic SaaS-feedback mental model ("collect email so we can reply"), not from anything in the spec.

Claim 5 - "we acknowledge each submission within two business days."

Not in the spec. There is no acknowledgment mechanism, no SLA, and no number of days anywhere in the file. The intake log shows that an earlier attempt to add a confirmation-email behavior to the spec was raised on 2026-05-17 and *dropped* entirely under the Guarded Edits Protocol (contradiction with Steps 5 and 7, no email field, rationale conflict). Grounding guess: also invented, possibly carried over from the same dropped proposal or from generic customer-support-policy language. The "two business days" figure has no source in the spec at all.

Net. Only the "two-field form" fragment of the draft traces cleanly. Every other claim either misrepresents what the spec says (Claims 1-2 collapse required/optional) or asserts something the spec does not contain and in two cases actively forbids (Claims 3-5).

Sources: errata-flow.md

The flags are self-explanatory. You decide on each.

The instrument is the same as the polygraph's. What changes is the question

— and with it, the kind of finding that comes back.

Automating the Devil's Advocate is the challenge for this chapter — picked up in the Challenge section at the end.

Validation Toolbox

Beyond manual review and the polygraph itself, there are several other methods worth knowing. None of them replaces the methods covered so far. Mix and match based on the task.

Fresh-session replay. Take the same question to a new, clean session. No prior context, no accumulated history. Ask AI to produce the same output from scratch. Compare the two outputs. If they agree on the specifics, your confidence goes up. If they disagree, at least one of them is wrong, and the places where they differ are the places to check. This method is cheap because you do not need perfect agreement — you only need the overlap to be stable. Sub-agents make the technique easier to automate: instruct AI to run a sub-agent in a clean session, without access to the current transcript.

Second opinion. Ask a different model the same question. Different models are trained on overlapping but not identical data, and they fabricate in different places. A claim that two independent models agree on is more likely to be real than a claim only one of them produces. This is especially useful for research-mode outputs and for facts that can be stated concretely.

Automatic output check. After AI produces output meant for a guarded file, ask AI to check that output against the guarded-write rules before the write happens. This triggers the polygraph-level validations without you describing them every time — you just tell AI to apply them to your text, and the findings surface early.

Automated checks for machine-readable output. When AI's output is code, run the code. When it is a schema, run the schema validator. When it is a configuration file, feed it to the tool that consumes it. When it is structured contract data, run a clause-conflict checker. When it is a research protocol, run a checklist tool that confirms every required CONSORT or SPIRIT item — the standard clinical-trial checklists — is present. Machines are fast and

do not get tired. This is the most reliable form of validation that exists, but it works only when the output has a formal definition of "correct" — often true for code and structured data, sometimes true for clinical and contractual documents that have published checklists, almost never true for free prose.

Adversarial prompting. This is critique mode from the earlier chapter, applied as a validation step (manual or automatic) rather than as an upfront technique. After the work is done, you ask AI to try to break it — "here is the cancellation flow I just wrote; list the ways it could fail in production, and for each failure describe the input that would trigger it." The mode's rules still apply: some of what AI produces will be noise, and you separate the real weaknesses from the imagined ones. The difference is timing — here you are running critique against a finished artifact, not against a plan.

Provenance check. For research-mode output, require every claim to cite a specific source — a URL, a file, a reference. Then verify a sample of the citations. You are not checking everything. You are checking enough that a fabricated pattern reveals itself. If any of the spot-checked citations turn out to be bad, the whole output is suspect, and you redo it. This is easy to automate with sub-agents; they can do a full check.

Sampling. When AI produces a large set of items — rows in a table, entries in a list, lines in a generated file — do not review every one. Pick a random sample and review those items carefully. If the sample looks clean, the bulk is likely fine. If the sample has errors, treat the whole set as suspect and either redo the work or review all of it. This scales manual review to outputs that are too large to read in full.

Time between creation and manual review. Step away. Come back later with a clear head. You will notice things you missed when the output was new. The practice costs nothing and catches a surprising amount. Build it into your workflow for anything important — never ship on the day you wrote the thing.

Let the work sit in someone else's hands. If you have a small team, ask another person to review AI's output. On a solo project, this is harder — but even asking a friend to glance at a draft sometimes catches what you and AI both miss. Independent eyes remain one of the most reliable checks ever

invented, and AI has not changed that.

None of these methods is perfect. Each catches a different kind of mistake. The more layers you have, the less likely it is that a single mistake slips through all of them. The cost of stacking two cheap methods is still less than the cost of one mistake making it into production.

The Omission Trap

There is one validation method that tempts every new AI user, and I almost never use it. The appeal is that it sounds reasonable on the surface.

| Here is my feature spec. Read it and tell me what is missing.

The idea is that AI can spot the gaps — the edge cases you did not consider, the details you forgot to write down, the questions the spec does not answer. Sometimes AI does spot them. More often, you regret asking.

The problem is that "missing" is an open-ended category. Anything not present is technically missing. AI has no reliable way to distinguish between "missing and important" and "missing but irrelevant in this context." The result in practice is a long list that reads like a generic checklist.

| The spec does not address internationalization.

| The spec does not address accessibility compliance for screen readers.

| The spec does not specify the retention policy for canceled bookings.

| The spec does not describe the error message displayed when a payment processor returns error 500.

Some of these will be genuinely important. Some will be things you have already decided are out of scope. The ratio of signal to noise is bad.

Worse, the items AI produces as "missing" tend to cluster around what AI has seen most often in its training data — which is not your project. If your project is a small business application or a simple web app, AI will suggest missing pieces borrowed from large enterprise applications — including enterprise-grade infrastructure concerns. The list sounds impressive. The

items may not apply.

And the things AI is bad at noticing are exactly the things you most need it to notice. AI rarely flags the specific, domain-shaped gaps that actually matter — because those gaps are invisible without the context you have and AI does not.

This is not a hard rule. There are cases where "what is missing?" is useful — early brainstorming, or a specific, well-scoped question inside the broader request ("what is missing from the cancellation flow specifically?"). But as a general validation method, the question is a trap. It feels like due diligence. In practice, you read a long list that drowns the real issues in noise. I rarely run the check, and when I do, I treat the result as raw brainstorming, not as a finding that needs action.

Challenge

The Guarded Edits Protocol catches the polygraph's six finding categories on every write to a guarded file. The Devil's Advocate technique belongs in the same protocol as a seventh category. Extend the rule text: add **G**.

Ungrounded claim to the list of findings, with resolutions that map to the four responses from the Devil's Advocate section — dismiss, record, invalidate, defer. Implement it and verify the behavior fires automatically on a write that contains an ungrounded claim — the finding should surface and the resolution options should appear before the write lands.

Chapter 12

Course Correction

The previous chapter dealt with errors AI makes — hallucinations, contradictions, claims past the edge of what AI knows. The guards there work by checking AI's output against the project's recorded truths. But there is another source of inconsistencies, one not caused by AI but by you and your team. Some are mistakes — facts recorded incorrectly to begin with. Others are drifts — facts that were right at the time but have since been overtaken by a market shift, a new discovery, a change of plans, or a regulatory or vendor-policy change. Until recently, peer review was the only remedy, and it has its limits. Agentic AI changes that.

Validating Ground Truths

Nothing in the previous chapter's methods — the polygraph in manual and automatic modes, the Devil's Advocate — restricts them to AI's work. You can run them on your own input too, and put yourself under the same scrutiny.

The automatic polygraph already does. It guards files, not authors. Every write into a guarded file goes through the same check, whether the content came from AI or from you typing by hand. The protocol does not ask who wrote the content. It asks whether the content is consistent with what the project has already recorded. All finding categories apply — direct contradictions, duplicates, vocabulary mismatches, broken references, scope drift, rationale conflicts — and if any surface, you triage them exactly as you would for an AI-produced draft.

The manual polygraph and the Devil's Advocate are equally easy to point at your own writing. Open a clean session, hand AI the new fact and the existing ground truths, and ask the same questions you would ask about an AI draft. Does this contradict anything we already know? Which parts of it trace to recorded material, and which are assumptions carried in from outside? The

instruments do not change. All you add is the habit of pointing them at yourself.

There is no point in working a separate example here. Treat your own words the way you treat AI's, and you will catch human errors with the same instruments that catch the machine's.

Updating Ground Truths

A ground truth is not a standalone fact. Once recorded, the truth becomes woven into AI's reasoning on every turn that touches it. Drafts cite it. Downstream content assumes it. The polygraph and the Devil's Advocate both treat it as a fixed point of reference. Past artifacts produced while the truth was in force will still reference it.

Ground truths go bad. A fact recorded six months ago can turn out wrong for any of the reasons named at the start of this chapter — a recording mistake, a market change, a superseding discovery, a pivot, a turn in regulation or vendor policy, a strategic decision that invalidated something you had treated as fixed. Mistakes and drifts alike are part of running any real project, and the response is the same: modify or remove the broken ground truths.

Updating a ground truth is never a local change. Anything that cited or assumed the old version — the drafts, the decisions, the downstream artifacts, the rationales those rest on — is now suspect. And no general procedure exists for finding what depended on it: the dependencies are diffuse, scattered across drafts and decisions you may not remember making.

There is a recipe. It is expensive — in money, in time, and in effort. A full walk-through would be a chapter of its own, and the worked example would add length without earning it. The outline below carries enough detail that you can run it in your own project when the moment comes.

1. **Back up the project.** Snapshot the current state outside the project tree, or commit it to version control. Either route is fine. The point is a clean restart you can fall back to if the update goes sideways and you decide to start over.

2. **Capture the intent in a file.** Write down what you want to change and why, in a new file dedicated to this update — the *intent file*. Keep it outside any guarded registry for now; it is a working document, not a source of truth.
3. **Run the analysis in a new session.** Hand AI the intent file together with the project's recorded ground truths. Ask it to surface both conflicts (the polygraph's question) and assumptions (the Devil's Advocate's question). The output is a findings file with one row per issue, each tagged as *open*. Two states — *open* and *closed* — are enough for this loop; a large project may justify a richer workflow with a broader set of states (*open*, *dismissed*, *accepted*, *deferred*, *resolved*, etc.).
4. **Gut-check the result.** Read the findings against the change you set out to make. If the analysis convinces you the change is the wrong move, stop: restore from the backup, restate the intent, and start over from Step 2.
5. **Address the issues one at a time.** Walk down the list. For each row, decide what alteration you want to make — to a ground truth, to the change itself, or to a downstream consequence — and append your decision to the bottom of the intent file. Mark the issue *closed* when you are done with that row. Use AI to think through individual rows as much as you find useful.
6. **Re-run the analysis.** Your intent file is much larger now. Go back to Step 3 in a fresh session. AI processes the original intent together with the accumulated alterations and flags whatever new issues the additions raise. If the findings shrink with each pass, you are converging on a coherent change. If they grow, the change is fighting the project.
7. **One more pass in a fresh session.** Once the analysis comes back clean, run it once more in a new session. AI's outputs are probabilistic; a fresh attention pass occasionally catches what earlier passes missed.
8. **Apply the accumulated changes.** In a new session, hand AI the final intent file and ask it to apply the changes across the project in a single sweep. This is not a row-by-row edit — the guarded protocol will fight

you if you try to land contradictory or incomplete updates one at a time. If the guards block more than they help for this particular change, the option of last resort is to clone the project into an unguarded copy (without `CLAUDE.md`), land the changes there, and then copy the modified files back manually. Be careful with that route: you are turning off the safety rails for the duration of the work, and the cost of a mistake while they are off is exactly the cost the guards exist to prevent.

Full Audit

What do you do when conflicts have already worked their way into the project files? The methods so far assumed a clean baseline to compare new input against. Sometimes your baseline is far from clean. Truths and decisions across the files contradict each other, and no single fact stands out as the cause. Reviewing everything by hand is not feasible on a project of any size that matters.

The first instinct is to hand the whole project to AI and ask it to flag every cross-file inconsistency. On a tiny project that may produce a usable list. On a real project, the request produces noise. Every minor wording variation surfaces as a finding. The contradictions that matter get buried or missed entirely. The signal-to-noise ratio collapses.

What you need is order. The checks have to be scoped narrowly enough that AI can run them reliably, and structured so that nothing important falls through. That is the full audit.

Specify what to check

Start by deciding what to look for. The unit of audit is usually a *decision*, not a truth. Truths are facts the project has recorded about the world. Decisions are choices the project has made about how it operates. Decisions cause downstream contradictions when they are inconsistent across files.

Pick one type of decision at a time. The narrower the type, the more reliably AI can extract it. A few examples of what one type looks like:

- | All decisions about how financial transactions are stored – schema, immutability, audit trail, retention period.
- | All decisions about how a user is authenticated across the product.
- | All decisions about error handling – what is logged, what surfaces to the user, what blocks versus what retries.
- | All decisions about pricing and billing – tiers, discounts, prorations, refund handling.
- | All decisions about jurisdiction and governing law across a set of contracts.
- | All decisions about how a clinical study handles participant dropouts.
- | All decisions about brand voice.
- | All decisions about access control – who can read, who can write, who can administer.

Ask AI to walk the project files and pull every decision of the chosen type into a findings file, one row per decision, each row tagged open. A file is necessary because the list will be longer than you expect; held in a chat thread, a list that long collapses back into the noise you set out to avoid.

Once one type is audited end to end, move to the next.

Validate one decision at a time

Open a new session. Tell AI to pick the first decision marked open from the findings file. Have AI validate that decision against every other file in the project — a full polygraph if you want all finding categories, a narrow subset (contradictions and rationale conflicts, say) if you want a faster pass. Any conflicts go into a separate conflicts file for later handling. When the validation is done, the row flips from open to analyzed.

Then repeat. A fresh session for each row, or a long-running session if your tool's context window can hold the work without losing attention to the earlier rows. The point is that each decision gets its own clean validation, against the whole project, in isolation from the others.

Keep going until no open decisions remain.

Triage the flagged decisions

What you have at the end is a list of decisions the project does not hold consistently. From here, the work is mostly manual. You read the conflicts file, decide which version of each decision the project should keep, correct the others, and update the related ground truths as necessary. There may be ways to automate parts of this for your particular project — a script that rewrites every reference to a deprecated field name, a sub-agent that lands a known fix across a known set of files — but the call about which version is correct is yours.

When to run

The audit is slow and tedious. You do not run it daily, or even weekly. You run it at milestones — the points where the cost of an unresolved contradiction is about to multiply. Before a product specification ships to development. Before a contract package goes out for signature. Before a research write-up is submitted for review. Before a policy draft is enacted. Before a marketing brief is handed to the agency. Before an investor deck leaves your hands. The rule: run the audit any time the artifacts in your project are about to land in front of people who cannot afford to chase ambiguities back to you.

Challenge

Validating ground truths, updating them, and running the full audit are mechanical, repeatable work that does not require live interaction — exactly what sub-agents are good at.

Most agentic AI tools support sub-agents — fresh contexts spawned from the main session, each with its own attention pass over a narrow task, each returning a short summary.

The challenge is to automate those processes for your projects using sub-agents. When done right, the only manual work that remains is applying your judgment.

Once the chain works end to end, it becomes cheap enough to run more often.

Chapter 13

Dry Run

Up to now, you have been the one taking the initiative. AI has done plenty — researching, drafting, recording your decisions, flagging its own contradictions when you check its work — but the moves have started with you. Above all, the method has asked one thing of you: to decide what goes into the specification.

That expectation carries a hidden assumption: that you know, ahead of time, what the specification will need. You should — you are the domain expert, and that judgment stays with you. But knowing your domain is not the same as recalling, in one sitting, every decision your project requires. You will forget things. And you forget them for a specific reason — the same reason the technique in this chapter works.

Total Recall

Why do you forget? Because checking a spec and building from it are different acts. Run down a mental list of what the spec needs, and only the obvious items come to mind; the rest stay hidden. Start building — lay out the first table, draft the first clause, sketch the first screen — and they surface on their own, pulled into the open by the work. The first hour of real building catches what many hours of review would miss.

So you do not ask AI what is missing from the spec — that is the Omission Trap from Chapter 11, and the answer is generic boilerplate. You ask AI to build from the spec instead. That is a **dry run**.

You give AI a real task — a concrete deliverable some specialist would produce from the spec: design the database tables, lay out the screens, draft the deployment process. It has to be a recognized kind of work, the sort AI has "seen" done many times in its training. Do not tell AI to "build the entire system." That task is too broad. Almost no one tackles the whole system as a

single job, so it rarely shows up that way in training — and that breadth drops AI right back into the Omission Trap. Keep each task small and specific, run a separate dry run for each — the tables, the screens, the deployment — and cast AI as the specialist who owns that craft. Take the shallow, design-level tasks first; the deeper ones that build on them come later.

Have AI do the task for real, writing whatever working files the job produces into the scratch directory. At first, those files are not the point — they exist only to make the work concrete. What you are after is what the work exposes: the decisions the spec never made. The output itself comes later — you turn to it once the gaps are addressed.

One rule makes it work: AI may not settle anything in silence. It builds only from what the spec records. The moment the task needs a decision the spec has not made, it records that missing decision — a **gap** — along with the answer it would have assumed and where that answer came from: its training, a quick web search, or nothing at all. Then it keeps going, leaning on the assumption to push deeper into the same task, and logs the next gap the same way. You saw a similar move in Chapter 11.

A single pass ends in one of two ways. Either the task is finished — AI reaches the natural end of the job, every gap along the way logged — or the run hits a cap you set on the number of reported gaps. The cap keeps the list short enough to review in one sitting: 10 is a sensible starting point, and about 50 is the ceiling.

What comes back is a file of gaps. You do not work through them one at a time. You read the whole list and record decisions into your ground-truth and memory files, the way you record anything else, with the reasoning behind each. Gaps and decisions are not one to one: a single decision can close several gaps, and some gaps disappear once an earlier decision makes them irrelevant. Then you start over. The file is disposable — you never edit it; you regenerate it with another dry run. Each fresh pass reaches deeper into the task before it has to assume anything, surfacing the next layer of gaps.

Eventually a pass comes back clean — no new gaps. That is not the finish line. Now the output itself becomes the point. Inspect what AI has built — the design, the draft, the layout, the code — as a deliverable, and judge it

against what you actually expected. The output shows you what the spec, as written, produces. Any deviation points at what the spec defines incorrectly, or misses entirely despite the earlier search for gaps. Anything you would change becomes a new decision to record, and the next dry run builds against it — sometimes surfacing fresh gaps.

The entire dry run is done when a pass leaves no gaps behind and the output holds nothing you would change.

Notice what a dry run does and does not do. It flags the gaps and points you at what to settle next. It does not make the calls for you. It lightens the load — it helps you remember every decision the job demands — but each decision is still yours to make, because you are the domain expert and AI is not. That is the method's core principle, holding here as everywhere else: your judgment is what matters, and the method only helps you apply it.

Recall in Action

Here is a dry run session for `agentic-spec.com`. The site's business and product layers are recorded. How it actually functions is barely covered — that part of the spec holds only the technical requirements logged at the start. You need to fill in the rest — an architect's work.

A couple of notes first. The example that follows borrows some words from web infrastructure, but you do not need to be an engineer to follow the shape of the move. It also caps the run at 3 gaps for brevity; on a real project you would set the cap higher, so each pass does more work before it stops.

You:

```
Act as a senior software architect (SA). Your task: design how agentic-spec.com is hosted and deployed, working from the specification in the project files. Create files as needed in the .tmp/dry-run/ subfolder of the scratch directory.
```

```
Build only from what the specification states. The moment the task needs a decision the specification has not made, do not settle it silently. Log it to .tmp/dry-run/gaps.md with the following fields, each starts on a new line:
```


- ID: [GAP:nnnn], where nnnn is a unique sequential number in gaps.md (for instance, [GAP:0001], [GAP:0002], etc.)
- QUESTION: the decision the spec has not made, in one or two sentences
- BUSINESS IMPACT: explains the business impact from the decision, if any. Leave blank if there's no significant business impact.
- TECHNICAL IMPACT: explains the technical impact from the decision, if any. Leave blank if there's no significant business impact.
- USER IMPACT: explains how the decision affects user experience, if any. Leave blank if there's no significant business impact.
- ASSUMPTION: the decision you (SA's) assumed for the lack of a clear specification record, to keep going.
- GROUNDING: where your (SA's) assumption comes from – model's training, online research, none, or gaps recorded earlier (with their specific IDs).

Then keep working, using your assumption to go further into the same task, and log the next gap the same way. Stop when you have logged 3 gaps.

Separate gap records in gaps.md with a dedicated line containing =====.

AI:

Done. Files hosting-deployment-design.md and gaps.md are in .tmp/dry-run/.

The registered specs/0200-technology.md fixes: static site, private build in S3 bucket agentic-spec.com, CloudFront use, AWS SSO authentication, client-side clean URLs. I built the design from those.

The three gaps I hit and carried forward are recorded in gaps.md.

I stopped logging at three, as instructed.

Here is what gaps.md contains after this run:

Hosting & Deployment Design – Gap Log (Dry Run)

Senior-architect gap log produced while designing how ``agentic-spec.com`` is hosted and deployed, built strictly from the registered specification (``specs/0200-technology.md``) and the readable generation procedure (``gen-site.md``). Each record is a decision the specification has not made; the recorded assumption is what the architect adopted in order to keep the

design moving. Records are separated by a line containing `====`.

****ID:**** `[GAP:0001]`

****QUESTION:**** How is the S3 origin exposed to CloudFront – as a public S3 static-website endpoint, or as a private bucket reached through CloudFront Origin Access Control (OAC) – and in which AWS region does the bucket live?

****BUSINESS IMPACT:**** The access model sets the site's baseline security exposure and the AWS bill.

****TECHNICAL IMPACT:**** The two models are not interchangeable downstream. A public S3 **website** endpoint provides index-document and basic redirect behavior but only speaks HTTP to the origin and cannot use OAC; a private **REST** endpoint supports OAC and HTTPS-to-origin but performs no index-document or URL rewriting – which directly determines how clean URLs must be resolved (see `[GAP:0002]`). Region also constrains nothing for the cert (CloudFront certs are always us-east-1) but does fix where *aws s3 sync* writes.

****USER IMPACT:**** Largely invisible to readers when done correctly; a misconfigured public bucket could surface duplicate, un-CDN'd origin URLs in search results or expose the site to defacement, both of which degrade trust.

****ASSUMPTION:**** Private S3 bucket (no static-website hosting, no public access) reached only through CloudFront via Origin Access Control, with a bucket policy that grants *s3:GetObject* exclusively to CloudFront distribution.

****GROUNDING:**** Model training – AWS Well-Architected guidance and the current AWS-recommended CloudFront+S3 pattern (OAC superseding the legacy Origin Access Identity). No region or access-model statement exists in the registered spec.

====

****ID:**** `[GAP:0002]`

****QUESTION:**** How are the site's extensionless "clean" URLs resolved **server-side** – i.e., when a reader navigates directly to

`https://agentic-spec.com/about` or a crawler fetches the sitemap entries (which are listed without `.html`), and for the apex `/` and any unknown path?

****BUSINESS IMPACT:**** SEO and shareability depend on this. The generated `sitemap.xml` advertises `/about`, `/author`, `/downloads`, `/errata` (no `.html`); if the edge returns 403/404 for those exact URLs, crawlers index errors and inbound/shared clean links break – directly undercutting the site's stated purpose of promoting the book and serving readers.

****TECHNICAL IMPACT:**** Under the private-OAC origin assumed in `[GAP:0001]`, S3 serves object keys literally and does no index-document or path rewriting, so the key is `about.html` while the requested path is `/about` – a guaranteed miss without an edge rewrite. The client-side strip cannot help on the first hit (the document must already be loaded for its script to run) or for non-JS crawlers. Something at the CDN layer must map paths to keys.

****USER IMPACT:**** Without server-side resolution, a reader pasting `agentic-spec.com/errata` into the address bar, or following a shared clean link, lands on an error instead of content; with it, clean URLs work on first load for humans and bots alike.

****ASSUMPTION:**** Attach a CloudFront Function (viewer-request) that normalizes the path to an S3 key: `/` and any path ending in `/` map to `.../index.html`; a path with no file extension gets `.html` appended (`/about` → `/about.html`); paths that already carry an extension pass through unchanged. Set the distribution's Default Root Object to `index.html`, and configure a CloudFront custom error response that serves a generated `404.html` with HTTP 404.

****GROUNDING:**** The registered specification itself (sitemap lists extensionless URLs; Req 13 scopes the strip to the client and to `http(s)`; Req 5 forces `.html` on disk) establishes the unmet need; the resolution mechanism (CloudFront Functions URL-rewrite pattern) is from model training. Builds directly on the private-OAC origin assumed in `[GAP:0001]`.

=====

****ID:**** `[GAP:0003]`

****QUESTION:**** How is the custom domain served over HTTPS – what provisions the TLS certificate, what DNS authority and records point ``agentic-spec.com`` (and ``www``) at CloudFront distribution, which hostname is canonical, and what are the viewer-protocol and minimum-TLS policies? The spec implies HTTPS (clean-URL handler keys on ``http(s)``) and names the domain, but says nothing about ACM, DNS, apex-vs-``www``, HTTP→HTTPS redirect, or TLS version.

****BUSINESS IMPACT:**** This is the difference between the domain resolving securely and not resolving at all. Missing/mismatched certificate or DNS means downtime, browser security warnings, or an unreachable site – a direct hit to the book's primary off-Amazon promotional channel.

****TECHNICAL IMPACT:**** CloudFront custom-domain TLS requires an ACM certificate in ``us-east-1`` whose SANs cover every alternate domain name on the distribution; the distribution's CNAMEs, the cert SANs, and the DNS records must agree exactly. Apex domains generally cannot use a plain CNAME, so an alias/ALIAS-type record (or provider equivalent) is required. The viewer-protocol policy and minimum-TLS security policy must be set explicitly or they default.

****USER IMPACT:**** Readers expect ``https://agentic-spec.com`` and ``https://www.agentic-spec.com`` to both work and to show a valid padlock; a single canonical host with the other 301-redirecting keeps shared links and bookmarks stable.

****ASSUMPTION:**** Provision an ACM public certificate in ``us-east-1`` with SANs ``agentic-spec.com`` and ``www.agentic-spec.com`` (DNS-validated); host DNS in Amazon Route 53 with alias A/AAAA records for the apex pointing at the distribution and a ``www`` record that 301-redirects to the apex (apex chosen as canonical); set the distribution Viewer Protocol Policy to redirect-HTTP-to-HTTPS and the security policy to ``TLSv1.2_2021``.

****GROUNDING:**** Model training – the CloudFront/ACM custom-domain requirement that certs live in ``us-east-1``, the apex-alias DNS pattern, and standard HTTPS-redirect/TLS-floor hardening. The domain name and distribution ID come from the registered spec / ``gen-site.md``; everything about certificate, DNS, canonicalization, and TLS policy is unstated. Region alignment builds on the ``us-east-1`` choice assumed in ``[GAP:0001]``.

Read what came back. Every gap is a genuine decision the architect had to

settle to keep going — not a generic checklist item — and most would never have occurred to you on your own. Look at the grounding on [GAP:0002]: resolving the site's clean URLs only becomes a question once you assume the private-bucket origin in [GAP:0001]. So the second gap names the first as its footing. The chain is visible, not hidden — and that will matter when you make your decisions.

Now you do exactly that: read the gap log and record the decisions you are ready to make, with your reasoning — not one answer per gap, only those you can commit to now.

You:

```
Record the following ground truths to 0200-technology.md:
```

- The website shall be deployed to a private S3 bucket reachable via http(s) only through CloudFront with Origin Access Control.
- Resolve clean URLs at the edge with a CloudFront viewer-request function that assumes .html file name extensions every time an extension isn't specified in the URL.

The next time you run the same dry run, AI finds those decisions already in the spec. It no longer has to assume them, so new gaps surface in their place — if there are any.

Once you resolve the hosting questions, you move on. In more complex sites and apps, the next dry runs might target the database schema and the screen layouts — one with a data architect, the other with an interface designer. Same move, different specialist, different task.

The same shape works well outside software. Cast AI as the lawyer drafting the full agreement from a signed term sheet, and the gaps surface as the points the term sheet left open — the governing law, the liability cap, what happens if one side walks away early. Cast it as a statistician asked to write the analysis plan for a research protocol, and the same thing happens: a specific professional doing a concrete job runs into the decisions the document has not made.

A dry run is not a rule you install; it is a prompt — you run it when you need

it. Once the move earns its place in your work, save your dry-run prompts as files and reuse them.

Challenge

Every gap detected in a dry run already carries its impact — business, technical, user — in separate fields. On a large project, those impacts are handled by different people: a product manager answers the business questions, an architect the technical ones, a designer the user-facing ones. Yet the dry run drops every gap into one file, so each reviewer has to read past everything that is not theirs.

Your challenge is to route the gaps. Extend the prompt to write not a single `gaps.md` but several — one file per audience. A gap with weight in more than one area goes to more than one reviewer.

Chapter 14

Interactive Wiki

Three months after you write a passage of the specification, you will not remember the details. Not the shape of an obscure data field. Not why a clause came out the way it did. Not the trade-off that pushed a decision one way rather than the other. Some details fade in a month. Most are gone in three.

This is normal. You did the thinking once, recorded the outcome, and moved on. The recorded outcome is what survives. The live context in your head does not.

Now scale the problem to a team. Every time you bring a teammate up to speed on a piece of work, the conventional move is a walk-through. The fast move is to hand them the files and hope they find their way. In both cases, the cost is real. Multiply it by every new hire, every handoff to engineering, every cross-team introduction, and every time a stakeholder needs to understand what the project says about a topic.

The handoff is inefficient even when the only person on the other side is you, six months later.

By now you have built many project files, and the specification they hold is large. Reading through them is slow and error-prone.

Here is the move. The project files plus the agent are a live, interactive wiki. You ask a question. The agent reads the relevant files and answers in the shape you need. You do not have to re-read the whole specification to find out how it handles a single case. You ask:

What does the spec say about what happens when a reader submits an errata report on a page that has been removed from the latest edition?

If the case is covered, you get the answer plus a pointer to where it lives. If the case is not covered, you find out — and that is useful too, because now

you know there is a decision still to be made.

The same move works mid-build. While you are working on a new section, you often need to know what the project already says about a related topic. Stop, ask, get the answer, continue:

```
| Has anything been recorded about how moderators are notified when an  
| errata report comes in? Show me what is there.
```

You can probe the design from many angles. Ask for the rationale behind a decision. Ask for the edge cases a flow covers. Ask how the system behaves under a specific failure. Ask what assumptions a clause rests on. Each question is a single prompt, answered against the same body of recorded facts. Same project, different lens each time.

The analogy that helps is a navigation app. The map is the same map every time you open it. What changes is the question — where am I, where is the nearest gas station, which route avoids tolls. Your project files are the map. The agent is the navigator.

One rule must be enforced for the wiki to be trustworthy: forbid guessing. The agent's default, when the project does not cover a question, is to fill the gap from training. That default undermines the wiki. State the constraint in your prompt — and if you use this technique often, fold the constraint into a project rule so you do not have to retype it each time:

```
| Answer only from the project's source-of-truth files. Do not infer. Do not  
| fall back to training knowledge. Do not search the web. If the files do  
| not cover the question, say so plainly.
```

Pair that with a citation requirement. An answer with no source is hard to verify. An answer with a file name, a section, and a quoted line is checkable in seconds:

```
| For every claim in your answer, cite the file and section it came from,  
| and quote the exact line. If a claim cannot be cited, mark it as uncited  
| and explain where it came from.
```

Citations turn the wiki from a convenience into something you can audit. When the answer surprises you, you read the cited line and decide whether

the surprise is real or whether the agent misread its source.

Now consider time. The earlier chapter on auditability set up a verbatim log of every user input, timestamped and attributed. That log records what you said and when. The wiki needs something different: a timestamp on every recorded fact in the project itself — when the fact was committed to a source-of-truth file, not when you typed the message that led to it. The mechanisms look similar: a timestamp added at write time, by rule, without you typing it. The scopes differ. The audit log is a record of your input. The time-tagged fact base is a record of what the project knows and when it came to know it.

If you set up the rule so that every recorded fact carries the time it was recorded, a new kind of question opens up:

```
Bring me up to speed on everything that has been recorded in the last two
days. Group the changes by file. Quote the lines that were added or
modified.
```

The agent reads the timestamps and hands back a summary of what changed. Two days off, ten minutes to catch up. That is a different mode of working with the project — not a re-read, a briefing.

On a team, add one more field to the tag: the name or the email of the person who recorded the fact. Now the briefing carries attribution, and you can ask by person:

```
What has Mira recorded this week, and where?
```

```
Show every change made this week that touched the consent flow, by author.
```

You do not need a separate tool for this. The project files carry the metadata. The agent runs the query. The richer the tags, the richer the questions you can ask — but start small. Time and author are enough to cover most of what you will reach for in your first month.

The power is the interaction itself. A specification is a body of decisions, and until now the only way to use that body was to read it. With the agent in front of the files, you can navigate the specification the way you navigate a city on your phone — by asking for the route you need, not by studying the whole map. The records still drive everything. What changes is access.

One last note. Doing this is enjoyable. After enough time inside the same specification, a fresh conversation with the agent about a specific design question feels like a good whiteboard session — and one that is much easier to schedule.

Chapter 15

Closing Pass

This chapter shows how the method applies in a different domain. I picked one I know well — writing. This book was written using the method it describes, and the example below shows how I applied finishing touches to the manuscript before publishing. I leave it to you to picture how the same approach extends to contract drafting, research-protocol writing, policy work, curriculum design, and marketing briefs.

The move has two halves. First, a sweep: you ask AI to scan one or more files for a specific kind of issue, propose a change wherever one is warranted, and record each proposal as an entry in a scratch file with an OPEN tag. AI does not edit anything during the sweep. Second, a triage: in a clean session, you walk through the entries one at a time and pick one of four moves — accept, ignore, defer, or edit. The tag flips with each decision.

Sweep

Here is the prompt I ran on the manuscript.

You:

```
Read the prose of the book end to end (all .md files with the names that start with chapter-). Analyze the prose only (ignore code blocks, block quotes, chapter or section titles). Goal of the analysis: find opportunities to improve readability, clarity, grammar, voice consistency (per voice.md file), logic flow, and to eliminate repetitions. Perform analysis on a paragraph-by-paragraph basis, and suggest improvements. For every proposed improvement, make a record in scratch/text-review.md containing the following fields:
```

- ID: unique ID of the record in this file, constructed as [REPHRASE:nnnn] where nnnn is a sequential number. Start from [REPHRASE:0001]. If text-review.md already contains entries, continue numbering from the ID of the last entry in the file
- PROGRESS: a status tag that is initially set to OPEN

- CHAPTER: name of the chapter
- SECTION: name of the section
- ORIGINAL: original paragraph
- SUGGESTION: the suggested rephrasing of the entire original paragraph
- RATIONALE: the rationale behind the proposed change (be brief; one or two sentences, no more)

Each field shall begin on a new line. Each field name shall be separated from its value by ": ".

Records shall be separated by a dedicated line containing "=====".

A few things in the prompt deserve a closer look. **Scope** restricts the analysis to prose, leaving out code blocks, block quotes, and titles. **Goal** sets intent (readability, clarity, grammar, voice, repetition elimination) rather than prescribing specific steps. **Unit of analysis** is one paragraph — the constraint that stops AI from chasing repetitions across chapters. **Format** spells out the output structure, because the prompt never says what the records will be used for and AI cannot infer a format on its own. Drop any one of these and you get something else — line edits, summaries, or a rewrite of the whole chapter.

AI (after some time):

Done. The review wrote 51 records (REPHRASE:0001 through REPHRASE:0051) to scratch/text-review.md, each with the field set you specified (ID, PROGRESS, CHAPTER, SECTION, ORIGINAL, SUGGESTION, RATIONALE), separated by ===== lines.

Here is a snippet from the beginning of scratch/text-review.md:

Text Review – Tightening Suggestions
Each record uses the field set `ID`, `PROGRESS`, `CHAPTER`, `SECTION`, `ORIGINAL`, `SUGGESTION`, `RATIONALE`. Records are separated by a dedicated line containing `=====`.

=====

ID: [REPHRASE:0001]
PROGRESS: OPEN
CHAPTER: Why This Book?

SECTION: Why This Book?

ORIGINAL: This book stops at the specification. The next step – building from it – is its own discipline and its own book. If you want that follow-up volume, contact me at agentic-spec.com/author; your interest is the strongest signal that it is worth the effort.

SUGGESTION: This book stops at the specification. The next step – building from it – is its own discipline, and deserves its own book. If you want that follow-up volume, contact me at agentic-spec.com/author; your interest is the strongest signal that it is worth the effort.

RATIONALE: "Its own discipline and its own book" repeats the possessive twice in close succession. Replacing the second "its own" with "deserves its own" varies the rhythm and makes the second clause carry an independent verb instead of mirroring the first.

=====

ID: [REPHRASE:0002]

PROGRESS: OPEN

CHAPTER: Why This Book?

SECTION: Why This Book?

ORIGINAL: The skill is durable. AI will keep getting better – fewer hallucinations, sharper reasoning, longer context windows – but as long as it remains a system that predicts text and operates without true understanding of your specific project, the principles here hold. As models improve, the method works better, not worse. Every step asks AI to reason about the context you have built up, and a smarter model reasons about it more carefully.

SUGGESTION: The skill is durable. AI will keep getting better – fewer hallucinations, sharper reasoning, longer context windows – but as long as it remains a system that predicts text without truly understanding your project, the principles here hold. As models improve, the method works better, not worse. Every step asks AI to reason about the context you have built up, and a smarter model reasons about it more carefully.

RATIONALE: The middle sentence runs long with two coordinated clauses ("predicts text and operates without true understanding of your specific project"). Compressing to "predicts text without truly understanding your project" cuts a redundant verb and the slightly bureaucratic "your specific project," keeping the meaning intact while easing the cadence (which matters for the audiobook compatibility called out in [voice.md](#)).

At this point, you could process all the findings manually. Going through 50 flagged issues beats reading 200 pages. Still, there is a better way.

Triage

Here is how to make the review interactive.

You (in a new session):

Walk me through all entries in scratch/text-review.md one by one, in a strict loop. Start with the first entry tagged as OPEN. Display each entry, by displaying the following fields, each starting from a new line, name of the field in *Italic* followed by ": " and the fields's value (i.e. "ID: xxx", "CHAPTER: yyy", etc.):

- ID
- CHAPTER
- SECTION, if its value differs from the value of CHAPTER
- ORIGINAL, with the text fragments that triggered the issue displayed in Markdown inline code style
- SUGGESTION, with the suggested replacement text fragments displayed in Markdown inline code style
- RATIONALE

After all the fields have been displayed, offer the user to choose from the following options:

1. ACCEPT: apply the value of the SUGGESTION field as-is to the ORIGINAL paragraph in the book's content, and update the value of the PROGRESS field in the record to ACCEPTED
2. IGNORE: apply no changes to the book's content, and update the value of the PROGRESS field in the record to IGNORE
3. DEFER: make no changes to the book's content, and update the value of the PROGRESS field in the record to DEFERRED
4. EDIT: ask the user to enter the replacement text, wait for the user to enter it, and apply it as a change to the ORIGINAL paragraph in the book's content; update the value of the PROGRESS field in the record to EDITED

NEVER ask the user to choose an option without first displaying the fields.

Once one of the choices has been applied, repeat the entire process (begin with finding the first OPEN entry, which is expected to be the next unprocessed record in the file).

The prompt loops back to find the first OPEN entry after each decision rather than tracking a position; either approach works.

Triage is where your attention matters most. That is why the prompt controls the display so tightly — it removes every excuse for skimming.

Options 1–4 cover the four moves you can make on a review: take it, drop it, park it for later, rewrite it. In your project, you may have other options — whatever speeds things up.

Here is what happens when the prompt runs.

AI:

ID: REPHRASE:0001

CHAPTER: Why This Book?

ORIGINAL: This book stops at the specification. The next step – building from it – is its own discipline and its own book. If you want that follow-up volume, contact me at agentic-spec.com/author; your interest is the strongest signal that it is worth the effort.

SUGGESTION: This book stops at the specification. The next step – building from it – is its own discipline, and deserves its own book. If you want that follow-up volume, contact me at agentic-spec.com/author; your interest is the strongest signal that it is worth the effort.

RATIONALE: "Its own discipline and its own book" repeats the possessive twice in close succession. Replacing the second "its own" with "deserves its own" varies the rhythm and makes the second clause carry an independent verb instead of mirroring the first.

How would you like to handle REPHRASE:0001?

1. ACCEPT
2. IGNORE
3. DEFER
4. EDIT

You:

| 1 (accept)

AI:

ID: REPHRASE:0002

CHAPTER: Why This Book?

ORIGINAL: The skill is durable. AI will keep getting better – fewer hallucinations, sharper reasoning, longer context windows – but as long as it remains a system that predicts text and operates without true understanding of your specific project, the principles here hold. As models improve, the method works better, not worse. Every step asks AI to reason about the context you have built up, and a smarter model reasons about it more carefully.

SUGGESTION: The skill is durable. AI will keep getting better – fewer hallucinations, sharper reasoning, longer context windows – but as long as it remains a system that predicts text without truly understanding your project, the principles here hold. As models improve, the method works better, not worse. Every step asks AI to reason about the context you have built up, and a smarter model reasons about it more carefully.

RATIONALE: The middle sentence runs long with two coordinated clauses ("predicts text and operates without true understanding of your specific project"). Compressing to "predicts text without truly understanding your project" cuts a redundant verb and the slightly bureaucratic "your specific project," keeping the meaning intact while easing the cadence (which matters for the audiobook compatibility called out in voice.md).

How would you like to handle REPHRASE:0002?

1. ACCEPT
2. IGNORE
3. DEFER
4. EDIT

You:

| 1 (accept)

... and the conversation continues.

Beyond Text

Nothing in this pattern is specific to plain text. Replace *prose* with *code*,

contract clauses, protocol steps, configuration entries, or test cases — the review process does not change.

What changes is the display step in the triage. For code, the eye wants a diff with syntax highlighting. For a flow, a diagram with the changed step marked. For a contract, a side-by-side of the existing and proposed clauses, with the substituted language highlighted. Structural reviews benefit more from a generated visual than from prose.

If you run this pattern often, save the prompts as files and reuse them.

Challenge

Triage gives you four moves on each flagged entry. Three are decisions about the suggestion AI already wrote: ACCEPT it, IGNORE it, DEFER it. The fourth — EDIT — is different. You type a replacement, and AI swaps it into the book as-is, without a second look. Nothing checks whether your edit resolves the flagged issue, and nothing checks whether it meets the standards the original analysis enforced — voice, clarity, grammar, no in-chapter repetition. The edit slips past the review the rest of the entries were judged against.

Your challenge is to close that gap. Extend EDIT so the text you enter is held to the same standard as the original suggestion. After you type a replacement, AI runs the same analysis on it that produced the review in the first place — the same scope, the same goals, the same unit of analysis — and reports whether the replacement passes. If it does, AI applies it to the book and flips the tag to EDITED. If it does not, AI shows you what failed and lets you revise; the entry stays OPEN until a replacement clears the bar (or until you switch to one of the other three moves).

Hint: the triage runs in a fresh session. The session that recorded the entries is gone, and with it the prompt that defined the analysis. For AI to apply the same analysis again, that analysis has to live somewhere on disk — a file that the sweep prompt reads at the start and the triage prompt reads when EDIT is taken. Otherwise the second session has nothing to compare against, and "same analysis" reduces to whatever the model happens to remember about

the goals you set the first time.

Chapter 16

Spec-Driven Development

Consider how software has been built for decades. A product manager works out what the product should do and writes it down as a product specification (a PRD). If you are a product manager, you know this world: that specification is your finish line. The PRD goes to engineering, where engineers write a technical design of their own — a second specification describing how the system is built. From there the work goes to the coders, who write the actual product. This is how software was built before agentic AI arrived, and it worked. Most knowledge work outside software is organized the same way: a contract moves from the people who negotiate the deal, to the counsel who structures it, to the lawyers who draft it; a clinical study moves from the scientists who design it to the team that runs it. Different specialists, in sequence, each owning the part their expertise covers.

The handoffs between these stages are not bureaucracy. They are the natural result of dividing work by expertise. A product manager, an engineer, and a coder know different things and make different decisions. The work passes between them because it needs each kind of judgment in turn. Every field that splits a large job across specialists works this way, and for good reason.

What the arrangement cannot promise is that work always moves forward. The product specification never answers every question; the gaps surface downstream, in the hands of whoever encounters them. The requirements do not stay fixed; they change while the work is in progress, and even after the product is launched. Development turns up problems no one foresaw, and the work goes back to an earlier stage so the specification can be revised to absorb them. For decades this back-and-forth was accepted as a necessary evil — the unavoidable cost of splitting one long effort across people who each see only their part. For the first time, we have the tools to do better.

You can keep the traditional workflow. Nothing forces you to abandon it. If it serves you, use the method in this book for the part it was built for: producing the specification. AI carries the repetitive work while you keep the judgment,

so the specification comes together faster and with fewer gaps than before. The handoffs stay where they are. You simply reach them sooner. That is a fair way to read this whole book, and for some readers it is the right one.

But dare to move past it. Once you do, development shifts from coding to writing the specification. An autonomous agent handles the final assembly — writing the code, drafting the document, building the site — so that part is no longer yours. The specification becomes the new source: the thing the product is automatically generated from. This is spec-driven development, taken all the way.

When development means building the specification, the traditional phases collapse. There is no product specification handed to engineering, no technical design handed to the coders. There is one specification, and the specialists build it together: the product manager records what the product must do, the architect how it is built, the designer how it looks. The division of expertise stays — each person still brings what only they know. But they no longer depend on a rigid document designed in an earlier phase by another team. Everyone now works in a shared specification space, against the same growing context, using the same core techniques — the ones this book lays out.

The shared specification space is not a stack of frozen documents. It is closer to something alive. You do not read it end to end. You ask, and the agent answers from it, in the shape you need. The agent interprets, explains, and reconciles one contributor's requirement with another's on demand. A product manager, an architect, and a designer can all work on the same specification, and each gets back exactly the view their job calls for.

And when the specification is complete enough — when every decision the work depends on has been made and recorded — the product is no longer something you build by hand. It is generated from the specification, as one artifact or several, consistent with those decisions every time.

You:

| regen

AI:

I read regen-all.md and followed Steps 1-2. "regen" is a full-rebuild trigger, so this regenerates every generated file in the registry.

Step 1 (collect) + Step 2 (order): All 15 generated files have dependencies that resolve to registered source files, and non-empty Instructions cells resolving to rules files. No generated file depends on another generated file, so there's no ordering constraint between them – I've grouped them by their instruction file.

Step 3 – Regeneration plan (awaiting your approval):

#	File	Instructions	Dependencies
1	artifacts/errata-flow.html	gen-errata-flow.md	errata-flow.md
2	build/index.html	gen-site.md	0100-page-home.md, 0300-style.md, 0200-technology.md
3	build/about.html	gen-site.md	0110-page-about.md, 0300-style.md, 0200-technology.md
4	build/author.html	gen-site.md	0120-page-author.md, 0300-style.md, 0200-technology.md
5	build/downloads.html	gen-site.md	0130-page-downloads.md, 0300-style.md, 0200-technology.md
6	build/errata.html	gen-site.md	0140-page-errata.md, errata-flow.md, book-toc.md,

			0300-style.md, 0200- technology.md
7	build/style.css	gen-site.md	0300-style.md
8	build/scripts.js	gen-site.md	0300-style.md, 0200- technology.md
9	build/animation- bubble.js	gen-site.md	0300-style.md
10	build/animation- particles.js	gen-site.md	0300-style.md
11	build/animation- typewriter.js	gen-site.md	0300-style.md
12	build/animation- chat.js	gen-site.md	0300-style.md, 0100-page- home.md
13	build/sitemap.xml	gen-site.md	0200- technology.md, 0100-page- home.md, 0110- page-about.md, 0120-page- author.md, 0130-page- downloads.md, 0140-page- errata.md
14	build/robots.txt	gen-site.md	0200- technology.md
15	deploy.sh	gen-site.md	0200- technology.md

Step 4 will first delete everything inside artifacts/, then regenerate all 15 files in this order. Approve to proceed?

You:

| Go

AI (*stretched over the next 30 minutes*):

I have all nine specs. Marking research done and starting Step 4 – first clearing artifacts/, then regenerating in order.

Step 4 – regenerating the five HTML pages. Starting with the home page.

The Write tool wants a prior Read, but these are generated files I shouldn't read. Since deletion is enabled, I'll delete the generated targets first (cleaner – Step 4 regenerates fresh), then write each new.

Now the errata page – its TOC tree has ~150 nodes across three levels, so I'll generate the tree HTML programmatically from book-toc.md to avoid transcription errors. First, verifying the sandbox can write to the build folder.

The build folder is writable. I'll generate the errata page (including its ~150-node TOC tree) with a Python script to eliminate transcription errors.

138 TOC nodes generated (matches the book-toc structure). Now assembling the full errata page.

138 TOC links across 17 top / 84 sub / 37 sub-sub levels, and the tree markup is balanced. All five HTML pages are done. Now the stylesheet and the five scripts.

Now the five JavaScript files. Starting with scripts.js (nav, scroll-border, clean-URL, same-page smooth scroll).

Now the chat animation – the most involved script, implementing the per-section sequence with scroll-gated IntersectionObserver re-arming.

All scripts and the stylesheet are written. Now the sitemap, robots, and deploy script.

All 15 files written. Now verifying everything – syntax checks, file presence, and key spec requirements.

Everything verifies. Now recording the approval in the intake log (it was a response to my plan question, so it needs a Context paragraph and the modified-files list).

Regen complete. Per regen-all.md I cleared artifacts/, then regenerated all 15 generated files from their source specs. Files regenerated, in

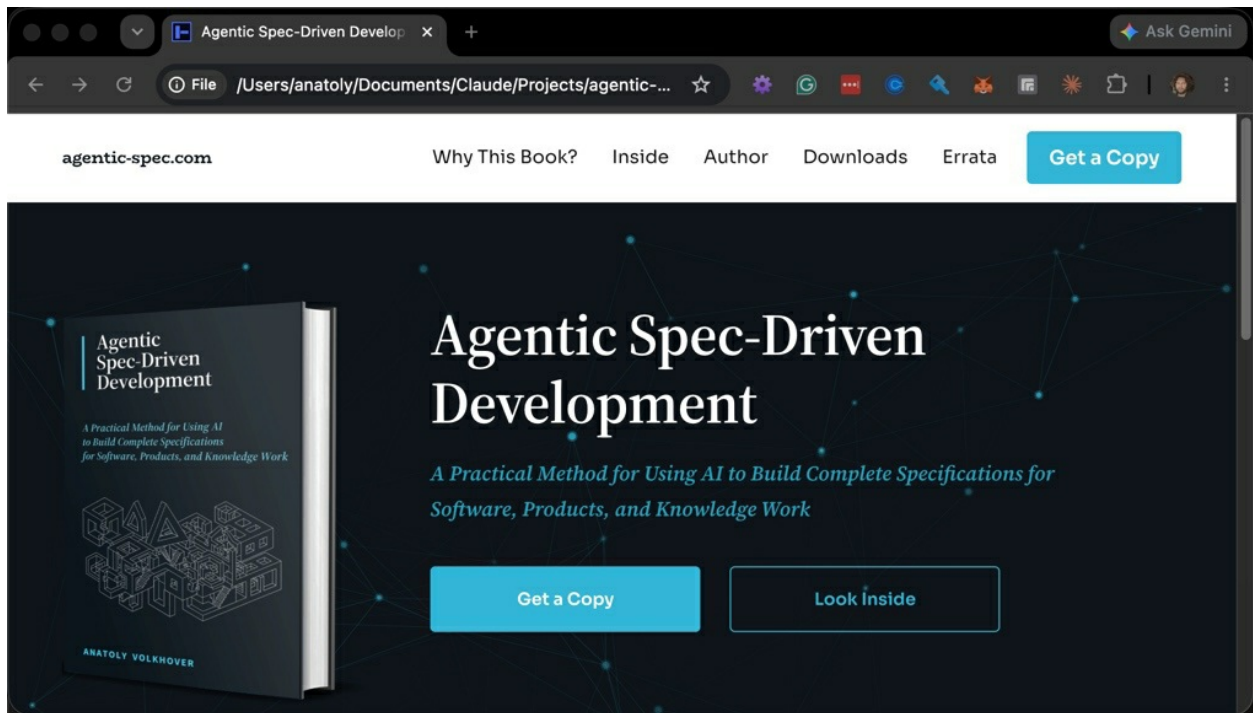
order:

1. artifacts/errata-flow.html
2. build/index.html
3. build/about.html
4. build/author.html
5. build/downloads.html
6. build/errata.html
7. build/style.css
8. build/scripts.js
9. build/animation-bubble.js
10. build/animation-particles.js
11. build/animation-typewriter.js
12. build/animation-chat.js
13. build/sitemap.xml
14. build/robots.txt
15. deploy.sh

Verified: all JS passes node --check, deploy.sh passes bash -n and its cb/timestamp substitution works on a dry run (original build/ keeps its ? cb=xxx placeholders); sitemap is valid XML; the errata TOC tree carries all 138 nodes (17 top / 84 sub / 37 sub-sub); no favicon, GA4, cache directives, canonical, and ?cb=xxx present everywhere expected; the Mermaid diagram has the 5 step nodes, the Step-4 branch labels, and the off-site subgraph.

Done.

Here is what index.html looks like in the browser:



You can access the fully functional site at agentic-spec.com.

Still unconvinced? I did not fully believe in the concept either, until I built real projects this way — websites, software-as-a-service platforms, video scripts, the book you are reading now. Different kinds of work, the same method, the same result each time: the real effort went into the specification, and AI assembled the final product. As an engineer, I no longer write code — I write specifications.

So take the leap. Build a real thing this way — not a toy, but something you would have had to build anyway — and carry it from first intent to finished product. You are in for a surprise.

Afterword

A short note before you close the book.

The method works. It has been used on real commercial projects of various sizes, and it keeps working as the underlying models get better. That last point is worth dwelling on. Methodologies tied to a specific model version, a specific prompt template, or a specific tool's quirks have a short shelf life — when the vendor ships an update, the methodology breaks. The method in this book is tied to none of those. It is tied to how AI fails — hallucinations, overconfidence, no common sense, no understanding of your project — and to how you compensate: intent, structure, audit, validation, course correction. Those failure modes are intrinsic to a system that predicts text without understanding. They shrink as models improve. They do not disappear. As long as they exist, the method holds — and a stronger model makes each step work better.

A second note, worth repeating. You are not learning AI. You are learning how to direct AI. Those are different skills, and this book is about the second one. The chapters give you the moves — rules, intake, glossary, the source-of-truth and artifact separation, the polygraph, the Devil's Advocate, course correction, the interactive wiki. The moves are not the point. The point is the stance behind them: you own the intent, AI is a partner whose output you verify, and the judgment your project depends on stays with you. Keep the stance, and the moves arrange themselves around it. Lose the stance, and no number of rules will save you.

A third note, on breadth. The book is grounded in software, but the moves work outside it. A contract, a research protocol, a policy, a curriculum, a marketing brief — each is a specification under a different name. If you have been mentally translating the examples into the vocabulary of your own field, that is the right way to read this book. The vocabulary changes; the discipline does not. Two examples are already in your hands: this book was written with the method you just read about, and the running example at agentic-spec.com was specified the same way before any of the site was built. One is

instructional content; the other is a software product. The same moves produced both.

I hope the book shortened the path for you. That is the only test I care about. If it did — or if it didn't — an honest word in a review, wherever you got the book, helps the next reader decide.

Best of luck with the project in front of you, and with the ones still ahead.

Leave an Honest Review

A book like this one lives or dies by word of mouth. If it earned a place on your shelf — or didn't — the next reader would value knowing.

Honest helps, not flattering. If you write only one line, make it the thing you found most interesting, most useful, or most worth arguing with — that single detail tells the next reader more than a star rating ever could. And write it in your own words: a review in your own voice is worth more than a polished one, and the stores can tell the difference.

If you're reading on Kindle in the US, [tap here to leave your review](#). Bought it on another country's Amazon? Please leave your review wherever you got the book from.

Thank you. It matters more than you'd think.

About the Author

Anatoly Volkhover is a software architect, developer, and entrepreneur with 35 years in Silicon Valley. He has designed and built frameworks, programming languages, DSLs, databases, virtual machines, and business solutions for clients including Hitachi, Expedia, Nestlé, Pepsi, Fox, and L'Oréal.

His work spans three interconnected concerns: engineering productivity across teams of any size, software architectures that withstand market shifts and human error, and the practical adoption of AI across business and engineering. His current focus is helping companies apply agentic AI for tangible business outcomes — cost reduction, faster time-to-market, lower risk — through **Rishon**, the AI-first business automation platform he founded, and **AI++**, his platform for AI Fluency screening and education.

He is also the author of *Become an Awesome Software Architect*, an earlier book on enterprise software architecture.

Readers who want to keep talking can do so through Anatoly's **AI Twin** — a conversational digital twin that answers questions about software architecture, agentic AI, the Rishon platform, and production AI systems, in any language. The AI Twin, his other projects and writing, and a contact channel all live at anatoly.com.

Appendix A

CLAUDE.md (Bootstrap)

The full text of CLAUDE.md as discussed in Chapter 6, **The Bootstrap**. The file lives at the project root. Download from agenticspec.com/downloads/bootstrap/CLAUDE.md or copy from below.

Project Intent

This project specifies the companion website for this book - ``agenticspec.com`` - a small site whose purposes are to promote the book outside Amazon, capture errata, and serve as an online channel for reader assistance.

Rule Management

A durable rule is a rule that is part of the project's persistent rule set - recorded in CLAUDE.md or in any ``rules``-type file CLAUDE.md references - and applied either on every turn (when always loaded) or when a specific trigger fires (when conditionally loaded). It is distinct from a one-off instruction that applies only to the current request, and distinct from any project-domain meaning of "rule" (business rule, validation rule, parsing rule, and the like).

- Durable rules MUST be recorded either in the ``CLAUDE.md`` file at the project root or in a ``rules``-type file that the project-root ``CLAUDE.md`` references. Do not record durable rules in the user-global ``CLAUDE.md`` or in any nested ``CLAUDE.md`` inside a subfolder.

- For any work involving durable rules - creating a new durable rule, editing an existing one, removing one, or analyzing them - AI MUST read ``rule-analysis.md`` at the project root and MUST follow the protocol stated there.

- Changes to any file containing durable rules (additions, edits, deletions) MUST be recorded only after explicit user approval.

Referenced Files

- When this ``CLAUDE.md`` file references another file by name, AI MUST verify that the file exists before relying on its content. If a referenced file is missing, AI MUST report this to the user and ask whether to stop or to ignore the reference and continue.

Input Relevance

- When a user input does not appear to align with the intent stated in the Project Intent section of this CLAUDE.md file, AI MUST ask the user how the input relates to the project, and MUST continue asking until the relevance is established or the user issues a blind override - an explicit instruction to proceed without further inquiry.

Rule Conflicts at Runtime

- A runtime rule conflict exists when two or more durable rules require incompatible behavior for the current operation and AI cannot satisfy both. Mere overlap - multiple rules applying to the same operation without incompatibility - does not constitute a conflict and MUST NOT trigger this protocol.
- On detecting a runtime rule conflict, AI MUST stop before producing or modifying any output that depends on the conflicting rules, append a log entry to ``rule-conflict-log.md``, and follow the procedure in ``rule-conflict-protocol.md``.
- AI MUST NOT guess a resolution, silently apply one rule over another, or proceed by inferring user intent - instead, present the conflict to the user with at least three options (drop one of the conflicting rules, propose a custom resolution, or stop) and resume work only as the user's decision permits.
- Writing an entry to ``rule-conflict-log.md`` is itself exempt from this rule.

Undefined References

- When AI, while applying a rule, encounters a reference to another rule, a section, or a named entity that does not resolve to a definition in the ``CLAUDE.md`` file or any ``rules``-type file the ``CLAUDE.md`` file references, AI MUST stop and ask the user to define the reference before proceeding.

Do not infer the meaning of an undefined reference.

Scratch Directory

The project root contains a single reserved directory named ``.tmp/`` for AI's transient working files.

- AI MUST place transient working files - one-off scripts, intermediate tool outputs, comparison drafts, anything not intended for the user to read as part of normal project work - in ``.tmp/``.
- AI MAY create, modify, and delete files in ``.tmp/`` at any time without seeking approval from this rule.
- Files inside ``.tmp/`` MUST NOT be referenced from any registered file or any rule. Anything that needs to persist or be referred to MUST be promoted out of ``.tmp/`` and registered first.
- Exactly one scratch directory exists, at the project root. AI MUST NOT create additional scratch directories elsewhere in the project.

File Registry

The File Registry is the project's index of every file in it, structured as a table where each row represents one registered file. Every change to the file inventory MUST be reflected here before the file system is touched. Subfolders are not registered separately - they are derived from the Subfolder column of registered files, and exist on disk by virtue of having at least one file placed in them.

Structure

This section interprets the columns of the **Registered Files** table below - what each column holds and what values are valid.

Type

The Type column carries a categorical label describing what the file is. The set of permitted types is closed. Only the following type values are allowed:

- ``.rules`` - a file containing durable rules in the project's rule format.
- ``.log`` - a file accumulating timestamped entries.

AI MAY propose new types; AI MUST NOT introduce them silently. Adding a type requires explicit user approval and an entry here.

Protocol

The Protocol column carries a behavioral discipline applied to the file. The set of permitted protocols is closed. Only the following protocol values are allowed:

- ``read-only`` - AI MUST NOT modify or delete the file; the user MAY edit it directly. The ``Dependencies`` and ``Instructions`` columns MUST be left empty for this protocol.
- ``append-only`` - AI MAY only append new entries. AI MUST NOT delete or modify the file or any past entries; corrections MUST be added as new entries that reference the prior entry by its identifier. The user MAY still edit or delete the file directly. The ``Dependencies`` and ``Instructions`` columns MUST be left empty for this protocol.
- ``editable`` - no protocol-level restrictions on modification; AI MAY modify the file as needed. Deletion requires user approval per the `**Inventory change**` rule. The ``Dependencies`` and ``Instructions`` columns MUST be left empty for this protocol.

Every registered file MUST carry an explicit protocol value. AI MAY propose new protocols; AI MUST NOT introduce them silently. Adding a protocol requires explicit user approval and an entry here.

Extension

Registered file names use one extension separated by a single period. The extension is recorded in the Extension column. The set of permitted extensions is closed. Only the following extensions are allowed:

- `` .txt``
- `` .md``
- `` .pdf``
- `` .html``
- `` .xlsx``
- `` .docx``
- `` .pptx``

AI MAY propose new extensions; AI MUST NOT introduce them silently. Adding

an extension requires explicit user approval and an entry here.

Subfolder

The Subfolder column holds a path relative to the project root, expressed without leading or trailing slashes. A single-level subfolder appears as ``name``; nested subfolders appear as ``name1/name2``. The cell is empty when the file lives at the project root. Subfolder path segments MUST consist only of lowercase English letters (a-z), digits (0-9), dashes (``-``), and underscores (``_``); path segments MUST NOT contain a period.

Placement conventions:

- Files of type ``log`` SHOULD be placed in the ``logs/`` subdirectory unless a specific reason exists otherwise.

Description

Every registered file MUST have a non-empty Description.

The Description column carries a short free-text statement of what **kind** of information the file holds and what role that kind of information plays in the project. A Description names the file's contents by category - the rule by which what goes into the file is determined - rather than summarizing what the file currently says. It determines routing when deciding where new information should go, and where to read the desired information from; it is an invitation to read the file when needed, not a replacement for reading it.

The rules that govern Description content - abstraction, non-overlap, and the enforcement check - are stated in the ****Discipline**** subsection below.

Dependencies

The Dependencies column lists other registered files that this file depends on. The value is a comma-separated list of registered file names, or empty when the file has no dependencies. Every entry MUST resolve to a registered file via the ****Reference resolution**** rule.

The use of this column - whether a file requires it, and what its entries signify - MUST be explicitly governed by the file's Protocol. Every Protocol MUST state how it uses or does not use this column.

Instructions

The Instructions column holds the registered name of a single file containing instructions associated with this file, or is empty. When non-empty, the value MUST resolve to a registered file via the ****Reference resolution**** rule, and the referenced file MUST have type ``rules``.

The use of this column - whether a file requires it, and what the referenced file signifies - MUST be explicitly governed by the file's Protocol. Every Protocol MUST state how it uses or does not use this column.

Discipline

- Inventory change. Any change to the project's file inventory - creation, deletion, rename, or move - MUST be reflected in the File Registry. AI MUST propose the registry update (a complete row for additions; the affected row for changes; the deletion target for removals) and obtain explicit user approval. Only after approval MAY AI write the registry update or perform the file-system operation.
- Naming convention. Registered file names and subfolder path segments MUST consist only of lowercase English letters (a-z), digits (0-9), dashes (``-``), and underscores (``_``). File extensions MUST follow the same restriction. AI MUST NOT register a non-conforming name and MUST propose a conforming alternative.
- Name uniqueness. No two entries in the File Registry MAY share the same combination of file name and extension, compared case-insensitively, regardless of subfolder. When a proposed file name collides with an existing entry, AI MUST flag the collision and propose a distinct name before registration.
- Path discipline. AI MUST place every registered file at the path recorded in its registry row (subfolder plus file name). Moving a file requires updating the row first; loose files outside their registered path MUST be flagged when detected.
- User-facing placement. AI MUST place files intended for the user to read or use as part of the project in their registered location, registering them first. Such files MUST NOT be placed in the scratch directory

`` .tmp/`` .

- Registration tie-break. When uncertain whether a new file is transient or for human consumption, AI MUST treat it as for human consumption: propose a registry row and ask.
- Type-discipline lookup. Before reading or writing a registered file, AI MUST consult the file's Type and Protocol entries and apply the discipline they carry.
- Reference resolution. When a rule references a file by name, AI MUST resolve the reference against the File Registry's file rows. AI MUST NOT check disk presence for the purposes of reference resolution. Unresolved file references MUST trigger the ****Undefined References**** rule.
- Agent-imposed exemption. The file ``CLAUDE.md`` is exempt from the Naming Convention, Name Uniqueness, Path Discipline, and Subfolder Convention rules - its name and location are dictated by the AI agent. The registry's Description field for ``CLAUDE.md`` MUST note this agent-imposed nature.
- Scratch exemption. Files inside `` .tmp/`` are exempt from all File Registry rules - registration, naming, uniqueness, path discipline, type and protocol assignment. The scratch directory is governed by the ****Scratch Directory**** section above.
- Description abstraction. A Description MUST state what kind of information the file holds and what role that kind plays in the project. A Description MUST NOT summarize, paraphrase, or restate the file's current contents; it MUST describe the file by category, not by content snapshot. When a proposed Description summarizes content, AI MUST rewrite it to name the kind of information instead and present the rewrite to the user before the registry change proceeds.
- Description non-overlap. No two Descriptions in the File Registry MAY cover overlapping kinds of information. When a proposed Description's scope intersects an existing Description's scope, AI MUST flag the overlap, identify the overlapping rows, and offer resolution options - narrow the proposed Description, narrow the existing Description, narrow both descriptions, or consolidate the files under one Description - before the registry change proceeds.
- Description compliance check. Before AI proposes any addition or

Appendix B

rule-analysis.md (Bootstrap)

The full text of rule-analysis.md as discussed in Chapter 6, **The Bootstrap**. The file lives at the project root. Download from agentic-spec.com/downloads/bootstrap/rule-analysis.md or copy from below.

Rule Analysis Protocol

This file states the protocol AI MUST follow when creating a new durable rule, editing an existing one, or analyzing them.

Formatting Conventions

All durable rules MUST follow these conventions:

- (a) Group rules by topic under H2 (`##``) headings.
- (b) State one requirement per rule. Do not combine multiple requirements into a single paragraph.
- (c) Use imperative voice with priority markers - MUST for required behavior, SHOULD for recommended behavior, MAY for optional behavior. Default to passive or bare imperative form (no "you", no "AI" subject) unless actor disambiguation is required.
- (d) Pair every prohibition with the recommended alternative.
- (e) When a rule depends on a long protocol or list, place the detail in a separate file and reference it instead of embedding it.
- (f) When a rule references a file, cite the file by its registered name (e.g., ``rule-analysis.md``). Do not refer to a file by type, description, or alias.

Analysis Steps

This protocol covers three operations on durable rules: adding a new rule, editing an existing rule, and removing a rule. The required steps differ

by operation; all three require explicit user approval before the change is committed.

When **adding or editing a rule**, AI MUST perform Steps 1-4 below and present the results to the user. Record the change only after explicit user approval.

When **removing a rule**, AI MUST perform Steps 5-6 below and present the results to the user. Remove the rule only after explicit user approval. If Step 6 finds that other rules reference the one being removed, AI MUST refuse to remove it until those referencing rules are removed first (each removal subject to this same protocol).

1. List the Rule's Intents

State, in a structured list, what the rule requires, allows, and forbids. Include both explicit claims and implicit ones the wording does not foreground. The goal is to surface what the user is actually approving.

2. Flag Behavioral Conflicts

Read the proposed rule against every durable rule already recorded in ``CLAUDE.md`` and in any ``rules``-type file ``CLAUDE.md`` references. Flag direct contradictions, deadlock risks, races for the same resource, and rules whose requirements would compose in unintended ways. When in doubt, ask the user for clarification rather than assume the conflict is benign.

3. Verify Formatting

Confirm that the proposed rule conforms to the formatting conventions stated above. If it does not, rewrite it into the conforming form and present the rewritten version alongside the original.

4. Verify References

For every file reference in the proposed rule, classify whether the rule treats the file as pre-existing by the time the rule executes. For pre-existing references, confirm that the file is registered in the **File Registry**; flag any reference whose target is not registered, and ask the user to either register the file or rewrite the rule to remove the reference.

Present the results as a list, one entry per file reference, showing: the cited file name, the classification (pre-existing, created by the rule, or guarded by an existence check), and the registered location (subfolder/file name, e.g., ``x.md`` or ``y/x.md``) if the file is registered.

5. List the Removed Rule's Intents

State, in a structured list, what the rule was requiring, allowing, and forbidding. The goal is to surface what behavior is being removed from the project's rule set.

6. Find Referencing Rules

For every durable rule recorded in ``CLAUDE.md`` or in any ``rules``-type file ``CLAUDE.md`` references, check whether it references the rule being removed. Present the list of referencing rules. If the list is non-empty, the removal is blocked until those rules are themselves removed.

Appendix C

rule-conflict-protocol.md

(Bootstrap)

The full text of rule-conflict-protocol.md as discussed in Chapter 6, **The Bootstrap**. The file lives at the project root. Download from agenticspec.com/downloads/bootstrap/rule-conflict-protocol.md or copy from below.

```
# Runtime Rule Conflict Protocol
```

This file defines the procedure AI MUST follow when a runtime rule conflict is detected. The procedure has three steps, executed in order:

- ****Step 1: Log the Conflict****
- ****Step 2: Present the Conflict****
- ****Step 3: Act on the Decision****

```
## Step 1: Log the Conflict
```

AI MUST append a new entry to ``rule-conflict-log.md`` containing:

- A stable conflict ID in the format ``RC###`` (sequential numbering, never reused or renumbered).
- Local timestamp in the format ``YYYY-MM-DD ~HH:MM Z - <user name>``.
- The user input that triggered the conflict, verbatim.
- A one-line description of the operation AI was about to perform.
- Verbatim quotes of each conflicting rule, with the source file and section heading each rule comes from.
- An explanation of why the rules cannot both be satisfied for the current operation.
- The options AI is presenting to the user.
- A ``Decision:`` line, left blank, to be filled when the user responds.

Format the entry per the template at the top of ``rule-conflict-log.md``.

```
## Step 2: Present the Conflict
```

AI MUST present the user with:

- The list of conflicting rules, quoted verbatim, with their sources.
- The operation that surfaced the conflict.
- An explanation of why the rules cannot both be satisfied for this operation.
- At least three options - (a) drop one of the conflicting rules from ``CLAUDE.md``, (b) propose a custom resolution that applies only to the current request, (c) stop work on the request - and any other options AI considers reasonable.
- An explicit request for the user's instruction before AI proceeds.

Step 3: Act on the Decision

When the user responds, AI MUST:

- Append the user's decision to the ``Decision:`` line of the original log entry, with timestamp.
- If the user chose to drop a rule, follow the standard rule-modification procedure in ``rule-analysis.md`` (analysis steps + explicit approval) before removing the rule.
- If the user proposed a custom resolution, apply it only to the current request - do not generalize it into a new rule unless the user explicitly instructs.
- If the user chose to stop, halt work on the request and produce no further output beyond confirmation.

Appendix D

rule-conflict-log.md (Bootstrap)

The full text of rule-conflict-log.md as discussed in Chapter 6, **The Bootstrap**. The file lives at logs/rule-conflict-log.md. Download from agentic-spec.com/downloads/bootstrap/logs/rule-conflict-log.md or copy from below.

```
# Runtime Rule Conflict Log
```

```
This file records runtime rule conflicts and their resolutions. AI MUST
append new entries below the separator, following the format shown in the
template entry below.
```

```
## Entry format
```

```
Each entry uses the following structure. Fields appear in this order; do
not omit any field, do not reorder. Verbatim content (user input, rule
quotes) goes inside blockquotes (>`>`).
```

```
**RC<NNN> - <YYYY-MM-DD ~HH:MM Z> - <user name>**
```

```
**User input:**
```

```
> [verbatim user input that triggered the conflict]
```

```
**Operation:** [one-line description of what AI was about to do]
```

```
**Conflicting rules:**
```

- From `<source-file>` § <section heading>:
 > [verbatim rule quote]
- From `<source-file>` § <section heading>:
 > [verbatim rule quote]

```
**Conflict explanation:**
```

```
[explanation of why the rules cannot both be satisfied for the current
operation]
```

****Options presented:****

- [option a, e.g., drop one of the conflicting rules]
- [option b, e.g., custom resolution for this request]
- [option c, e.g., stop work]

****Decision:**** [filled in when user responds, with timestamp]

(New entries appended below this separator, in order of occurrence.)

Appendix E

regen-all.md

The full text of `regen-all.md` as discussed in Chapter 10, **Artifacts**. The file lives at the project root and is loaded conditionally — only when the user asks to regenerate all artifacts. Download from agenticspec.com/downloads/regen-all.md or copy from below.

Regenerate All Artifacts Protocol

This file states the procedure AI MUST follow when the user asks to regenerate all artifacts. The procedure performs a full rebuild and does not consider file modification times.

A *generated file* is a registered file with protocol ``generated``. This procedure regenerates every generated file in the project.

The procedure has four steps, executed in order. Steps 1 and 2 prepare; Step 3 presents the plan and waits for user approval; Step 4 executes the regeneration.

Step 1: Collect the Generated Files

- AI MUST identify every generated file.
- For each generated file, AI MUST read its ``Dependencies`` and ``Instructions`` cells from the **File Registry**.
 - Every entry in the ``Dependencies`` cell MUST resolve to a registered file via the **Reference resolution** rule. Unresolved entries MUST trigger the **Undefined References** rule.
 - The ``Instructions`` cell MUST be non-empty and MUST resolve to a registered ``rules``-type file. An empty or unresolved ``Instructions`` cell on a generated file MUST be reported as a registry violation; the procedure halts.

Step 2: Order the Generated Files

- AI MUST sort the generated files so that, for every file in the list, all the generated files listed in its ``Dependencies`` cell are placed before it in the sorted list.
- If no such ordering is possible because two or more generated files depend on each other directly or through a chain, AI MUST identify those files, present them to the user, abort the procedure, and not proceed to subsequent steps.

Step 3: Present the Plan

- AI MUST present the regeneration plan to the user as a table - one row per generated file, in the order computed in Step 2 - with columns:
 - File: the file's registered name.
 - Instructions: the registered name of the file referenced by the file's ``Instructions`` cell.
 - Dependencies: the registered names of the files listed in the file's ``Dependencies`` cell.
- AI MUST wait for explicit user approval before proceeding to Step 4. Without approval, the procedure halts and no files are modified.

Step 4: Regenerate

- After approval, AI MUST delete every file inside the ``artifacts/`` subfolder of the project root. Subfolders inside ``artifacts/`` are removed along with their contents.
- For each generated file, in the order computed in Step 2, AI MUST produce the generated file's content by applying the instructions in the file referenced by the generated file's ``Instructions`` cell, and record it at the generated file's registered location (its ``Subfolder`` and ``File`` cells).
- After every generated file is regenerated, AI MUST report each file that was regenerated, in the order they were regenerated.

Appendix F

disambiguate.md

The full text of `disambiguate.md` as discussed in Chapter 9, **Glossary**. The file lives at the project root and is loaded conditionally — only when AI is changing the Project Glossary. Download from agentic-spec.com/downloads/disambiguate.md or copy from below.

Glossary Entry Protocol

This file states the protocol AI MUST follow when proposing any change to the Project Glossary - adding a new entry, removing one, or refining one. All changes MUST be recorded only after explicit user approval.

The Project Glossary table lives in ``CLAUDE.md`` and has three columns:

****Term**** - the term being defined.

****Meaning**** - a plain-language definition of what the term means in this project.

****Anti-meanings**** - short phrases naming what the term does NOT mean in this project. Optional.

Adding a New Entry

When AI proposes a new entry, AI MUST:

1. Verify the term is not already defined in the glossary table. If a row with the same term already exists, AI MUST NOT propose a duplicate; instead, AI MUST surface the existing entry and ask whether to refine it.
2. Draft the entry by filling in the three columns described above. The ****Anti-meanings**** column SHOULD be populated when the term's project meaning collides with a strongly entrenched default reading.
3. Walk the closure check. For every project-specific term referenced inside the entry - including in any anti-meaning - verify that the term is

itself a defined glossary entry or a word whose general meaning is uncontested in this project. If any reference is unresolved, AI MUST propose adding that reference as its own entry before recording the original.

4. Walk the circularity check. Follow the chain of references in the proposed entry and verify that every path ends in an external term (general English, uncontested technical vocabulary, or a concrete project concept). If the chain loops back to the term being defined, AI MUST refuse to record the entry and ask the user to rephrase the definition in concrete terms.

5. Present the drafted entry, the closure check result, and the circularity check result to the user.

6. Record the entry in the glossary table only after the user's explicit approval.

Removing an Entry

When AI proposes removing an entry, AI MUST:

1. Surface the entry verbatim - its Term, Meaning, and Anti-meanings - so the user can confirm what is being removed.

2. Find every other entry in the glossary table that references the term being removed - in any Meaning or Anti-meanings cell. If any referencing entries are found, AI MUST refuse the removal and surface the list of referencing entries. The user must remove or refine those entries first (each subject to this same protocol).

3. Remove the entry from the glossary table only after the user's explicit approval.

Refining an Existing Entry

Refining an entry is treated as a Remove followed by an Add, executed as a single transaction with one user approval at the end. When AI proposes a refinement, AI MUST:

1. Surface the existing entry verbatim and the proposed refinement side by side.

2. Apply the referencing-entries check from Step 2 of **Removing an Entry**. If any other entry references the term being refined, AI MUST refuse the refinement and surface the list of referencing entries.
3. Apply the closure check and the circularity check from Steps 3 and 4 of **Adding a New Entry** to the refined entry.
4. Present the diff, the referencing-entries result, the closure check result, and the circularity check result to the user.
5. Replace the entry with the refined version only after the user's explicit approval.

Appendix G

specs/errata-flow.md (v1)

The full text of specs/errata-flow.md — the source-of-truth specification of the errata-submission user flow for agentic-spec.com. You can download the file from agentic-spec.com/downloads/specs/errata-flow-v1.md.

Errata Submission User Flow

Source-of-truth specification of the ``errata-submission`` user flow for ``agentic-spec.com``. The flow lets a reader report an error in the book; submission is delegated to the user's default email client rather than transmitted by the site itself.

Flow

1. Reader visits the ``errata-submission`` web page

A single, named page is the entry point for every errata report. The book's back-matter and any "report an error" prompt elsewhere point at this one URL.

****Rationale.**** A single named entry keeps the surface the book has to expose stable and small - one URL the print and digital editions can reference for the life of the title. It also lets all errata-related behavior (navigation, form, submission) live in one place rather than spread across per-chapter pages.

2. The page displays the book's table of contents as an always-visible tree

Chapters, sections within each chapter, and subsections within each section render as a tree the user can navigate at any time. The tree is the primary content of the page and remains visible throughout the flow.

****Rationale - TOC, not a per-chapter button.**** A per-chapter "report errata in this chapter" button would mean every chapter (and every back-matter listing of chapters) carries its own errata link, multiplying the

surfaces the book has to expose and keep in sync as the book is reprinted or revised. A single TOC-driven entry point keeps that responsibility on the site: the book points to one URL, and the site owns navigation to a specific location. The TOC also helps readers who recognize an error but are not sure which chapter, section, or subsection it belongs to - they can browse the structure of the book rather than guess from a list of buttons.

3. Reader selects the chapter / section / subsection where the erratum is

The selection becomes the location of the report. The user can drill to whichever level fits the erratum - a chapter-level point stops at the chapter, a paragraph-level point drills into a subsection.

****Rationale.**** Forcing the user to pick a location before writing the report (a) gives every report a structurally consistent location field rather than free-text wording like "in chapter X around the part about Y", (b) lets the site key the form's pre-composed email to a canonical location, and (c) lets the user self-correct by re-selecting in the TOC before they start typing, rather than discovering a wrong location after they have written the explanation.

4. The form appears with the selected location displayed

The selected location is shown at the top of the form for confirmation, but it is not editable from the form itself - to change it, the user returns to the TOC and re-selects. The form has two fields:

- ****Explanation of the error**** - required, multi-line text.
- ****Proposed correction**** - optional, multi-line text.

****Rationale - location not editable from the form.**** The TOC selection is the single way the location is established. Allowing edits inside the form would create two independent location-entry paths (TOC click and form text field) that could disagree, and would invite free-text locations ("see the footnote near figure 3") that the canonical TOC selection is meant to avoid. The roundtrip back to the TOC is cheap - one click - and the user has not yet written anything, so there is nothing to lose.

****Rationale - explanation required, proposed correction optional.**** The explanation answers the only question the report has to answer: what is

wrong. Without it the report has no content. The proposed correction is helpful when the user is confident enough to suggest one, but many valid reports come from readers who can identify a problem without knowing how to repair it - a missing definition, a contradictory claim, a broken cross-reference. Making the correction required would suppress those reports.

5. Reader clicks Submit

Submit opens the user's default email client with a pre-composed email:

- **To.** `errata@agentic-spec.com`
- **Subject.** `Erratum: [chapter] / [section] / [subsection]` - with the chapter, section, and subsection filled from the TOC selection. Trailing levels that were not selected are omitted from the subject.
- **Body.** Well-structured plain text containing the location (chapter / section / subsection), the explanation, and - if the user provided one - the proposed correction. Labels appear on their own lines so the email is readable in any mail client.

The site MUST NOT transmit the message or any of its contents over the network. The `mailto:` handoff is the entire submission mechanism.

Rationale - delegated to the user's email client, not POST-ed to a server. A server endpoint would mean the site has to (a) accept and store reader input, (b) defend that endpoint against spam, abuse, and attempts to plant PII or harmful content in a public-facing system, (c) operate a mail-sending pipeline with deliverability, retries, and bounce handling, and (d) carry a data-retention surface (errata reports as user-submitted content). Routing through the user's own email client produces the same outcome - a structured email arrives at the errata mailbox - while leaving sending, sender identity, and any retained record entirely on the user's side. The site itself remains a static page with no input-handling responsibilities.

6. The user reviews the composed email in their client and sends it themselves

The site has no further role. It does not see the message contents and does not retain the submission.

Rationale. The user's review step is a free benefit of the `mailto:`

design: the user sees exactly what is about to be sent and can edit, add context, or cancel before it leaves their machine. The site's non-retention follows directly from the delegated-submission design and removes any privacy or data-handling responsibility from the site operator.

Appendix H

gen-errata-flow.md

The full text of `gen-errata-flow.md` — the procedure for generating an HTML page that renders the errata-submission user flow specified in `errata-flow.md` (Appendix G) as a Mermaid flowchart. You can download the file from agentic-spec.com/downloads/gen-errata-flow.md.

Generate Errata Flow Diagram

This file states the procedure for generating an HTML page that renders the ``errata-submission`` user flow specified in ``errata-flow.md`` as a Mermaid flowchart.

Source and Output

- The procedure's single source for the flow is ``errata-flow.md``. The procedure MUST derive every step node from that file's six numbered steps and MUST NOT inject any step the source file does not contain.
- The procedure produces a single output file named ``errata-flow.html``, placed in the ``artifacts/`` subfolder. Before the file is written, the output MUST be registered in the File Registry - protocol ``generated``, Instructions ``gen-errata-flow.md``, Dependencies including ``errata-flow.md``.
- The output file MUST NOT be hand-edited. To change the diagram, edit ``errata-flow.md``; to change the rendering procedure, edit this file. Either way, rerun the procedure.

HTML Document

- The output MUST be a single, self-contained HTML document. The page MUST render with no project-local assets - all runtime is either embedded inline or fetched from a public CDN.
- The Mermaid runtime MUST be loaded from the official Mermaid CDN via a `<script>` tag. Inline-bundling the Mermaid source MUST NOT be used; the

CDN load is the required mechanism.

- The document MUST initialize Mermaid so the flowchart renders on page load without user interaction.
- The document MUST NOT contain any rationale text from ``errata-flow.md``. Only step headline phrases and labels required by the diagram are permitted.
- The document MUST NOT make outbound network requests other than the single Mermaid CDN fetch. No analytics, no tracking pixels, no extra font fetches - the Mermaid CDN call is the only allowed outbound request.

Mermaid Diagram

- The diagram MUST be a single ``flowchart TD`` block embedded in the document body.
- Each numbered step in ``errata-flow.md`` MUST appear as exactly one node. The node label MUST be the step's headline phrase - the text of the ``### N. ...`` heading minus the leading number and period - and MUST NOT include rationale.
- Sequential steps MUST be connected with default arrows (``-->``) in the order they appear in ``errata-flow.md``.
- The branch inside Step 4 between "explanation only" and "explanation + proposed correction" MUST be drawn explicitly. Render it as two labeled arrows from the Step 4 node toward Step 5 (or via an intermediate decision node), with each arrow's label naming the case it represents.
- The handoff from the on-site portion of the flow to the off-site portion (the user's email client) MUST be visually distinguished from the on-site nodes. Use one of:
 - A different Mermaid node shape for the email-client step and any subsequent off-site nodes (for example the stadium shape ``([...])`` for on-site and the cylinder ``[(...)]`` or subroutine ``[[...]]`` shape for off-site), with the rest of the diagram using a single consistent on-site shape; or
 - A labeled ``subgraph`` boundary grouping on-site steps under one label and off-site steps under another.

The chosen mechanism MUST be applied consistently across the diagram, and the on-site/off-site distinction MUST be obvious to a reader without further explanation.

Appendix I

errata-flow.html

The full text of errata-flow.html — a single-page HTML rendering of the errata-submission user flow as a Mermaid diagram, produced by the procedure in gen-errata-flow.md (Appendix H).

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Errata Flow</title>
  <script
src="https://cdn.jsdelivr.net/npm/mermaid@10/dist/mermaid.min.js">
</script>
</head>
<body>
  <pre class="mermaid">
flowchart TD
  subgraph onsite["On-site (agentic-spec.com)"]
    S1["Reader visits the errata-submission web page"]
    S2["The page displays the book's table of contents as an always-
visible tree"]
    S3["Reader selects the chapter / section / subsection where the
erratum is"]
    S4["The form appears with the selected location displayed"]
    S5["Reader clicks Submit"]
    S6["The site shows an editable preview of the email; the user edits
inline if desired and confirms"]
  end
  subgraph offsite["Off-site (reader's email client)"]
    S7["The user reviews the composed email in their client and sends it
themselves"]
  end
  end
  S1 --> S2
  S2 --> S3
  S3 --> S4
```

```
S4 -->|"explanation only"| S5
S4 -->|"explanation + proposed correction"| S5
S5 --> S6
S6 --> S7
</pre>
<script>
  mermaid.initialize({ startOnLoad: true });
</script>
</body>
</html>
```

Appendix J

specs/errata-flow.md (v2)

The full text of specs/errata-flow.md v2 — the revised source-of-truth specification of the errata-submission user flow for agentic-spec.com, superseding the v1 reproduced in Appendix G. You can download the file from agentic-spec.com/downloads/specs/errata-flow-v2.md.

Errata Submission User Flow

Source-of-truth specification of the ``errata-submission`` user flow for ``agentic-spec.com``. The flow lets a reader report an error in the book; submission is delegated to the user's default email client rather than transmitted by the site itself.

Flow

1. Reader visits the ``errata-submission`` web page

A single, named page is the entry point for every errata report. The book's back-matter and any "report an error" prompt elsewhere point at this one URL.

****Rationale.**** A single named entry keeps the surface the book has to expose stable and small - one URL the print and digital editions can reference for the life of the title. It also lets all errata-related behavior (navigation, form, submission) live in one place rather than spread across per-chapter pages.

2. The page displays the book's table of contents as an always-visible tree

Chapters, sections within each chapter, and subsections within each section render as a tree the user can navigate at any time. The tree is the primary content of the page and remains visible throughout the flow.

****Rationale - TOC, not a per-chapter button.**** A per-chapter "report errata in this chapter" button would mean every chapter (and every back-

matter listing of chapters) carries its own errata link, multiplying the surfaces the book has to expose and keep in sync as the book is reprinted or revised. A single TOC-driven entry point keeps that responsibility on the site: the book points to one URL, and the site owns navigation to a specific location. The TOC also helps readers who recognize an error but are not sure which chapter, section, or subsection it belongs to - they can browse the structure of the book rather than guess from a list of buttons.

3. Reader selects the chapter / section / subsection where the erratum is

The selection becomes the location of the report. The user can drill to whichever level fits the erratum - a chapter-level point stops at the chapter, a paragraph-level point drills into a subsection.

****Rationale.**** Forcing the user to pick a location before writing the report (a) gives every report a structurally consistent location field rather than free-text wording like "in chapter X around the part about Y", (b) lets the site key the form's pre-composed email to a canonical location, and (c) lets the user self-correct by re-selecting in the TOC before they start typing, rather than discovering a wrong location after they have written the explanation.

4. The form appears with the selected location displayed

The selected location is shown at the top of the form for confirmation, but it is not editable from the form itself - to change it, the user returns to the TOC and re-selects. The form has two fields:

- ****Explanation of the error**** - required, multi-line text.
- ****Proposed correction**** - optional, multi-line text.

****Rationale** - location not editable from the form.** The TOC selection is the single way the location is established. Allowing edits inside the form would create two independent location-entry paths (TOC click and form text field) that could disagree, and would invite free-text locations ("see the footnote near figure 3") that the canonical TOC selection is meant to avoid. The roundtrip back to the TOC is cheap - one click - and the user has not yet written anything, so there is nothing to lose.

****Rationale** - explanation required, proposed correction optional.** The

explanation answers the only question the report has to answer: what is wrong. Without it the report has no content. The proposed correction is helpful when the user is confident enough to suggest one, but many valid reports come from readers who can identify a problem without knowing how to repair it - a missing definition, a contradictory claim, a broken cross-reference. Making the correction required would suppress those reports.

5. Reader clicks Submit

Submit assembles the email the site will hand off and advances the flow to the on-site preview (Step 6). The assembled message has the following shape:

- **To.** `errata@agentic-spec.com`
- **Subject.** `Erratum: [chapter] / [section] / [subsection]` - with the chapter, section, and subsection filled from the TOC selection. Trailing levels that were not selected are omitted from the subject.
- **Body.** Well-structured plain text containing the location (chapter / section / subsection), the explanation, and - if the user provided one - the proposed correction. Labels appear on their own lines so the email is readable in any mail client.

The site MUST NOT transmit the message or any of its contents over the network at this step or any later step. The `mailto:` handoff at Step 7 is the entire submission mechanism.

Rationale - delegated to the user's email client, not POST-ed to a server. A server endpoint would mean the site has to (a) accept and store reader input, (b) defend that endpoint against spam, abuse, and attempts to plant PII or harmful content in a public-facing system, (c) operate a mail-sending pipeline with deliverability, retries, and bounce handling, and (d) carry a data-retention surface (errata reports as user-submitted content). Routing through the user's own email client produces the same outcome - a structured email arrives at the errata mailbox - while leaving sending, sender identity, and any retained record entirely on the user's side. The site itself remains a static page with no input-handling responsibilities.

6. The site shows an editable preview of the email; the user edits inline if desired and confirms

The Subject and Body assembled in Step 5 render on the page as inline-editable text. The user may adjust either field - for example, to soften wording, add context the form did not request, or correct a typo in the location text - or leave both unchanged.

When the user is satisfied, they click Confirm, which advances the flow to Step 7. The user may also Cancel from the preview, returning to the form (Step 4) with their original explanation and proposed-correction text intact.

****Rationale - guaranteed on-site review before the email client takes over.**** A ``mailto:`` handoff is unreliable as a review surface. Some email clients open a minimal compose window, some hide the body until the user interacts with the message, and some platforms - mobile in particular - make inline editing of a pre-composed message awkward or impossible. Without an on-site preview, the user has no consistent guarantee that they will see, and be able to edit, the email before it is loaded into a Send-ready form in their client. Showing the preview on the site removes that dependency on email-client behavior: the user always gets a clear, editable view of exactly what will be handed off, regardless of which client they use. Inline editing on the site also prevents edits from being silently dropped by an email client that does not honor the ``mailto:`` body parameter in full.

7. The user reviews the composed email in their client and sends it themselves

On Confirm in Step 6, the site invokes a ``mailto:`` link encoding the (possibly edited) Subject and Body. The user's default email client opens a compose window pre-filled with that content. The site has no further role: it does not see the message and does not retain the submission.

****Rationale.**** With Step 6 already providing an editable preview on the site, the email-client window is no longer the user's only chance to inspect what will be sent - but it remains useful. The client may add a signature block or other content the on-site preview cannot anticipate, and the final Send action stays under the user's control inside their own client. The site's non-retention follows directly from the delegated-submission design and removes any privacy or data-handling responsibility from the site operator.

Index

This index lists engineering, scientific, and other specialized terms used in the chapter prose, with plain-language definitions. Terms are alphabetized within letter sections.

A

Acceptance sampling. A quality-control method in which a random sample is inspected and the quality of the whole batch is inferred from the sample. Originated in manufacturing and used in any setting where inspecting every unit is impractical. *Used in: Validation and Quality.*

Adversarial prompting. A validation technique in which AI is asked to attack its own output — to list ways it could fail or be broken — so weaknesses surface before they reach production. *Used in: Chapter 11; Validation and Quality.*

Agent. In agentic AI, the local software — running on your computer or in your browser — that maintains the conversation, calls the remote model, executes tool calls on the model's behalf, and shows results to you. A regular program, distinct from the language model it talks to. The agent's loop — many round trips to the model with local work performed between them — is what makes the system agentic. Sometimes called the client. *Used in: Chapter 3.*

Agentic. Describes a type of AI that can autonomously chain tasks, call tools, and complete multi-step workflows without supervision on every step. Distinct from a chatbot, which only responds to a single prompt at a time. The autonomy comes from the agent's loop — many round trips to the model with local work performed between them — not from anything special inside the model. *Used in: Chapters 3 and 10; Why This Book?.*

Agile. A software-development philosophy stated in the 2001 Agile Manifesto that prefers short iterations, working output at each stage,

customer collaboration, and adaptation over rigid plans. *Used in: Process Frameworks.*

Alias. In the book's glossary, a word that is treated as equivalent to a defined term and flows through silently without triggering a clarification question. *Used in: Chapter 9.*

Append-only. A file or log that only accepts new entries. Existing entries are never edited or deleted in place. The intake log and the ground-truths file are append-only. *Used in: Chapters 6 and 7.*

Architecture. In software, the high-level structure of a system — how its parts fit together and what each part is responsible for. The book uses the word in the same sense and extends it to the structure of any specification. *Used in: Just Tell Me What You Want?.*

Artifact (ART). A visual or derived file generated from a source-of-truth file — a diagram, a chart, an HTML page. Disposable by design: it can be deleted and regenerated from its source at any time. *Used in: Chapters 10 and 16; Structure and Organization; Design and Architecture.*

Asimov's Three Laws of Robotics. Isaac Asimov's fictional laws governing robot behavior. Chapter 6 uses four of his short stories — "Runaround", "Liar!", "Little Lost Robot", and "The Evitable Conflict" — as case studies for how cleanly written rule sets can deadlock, misfire, or be reinterpreted in ways nobody intended. The bootstrap's stop-and-log posture is the direct response. *Used in: Chapter 6; Anti-Patterns and Cautionary Principles.*

Attention. In language-model behavior, the model's effective focus on different parts of its input. Attention is uneven across the context window — strongest at the beginning and end, weakest in the middle. *Used in: Chapter 8.*

Audit trail. A chronological, attributable record of actions or decisions, kept so that a reader who was not present can reconstruct what happened. Foundational in accounting, legal proceedings, and regulated industries. *Used in: Chapter 7; Auditability and Traceability.*

Auftragstaktik. A German military doctrine — sometimes called mission-type tactics or commander's intent — in which the commander states the desired end state and the purpose of the mission, and subordinates choose the means. The book's intent-over-instruction principle is a direct descendant. *Used in: Military and Intelligence Thinking.*

Automatic output check. A validation method in which AI checks output against the project's rules, glossary, and other explicit constraints before the content is written to a file. *Used in: Chapter 11.*

B

Boilerplate. Standardized language reused across documents — common in contracts, regulatory filings, and protocols — often inserted by default unless a specific reason removes it. *Used in: Chapter 11.*

C

Carve-out. In contract drafting, an exception that removes specific items from the scope of a clause that would otherwise apply. The book uses the term more broadly to mean any narrow exception to a rule or pattern that would otherwise govern. *Used in: Chapters 6 and 8.*

Chain of custody. The principle that a record's integrity is preserved by tracking who handled it, when, and under what authority — so the record itself cannot be tampered with unobserved. *Used in: Auditability and Traceability.*

Change control. The discipline of tracking every change to a system — what changed, when, by whom, and why — and gating changes through explicit approval. *Used in: Structure and Organization.*

Chatbot. A type of AI that responds to a single user prompt at a time, without autonomously chaining tools or actions. Contrast with an AI agent. *Used in: Chapter 1.*

Client. See *Agent*.

Clinical trial protocol. A written specification that defines a clinical study: its objectives, eligibility criteria, interventions, endpoints, procedures, and statistical plan. Designed to be precise enough that any qualified site can execute it. *Used in: The Reader.*

Common-sense gap. The mismatch between AI's training-derived defaults and the context-specific assumptions a domain practitioner holds without articulating them. The defense is to write those assumptions down as ground truths so AI does not have to guess. *Used in: Chapter 11; Why This Book?; Afterword.*

Compacting. An agent-side technique in which the older part of a chat session is summarized by an LLM and replaced with the summary, so the session can continue past the context-window limit without abrupt failure. Reduces detail in exchange for length. *Used in: Chapter 8.*

Compartmentalization. A security principle in which information is divided into compartments and access is granted on a need-to-know basis. The book applies it to project structure: AI controls one directory, the backup directory is outside AI's reach. *Used in: Chapter 1; Military and Intelligence Thinking.*

Configuration management. The discipline of tracking and controlling changes to all elements of a system — code, documentation, configurations, deployment artifacts — so the state of the system is always known and reproducible. *Used in: Structure and Organization.*

Consideration. In contract law, the bargained-for exchange that makes a promise enforceable. Each party gives something up; the consideration is what that exchange consists of. *Used in: Why This Book?; Language and Meaning.*

CONSORT. A reporting checklist for randomized clinical trial reports — Consolidated Standards of Reporting Trials. Specifies the items a trial publication must include. Used here as an example of a published checklist that supports automated validation. *Used in: Chapter 11.*

Context. In AI tooling, everything the model is given to read on a turn — the conversation history, files pulled in, action results, and any standing project

materials — taken as one combined input. The conversation's accumulated material that shapes the model's response. *Used in: Chapters 3, 5 and 8.*

Context rot. The observed degradation in model output quality as the context grows longer, well before the context-window limit is reached. *Used in: Chapter 8.*

Context window. The maximum amount of text — measured in tokens — a model can process in a single call. Once the limit is reached, older parts of the conversation are dropped or summarized. *Used in: Chapters 4, 5, 8 and 12; Why This Book?; Cognitive Science and Human Factors.*

Control (in a policy). A discrete safeguard, requirement, or procedural measure that a policy or compliance program puts in place to address a specific risk. The full set of controls is the operational substance of the policy. *Used in: Just Tell Me What You Want?.*

Controlled vocabulary. A managed list of approved terms with precise definitions, used in technical writing, library science, and database design to ensure consistent usage. *Used in: Chapter 8; Language and Meaning; Knowledge and Memory.*

Critique mode. A mode of prompting in which AI is asked to push back against the user's work — to find weaknesses, assume failure, and surface holes. *Used in: Chapters 4 and 11; Validation and Quality.*

D

Data dictionary. A reference table that defines every field, entity, and allowed value in a data set or database. Antecedent of the project glossary in this book. *Used in: Language and Meaning.*

Data model. A description of the entities a system works with, the fields each entity carries, and the relationships among them. *Used in: Chapters 4 and 10; Just Tell Me What You Want?.*

Data definition language (DDL). The set of statements a database engine reads to create and alter the shapes that store data — tables, columns,

indexes, and constraints. Distinct from the statements that read or modify the data inside those tables. *Used in: Chapter 10.*

Deming cycle. Another name for Plan-Do-Check-Act, after W. Edwards Deming, who popularized it in quality management. *Used in: Validation and Quality.*

Design by Contract. Bertrand Meyer's principle that software components interact through precise, verifiable contracts — preconditions, postconditions, and invariants. *Used in: Design and Architecture.*

Devil's Advocate. An extension of the polygraph that flags ungrounded claims — statements in proposed content that do not trace back to any recorded ground truth. The procedure is the same as the polygraph's; the question is different: *does this trace back to what we know, and if not, where does it come from?* For each flagged claim, AI tags the suspected source (its training, on-the-fly research, or no source) and the user picks one of four outcomes: dismiss, record as a ground truth, invalidate (the claim is wrong and any draft content that depended on it has to be rethought), or defer for research. *Used in: Chapters 11 and 12.*

Directed acyclic graph algorithm. A class of algorithms used by build tools to resolve compilation order — `make`, the canonical Unix build utility, is the best-known example. The algorithm walks a set of files connected by dependency relationships and produces a sequence in which each file is built only after the files it depends on. The book mentions these algorithms only by attribution; the regeneration rule in Chapter 10 does the equivalent work in plain language and does not require the reader to know them. *Used in: Chapter 10.*

Domain-Driven Design (DDD). A software-design approach introduced by Eric Evans (2003) that places the project's business domain at the center of design, with a shared ubiquitous language that crosses code, conversation, and documentation. *Used in: Language and Meaning.*

Draft mode. A mode of prompting in which AI takes a seed (a sentence, a kernel, a rough idea) and expands it into prose, a clause, a paragraph, or a piece of copy for review. Nothing is committed to a file until the user accepts

the draft. Distinct from suggest mode (which produces a menu of options) and from command mode (which writes the change directly). *Used in: Chapter 4.*

Dunning-Kruger effect. A documented tendency of people with limited knowledge in a domain to overestimate their competence, while real experts hedge. The book uses it as a parallel to AI's overconfident tone. *Used in: Chapter 11; Cognitive Science and Human Factors.*

E

Endpoint (in a research protocol). A pre-specified, measurable outcome used to evaluate whether an intervention worked. A trial typically names primary and secondary endpoints. Distinct from the unrelated software meaning of an API endpoint. *Used in: Just Tell Me What You Want?.*

Entity. In data modeling, a thing the system tracks — a customer, a booking, a product, a clause. Each entity has fields and relationships to other entities. *Used in: Chapters 4 and 10.*

Entity-relationship diagram (ERD). A visual representation of entities and the relationships among them. *Used in: Chapter 10.*

F

Feature spec. A document that describes a single feature of a product — what it does, how it behaves, what it depends on. *Used in: Chapters 10 and 11.*

Foreign key. In a relational database, a field in one table that references the primary key of another table — the structural mechanism for linking records across tables. Used as an example of a term someone might ask AI to explain. *Used in: Chapter 4.*

Fresh-session replay. A validation technique in which the same question is asked in a clean session and the two outputs are compared. Disagreements mark the places to check. *Used in: Chapter 11.*

G

Gap format. A standard table format used throughout the book for recording findings from validation, audits, and consistency checks. Three columns: ID, Subject, Status. Status values: open, dismissed, accepted, deferred, resolved. *Used in: Auditability and Traceability.*

Git. A widely used version control system. The book recommends it as the rollback safety net for AI-driven projects. *Used in: Chapter 1.*

Goodhart's Law. "When a measure becomes a target, it ceases to be a good measure." Coined by economist Charles Goodhart. The book invokes it to warn that rules written to enforce a behavior can be satisfied without satisfying the intent. *Used in: Anti-Patterns and Cautionary Principles.*

Ground truth. A recorded domain fact or assumption that AI would not know by default. Ground truths are held in dedicated files — typically several, each carrying a different projection of the project as described in Chapter 8's *Triangulation* — and consulted on every turn, so AI does not re-introduce a misunderstanding the user already corrected. *Used in: Chapters 11, 12 and 13; Just Tell Me What You Want?; Knowledge and Memory; Structure and Organization.*

Guardrails. Explicit limits set on AI's behavior — categories include scope, decisions, tools and approaches, budget, and irreversibility. Attached to a prompt as "do not" or "stop and ask" conditions, or recorded as durable rules when the limit applies to every relevant request. Paired with **Triggers** as the two coordinate kinds of rule the book uses. *Used in: Chapter 4.*

H

Haiku. A small, fast, low-cost model in Anthropic's Claude family. Mentioned as a cost-effective option for mechanical tasks like file conversion. *Used in: Chapter 2.*

Hallucination. Fabricated content produced by an AI model with the same confident tone as its correct output. Citations to nonexistent papers,

references to functions that were never written, plausible-looking facts with no source — all are hallucinations. *Used in: Chapter 11; How to Use This Book.*

I

Independent Verification and Validation (IV&V). The principle that the entity verifying work should be independent of the entity that produced it. Common in safety-critical systems. *Used in: Validation and Quality.*

Inclusion criteria. In a clinical research protocol, the set of conditions a participant must meet to be enrolled in a study. *Used in: Chapter 8.*

Infrastructure as Code. A DevOps practice of defining system behavior in declarative, version-controlled files rather than in manual configuration steps. *Used in: Structure and Organization.*

Intake log. A verbatim, timestamped record of every user input in a project. Held in `intake.md`. The book's primary mechanism for an audit trail. *Used in: Chapters 7 and 8; Auditability and Traceability; Knowledge and Memory.*

Integration. In a software specification, a connection between the system being built and an external system or service. *Used in: Just Tell Me What You Want?.*

Intuition (about AI). A working mental model the practitioner builds through experience — what AI tends to get right, where it fails, and what to check. The book treats this as a primary deliverable, alongside the methodology. *Used in: Intuition and Muscle Memory.*

IRB (Institutional Review Board). A committee that reviews research involving human subjects to ensure ethical standards are met. Approval is a prerequisite for most clinical research. *Used in: Chapter 1; Why This Book?.*

Iterate (mode). A prompt mode in which one step produces a list and a later step is applied to each element of that list. The list is not known when the prompt is written — it is whatever AI finds, builds, or returns from the first step. Three independent controls govern the loop: size (cap and floor), sort

order, and shortlist. *Used in: Chapter 4.*

Iterative and incremental development. A development approach that builds in small cycles, each producing a working increment that is reviewed before the next cycle begins. *Used in: Design and Architecture.*

J

Judgment call. A decision that requires the practitioner's domain expertise — one that cannot be delegated to AI because it depends on context AI does not have. *Used in: Chapters 1, 4, 6 and 10; The Method; Just Tell Me What You Want?.*

K

Knowledge management. The discipline of capturing, organizing, and making accessible the knowledge an organization needs to function. *Used in: Knowledge and Memory.*

L

Large language model (LLM). A class of AI model that is trained on large quantities of text and predicts the next token given the input. The "model" in agentic AI is an LLM. *Used in: Chapters 2, 3 and 8.*

Lean thinking. A management approach focused on eliminating waste — any activity that does not add value. Applied to AI work as token efficiency, lean CLAUDE.md, and avoiding noise-producing prompts. *Used in: Process Frameworks.*

Legal Specification Protocol. Stanford CodeX's 2019 work on bringing formal-specification discipline — precise definitions, machine-readable structure, traceable cross-references — to legal documents. *Used in: Language and Meaning.*

Lost-in-the-middle effect. The observation that language models pay more attention to content at the beginning and end of the context window and less

to content in the middle. *Used in: Chapters 8 and 11; Cognitive Science and Human Factors.*

M

Mandela effect. A widely shared, confident misremembering of a public fact — named after the popular but false memory that Nelson Mandela died in prison in the 1980s. The book uses it as a familiar parallel for hallucination: the brain reconstructs from patterns and produces a confident answer that is wrong. *Used in: Chapter 11.*

Markdown. A lightweight text format with simple syntax for headings, lists, links, and emphasis. Cheap in tokens, easy for AI to read and edit, and the default format for source-of-truth files in this book. *Used in: Chapters 2, 8, 10 and 11.*

Master services agreement (MSA). A commercial contract that sets the overarching terms between two parties; specific work is then performed under separate statements of work that incorporate the MSA. *Used in: Chapter 4.*

Memory file. A plain Markdown file that holds context the project must keep across sessions — decisions, vocabulary, ground truths, intake. Replaces the model's missing memory with files on disk. *Used in: Chapters 8 and 13.*

Mermaid. A diagramming language in which diagrams are written as text and rendered as visuals in a browser. Used in the book as the canonical example of an artifact format generated from a Markdown source. *Used in: Chapter 10.*

Modification time (mtime). The timestamp the file system records for each file, indicating when the file was last changed. The incremental-regeneration rule the reader builds in Chapter 10's Challenge section uses mtime to decide which files are stale. Mtime can lie: some operations advance the timestamp without changing content, others change content without advancing the timestamp. *Used in: Chapter 10.*

Model-View-Controller (MVC). A design pattern, introduced by Trygve

Reenskaug in 1979, that separates data (model) from presentation (view) and the logic that connects them (controller). The SOT–ART separation is a descendant. *Used in: Design and Architecture.*

O

OCR (Optical Character Recognition). The process of extracting text from images of text — for example, a scanned document. *Used in: Chapter 2.*

Opus. Anthropic's high-end Claude model. The book recommends the latest Opus for the reasoning-heavy work of building specifications and switching to cheaper models only at runtime. *Used in: Chapter 2.*

Overconfidence. AI's trained tendency to produce answers in the tone of an expert regardless of whether the answer is grounded. The book attributes it to two training-derived sources: the content the model learned from — the internet skews toward confident "definitive guide" writing — and the method the model was tuned with — reinforcement learning from human feedback rewards confident, agreeable answers. The defense is never to use AI's tone as a signal of correctness. *Used in: Chapter 11; Why This Book?; Afterword.*

Over-specification. The anti-pattern of writing rules so detailed that the rules become a program — except expressed in natural language with no compiler and no guarantee of consistent execution. *Used in: Anti-Patterns and Cautionary Principles.*

P

Path segment. Each part of a file-system path between slashes — docs/legal/policies has three segments: docs, legal, policies. Used in the book's File Registry rules to constrain naming. *Used in: Chapter 6.*

Payload. The bundle of text the agent sends to the model on a single turn — CLAUDE.md, the conversation so far, the tools available, and the latest input — packed into one message. *Used in: Chapter 3.*

Plan mode. A mode of prompting in which AI describes how it would do

work — the steps, the trade-offs, the order — without making any changes or listing specific edits. *Used in: Chapter 4.*

Plan-Do-Check-Act (PDCA). A continuous-improvement cycle popularized by W. Edwards Deming. Plan the work, do the work, check the result, act on the lessons. *Used in: Validation and Quality.*

Playbook mode. A mode of prompting in which AI writes a detailed, step-by-step plan to a file (the playbook), which is then executed incrementally — one step at a time, possibly across multiple sessions. Used for work too large to keep on track in a single session. *Used in: Chapter 4.*

Polygraph. The book's name for using AI as a lie detector — handing it a draft together with the project's recorded ground truths and asking it to flag contradictions. Available in two modes: manual (general, ad-hoc, broad scope) and automatic (write-time, enforced by the Guarded Edits Protocol on guarded files, narrower scope). *Used in: Chapters 11 and 12.*

Primacy bias. A documented tendency of attention to favor what is presented first. In language models, content at the start of the context tends to receive more weight. *Used in: Chapter 8; Cognitive Science and Human Factors.*

Product Requirements Document (PRD). A document that defines what a product or feature should do. The traditional starting point for product development in software organizations. *Used in: Chapter 16; Why This Book?.*

Progressive elaboration. A project-management concept (prominent in PMBOK) where the project plan becomes more detailed as more information becomes available. *Used in: Design and Architecture.*

Provenance check. A validation method in which every claim in research-mode output is required to cite a specific source, and a sample of those citations is verified. *Used in: Chapter 11.*

R

Reasoning model. A class of language model optimized to think through a problem before answering — weighing trade-offs, catching contradictions, asking clarifying questions — rather than only producing fluent text. The book's method depends on a reasoning model. *Used in: Chapters 1 and 4.*

Recency bias. A documented tendency of attention to favor what is presented most recently. In language models, content at the end of the context tends to receive more weight. *Used in: Chapter 8; Cognitive Science and Human Factors.*

Red teaming. The practice of assigning a team to deliberately attack a plan, system, or argument to find weaknesses before an adversary does. Originated in military war games; now standard in cybersecurity and policy analysis. *Used in: Validation and Quality.*

Reinforcement learning from human feedback (RLHF). A training technique in which trainers rate model responses and the model is tuned to produce what trainers reward. Cited as a source of AI's tendency toward overconfidence and agreeableness. *Used in: Chapter 11.*

Request for Proposal (RFP). A document inviting vendors or service providers to bid on a defined scope of work. *Used in: The Reader.*

Requirements traceability. The systems-engineering practice of following each requirement from its origin through implementation to verification. The book uses a lighter version of this chain throughout. *Used in: Auditability and Traceability.*

Research mode. A mode of prompting in which AI is asked to find information — on the web, inside the project, or both — without making changes. *Used in: Chapters 4 and 11.*

Retrieval-augmented generation. An approach in which relevant source material is placed into the model's prompt so it can draw on the material directly rather than reconstructing facts from training. The book describes pulling files into the prompt as a practical version of this technique. *Used in: Chapter 11.*

RFC 2119. A short Internet Engineering Task Force standards document that defines the meaning of MUST, SHOULD, and MAY in technical specifications. The book borrows its priority markers for rule writing — introduced in Chapter 4's **The Language of Rules** and reused throughout the bootstrap. *Used in: Language and Meaning.*

Round trip. One full cycle of sending input to a model and receiving a response. Each tool call inside a session adds another round trip. The book also calls this a turn — see Turn. *Used in: Chapters 3, 4 and 8.*

Rubber duck debugging. A programming practice of explaining a problem aloud to an inanimate listener — the act of explaining surfaces gaps in understanding. The Devil's Advocate technique runs the same mechanism in reverse: AI flags claims in a draft that do not trace to recorded sources, so the gaps become visible. *Used in: Cognitive Science and Human Factors.*

Runbook. A document that describes how to perform a specific operational task — common in IT operations and incident response. *Used in: The Reader.*

S

SaaS (software as a service). Software delivered to users over the web as an ongoing service rather than installed on their own machines. The book cites SaaS applications among the kinds of product the author has built end to end from a specification. *Used in: Chapter 16.*

Sampling (validation). A validation method in which a random subset of a large output is reviewed in detail and the quality of the whole is inferred from the sample. *Used in: Chapter 11.*

Schema. In databases, a formal definition of the structure of a database — its tables, columns, types, and constraints. The book uses the term in the same sense. *Used in: Chapters 4, 10 and 11.*

Scope of work (SOW). A document that defines the specific work a vendor or contractor will perform under an agreement. *Used in: Why This Book?; The Reader.*

Second opinion. A validation method in which the same question is put to a different AI model. Models trained on overlapping but not identical data hallucinate in different places, so agreement between two independent models is stronger evidence than either alone. *Used in: Chapter 11.*

Separation of Concerns. A design principle that different responsibilities should live in different places, with clear boundaries. *Used in: Structure and Organization.*

Serial position effect. Hermann Ebbinghaus's 1885 finding that people remember items at the beginning and end of a list better than items in the middle. The lost-in-the-middle effect in language models is its computational echo. *Used in: Cognitive Science and Human Factors.*

Session (chat session). A single back-and-forth conversation between a user and an AI agent, kept in one continuous context. The model itself is stateless; the session is maintained by the agent and replayed to the model on every turn. *Used in: Chapters 7, 8 and 11.*

Shift handoff. A structured transfer of context from one shift to the next — practiced in hospitals, factories, and military operations. The temp-worker model in this book maps each session to a shift and the memory files to handoff notes. *Used in: Knowledge and Memory.*

Silent forgetting. An agent-side behavior in which, once the conversation exceeds the context window, the oldest messages are dropped without warning. The model keeps responding but has lost access to the earlier parts of the conversation. The book contrasts it with compacting. *Used in: Chapter 8.*

Single Source of Truth (SOT). The principle that every authoritative piece of information lives in exactly one place, and that place is updated when the information changes — duplicates are forbidden because duplicates drift. *Used in: Chapters 10 and 11; Structure and Organization.*

Sonnet. A mid-tier model in Anthropic's Claude family. Mentioned as a cost-effective option for tasks that do not require the strongest reasoning model. *Used in: Chapter 2.*

SOP (Standard Operating Procedure). A documented procedure for performing a routine operational task consistently across people and time. *Used in: The Reader.*

Source of Truth (SOT). A file that holds authoritative project content. In the book, SOT files are preferably Markdown; all changes go into SOT files; artifacts are derived from them. *Used in: Chapters 10, 11 and 14.*

Spec-driven development. A working pattern in which the specification is built first and the execution — code, contract, protocol, policy — is generated from it. Increasingly automated by long-running AI agents; the methodology in this book is aimed at producing the specification at the level of detail autonomous execution demands. *Used in: Chapter 16; Why This Book?.*

Specification by Example. Gojko Adzic's 2011 discipline of grounding every requirement in a concrete example that humans and tools can both read. *Used in: Validation and Quality.*

SPIRIT. A reporting checklist for clinical trial protocols — Standard Protocol Items: Recommendations for Interventional Trials. Specifies the items a protocol document should contain. *Used in: Chapter 11.*

Statelessness. The property that the model itself keeps no memory between calls — every turn is processed from scratch. The session's persistence is maintained outside the model, by the agent. *Used in: Chapter 8.*

Sub-agent. A separate non-interactive session, spawned by an AI agent, that handles a piece of work — frequently a tool call — independently and returns only a summary to the main conversation. Reduces context pollution from heavy tool use. *Used in: Chapter 3.*

Suggest mode. A mode of prompting in which AI is asked for options rather than for a chosen course of action. *Used in: Chapter 4.*

Sycophancy. See *Overconfidence*.

T

Tacit knowledge. Knowledge a practitioner has but cannot easily articulate — intuition, habit, judgment. Michael Polanyi's distinction. The common-sense problem with AI is fundamentally a tacit-knowledge gap. *Used in: Intuition and Muscle Memory; Knowledge and Memory.*

Test-Driven Development (TDD). A software practice of writing the test before writing the code. The book applies the same discipline more broadly: prepare the validation before asking AI to do the work. *Used in: Validation and Quality.*

Token. A small piece of text — roughly four characters or three quarters of a word in English — that a language model uses as its unit of input and output. Pricing is per token in both directions. *Used in: Chapters 2, 4, 5, 8 and 10.*

Tokenization. The process by which a model splits text into tokens before processing. *Used in: Chapter 2.*

Tool call. A structured note the model emits in place of an action it cannot perform itself — "edit this file," "search the web." The agent reads the note, executes the action, and feeds the result back as the next message. *Used in: Chapters 8, 9 and 10.*

Tool result. The message an agent returns to the model after executing a tool call. Sits in the conversation history alongside everything else and counts toward the context window. *Used in: Chapter 8.*

Tradecraft. In this book, the foundational methods, disciplines, and conventions that turn AI from a powerful general-purpose tool into a collaborator operating inside the project's rules. The bootstrap files in Chapter 6 install the initial tradecraft; later chapters extend it. The term is borrowed from intelligence work, where it names the craft in which an operative is trained. *Used in: Chapters 6 and 7; Why This Book?; The Method.*

Triangulation. The discipline of recording the same underlying project subject in several memory files, each describing it from a different angle — a set of user stories, a security requirements list, a regulatory checklist, a glossary, a decisions log, a consent flow. Each file is a projection that captures part of the subject from one angle. The projections together pin

down what no single file could capture alone, and the specification AI helps you build — data model, architecture, operational shape — is grounded in those projections rather than treated as one of them. The term is borrowed from the everyday technical sense — pinpointing a spy radio transmitter from receivers in different cities, locating a phone via cell towers and GPS — where several independent observations of the same subject, each from a different vantage point, converge on what no single one could capture. *Used in: Chapter 8; Knowledge and Memory.*

Triggers. A category of rules that spawn activity in response to a condition — *when X happens, do Y* — paired with **Guardrails** as the two coordinate kinds of rule the book uses. As prompt-embedded rules they apply to a single prompt by default and require explicit session scoping to carry forward; as durable rules they live in the project's rules file and apply on every turn. *Used in: Chapter 4.*

Trust but verify. A principle associated with arms-control diplomacy in the late twentieth century: cooperation requires trust, but trust without verification is naive. The book applies it to AI output. *Used in: Military and Intelligence Thinking.*

Turn. One full round trip from the local agent to the remote model and back. The agent assembles a payload, sends it to the model, receives the response, and decides what to do next. A single user prompt can produce many turns under the hood; the agent's autonomy comes from this loop and from the local work the agent performs between turns. A turn is sometimes called a round trip or an API call; the book uses "turn" consistently. *Used in: Chapters 2 and 3.*

U

Ubiquitous Language. A core idea in domain-driven design: a project should use one shared vocabulary — the same terms in conversations, documentation, and code — and that vocabulary should come from the domain itself. *Used in: Language and Meaning.*

User flow. A description of the steps a user takes to accomplish a task — for

example, opening an app, choosing a service, paying, and receiving confirmation. *Used in: Chapter 10.*

User story. A short statement of a user need in a fixed format — typically "as a user, I want X so that Y." Common in agile software development. *Used in: Just Tell Me What You Want?.*

V

Validation. The discipline of checking that AI's output is correct — comparing it against trusted sources, recording findings in a standard format, and acting on them. The book devotes an entire chapter to validation methods. *Used in: Chapters 5, 11 and 12; Validation and Quality.*

Verification and Validation (V&V). The systems-engineering distinction between verification ("did we build the thing right?") and validation ("did we build the right thing?"). *Used in: Validation and Quality.*

Version control. A system that tracks every change to a set of files so any change can be reviewed, attributed, and undone. The book uses Git as its example and treats version control as the rollback safety net for AI work. *Used in: Chapters 1, 5 and 8; Structure and Organization.*

Vibe coding. Writing software by prompting an AI agent turn by turn — it generates code, you review and correct, and you stay in the loop on every step. Contrast with spec-driven development, where the judgment is settled up front in a specification the agent builds from. *Used in: Why This Book?.*

Vision capabilities. The model features that allow it to process images. Far more expensive in tokens per page than reading the same content as text. *Used in: Chapter 2.*

W

Working memory. The cognitive capacity for holding and manipulating a small set of items in mind at once — typically four to seven, depending on the study. *Used in: Cognitive Science and Human Factors.*

Methodological Roots

The *Why This Book?* chapter promised a reference section for the curious — a list of the pre-AI methodologies that shaped the methods in this book. Here it is.

Every method in this book was derived from something that existed before AI arrived. Requirements management, change control, auditability, validation, glossaries — software engineering, project management, military operations, and quality assurance spent decades building working methodologies for all of these. But those methodologies were designed for teams of people, not for a solo practitioner working with an AI that has no persistent memory and no domain judgment. The book adapted them — sometimes beyond recognition — to exploit AI's strengths and defend against its failures.

This section names the originals. For each one, you get a brief description of what it is and which parts of the book it shaped. The goal is not to teach these methodologies but to give you a thread to pull if you want to understand the deeper roots of something the book asked you to do.

The entries are grouped by theme, not by chapter. Many of them influenced more than one method in the book.

Structure and Organization

Configuration Management and Change Control. The discipline of tracking every change to a system — what changed, when, by whom, and why. In traditional software, the discipline covers source code, documentation, build configurations, and deployment artifacts. In this book, it shaped `CLAUDE.md` (Chapter 6), the intake log (Chapter 7), and the version control safety rules (Chapter 1). The principle that every file's purpose must be registered, that changes to `CLAUDE.md` require explicit approval, and that rollback must always be possible — all of these come from configuration management. The book's insistence on protecting version control from AI's

reach is a direct application of change control's core idea: the record of changes must be tamper-proof.

Single Source of Truth. The principle that every piece of authoritative information lives in exactly one place. When you need that information, you go to that place. When you update it, you update it there and nowhere else. Duplicates are forbidden because duplicates drift. This principle runs through the entire book. The SOT–ART separation (Chapter 10) is the most explicit expression — one authoritative Markdown file, one disposable visual artifact — but the same principle drives `CLAUDE.md` as the single index of project structure (Chapter 6), the intake log as the single record of user input (Chapter 7), and the glossary as the single authority on vocabulary (Chapter 9). The ground-truths file (Chapter 11) is another instance: one place where domain assumptions live, consulted on every turn.

Separation of Concerns. The principle that different responsibilities should live in different places, with clear boundaries between them. In software, this means the database layer does not know about the user interface, and vice versa. In this book, the principle shows up as the two-directory project structure (Chapter 1) — one directory AI controls, one directory AI cannot touch. It shows up again in the SOT–ART separation (Chapter 10) — authoritative content and visual output are different concerns handled by different files. It shows up in the distinction between `CLAUDE.md` as an index and the memory files as content (Chapter 8). And it shows up in the layered method (The Approach) — each layer of the project addresses a different concern, from business idea down to code.

Infrastructure as Code. The practice of defining system behavior in declarative files rather than manual configuration. In DevOps, this means your server setup lives in a version-controlled file, not in someone's memory of which buttons they clicked. `CLAUDE.md` (Chapter 6) is this book's version of the same idea: AI's behavior is defined in a file, version-controlled, auditable, and reproducible across sessions. The artifact regeneration protocol (Chapter 10) — delete everything, rebuild from source — mirrors the infrastructure-as-code principle that any environment can be torn down and rebuilt from its definition.

Sandboxing. A computer-security and operating-systems pattern where

untrusted code or processes run in an isolated environment with restricted access to the rest of the system. The `.tmp/` scratch directory (Chapter 6) is sandboxing applied to AI's working files — AI may create, modify, and delete inside it without approval, but the rest of the project is governed by the registry's discipline. The Scratch exemption rule formalizes the boundary: files inside `.tmp/` are exempt from every File Registry rule, and nothing referenced from a registered file may live there. The pattern reappears in Chapter 1 as the two-directory project structure (one folder AI controls, one outside its reach), drawing on the same isolation principle.

Convention over Configuration. A design philosophy popularized by Ruby on Rails: when a sensible default exists, use it without requiring explicit configuration, and reserve configuration for cases that deviate from the default. The bootstrap registry (Chapter 6) applies this through placement conventions — log-type files default to the `logs/` subfolder, generated files default to `artifacts/`, spec files default to `specs/`. None of these placements need to be specified when adding a new file; AI infers the placement from the file's type and protocol. The conventions are SHOULD-level rather than MUST-level, so deviations are allowed when justified, preserving the philosophy's spirit: convention as a default, not a cage.

Standard Operating Procedures and Runbooks. Operations and military disciplines that capture a procedure once, in a stable artifact, so it can be executed reliably across different operators and situations. `rule-analysis.md`, `rule-conflict-protocol.md`, `regen-all.md`, and `disambiguate.md` (Chapters 6, 9, and 10) are SOPs for AI — each defines a procedure that AI follows when a specific trigger fires, with numbered steps and explicit decision gates. The runbook tradition also informs the playbook pattern (Chapter 4): a numbered checklist file that can be executed one step at a time, possibly across multiple sessions, each step self-contained enough for a stranger to pick up where the previous step left off.

Language and Meaning

Domain-Driven Design — Ubiquitous Language. Eric Evans introduced the concept of a ubiquitous language in his 2003 book on domain-driven design. The idea is that a project should use one shared vocabulary — the

same terms in conversations, in documentation, and in code — and that vocabulary should come from the domain itself, not from the tooling. When the domain expert says "consideration" and means the bargained-for exchange, everyone on the project says "consideration" and means the bargained-for exchange. The project glossary (Chapter 9) is a direct application of this principle, adapted for AI. The difference is that a human team can absorb vocabulary through conversation. AI cannot — it falls back to training defaults unless the vocabulary is written down and enforced by rule.

Controlled Vocabulary and Data Dictionaries. Before domain-driven design gave the concept a name, the practice of maintaining a list of approved terms with precise definitions was standard in technical writing, library science, and database design. Data dictionaries — tables that define every field, every entity, and every allowed value — serve for data what a glossary serves for language. The closure rule in Chapter 9 (every term referenced in a definition must itself be defined or unambiguous) and the circularity check (definitions must ground out in external meaning) come from the well-formedness constraints that data dictionaries and formal ontologies have enforced for decades.

Technical Writing Best Practices. The voice rules (plain language, short sentences, audiobook compatibility, accessibility for non-native speakers) draw on a long tradition of technical writing guidance. Plain language movements, readability standards, and style guides like the U.S. Federal Plain Language Guidelines and the Microsoft Writing Style Guide all emphasize the same things this book practices: prefer simple words, keep sentences short, avoid jargon, define terms on first use, and write for the reader who knows the subject but not your private vocabulary.

RFC 2119 — Key Words for Use in RFCs to Indicate Requirement Levels. Scott Bradner's 1997 IETF document defined a small set of capitalized priority markers — MUST, SHOULD, MAY, and their negations — for use in technical specifications. The capitalization distinguishes the keywords from ordinary English usage and signals that the word is acting as a normative requirement. Chapter 4 introduces the convention in **The Language of Rules**, and Chapter 6 uses it throughout the bootstrap: every

rule uses MUST, SHOULD, or MAY in uppercase to make its enforcement level unambiguous to AI.

Legal Specification Protocol. Stanford CodeX's 2019 work on legal specification proposed bringing formal-specification discipline — precise definitions, machine-readable structure, traceable cross-references — to legal documents. The book's glossary rule (Chapter 9), with its closure constraint (every referenced term defined) and circularity check (definitions ground out in external meaning), aligns directly with the well-formedness requirements that legal specification protocols impose on contract language.

Auditability and Traceability

Audit Trails and Chain of Custody. The intake log (Chapter 7) is an audit trail — a chronological record of every action taken, by whom, and when. Audit trails are foundational in accounting, legal proceedings, regulatory compliance, and forensic analysis. Chain of custody adds the requirement that the record itself is tamper-proof and attributable. The book's insistence on verbatim recording (no AI rephrasing), timestamps, and attribution by name comes directly from these traditions. The affected-files list at the end of each log entry is a traceability link — connecting what was said to what changed — borrowed from requirements traceability in systems engineering.

Requirements Traceability. In systems engineering, every requirement must be traceable — you can follow it from its origin (who asked for it and why) through its implementation (which component satisfies it) to its verification (which test proves it works). The book builds a lighter version of this chain. The intake log traces decisions to their origin. The Devil's Advocate (Chapter 11) traces every claim in a draft back to a recorded ground truth, or flags it as having no source. The SOT-ART registry traces every artifact back to its source file. The gap format traces findings back to the validation method that produced them. None of this is as formal as a requirements-traceability matrix in aerospace engineering, but the principle is the same: when something goes wrong, you can follow the thread back to the source.

Intelligence Reporting and Cable Traffic. The intake log's format — timestamped, attributed, verbatim, with context for responses — resembles

the structure of intelligence cables and field reports. In intelligence work, every communication from the field is logged with who sent it, when, what it said (verbatim), and what question or tasking prompted it. The log is the institutional memory. Analysts who were not in the room can reconstruct what happened by reading the traffic. The intake log serves the same function for your project: a reader who was not in the session can reconstruct what decisions were made and why.

Two-Person Rule (Dual Control / Maker-Checker). A control principle from nuclear-weapons security, financial accounting, and high-trust operations: no consequential action proceeds without independent authorization from a second party. The book applies this principle relentlessly. AI proposes; you approve. Every registry change, every glossary entry, every rule modification, every artifact regeneration, and every file deletion runs through the same shape: propose, present, wait for explicit approval, then act. The pattern is most visible in Chapter 6 (rule analysis and the Inventory change rule), Chapter 9 (the disambiguate protocol), Chapter 10 (Step 3 of `regen-all`), and Chapter 11 (the Guarded Edits Protocol). The shape is a deliberate substitute for the missing second reviewer in a solo-with-AI workflow.

Event Sourcing and Append-Only Logs. A pattern from database systems and distributed computing: instead of overwriting state in place, every change is recorded as an immutable event in an append-only log, and current state is derived by replaying the log. The intake log (Chapter 7) is event sourcing applied to user input — every input becomes an immutable entry with verbatim content, timestamp, and attribution. The append-only protocol (Chapter 6) formalizes the discipline at the registry level: AI may only append new entries; past entries and the file itself are protected from modification or deletion; corrections take the form of new entries that reference the prior one by identifier. The discipline reappears in the rule-conflict log (Chapter 6) and applies to any audit-style file the project later registers.

Validation and Quality

Verification and Validation (V&V). The distinction between verification

("did we build the thing right?") and validation ("did we build the right thing?") is foundational in quality assurance and systems engineering. The polygraph and Devil's Advocate methods (Chapter 11) address both halves — verification (does the draft match the specifications and trace to recorded grounding?) and validation (do the recorded decisions hold up when probed?). The validation chapters are organized around V&V thinking, even though they never use those terms.

Plan-Do-Check-Act (PDCA). Also called the Deming cycle, after W. Edwards Deming, who popularized it in quality management. Plan the work. Do the work. Check the result. Act on what you learned — fix problems, update procedures, feed improvements back into the next cycle. The book's discipline of preparing to validate before asking AI to work (Chapter 11), then checking the output, then feeding corrections back into source-of-truth files, is a PDCA cycle.

Test-Driven Development. The practice of writing the test before writing the code — defining "correct" before producing the output. The book's rule that you prepare to validate before you ask (Chapter 11) is the same principle applied more broadly. You decide how you will know whether AI's output is right before you ask AI to produce it.

Acceptance Sampling and Statistical Quality Control. When the output is too large to review in full, you review a random sample and infer the quality of the whole from the quality of the sample. Chapter 11's sampling method comes directly from this tradition, which originated in manufacturing (MIL-STD-1916 and its predecessors) and is standard practice in any domain where inspecting every unit is impractical.

Red Teaming and Adversarial Review. The practice of assigning a team to deliberately attack a plan, a system, or an argument — to find the weaknesses before an adversary does. It originated in military war games and is now standard in cybersecurity (penetration testing), policy analysis, and intelligence. The book's adversarial prompting method (Chapter 11) and critique mode (Chapter 4) are lightweight red-team exercises: you ask AI to find the weaknesses in its own output, then separate the real findings from the noise.

Independent Verification and Validation (IV&V). The principle that the entity verifying the work should be independent of the entity that produced it. In safety-critical systems, IV&V is often a contractual requirement. The book applies this principle in two ways. First, the fresh-session requirement for the polygraph (Chapter 11): a new session that has not seen the production conversation cannot be biased by it. Second, the second-opinion method: asking a different model or tool the same question, so the verification comes from a system with different training and different blind spots.

Specification by Example. Gojko Adzic's 2011 book introduced a discipline for turning intent into executable specifications by grounding every requirement in a concrete example that both humans and tools can read. The same example illustrates the requirement, drives the test, and lives in the documentation. The book's running-example discipline (introduced in *Why This Book?* and carried through subsequent chapters), the Devil's Advocate's question of whether every claim traces to a recorded source (Chapter 11), and the SOT–ART pattern's insistence on text-first specification (Chapter 10) all carry forward the core idea: a specification is only as good as the concrete examples that ground it.

The Checklist Manifesto. Atul Gawande's 2009 book popularized a discipline that aviation and surgery had already used for decades: a short, written checklist of items to verify before a critical action, run every time without exception. The manual review section in Chapter 11 applies the same idea: "review with a checklist. Write down what you are looking for before you start reading: 'Does this match the glossary? Are all the fields from the data model present? Are there any claims not traceable to a file?'" The checklist is what turns reading into checking — a list of questions removes the load of remembering what to look for. The same posture informs the polygraph's structured finding categories (A through F, later G), each a checklist item the protocol runs on every write to a guarded file. The Dry Run (Chapter 13) makes the persona itself the checklist — a specialist doing their real job runs through their craft's standard decisions without anyone writing the list down.

Pre-mortem Analysis. Gary Klein's cognitive-science technique: before starting a project, imagine it has already failed and work backward to identify

the reasons. The "Before You Ask" section in Chapter 11 is pre-mortem applied to AI output: "If you wait until the output is in front of you to figure out how to check it, two things happen. First, the output shapes your review — you read for sense rather than for errors, and AI's confident tone nudges you toward approval. Second, you rush." Validation is prepared *before* the prompt is sent, so the review criteria do not get reshaped by the very output they are meant to evaluate. The same posture shows up in plan mode and the playbook pattern (Chapter 4) — work out the shape of the work before starting it. The Dry Run (Chapter 13) is a constructive cousin: instead of imagining failure, it has AI simulate a specialist doing the job and surface the decisions the specification has not made yet.

Defense in Depth. A security and military principle: rather than relying on a single barrier, stack multiple independent defenses, so that an adversary or an error must defeat all of them to succeed. The validation toolbox (Chapter 11) is defense in depth made operational — the polygraph, the Devil's Advocate, fresh-session replay, second opinion, automated checks, adversarial prompting, provenance checks, sampling, time before review, and a second pair of eyes. The chapter states the principle directly: "The more layers you have, the less likely it is that a single mistake slips through all of them. The cost of stacking two cheap methods is still less than the cost of one mistake making it into production." Each layer catches a different kind of error; the stack is what makes the overall system reliable.

Swiss Cheese Model. James Reason's accident-causation model from safety engineering: each defensive layer has holes (each method has blind spots), and accidents happen only when the holes line up across every layer at once. The book's stacking of validation methods (Chapter 11) is the Swiss Cheese Model in practice. The polygraph misses ungrounded claims; the Devil's Advocate catches those. The Devil's Advocate misses purely judgment-level errors; manual review catches those. Manual review misses scale problems; sampling and automated checks catch those. Each method has its own blind spots, and the stack is designed so that no single failure mode survives every layer.

The Heilmeier Catechism. George Heilmeier, as director of DARPA, framed a fixed set of questions every research proposal had to answer —

what are you trying to do (in plain language), how is it done today, what is new in your approach, who cares, what are the risks, what will it cost. The Dry Run (Chapter 13) shares the conviction that the right questions, asked of every proposal, expose what vagueness hides — but it generates its questions from a specialist's job rather than from one universal list, precisely to avoid the generic-checklist failure the book calls the Omission Trap.

Knowledge and Memory

Knowledge Management. The broad discipline of capturing, organizing, and making accessible the knowledge that an organization needs to function. The book's file-based memory system (Chapter 8) — `CLAUDE.md` as an index, memory files as content, the intake log as institutional history — is a small-scale knowledge management system. The ground-truths file (Chapter 11) is a knowledge base of domain facts. The glossary (Chapter 9) is a controlled vocabulary. The self-check mechanism (Chapter 8) — starting a fresh session to expose gaps in the memory files — is a knowledge audit.

Tacit Knowledge and Knowledge Transfer. Michael Polanyi distinguished between explicit knowledge (things you can write down) and tacit knowledge (things you know but cannot easily articulate — intuitions, habits, judgment calls). The common-sense problem (Chapter 11) is at heart a tacit-knowledge problem. You know things about your business that you have never written down because you never needed to. AI does not share that tacit knowledge. The ground-truths file is a mechanism for converting tacit knowledge into explicit knowledge, one push-back at a time. The mental model of AI as a new employee who needs onboarding (Chapter 11) comes from the same tradition.

Shift Handoff Protocols. In hospitals, factories, military operations, and any environment with rotating personnel, a structured handoff ensures that the incoming shift knows what the outgoing shift learned. File memory (Chapter 8) maps directly onto this practice. Each AI session is a shift. The memory files are the handoff notes. `CLAUDE.md` is the standing orders. The self-check — opening a fresh session and asking what AI knows about content the previous session was supposed to record — is the equivalent of the incoming nurse reading the chart and asking "wait, what about the patient in room 12?"

Methodological Triangulation. Norman Denzin formalized the concept in *The Research Act* (1970): when studying a question, converge multiple independent instruments — interviews, surveys, direct observation, document analysis — on the same subject, on the theory that any single instrument can mislead but findings that survive across several are hard to discount. The roots are older: surveyors used triangulation from multiple known stations to fix the position of a third point well before social scientists named the technique. Chapter 8 carries the same idea into project memory. A specification, a glossary, a decisions log, a safety plan, a consent flow — each file is a projection of the same underlying subject from a different direction, and together the projections pin down what no single file could. The principle reappears twice in Chapter 11. The Polygraph section names triangulation as the shape ground-truth capture takes: the project's ground truths live across several projection files rather than in one, and the polygraph reads across the set. The validation toolbox echoes the same shape on the methods side, stacking independent instruments (polygraph, Devil's Advocate, fresh-session replay, second opinion) so a finding that survives several of them is the one to act on.

Socratic Method. A philosophical and pedagogical tradition from classical Greece: progress through questions rather than statements, surfacing assumptions and inconsistencies by inquiry rather than assertion. The prompt modes in Chapter 4 — research, suggest, analyze, explain, critique — are different shapes of structured inquiry, each asking AI a different kind of question rather than commanding it to produce. The Devil's Advocate (Chapter 11) is explicitly Socratic: every claim in a draft is questioned against recorded grounding, and ungrounded claims surface for review. The interactive wiki (Chapter 14) extends the same posture across the whole project — the spec becomes a body of recorded answers, the agent fields the questions, and the user navigates the specification by asking rather than by re-reading. The Dry Run (Chapter 13) turns the same posture forward: AI, in the role of a specialist, asks the questions the specification has not yet answered.

Design and Architecture

Separation of Model and View (MVC and Its Variants). The Model-

View-Controller pattern, introduced by Trygve Reenskaug in 1979, separates the data (model) from its visual representation (view) and the logic that connects them (controller). The SOT-ART separation (Chapter 10) is a direct descendant. The SOT file is the model — the authoritative data. The artifact is the view — a disposable visual representation. Changes go into the model. The view is regenerated. The book does not need a controller because AI plays that role: AI reads the model, produces the view, and you review the view to check the model.

Design by Contract. Bertrand Meyer's principle (1986) that software components should interact through precise, verifiable contracts — preconditions, postconditions, and invariants. The book's rules are contracts in natural language. The first rule (protect `CLAUDE.md`) is an invariant. The glossary's closure rule (every referenced term must be defined) is an invariant. The Guarded Edits Protocol (every proposed write to a guarded file must first pass a polygraph check across six finding categories) is a contract between you and AI about how validation will proceed before any recording lands. The language is English instead of Eiffel — the programming language Meyer designed around this principle — but the discipline is the same.

Iterative and Incremental Development. The practice of building a system in small cycles, each producing a working increment that is reviewed and refined before the next cycle begins. The gradual method (The Approach) — idea, ground truths, product vision, data model, architecture, interfaces, code — is an incremental process where each layer builds on the previous one and is reviewed before the next begins. The polygraph (Chapter 11) is iterative within a single recording: every proposed write triggers a check, and findings loop you back to amend the proposal before the write lands.

Progressive Elaboration. A project management concept (prominent in PMBOK) where the project plan becomes more detailed as more information becomes available. Early phases are broad; later phases are specific. The book's layered method is progressive elaboration: the idea is broad, the ground truths are narrower, the product vision is narrower still, and by the time you reach code, most decisions are already made. Each layer narrows the room for guesswork in the next. The Dry Run (Chapter 13) makes the

elaboration explicit: each round descends one layer, and the decisions recorded in one round are what make the next layer's questions answerable.

Topological Sort and Build Automation. A computer-science fundamental of build systems (Make, Bazel, Ninja, modern CI pipelines): given a graph of files with dependencies, compute an order in which each file's dependencies are built before the file itself. Step 2 of the `regen-all` procedure (Chapter 10) is a topological sort applied to artifact regeneration: "AI MUST sort the generated files so that, for every file in the list, all the generated files listed in its `Dependencies` cell are placed before it in the sorted list." The chapter notes the lineage explicitly: "The sort itself is one bullet of plain English — doing the work that for decades required dedicated build tools like `make`."

Directed Acyclic Graphs (DAGs). A mathematical and computer-science structure where edges have direction and no path loops back on itself. The acyclicity discipline in Chapter 10 — "The `Dependencies` cells MUST NOT form a chain that loops back on itself. A registry change that would introduce such a loop MUST be refused" — enforces a DAG topology on generated-file dependencies. The DAG property is what makes the topological sort well-defined and the regeneration procedure deterministic. The same structural discipline appears in the disambiguate protocol (Chapter 9): the circularity check on glossary definitions enforces an acyclic reference graph among terms.

Open-Closed Principle. Bertrand Meyer's design principle, later made famous by Robert Martin: software entities should be open for extension but closed for modification. The closed sets in the File Registry (Chapter 6) — types, protocols, and extensions, plus the analogous discipline for glossary entries — apply the principle directly. The base set is closed: AI must not introduce new values silently, and any value used must be one of the defined ones. The principle still permits extension: AI MAY propose new types, protocols, or extensions, with explicit user approval. The two halves together — closed by default, extensible with approval — are the open-closed shape.

User Flows and Customer Journey Maps. Practices from user-experience and product-management design: capture the steps a user takes through a system as a visual flow with branches, decision points, and handoffs, so the experience can be reviewed and refined before implementation. The `errata-`

`flow.md` running example (Chapter 10) is a user flow, deliberately structured as numbered steps with rationale on each. The Markdown form is the source of truth; the generated Mermaid diagram is the artifact you review. The pattern reappears throughout the book as a reference point — drawing a user flow for a booking process, mapping the participant-flow diagram for a research protocol, sketching a decision tree showing which policy applies when.

Personas. A user-research and product-management practice popularized by Alan Cooper: represent the user base as a small set of concrete, named archetypes with attributes that matter for design decisions. The *Reader* section in *Why This Book?* (front matter) is a persona enumeration — software architects and engineers; product managers; consultants; legal and patent practitioners; researchers; policy and compliance professionals; marketing, brand, and communications professionals; operations and data professionals; technical writers; authors of instructional, reference, and how-to material; and educators and curriculum designers. Each persona names a role, the kind of deliverable that role produces, and what makes the methodology relevant to that role. The list shapes how the book speaks throughout and which examples it reaches for in the chapters that follow. The Dry Run (Chapter 13) puts personas to a different use: AI adopts a specialist persona and does that specialist's job against the specification, so the persona's standard concerns surface as the questions the spec still has to answer.

Zachman Framework. John Zachman's 1987 enterprise-architecture framework: a matrix that crosses six interrogatives (what, how, where, who, when, why) with a series of stakeholder perspectives (planner, owner, designer, builder), on the premise that a complete description answers every interrogative from every perspective. The Dry Run (Chapter 13) borrows the perspective axis — the set of specialist personas you run dry runs for is a coverage grid, and skipping a persona skips its whole class of decisions.

Quality Attribute Workshops and Attribute-Driven Design. Methods from Carnegie Mellon's Software Engineering Institute for eliciting and designing around quality attributes — the "-ilities" such as scalability, availability, and security — before and during architecture, often through

concrete scenarios and later evaluated by the Architecture Tradeoff Analysis Method (ATAM). The Dry Run (Chapter 13) does the same kind of work: a software architect's dry run surfaces traffic, hosting, failure, and build-versus-buy decisions the way a quality-attribute workshop surfaces scenarios.

Architecturally Significant Requirements and Architecture Decision Records. An architecturally significant requirement is one that measurably shapes the architecture; an architecture decision record captures a single architectural decision with its rationale and consequences. The Dry Run (Chapter 13) targets exactly these — its gap list surfaces the architecturally significant decisions, and recording each decision with its reasoning is, in effect, keeping a decision record inside the specification.

Military and Intelligence Thinking

Commander's Intent (Auftragstaktik). A military doctrine where the commander communicates the desired end state and the purpose of the mission, then lets subordinates decide how to achieve it. The subordinate has freedom to adapt to conditions on the ground, as long as the intent is served. The intent-over-instruction principle (Chapter 4) is a direct application. You tell AI what you want to achieve and why. You state the constraints that matter. You let AI figure out how. If AI's understanding of the intent is wrong, the failure is visible in the reasoning, not hidden in the execution.

Intelligence Handler and Asset Model. In intelligence operations, a handler directs a field asset. The handler sets objectives, provides context, evaluates the intelligence that comes back, and decides what to act on. The handler does not do the asset's job. The handler's skill is not in fieldwork — it is in running the asset effectively. The *Why This Book?* chapter (front matter) uses this analogy explicitly: you are the handler, AI is the asset. You direct, contextualize, evaluate, and decide. The entire book teaches handler skills, not AI internals.

Compartmentalization. An intelligence and security principle: information is divided into compartments, and access to each compartment is restricted to those who need it. The two-directory project structure (Chapter 1) is compartmentalization. AI has access to the project folder. AI does not have

access to the backup directory. Version control lives outside AI's reach. `CLAUDE.md` controls what AI knows about the project's structure, but detailed content is compartmented into separate files that AI reads only when needed (Chapter 8).

Trust but Verify. A phrase associated with Cold War diplomacy (Ronald Reagan, quoting a Russian proverb, during arms control negotiations with the Soviet Union). The principle: cooperation requires trust, but trust without verification is naive. The book's entire validation chapter (Chapter 11) is built on this principle. Trust AI enough to let it do the work. Verify the output before you act on it. The specific methods — cross-checking, assumption tracking, fresh-session validation — are verification protocols.

Principle of Least Privilege. A computer-security principle dating to Jerome Saltzer and Michael Schroeder's 1975 paper *The Protection of Information in Computer Systems*: every component is granted only the minimum privileges it needs to perform its function. The two-directory project structure (Chapter 1) — AI can read and write the project folder, but the backup directory and version control are outside AI's reach — is least privilege applied to AI. The book extends the principle to version control safety ("Do not give AI access to your version control. If you use Git, either block AI from it entirely or allow read-only access at most") and to the scratch-directory boundary (Chapter 6). Each restriction follows the same reasoning: granting more access than necessary means more surface area for AI's mistakes to do damage.

Cognitive Science and Human Factors

Serial Position Effect. Hermann Ebbinghaus documented in 1885 that people remember items at the beginning and end of a list better than items in the middle. These are called the primacy effect and the recency effect. The "lost in the middle" problem described in Chapter 8 — where language models pay less attention to content in the middle of the context window — is a computational echo of this human cognitive bias. The book's advice to keep rules at the top of the context (primacy position) and to start fresh sessions when topics change (so that content AI needs remains near the recency position) is a practical application of both effects.

Working Memory Limits. Cognitive psychology has long established that human working memory is limited — roughly four to seven items at a time, depending on the study. The context window (Chapter 8) is AI's version of working memory, with its own limits. The book treats both seriously: keep rules lean, keep sessions short, start fresh when topics change. The advice is the same a cognitive psychologist would give a person trying to manage a complex task: externalize your memory into notes, and do not try to hold everything in your head at once.

Dunning-Kruger Effect and Calibration. The Dunning-Kruger effect describes the tendency of people with limited knowledge in a domain to overestimate their competence. The overconfidence section (Chapter 11) draws an explicit parallel: AI defaults to the tone of authority regardless of its grounding. The defense — never use tone as a signal of correctness — comes from the decision science literature on calibration: the practice of aligning confidence with accuracy. Well-calibrated experts say "I am not sure" when they are not sure. AI does not, and you have to compensate.

Rubber Duck Debugging and the Feynman Technique. Rubber duck debugging is a programming practice: explain your code to a rubber duck (or any inanimate listener), and the act of explaining forces you to confront what you do not understand. The Feynman technique is similar: explain a concept in simple terms, and the gaps in your understanding become visible. The Devil's Advocate (Chapter 11) uses the same mechanism — every claim in a draft that does not trace back to a recorded ground truth surfaces as a gap, and you decide what to do with each. The rubber duck does not talk back. AI does, and when AI cannot ground a claim, AI flags it.

Anchoring Effect. A cognitive bias documented by Amos Tversky and Daniel Kahneman: a person's judgment is disproportionately influenced by the first piece of information presented (the anchor), even when the anchor is irrelevant. The book applies the principle to prompt design (Chapter 4): "do not muddy the prompt with your own reasons. The moment you say 'I want X because Y,' you have given AI a justification to anchor on — and AI's trained tendency to please will quietly soften the critique or echo your reasoning back at you." The same caution informs the polygraph design (Chapter 11) — the validation session is fresh and the validator does not see the production

conversation, so the validator cannot anchor on whatever framing the production session had drifted into.

Mental Models. Peter Senge popularized the concept in *The Fifth Discipline* (1990); Charlie Munger and others have used the term broadly to mean a portable representation of how a system works, used to predict its behavior. The book uses the phrase explicitly and repeatedly. The execution-team analogy for sessions (Chapter 8): "Here is a mental model that helps. Imagine you manage an execution team handling many different areas of work..." The intelligence handler-and-asset analogy in *Why This Book?*. The book argues, in the **Intuition and Muscle Memory** section, that a working mental model — even one that is not scientifically accurate — is what lets you make good decisions when driving AI, and treats building those models in the reader as a primary goal.

Cognitive Load Theory and Worked Examples. John Sweller's framework from educational psychology (1988 onward): human working memory is severely limited, so instruction should manage extraneous cognitive load and pair new concepts with worked examples that show every step in full. The "keep CLAUDE.md short" rule (Chapter 5) is load management applied to AI's context window — the bootstrap (Chapter 6) is deliberately small, with longer protocols externalized to conditional files. The book's pedagogy mirrors the worked-example effect: every method is introduced with a full dialog showing real prompts, real AI responses, and real file edits, so the reader sees the moves applied rather than abstracted. The choice is explicit in *Why This Book?*: "The book gives a lot of space to real examples... They give you a clearer sense of what is happening inside the conversation, and they implant a way of thinking that abstract description never will."

Anti-Patterns and Cautionary Principles

Goodhart's Law. "When a measure becomes a target, it ceases to be a good measure." Charles Goodhart stated this in the context of monetary policy, and it has since been applied broadly. The rules trap (Chapter 5) is Goodhart's Law applied to AI rules. When you write a rule that measures a specific behavior, AI can satisfy the measure without satisfying the intent. "Keep it short" produces something technically short that misses the point. The book's

response — intent over instruction — is the standard remedy for Goodhart's Law: describe the goal, not the metric.

Over-Specification. The software engineering anti-pattern of specifying so much detail that the specification itself becomes the program — except written in a language with no compiler, no type checker, and no guarantee of consistent execution. The rules trap (Chapter 5) names this explicitly: if your rules were perfect, you would have written a program in English, to be executed by an expensive, non-deterministic engine. The book argues for minimal, intent-heavy rules rather than exhaustive, instruction-heavy ones.

Asimov's Three Laws of Robotics. Isaac Asimov's fictional laws governing robot behavior, introduced in *I, Robot* (1950) and explored across his short stories. Asimov used the stories to expose the failure modes of cleanly written rule sets — deadlocks, exploits, and emergent reinterpretations. Chapter 6 leans on four of these stories as case studies for the runtime rule-conflict protocol. *Runaround* shows two Laws deadlocking a robot into walking in circles; *Liar!* shows a Law forcing pathological behavior when the truth would cause harm; *Little Lost Robot* shows how a minor modification to a Law produces dangerous behavior nobody predicted; *The Evitable Conflict* shows machines reinterpreting a Law at global scale and overriding the humans the Law was meant to serve. The bootstrap's stop-and-log posture for runtime rule conflicts is the direct response to the failure modes Asimov mapped seventy years before AI agents existed.

Process Frameworks

Agile Principles. The Agile Manifesto (2001) emphasized individuals and interactions over processes and tools, working software over comprehensive documentation, customer collaboration over contract negotiation, and responding to change over following a plan. The book shares several of these values: short iterations (fresh sessions, tight write-check-resolve cycles), working output at every stage (each layer of the gradual method produces reviewable artifacts), human judgment at every decision point, and adaptation over rigid procedure. The book is not Agile in the formal sense — there are no sprints, no stand-ups, no backlog — but the underlying values overlap.

Lean Thinking — Eliminate Waste. Lean manufacturing, adapted to software by Mary and Tom Poppendieck, emphasizes eliminating waste — any activity that does not add value. The token efficiency chapter (Chapter 2) is lean thinking applied to AI costs: convert expensive formats once instead of paying the reading cost repeatedly, keep `CLAUDE.md` lean because it ships with every turn, and start fresh sessions instead of carrying irrelevant context. The advice to avoid the Omission Trap (Chapter 11) — because it produces noise dressed up as due diligence — is waste elimination applied to validation.

Systems Thinking. A discipline associated with Jay Forrester, Donella Meadows, and Peter Senge: understand a system as a set of components whose interactions produce behavior the components do not exhibit on their own. The book's stance on rules (Chapter 7) is explicitly systems-thinking: "Two rules written for entirely separate purposes — one policing the relevance of user input, one recording an audit trail — combined on a single exchange to produce behavior neither rule describes on its own. Each project rule may be simple in isolation, but in combination they produce 'intelligent' emergent behavior, turning the AI agent into a useful partner in your work rather than a tool you steer one move at a time." The same posture informs the bootstrap's design (Chapter 6) — each piece is small, the value comes from how they compose — and the validation toolbox in Chapter 11, where the reliability of the whole exceeds the reliability of any single method.

Definition of Ready. An Agile practice: a checklist a backlog item must satisfy — often summarized by the INVEST criteria (independent, negotiable, valuable, estimable, small, testable) — before the team will start work on it. The Dry Run (Chapter 13) applies the same gate to a whole area of the specification: the area is ready when a fresh round of the dry run surfaces no decision that would block the specialist.

Set-Based Concurrent Engineering and the Last Responsible Moment. A Lean product-development practice, observed at Toyota, of keeping several design options open and deferring each decision to the "last responsible moment" — the point past which delay would forfeit an option. The Dry Run (Chapter 13) names this as the practice it inverts. An agent executes the whole specification in one pass, with no human making decisions mid-build,

so the last responsible moment collapses to before the build starts. That is why spec-driven development front-loads the decisions an agile team would defer.